Chronos-ESM

A Differentiable Earth System Model

Theory, Numerics, and Climate Applications

Bijan Fallah
and the Chronos-ESM contributors

Version 0.1.0 (research preview)
Compiled June 23, 2026

Source: https://github.com/bijanf/ESMB

Preface

Chronos-ESM is a fully *differentiable*, GPU- and CPU-capable coupled Earth System Model written in JAX. It couples a multi-level spectral primitive-equation atmosphere (built on Google's DINOSAUR dynamical core), a z-level primitive-equation ocean, a thermodynamic sea ice model, and a land-surface scheme, at a teaching-friendly T31 ($\approx 3.75°$) resolution that runs in minutes-to-hours on a single GPU.

This book has three aims. First, to be **equation-complete and honest**: every prognostic equation, closure, and numerical scheme is derived from, and cross-referenced to, the actual source code — including its approximations and known biases. Climate-model documentation too often hides the gap between the textbook equations and what the code really integrates; here the two are presented together, with the code location given beside each equation. Second, to be **pedagogical**: the model is small and fast enough that a student can run a coupled climate experiment, change an equation, and see the result the same afternoon. Each chapter ends with exercises that use the model as a laboratory. Third, to showcase **differentiable Earth-system modeling**: because the entire model is written in JAX, one can take the gradient of any output (global temperature, AMOC strength, a tipping threshold) with respect to any input or parameter — enabling calibration, sensitivity analysis, and data assimilation that are impractical with conventional models.

The book is organized in five parts: *Foundations* (the continuous and discrete mathematics, and the differentiable-programming paradigm); *The Component Models* (ocean, atmosphere, ice, land, and the coupler, each derived from the code); *Forcing, Sensitivity, and Data Assimilation*; *Canonical Climate Test Cases* (the standard phenomena every climate model must reproduce — mean climatology, the AMOC and its tipping, mid-latitude jets and storm tracks, the ITCZ and monsoons); and *Community, Standards, and Teaching*, including a roadmap toward CMIP7 participation and a guide to using Chronos-ESM in the classroom.

A note on status. Chronos-ESM is research software under active development. Where a component is a deliberate simplification or a work in progress, this book says so explicitly — the honest documentation of an evolving model is part of its scientific value.

Contents

List of Figures

Part I

Foundations

Chapter 1

Introduction: A Differentiable Earth System Model

1.1 What is Chronos-ESM?

Chronos-ESM is a coupled Earth System Model (ESM): a numerical representation of the physical climate system in which an *atmosphere*, an *ocean*, *sea ice*, and a *land surface* evolve together, exchanging heat, water, and momentum. What distinguishes it from the large community models (CESM, MPI-ESM, IPSL-CM, and the rest of the CMIP fleet) is not the physics it contains but *how it is written* and *how fast it runs*:

- **It is differentiable.** The entire model — every flux, every time step, every spectral transform — is written in JAX (Bradbury et al., 2018), a system for composable program transformations. As a consequence one can compute the exact gradient of *any* scalar diagnostic (global-mean temperature, the strength of the Atlantic overturning, the freshwater flux at which that overturning collapses) with respect to *any* input or tunable parameter, by automatic differentiation. This turns calibration, sensitivity analysis, and variational data assimilation from heroic, model-specific efforts into one-line operations (Part III).
- **It runs on a laptop or a GPU.** At its default T31 spectral resolution ($\approx 3.75°$, roughly 96×48 horizontal grid points) a coupled century integrates in minutes-to-hours on a single GPU, and runs — more slowly but identically — on a CPU. There is no MPI and no bespoke build system: `pip install` and run.
- **It is small enough to understand.** The whole model is about 8,000 lines of Python. A student can read the ocean's tracer-advection routine, change the equation of state, and watch the climate respond the same afternoon.

These three properties — differentiable, fast, legible — are what make Chronos-ESM suitable both as a research instrument for the emerging field of differentiable Earth-system science and as a teaching model for graduate climate courses. This book serves both audiences.

1.2 The model at a glance

Chronos-ESM couples four components on a sphere of radius a rotating at rate Ω:

Atmosphere
A multi-level spectral primitive-equation dynamical core built on Google's differentiable DINOSAUR dycore — the dynamical engine behind NeuralGCM (Kochkov et al., 2024). It supports genuine baroclinic instability (growing synoptic eddies, a mid-latitude jet) and carries moisture for a tropical rain band. A legacy single-level spectral model is also provided for the cheapest experiments (Chapters 9–10).

Ocean
A z-level, hydrostatic, Boussinesq primitive-equation ocean with Gent–McWilliams eddy transport (Gent and McWilliams, 1990), initialized from the World Ocean Atlas climatology (Chapters 4–8).

Sea ice
A Semtner-style thermodynamic ice model (Semtner, 1976) (Chapter 11).

Land
A slab/bucket land-surface scheme with runoff routing (Chapter 12).

The components exchange fluxes through a coupler (Chapter 13) using bulk aerodynamic formulae, with an optional flux correction that allows the coupled system to be run either as a tightly observationally-constrained *control* or as a freely-evolving, forcing-responsive model.

1.3 Philosophy: equations and code, side by side

A recurring frustration in climate science is the distance between the equations a model is *said* to solve and the equations its code *actually* integrates — the discretizations, the limiters, the clamps, the approximations adopted for stability. This book takes the unusual stance that those details *are* the model and deserve first-class documentation. Throughout, each governing equation is presented in its continuous form, then in the discrete form the code uses, with a pointer to the exact source location:
↪ *Code:* `chronos_esm/ocean/veros_driver.py` — the ocean time step `step_ocean`
Where a scheme is a deliberate simplification — the single-level atmosphere's lack of vertical structure, or the interim thermohaline closure that stands in for a prognostic overturning until the momentum core is wired in — we say so plainly and quantify the consequence. Honest documentation of an evolving model is itself a scientific contribution.

1.4 Why differentiability matters

Conventional ESMs are evaluated and tuned by running them many times and comparing outputs — an expensive, derivative-free search over a high-dimensional parameter space. Because Chronos-ESM is differentiable, the sensitivity of any diagnostic J to any parameter vector $\boldsymbol{\theta}$ is available directly as $\partial J/\partial \boldsymbol{\theta}$ at the cost of roughly one extra model evaluation, by reverse-mode automatic differentiation. This enables:

- *gradient-based calibration* — fit parameters to observations by descent rather than by grid search;
- *variational data assimilation* (4D-Var) — find the initial state that best fits a window of observations;
- *rigorous sensitivity and tipping analysis* — e.g. locating a bifurcation by differentiating through the steady-state condition rather than by brute-force sweeps (Chapters 15 and 17).

A theme of Part III is that differentiability is powerful but not free: some targets — notably bifurcations such as an AMOC collapse — are non-smooth, and require the right formulation (fold continuation) rather than naive backpropagation through a long integration.

1.5 How to read this book

Part I develops the continuous and discrete mathematics common to all components, and the differentiable-programming paradigm. Part II documents each component model in equation-complete detail, derived from the code. Part III covers forcing, sensitivity, and data assimilation. Part IV evaluates the model against the canonical phenomena every climate model must reproduce. Part V looks outward: a roadmap toward CMIP7 participation, and a guide to using Chronos-ESM in the classroom. Appendices collect the notation, the parameter values, a map from code modules to chapters, and installation instructions.

Readers wanting a fast orientation may read this chapter, then jump to the test cases of Part IV to see what the model produces, returning to Part II for the mechanisms. Instructors will find a suggested course outline in Chapter 21.

Exercise 1.1. Install Chronos-ESM (Appendix D) and run the shortest coupled integration in the repository. Plot the global-mean surface temperature over the run. How long did it take, and on what hardware?

Exercise 1.2. List three scientific questions you could ask with a differentiable climate model that would be impractical with a conventional one. For each, identify the scalar diagnostic J and the parameter $\boldsymbol{\theta}$ whose gradient $\partial J/\partial \boldsymbol{\theta}$ you would compute.

Chapter 2

Mathematical and Numerical Preliminaries

This chapter assembles the mathematical scaffolding the rest of the book stands on: the spherical geometry every component shares, the discrete grids on which the continuous fields live, the metric factors that the curvature of the sphere forces into every gradient and divergence, the distinction between *prognostic* variables (which the model time-steps) and *diagnostic* ones (which it reconstructs each step), and the family of time-integration schemes the code actually uses — with their stability limits stated honestly. Wherever a quantity appears in the source it is pinned to its location with a code reference. The notation fixed here is used unchanged throughout Parts II–V.

2.1 Spherical coordinates and the metric

Chronos-ESM lives on a sphere of radius a (`EARTH_RADIUS` $= 6.371 \times 10^6$ m) rotating at rate Ω (`OMEGA` $= 7.2921 \times 10^{-5}\,\mathrm{rad\,s^{-1}}$). A point is addressed by longitude $\lambda \in [0, 2\pi)$, latitude $\phi \in [-\pi/2, \pi/2]$, and a vertical coordinate (geometric depth z in the ocean, a terrain-following σ coordinate in the multi-level atmosphere). The horizontal velocity is $\mathbf{u} = (u, v)$, with u eastward and v northward, and w is the vertical component.

↪ *Code:* `chronos_esm/config.py` — physical constants a, Ω, g and grid sizes

The essential complication of a sphere is that the zonal arc length shrinks toward the poles. An eastward displacement $\mathrm{d}\lambda$ subtends a physical distance $a\cos\phi\,\mathrm{d}\lambda$, whereas a northward displacement $\mathrm{d}\phi$ always subtends $a\,\mathrm{d}\phi$. The line element is

$$\mathrm{d}s^2 = a^2\cos^2\phi\;\mathrm{d}\lambda^2 \;+\; a^2\,\mathrm{d}\phi^2 \;+\; \mathrm{d}z^2. \tag{2.1}$$

Every horizontal differential operator inherits the $\cos\phi$ factor from (2.1). For a scalar f and horizontal vector $\mathbf{u}$,

$$\boldsymbol{\nabla} f = \left(\frac{1}{a\cos\phi}\frac{\partial f}{\partial \lambda},\; \frac{1}{a}\frac{\partial f}{\partial \phi} \right), \tag{2.2}$$

$$\boldsymbol{\nabla}\cdot\mathbf{u} = \frac{1}{a\cos\phi}\left[\frac{\partial u}{\partial \lambda} + \frac{\partial (v\cos\phi)}{\partial \phi} \right], \tag{2.3}$$

$$\nabla^2 f = \frac{1}{a^2\cos^2\phi}\frac{\partial^2 f}{\partial \lambda^2} + \frac{1}{a^2\cos\phi}\frac{\partial}{\partial \phi}\left(\cos\phi\,\frac{\partial f}{\partial \phi} \right). \tag{2.4}$$

The $\cos\phi$ in the denominators is the source of the polar singularity that haunts every latitude–longitude model: as $\phi \to \pm\pi/2$, a fixed zonal grid increment maps to a vanishing physical distance, so $\partial/\partial\lambda$ and $\partial^2/\partial\lambda^2$ amplify any small-scale zonal structure without bound. Section 2.2.3 shows exactly how the two grids of Chronos-ESM cope — one by capping the metric, the other by working in a basis in which the singularity never appears.

The planetary rotation enters through the Coriolis parameter

$$f = 2\Omega\sin\phi, \qquad \beta \equiv \frac{\partial f}{\partial y} = \frac{2\Omega\cos\phi}{a}, \tag{2.5}$$

the second of which is the β that controls Rossby propagation and the western intensification of ocean gyres. In the ocean driver f is built directly from a uniform latitude vector, `f_cor = 2*OMEGA*sin(lat_rad)`, and is floored away from zero near the equator (to $|f| \geq 10^{-5}\,\mathrm{s}^{-1}$) so that the geostrophic/Stommel division by f stays finite there.
↪ *Code:* `chronos_esm/ocean/veros_driver.py` — f from latitude, equatorial floor in `step_ocean`

2.2 The model grids

Chronos-ESM runs at spectral truncation **T31** by default, an option set by the environment variable `CHRONOS_RESOLUTION` (`T31` or `T63`). At T31 the horizontal grid is $N_\lambda \times N_\phi = 96 \times 48$, i.e. a nominal 3.75° spacing.
↪ *Code:* `chronos_esm/config.py` — `RESOLUTION`, `NLAT=48`, `NLON=96`

2.2.1 The ocean grid: latitude–longitude, z-level

The ocean is discretized on a regular latitude–longitude mesh with 96 cells in longitude and 48 in latitude, the row centres laid out by `linspace(-90, 90, ny)`. Longitude spacing is uniform, $\Delta\lambda = 2\pi/N_\lambda$; the corresponding physical zonal spacing is latitude dependent,

$$\Delta x(\phi) = a\cos\phi\,\Delta\lambda, \qquad \Delta y = \frac{\pi\, a}{N_\phi} \ \text{(uniform)}, \tag{2.6}$$

exactly the metric of (2.1). The driver receives Δx as an array (it varies down the column) and Δy as a scalar.

Vertically the ocean carries $N_z = 15$ levels on a *stretched* grid, not a uniform one. A uniform $5000/15 \approx 333\,\mathrm{m}$ surface layer would give the sea surface an enormous heat capacity and a sluggish response to fluxes, because surface forcing is applied as flux$/(\rho_0 c_p\,\Delta z_0)$. Instead the interfaces are placed at

$$z_i = H_{\mathrm{tot}}\left(\frac{i}{N_z}\right)^{s}, \quad i = 0,\dots,N_z, \qquad H_{\mathrm{tot}} = 5000\,\mathrm{m}, \quad s = 1.7, \tag{2.7}$$

so that $\Delta z_k = z_{k+1} - z_k$ grows from a thin ($\approx 50\,\mathrm{m}$) surface layer to a thick abyssal one, with layer-centre depths $\frac{1}{2}(z_k + z_{k+1})$ used as the coordinate onto which the World Ocean Atlas initial state is interpolated.
↪ *Code:* `chronos_esm/config.py` — `_ocean_vertical_grid`, `OCEAN_DZ`, `OCEAN_DEPTH_CENTERS`

Remark. The ocean driver computes horizontal derivatives with plain centred differences of the form $(\texttt{roll}(F,-1) - \texttt{roll}(F,1))/(2\Delta x)$ and treats $\Delta x, \Delta y$ as local Cartesian spacings; the spherical *convergence-of-meridians* term in the divergence, $\partial(v\cos\phi)/\partial\phi$ of (2.3), is approximated by the Cartesian $\partial v/\partial\phi$ (no explicit $\cos\phi$ metric in the meridional divergence). This is a known simplification; the diagnostic vertical velocity comment in the source flags it explicitly. It is acceptable at this resolution but is one of the contributors to the spurious net meridional transport that the per-latitude corrector of Chapter 8 removes.

2.2.2 The atmosphere grids: spectral on a Gaussian mesh

The atmosphere is *spectral*: prognostic fields are represented not by their grid-point values but by coefficients in a basis of spherical harmonics $Y_l^m(\phi,\lambda)$, and the "T31" label is the triangular truncation total wavenumber $l \leq 31$. Two atmospheres coexist in the code.

Single-level (legacy). The barotropic model in `atmos/dynamics.py` is a hybrid pseudo-spectral scheme: it takes the FFT in longitude (exact for the periodic direction) and uses fourth-order finite differences in latitude. Its grid is the same regular 96×48 latitude–longitude mesh as the ocean, with the metric of (2.6).
↪ *Code:* `chronos_esm/atmos/spectral.py` — `compute_gradients` — FFT in λ, 4th-order FD in ϕ

Multi-level (active). The production atmosphere is Google's differentiable DINOSAUR dycore (the engine behind NeuralGCM, Kochkov et al., 2024), run at T31 with 24 σ-levels and full spherical-harmonic transforms. Its horizontal grid is a *Gaussian* mesh: the latitude rows sit at the roots of the Legendre polynomial $P_{N_\phi}(\sin\phi)$, chosen so that Gauss–Legendre quadrature integrates the truncated harmonics

exactly. Grid→spectral and spectral→grid maps are `coords.horizontal.to_modal` / `to_nodal`, and the eastward/northward winds are recovered from the modal state as

$$u = \frac{\widehat{(u\cos\phi)}_0}{\cos\phi}, \qquad v = \frac{\widehat{(u\cos\phi)}_1}{\cos\phi}, \tag{2.8}$$

i.e. the dycore carries the momentum-like quantity $\mathbf{u}\cos\phi$ (regular at the poles) and divides out $\cos\phi$ only for diagnostics — the spectral route never divides by $\cos\phi$ internally, which is precisely why it has no polar instability.
↪ *Code:* `chronos_esm/atmos/dino_atmos.py` — T31, 24 σ-levels; `compute_diagnostic_state` for u, v

2.2.3 Metric terms in the discrete code

The single-level model must build the metric by hand. It forms the per-row zonal spacing $\Delta x(\phi) = 2\pi a\cos\phi/N_\lambda$ and $\Delta y = \pi a/N_\phi$ directly. The polar catastrophe implicit in the $1/\cos\phi$ factors of (2.2)–(2.4) is dealt with by a hard floor on the cosine used in the *dynamics*: rather than letting $\Delta x \to 0$ (it reaches $\approx 4\,\mathrm{m}$ in the polar row when $\cos\phi$ is floored to 10^{-5}), the metric is capped at its 80° value,

$$\cos\phi\big|_{\mathrm{dyn}} = \max\big(\cos\phi,\ \cos 80^\circ\big), \qquad \Delta x = \frac{2\pi a\ \cos\phi\big|_{\mathrm{dyn}}}{N_\lambda}, \tag{2.9}$$

so the handful of pole-most rows integrate with the (still small) 80° metric instead of detonating. A separate, slightly different floor is applied to the diffusion Δx so the explicit hyperdiffusion stencil $\propto 1/\Delta x^4$ stays inside its stability bound. This is a pragmatic fix, not a clean discretization: without it the mean sea-level pressure blew up at the poles to $\sigma \sim 150\,\mathrm{hPa}$ against an observed $\sim 12\,\mathrm{hPa}$. The DINOSAUR atmosphere needs no such floor because, as in (2.8), the singular $1/\cos\phi$ is never formed.
↪ *Code:* `chronos_esm/atmos/dynamics.py` — `cos_lat_dyn`/`cos_lat_diff` floors in `step_atmos`

2.3 Prognostic and diagnostic variables

A central organizing idea is the split between **prognostic** variables — those that carry a time derivative and are advanced by the integrator — and **diagnostic** variables — those reconstructed each step from the prognostic state by solving a (usually elliptic) balance. The split is not cosmetic: it is what makes a step well-posed, and it determines what must be saved in a checkpoint to restart bit-for-bit.

In the ocean, the `OceanState` carries eight fields, but only the three tracers are truly prognostic in the time-stepping sense. Temperature θ, salinity S, and dissolved inorganic carbon are advanced by an explicit Runge–Kutta step (Section 2.4); density, the velocities (u, v, w), and the barotropic streamfunction ψ are *diagnosed* each step from the tracers via the equation of state, thermal wind, continuity, and a Stommel balance respectively. Table 2.1 summarizes the split.
↪ *Code:* `chronos_esm/ocean/veros_driver.py` — `OceanState`; tracers stepped, $\mathbf{u}, w, \rho, \psi$ diagnosed

Table 2.1: Prognostic vs. diagnostic variables in the ocean and the active atmosphere.

Component	Prognostic (time-stepped)	Diagnostic (reconstructed)
Ocean	θ, S, DIC	ρ, u, v, w, ψ
Atmosphere (DINOSAUR)	ζ, δ, T', $\ln p_s$, humidity	u, v (from (2.8)), θ, geopotential

The diagnostic velocities follow from the linear equation of state

$$\rho = \rho_0 - \alpha(\theta - \theta_0) + \beta(S - S_0), \quad \rho_0 = 1025,\ \alpha = 0.2,\ \beta = 0.8, \tag{2.10}$$

followed by a thermal-wind/geostrophic relation for the baroclinic shear and a Stommel streamfunction for the barotropic flow; these are derived in Chapters 4 and 6. Because the velocities are diagnostic, a restart need only store the tracers (plus ψ as a warm start for the elliptic solve); the rest is rebuilt on the first step. The same logic governs DINOSAUR checkpoints, whose prognostic modal state is vorticity ζ, divergence δ, temperature variation T', and log surface pressure $\ln p_s$; u, v are diagnostic. (A subtle consequence: restoring `sim_time=None` on resume is mandatory, because the ImEx integrator adds a tendency *array* to the clock and a stored scalar would break that add.)
↪ *Code:* `chronos_esm/ocean/veros_driver.py` — `equation_of_state` — linear EOS, α, β

2.4 Time-integration schemes

Chronos-ESM uses different time integrators in different components, each chosen for the stiffness and conservation demands of the equations it solves. We state each in the canonical $\frac{\mathrm{d}\boldsymbol{\Phi}}{\mathrm{d}t} = \mathcal{R}(\boldsymbol{\Phi})$ form, where $\boldsymbol{\Phi}$ is the prognostic state and $\mathcal{R}$ its tendency.

2.4.1 Forward Euler — the single-level atmosphere

The legacy barotropic atmosphere advances with the simplest possible scheme,

$$\boldsymbol{\Phi}^{n+1} = \boldsymbol{\Phi}^{n} + \Delta t\, \mathcal{R}(\boldsymbol{\Phi}^{n}), \tag{2.11}$$

with $\Delta t_{\mathrm{atmos}} = 30\,\mathrm{s}$ (`DEFAULT_DT_ATMOS`). Forward Euler is first-order accurate and only conditionally stable: for an oscillatory mode it is *unconditionally* amplifying in the strict linear sense, but at this tiny step the per-step growth on the fastest physical waves is negligible. The relevant constraint is the gravity-wave Courant number

$$C = \frac{c\,\Delta t}{\Delta x_{\min}}, \qquad c = \sqrt{gH}, \tag{2.12}$$

which at $\Delta t = 30\,\mathrm{s}$ and the metric of (2.9) is $C \sim 0.06$ — far inside the stability region — so the dominant error is dissipative drift, not blow-up. Stability of the single-level model is bought elsewhere: by strong ∇^4 hyperdiffusion that damps the small scales (Chapter 10) and by velocity clamps ($|u| \le 80\,\mathrm{m\,s^{-1}}$) that act as a tripwire flagging instability rather than as physics.

↪ *Code:* `chronos_esm/atmos/dynamics.py` — `step_atmos` — forward Euler, ∇^4 filter, $80\,\mathrm{m\,s^{-1}}$ clamp

Remark. A semi-implicit option exists but is *not wired in*: `solve_helmholtz` in `atmos/spectral.py` solves $(\nabla^2 - \gamma)x = b$ via FFT-in-λ plus a tridiagonal solve in ϕ, with $\gamma = \Delta t^2 R T_0$ the implicit gravity-wave coupling. It would be needed only if Δt were raised enough to make C approach unity; at 30 s it buys nothing, so the live path stays explicit. The book documents it (Chapter 6) because the same Helmholtz machinery underlies the elliptic solves elsewhere.

2.4.2 Fourth-order Runge–Kutta — ocean tracers

The ocean tracers are advanced with the classical four-stage Runge–Kutta scheme,

$$\begin{aligned} k_1 &= \mathcal{R}(\boldsymbol{\Phi}^{n}), & k_2 &= \mathcal{R}\big(\boldsymbol{\Phi}^{n} + \tfrac{\Delta t}{2} k_1\big), \\ k_3 &= \mathcal{R}\big(\boldsymbol{\Phi}^{n} + \tfrac{\Delta t}{2} k_2\big), & k_4 &= \mathcal{R}\big(\boldsymbol{\Phi}^{n} + \Delta t\, k_3\big), \end{aligned} \qquad \boldsymbol{\Phi}^{n+1} = \boldsymbol{\Phi}^{n} + \frac{\Delta t}{6}\big(k_1 + 2k_2 + 2k_3 + k_4\big), \tag{2.13}$$

applied jointly to $(\theta, S, \mathrm{DIC})$ with $\Delta t_{\mathrm{ocean}} = 900\,\mathrm{s}$ (`DT_OCEAN`, 15 minutes). RK4 is fourth-order accurate and has a comfortably large stable region; the binding constraint here is again advective CFL, $|\mathbf{u}|\Delta t/\Delta x \lesssim 1$, easily satisfied by the slow ocean currents. The tendency $\mathcal{R}$ combines upwind horizontal advection, upwind vertical advection (whose vertical Courant number $|w|\Delta t/\Delta z \sim 10^{-3}$ is tiny), Laplacian diffusion, and surface fluxes injected into the top wet cell.

↪ *Code:* `chronos_esm/ocean/veros_driver.py` — `tendencies`, the four-stage RK4 update in `step_ocean`

After the RK4 update the tracers pass through three post-processing operators that are *not* part of the differential equation but are essential for a multi-century integration: an optional scale-selective ∇^4 Shapiro-type filter (off by default, removing only the $2\Delta x$ checkerboard mode); hard physical clamps $S \in [30, 38]$ psu, $\theta \in [250, 320]$ K, and a sea-ice freezing floor $\theta \ge 271.35\,\mathrm{K}$; and an additive global-mean salinity renormalization to 34.7 psu. The clamps are honest band-aids — the book quantifies where they bite (rarely, in well-tuned runs) in Chapter 16.

2.4.3 Semi-implicit / ImEx — the multi-level atmosphere

The DINOSAUR dycore solves the dry primitive equations with an *implicit–explicit* Runge–Kutta scheme (specifically the strong-stability three-stage ImEx-RK of the dycore, `imex_rk_sil3`). The state is split into a stiff linear part $L\boldsymbol{\Phi}$ — the fast gravity- and acoustic-like waves carried by the divergence/pressure/temperature coupling — and a slow nonlinear part $N(\boldsymbol{\Phi})$:

$$\frac{\mathrm{d}\boldsymbol{\Phi}}{\mathrm{d}t} = \underbrace{L\,\boldsymbol{\Phi}}_{\text{implicit}} + \underbrace{N(\boldsymbol{\Phi})}_{\text{explicit}}. \tag{2.14}$$

Each stage treats L implicitly (solved exactly in spectral space, where L is diagonal in the vertical-mode/horizontal-wavenumber basis) and N explicitly. The point of the split is that the CFL limit (2.12) no longer involves the fast gravity-wave speed c (handled implicitly, hence unconditionally stable) but only the much slower advective speed, so a far larger step is admissible: $\Delta t = 20\,\mathrm{min}$ at T31. After each macro-step a horizontal hyperdiffusion filter is applied as a separate operator,

$$\widehat{\mathbf{\Phi}}_l \leftarrow \widehat{\mathbf{\Phi}}_l \exp\left[-\frac{\Delta t}{\tau}\left(\frac{l(l+1)}{L(L+1)}\right)^p\right], \tag{2.15}$$

with diffusion order p and time-scale $\tau = 6\,\mathrm{h}$ chosen *weak enough* that the T31 baroclinic eddies ($l \sim 10$–15) are allowed to grow — a deliberate departure from the single-level model, where eddies were spurious and diffusion was kept strong (see the decoupling principle discussed in Chapters 9 and 18).
↪ *Code:* `chronos_esm/atmos/dino_atmos.py` — `imex_rk_sil3`, `step_with_filters`, `horizontal_diffusion_step_filter`

Table 2.2: Time integrators used across Chronos-ESM (default T31).

Component	Scheme	Order	Δt
Single-level atmosphere	Forward Euler + ∇^4 filter	1	30 s
Ocean tracers	Classical RK4	4	900 s
Multi-level atmosphere	ImEx-RK (DINOSAUR) + spectral filter	3	20 min
Coupler exchange	—	—	3600 s

The components are synchronized by a coupler that exchanges fluxes every $\Delta t_{\mathrm{cpl}} = 3600\,\mathrm{s}$ (`COUPLING_INTERVAL`); within a coupling window each component sub-cycles at its own step (Table 2.2). The coupling itself is a loose, first-order-in-time exchange of time-mean surface fields, documented in Chapter 13.

2.5 Notation conventions

We close with the conventions used throughout. Operators $\boldsymbol{\nabla}$, $\boldsymbol{\nabla}\cdot$, $\boldsymbol{\nabla}\times$, ∇^2 are the spherical forms (2.2)–(2.4) unless a Cartesian tangent-plane form is explicitly noted. The material derivative is $\frac{\mathrm{D}f}{\mathrm{D}t} = \frac{\partial f}{\partial t} + \mathbf{u}\cdot\boldsymbol{\nabla} f + w\,\frac{\partial f}{\partial z}$. Subscripts 0 denote reference constants (ρ_0, θ_0, S_0); primes denote departures from a reference or zonal mean (T'). A circumflex $\widehat{(\cdot)}$ denotes a spectral (modal) coefficient. A superscript n is the time level, so $\mathbf{\Phi}^{n+1}$ is the updated state. Transport is reported in Sverdrups, $1\,\mathrm{Sv} = 10^6\,\mathrm{m^3\,s^{-1}}$. Units are written in plain roman math, e.g. $\mathrm{m\,s^{-1}}$. Discrete grid operations name the array axes as the code does: for the ocean state of shape (N_z, N_ϕ, N_λ), `axis=2` is longitude, `axis=1` is latitude, and `axis=0` is the vertical. Code references appear inline as ↪ *Code:* pointers. Where a smooth surrogate replaces a non-differentiable operation (e.g. softplus for a positive part, or a soft clip), it is named at the point of use; the rationale is the subject of Chapter 3.

Example 2.1. At T31 the polar-row physical zonal spacing is $\Delta x = a\cos\phi\,(2\pi/N_\lambda)$. For the row centred near $\phi = 86.25°$, $\cos\phi \approx 0.0654$, giving $\Delta x \approx 27\,\mathrm{km}$ — already 15× smaller than the equatorial $\approx 417\,\mathrm{km}$. Push to $\phi \to 90°$ and $\Delta x \to 0$: this is the singularity (2.9) caps, and the reason the DINOSAUR dycore works in the harmonic basis instead.

Exercise 2.1. Using `config.py`, reconstruct the ocean vertical grid (2.7) for $N_z = 15$, $H_{\mathrm{tot}} = 5000\,\mathrm{m}$, $s = 1.7$. Tabulate the 15 layer thicknesses Δz_k. Confirm the surface layer is $\approx 50\,\mathrm{m}$ and the deepest $\approx 900\,\mathrm{m}$. Then recompute with $s = 1.0$ (uniform) and explain, using $\partial\theta/\partial t \sim \mathrm{flux}/(\rho_0 c_p \Delta z_0)$, why the stretched grid gives a more responsive sea surface temperature.

Exercise 2.2. Evaluate the gravity-wave Courant number (2.12) for the single-level atmosphere using $H = 10\,\mathrm{km}$, $g = 9.81\,\mathrm{m\,s^{-2}}$, $\Delta t = 30\,\mathrm{s}$, and the metric-floored polar Δx from (2.9) at $\phi = 80°$. How large could Δt become before $C \to 1$? Explain why, despite the headroom, the production atmosphere still moved to the ImEx DINOSAUR dycore rather than simply raising Δt here.

Exercise 2.3. Classify each field of `OceanState` (Table 2.1) as prognostic or diagnostic by inspecting `step_ocean`. For a checkpoint that must restart the run, which fields are strictly necessary and which can be regenerated on the first step? Verify your answer against the checkpoint-restart discussion in Chapter 6.

Exercise 2.4. The ImEx split (2.14) treats the fast linear waves implicitly. Write the linear shallow-water gravity-wave system $\partial_t u = -g\partial_x h$, $\partial_t h = -H\partial_x u$ as $\frac{\mathrm{d}\boldsymbol{\Phi}}{\mathrm{d}t} = L\boldsymbol{\Phi}$, find the dispersion relation, and show that a fully implicit (backward-Euler) treatment of L is unconditionally stable while forward Euler (2.11) is not. Relate this to why DINOSAUR can take a 20 min step where the explicit single-level model is pinned at 30 s.

Chapter 3

The Differentiable-Programming Paradigm

What makes Chronos-ESM different from a conventional climate model is not its physics — the governing equations of the next chapters are the same primitive equations every ocean and atmosphere model solves — but the substrate it is written in. The entire model, from the spectral atmosphere through the elliptic ocean solver down to the bulk flux formulae, is a single JAX program (Bradbury et al., 2018); a differentiable JAX ocean GCM is adjacent prior art for this approach (Häfner et al., 2018). Every floating-point operation is recorded on a computation graph, and that graph can be *differentiated*: for any scalar diagnostic $\mathcal{L}$ computed from a multi-decade run we can obtain the exact gradient $\partial\mathcal{L}/\partial\boldsymbol{\theta}$ with respect to *every* tunable parameter $\boldsymbol{\theta}$ at once. This chapter explains what that means mathematically, what it costs, and which numerical idioms the model must obey to keep the gradient correct. The applications it unlocks — calibration, sensitivity, and data assimilation — are developed in Chapters 15, 14, and 17.

3.1 The model as a composition of differentiable maps

A coupled climate model is, formally, a discrete dynamical system. Write the full state as a vector $\mathbf{s} \in \mathbb{R}^N$ — in Chronos-ESM this is the `DinoCoupledState` pytree, which carries the DINOSAUR spectral atmosphere (Google Research, 2024) (vorticity, divergence, temperature variation, log surface pressure, humidity), the ocean $(u, v, w, \theta, S, \psi, \rho, \mathrm{DIC})$, the land, and the sea-ice fields side by side (
↪ *Code:* `chronos_esm/coupler/dino_step.py` — `DinoCoupledState`
). One coupling interval is a map

$$\mathbf{s}_{n+1} = \mathcal{M}(\mathbf{s}_n;\, \boldsymbol{\theta},\, c_n), \tag{3.1}$$

where $\boldsymbol{\theta}$ collects static parameters (diffusivities, the haline gain, drag coefficients) and c_n collects time-varying forcings (CO_2, hosing flux). A run of K intervals is the K-fold composition $\mathcal{M}_K \circ \cdots \circ \mathcal{M}_1$, and a scalar diagnostic is $\mathcal{L}(\mathbf{s}_0; \boldsymbol{\theta}) = \ell(\mathbf{s}_K)$ for some reduction ℓ (e.g. the time-mean AMOC at 26.5°N, or a misfit to observations).

Because $\mathcal{M}$ is built entirely from primitive JAX operations — additions, multiplications, `roll`-based stencils, fast spherical-harmonic transforms, `lax.scan` loops — it is a *differentiable map* almost everywhere. The chain rule applied to the composition gives the exact sensitivity of the whole trajectory,

$$\frac{\partial\mathcal{L}}{\partial\boldsymbol{\theta}} = \frac{\partial\ell}{\partial\mathbf{s}_K} \prod_{n=K-1}^{0} \left[\frac{\partial\mathcal{M}}{\partial\mathbf{s}_n}\right] \frac{\partial\mathbf{s}_0}{\partial\boldsymbol{\theta}} \;+\; \sum_{n=0}^{K-1} (\text{direct } \boldsymbol{\theta}\text{-dependence of } \mathcal{M}_n)\,, \tag{3.2}$$

where $\partial\mathcal{M}/\partial\mathbf{s}_n$ is the (huge, never-formed) tangent-linear Jacobian of one step. The classical climate-science machinery for Eq. (3.2) — hand-coding a tangent-linear and adjoint model — is replaced here by automatic differentiation (AD), which constructs the same operator from the forward code mechanically and exactly (to machine precision, not by finite differences).

3.2 Forward mode versus reverse mode

AD comes in two modes, distinguished by the order in which the Jacobian factors of Eq. (3.2) are multiplied. Let $\mathbf{J} = \partial\mathcal{L}/\partial\boldsymbol{\theta}$ be conceptually an $M \times P$ Jacobian of M outputs in P parameters.

Forward mode (`jax.jvp`) propagates a perturbation *with* the computation: it carries a tangent vector $\dot{\mathbf{s}}$ alongside the state and applies each $\partial\mathcal{M}/\partial\mathbf{s}$ left-to-right as the model runs. One forward pass yields one *column* of $\mathbf{J}$ — the directional derivative along one input direction. Its cost scales with the number of inputs P.

Reverse mode (`jax.grad`, `jax.vjp`) propagates a sensitivity *against* the computation: it runs the model forward, stores intermediate values, then sweeps backward applying each transposed Jacobian $(\partial\mathcal{M}/\partial\mathbf{s})^\top$ right-to-left. One reverse pass yields one *row* of $\mathbf{J}$ — the gradient of one scalar output with respect to *all* inputs at once. Its cost scales with the number of outputs M.

Proposition 3.1 (Why climate calibration uses reverse mode)**.** *A calibration or data-assimilation objective is a single scalar $\mathcal{L}$ ($M = 1$) depending on many parameters ($P \gg 1$). Reverse mode delivers the entire gradient $\partial\mathcal{L}/\partial\boldsymbol{\theta} \in \mathbb{R}^P$ in one backward sweep, at a cost independent of P. Forward mode would need P separate passes. This asymmetry — cheap gradients of scalars in high-dimensional parameter spaces — is the entire reason a differentiable climate model is worth building.*

Remark. Finite differences would also estimate $\partial\mathcal{L}/\partial\boldsymbol{\theta}$, but at $P{+}1$ model runs and with truncation/round-off error that is fatal for a chaotic, multi-decadal integration. Reverse-mode AD gives the *exact* gradient of the discretised model in roughly the cost of a handful of forward runs (Section 3.4).

3.3 The four jax transformations used in the model

Chronos-ESM relies on four composable program transformations (Bradbury et al., 2018; Kochkov et al., 2024).

`jit` (just-in-time compilation). Traces a Python function once into an intermediate representation and compiles it to a single fused kernel (CPU/GPU). In Chronos-ESM the *entire* coupling interval — SST regrid, the spectral atmosphere, diagnostics, land, ice, bulk fluxes, and the ocean sub-stepping — is compiled as one program (`_step_fast`), roughly ten times faster than eager dispatch because it avoids re-launching each spectral transform and each of the ~ 96 ocean sub-steps separately (
↪ *Code:* `chronos_esm/coupler/dino_step.py` — `step_fast`
). Static (non-traced) data — grid metrics, masks, the atmosphere object — are closed over so the only traced argument is the state pytree.

`vmap` (vectorising map). Adds a batch axis to a function without a Python loop. The model uses it to extract the diagonal of a linear operator for the Jacobi preconditioner, applying the operator to every standard basis vector at once (
↪ *Code:* `chronos_esm/ocean/solver.py` — `create_diagonal_preconditioner`
). The same transform vectorises ensemble runs (perturbed parameters or initial conditions) for free.

`lax.scan` (compiled, differentiable loop). A structured loop with a fixed trip count that compiles to one rolled kernel and — crucially — has a defined, efficient adjoint. The ocean sub-stepping inside one coupling interval is a `scan` of length n_{sub} (
↪ *Code:* `chronos_esm/coupler/dino_step.py` — ocean `lax.scan`
); the atmosphere interval is an analogous fixed-length iteration of the ImEx Runge–Kutta step (
↪ *Code:* `chronos_esm/atmos/dino_atmos.py` — `_run_interval`, `ti.repeated`
). Section 3.5 explains why `scan` (not a `while` loop) is mandatory for the iterative solvers.

`checkpoint`/`remat` (gradient checkpointing). Trades recomputation for memory in the backward pass, discussed next.

3.4 The cost model of a gradient, and gradient checkpointing

Reverse-mode AD must access intermediate values from the forward pass during the backward sweep. Naively, every intermediate of a K-step run is stored, so memory grows like $\mathcal{O}(K \cdot N)$. For a multi-year coupled run this is hopeless: the activations of thousands of spectral transforms and ocean sub-steps would exhaust device memory.

Gradient checkpointing (rematerialisation) breaks the trade-off. A function wrapped in `jax.checkpoint` does not store its internal activations; instead it *recomputes* them during the backward pass from a saved input. Chronos-ESM applies this at two granularities in the differentiable step: the atmosphere interval is wrapped (`_atmos_remat = jax.checkpoint(self.atm._run_interval, static_argnums=(2,))`) and so is each ocean sub-step body inside the `scan` (
↪ *Code:* `chronos_esm/coupler/dino_step.py` — `_advance(..., remat=True)`
). The result is the standard cost model: with $\sqrt{K}$-spaced checkpoints, memory drops to $\mathcal{O}(\sqrt{K}\,N)$ at the price of computing the forward pass roughly twice. As a rule of thumb,

$$\text{cost}(\nabla\mathcal{L}) \;\approx\; 2\text{–}4\times\;\text{cost}(\mathcal{L}), \tag{3.3}$$

independent of the number of parameters P — the celebrated "cheap gradient principle". The fast forward step deliberately omits `remat`, because rematerialisation blocks kernel fusion and would slow a plain forward run; the model therefore keeps two paths — a fused `step_fast` for control/forced integrations and a remat'd `step` for gradients and DA (
↪ *Code:* `chronos_esm/coupler/dino_step.py` — `step` vs `step_fast`
).

3.5 Why iterative solvers must be fixed-length scans

The single most important rule for keeping a physics model differentiable is how its *iterative* solvers are written. Chronos-ESM's ocean needs to invert an elliptic (streamfunction / barotropic-vorticity) operator each step. The continuous problem is the variable-coefficient elliptic equation

$$\boldsymbol{\nabla}\cdot\big(H^{-1}\boldsymbol{\nabla}\psi\big) = \zeta, \tag{3.4}$$

with H the column depth and ζ the vorticity forcing (
↪ *Code:* `chronos_esm/ocean/solver.py` — module docstring; `solve_elliptic_varcoef_sphere`
). Discretised in symmetric face-conductance form on the sphere this becomes a sparse symmetric positive-definite linear system $A\psi = b$, solved by preconditioned conjugate gradients (CG).

The natural way to write CG is "iterate *until* the residual norm $\|r\|$ falls below a tolerance." In JAX that is `lax.while_loop` — and it is **not reverse-mode differentiable**, because the number of iterations is data-dependent and so the backward pass has no fixed graph to transpose. The model therefore writes CG as a *fixed-length* `lax.scan` of exactly `max_iter` steps, using `jnp.where` to turn each iteration into a no-op once converged (
↪ *Code:* `chronos_esm/ocean/solver.py` — `solve_cg`, the `scan_body` convergence freeze
):

```
def scan_body(state, _):
    converged = r_norm <= tol_abs
    new_state = body_fun(state)          # one CG iteration
    out_state = jax.tree.map(
        lambda old, new: jnp.where(converged, old, new), state, new_state)
    return out_state, None               # frozen once converged
final_state, _ = jax.lax.scan(scan_body, init, None, length=max_iter)
```

Listing 3.1: The AD-safe convergence freeze: a fixed-length scan that stops updating.

Each iteration applies the operator and updates the search direction with the standard CG recurrences — step length $\alpha = r^\top z/(p^\top Ap)$, conjugacy update $\beta = r_{\text{new}}^\top z_{\text{new}}/r^\top z$, with division guards (`jnp.where` on tiny denominators) so the graph never produces a `NaN` that would poison the adjoint (
↪ *Code:* `chronos_esm/ocean/solver.py` — `body_fun`
). The price is that the forward solve always runs `max_iter` iterations (the converged ones are cheap no-ops), but in exchange the linear solve has a clean, transposable adjoint and gradients flow

through the inverted streamfunction. The same fixed-trip-count discipline is what makes the atmosphere's ImEx Runge–Kutta interval (`ti.repeated(step, n_steps)`) and the ocean sub-step `scan` differentiable end-to-end.

Remark (The adjoint of a linear solve). For a converged linear solve $A\psi = b$ one need not differentiate through every CG iteration: the implicit-function theorem gives the adjoint directly as a *second* solve with $A^\top$. Chronos-ESM's solver instead differentiates the unrolled fixed-length scan, which is simpler to wire and, for the modest `max_iter` used here, comparably cheap. Either way, the `while`-loop-free structure is what makes an adjoint exist at all.

3.6 Implications: calibration, sensitivity, data assimilation

End-to-end differentiability turns the model into an instrument that can be *inverted*, not just run forward.

Calibration. Choosing parameters $\boldsymbol{\theta}$ (diffusivities, gains, drag) to minimise a misfit $\mathcal{L}(\boldsymbol{\theta})$ to observations becomes gradient descent: a handful of reverse-mode passes deliver $\partial\mathcal{L}/\partial\boldsymbol{\theta}$ for the whole parameter vector at once, replacing parameter sweeps that scale exponentially in dimension. Chapter 15 carries this out; Chapter 17 shows the important exception — a *bifurcation* threshold cannot be obtained by differentiating *through* the fold, because the trajectory's sensitivity diverges there, and one must instead differentiate the steady-state (fold-continuation) condition.

Sensitivity. The gradient $\partial\mathcal{L}/\partial\boldsymbol{\theta}$ *is* a complete sensitivity ranking: it says which parameter each diagnostic actually depends on, and how strongly, in one computation rather than one perturbation experiment per parameter. The same machinery gives sensitivities to the forcing c_n and to the initial state $\mathbf{s}_0$.

Data assimilation. Four-dimensional variational assimilation (4D-Var) minimises a trajectory-versus-observations cost over the initial state $\mathbf{s}_0$. Its gradient $\partial\mathcal{L}/\partial\mathbf{s}_0$ is exactly the adjoint sweep of Eq. (3.2) — historically the most expensive code in operational forecasting to write and maintain by hand. In a differentiable model it is `jax.grad`; the hand-built tangent-linear/adjoint model disappears. The standing caveat (developed in the applications chapters) is *chaos*: for a turbulent atmosphere the adjoint gradient of a long trajectory grows exponentially and becomes a poor descent direction, so the useful differentiable targets are short windows, time-mean statistics, or the better-conditioned ocean/coupled steady states.

Exercise 3.1 (Forward vs reverse cost)**.** A configuration has $P = 12$ scalar parameters and you want the gradient of a single time-mean AMOC diagnostic. Estimate the number of model evaluations needed by (i) finite differences, (ii) forward-mode AD, and (iii) reverse-mode AD. Using Eq. (3.3), state which is cheapest and why the answer is independent of P for reverse mode.

Exercise 3.2 (Break the gradient on purpose)**.** In
↪ *Code:* `chronos_esm/ocean/solver.py — solve_cg`
, the iteration is a fixed-length `lax.scan` with a `jnp.where` convergence freeze. Explain what would go wrong — for the *gradient*, not the forward solution — if it were rewritten with `lax.while_loop` that exits at the tolerance. Then explain why replacing the `jnp.where` freeze with a Python `if r_norm <= tol: break` is also not an option inside a jitted scan.

Exercise 3.3 (Memory vs compute)**.** A differentiable run uses `jax.checkpoint` on both the atmosphere interval and each ocean sub-step (
↪ *Code:* `chronos_esm/coupler/dino_step.py — _advance(..., remat=True)`
). For a K-interval run, write down the forward-pass memory and the wall-clock overhead with and without checkpointing, and explain why `step_fast` deliberately omits it.

Exercise 3.4 (Sensitivity by AD)**.** Using the differentiable `step`, compute $\partial(\text{AMOC at } 26.5°\text{N})/\partial(\texttt{thc\_haline\_gain})$ and $\partial(\cdot)/\partial(\texttt{thc\_k\_vel})$ from a short coupled run with `jax.grad`. Rank the two parameters by magnitude and compare your ranking to the fold-continuation result of Chapter 17.

Part II

The Component Models

Chapter 4

The Ocean: Governing Equations

This chapter sets out the governing equations of the Chronos-ESM ocean component and, crucially, the *discrete* system the code actually integrates. The ocean is a hydrostatic, Boussinesq, z-level model on a latitude–longitude grid at T31 resolution (48×96 horizontal cells, 15 vertical levels; see `config.py`). The reference text for the continuous theory is Vallis (2017); for the numerical formulation and the design choices of z-coordinate ocean models we follow Griffies (2004). Throughout, we are honest about where the implementation simplifies, clamps, or filters the physics: Chronos-ESM is a *differentiable* model first, a high-fidelity GCM second, and several of its closures are diagnostic stand-ins for prognostic dynamics. Those diagnostic closures are taken up in detail in Chapters 5–8; here we establish the equation set and the grid on which it lives.

4.1 The hydrostatic Boussinesq primitive equations

The ocean is nearly incompressible and its density varies by less than 3% about a reference value. The Boussinesq approximation exploits this: density variations are neglected everywhere except in the buoyancy (gravity) term, and the continuity equation reduces to non-divergence of the velocity field. Let $\mathbf{u} = (u, v)$ be the horizontal velocity, w the vertical velocity (positive up), p the pressure, ρ the in-situ density and ρ_0 the constant reference density (`RHO_WATER`$= 1025\,\mathrm{kg\,m^{-3}}$). With $f = 2\Omega \sin\phi$ the Coriolis parameter, g gravity, and $\mathbf{F}$ the frictional (viscous) stress, the hydrostatic primitive equations are (Vallis, 2017)

$$\frac{\mathrm{D}\mathbf{u}}{\mathrm{D}t} + f\,\mathbf{k}\times\mathbf{u} = -\frac{1}{\rho_0}\boldsymbol{\nabla}_h p + \mathbf{F}, \tag{4.1}$$

$$\frac{\partial p}{\partial z} = -\rho\, g, \tag{4.2}$$

$$\boldsymbol{\nabla}\cdot\mathbf{u} + \frac{\partial w}{\partial z} = 0, \tag{4.3}$$

where $\boldsymbol{\nabla}_h = (\partial_x, \partial_y)$ is the horizontal gradient and $\mathrm{D}/\mathrm{D}t = \partial_t + \mathbf{u}\cdot\boldsymbol{\nabla}_h + w\,\partial_z$ is the material derivative. Equation (4.2) is the hydrostatic balance: the vertical momentum equation is replaced by the statement that pressure at depth is the weight of the overlying water. Equation (6.10) is Boussinesq continuity — the three-dimensional velocity is non-divergent.

What the code integrates instead. Chronos-ESM does *not* time-step the horizontal momentum equation (4.1). Instead it *diagnoses* the velocity each ocean step from a split into a barotropic and a baroclinic part. This is a deliberate simplification: a prognostic-momentum core is the subject of Chapter 7 and is, at the time of writing, the model's critical-path development. The diagnostic split is

$$\mathbf{u} = \mathbf{u}_{\mathrm{bt}}(\mathbf{x}) + \mathbf{u}_{\mathrm{bc}}(\mathbf{x}, z), \qquad \int_{-H}^{0} \mathbf{u}_{\mathrm{bc}}\,\mathrm{d}z = 0. \tag{4.4}$$

The depth-independent (barotropic) part $\mathbf{u}_{\mathrm{bt}}$ comes from a Stommel streamfunction; the depth-varying (baroclinic) part $\mathbf{u}_{\mathrm{bc}}$ comes from thermal-wind balance.

Barotropic flow: a Stommel streamfunction. The vertically integrated, frictional, steady vorticity balance forced by the wind-stress curl is the classic Stommel problem (Stommel, 1961). Chronos-ESM writes the depth-integrated transport as a streamfunction ψ, $\mathbf{u}_{\mathrm{bt}} = \mathbf{k} \times \boldsymbol{\nabla}\psi / H$, and solves a Poisson problem for it,

$$\nabla^2 \psi = \frac{\boldsymbol{\nabla} \times_z \boldsymbol{\tau}}{\rho_0 \, H \, r_{\mathrm{bt}}}, \qquad r_{\mathrm{bt}} = \frac{1}{25 \times 86400\,\mathrm{s}}, \tag{4.5}$$

where $\boldsymbol{\tau} = (\tau_x, \tau_y)$ is the surface wind stress, $H = \sum_k \Delta z_k$ the total depth, and r_{bt} a tuned 25-day bottom-drag rate. The right-hand side is built with centred differences for the curl and the elliptic solve is an iterative masked Poisson solver. The geostrophic velocity then follows from finite differences of ψ.
↪ *Code:* `chronos_esm/ocean/veros_driver.py` — `step_ocean`: `curl_tau`, `rhs_psi`, `solve_poisson_2d`, and $u_{\mathrm{bt}}, v_{\mathrm{bt}}$ from ψ

Baroclinic flow: thermal wind. The vertical shear of the geostrophic current is set by the horizontal density gradient (thermal-wind balance). Chronos-ESM integrates the hydrostatic relation (4.2) downward to get the hydrostatic pressure from the density anomaly $\rho' = \rho - \rho_0$,

$$p(z) = g \int_0^z \rho' \, \mathrm{d}z' \xrightarrow{\text{discrete}} p_k = g \sum_{j \le k} \rho'_j \, \Delta z_j, \tag{4.6}$$

and then forms a damped-geostrophic velocity with a Rayleigh drag r to keep the equatorial denominator finite (the Coriolis parameter f is floored at $10^{-5}\,\mathrm{s}^{-1}$ near the equator):

$$u_{\mathrm{geo}} = -\frac{r\,\partial_x p + f\,\partial_y p}{\rho_0\,(f^2 + r^2)}, \qquad v_{\mathrm{geo}} = \frac{f\,\partial_x p - r\,\partial_y p}{\rho_0\,(f^2 + r^2)}. \tag{4.7}$$

The baroclinic part is the deviation of $\mathbf{u}_{\mathrm{geo}}$ from its wet-column vertical mean, enforcing the zero-mean constraint in (4.4) exactly per column.
↪ *Code:* `chronos_esm/ocean/veros_driver.py` — `step_ocean`: `pressure_hydro`, `u_geo/v_geo`, `f_clamped`, `u_bc/v_bc`

Remark. A flat-bottom Stommel streamfunction laid over *variable* bathymetry does not conserve mass column-by-column: because $\partial_x(\psi H) \neq H\,\partial_x \psi$, the zonally- and vertically-integrated meridional transport is spuriously $\mathcal{O}(\pm 350\,\mathrm{Sv})$ where a closed basin requires ≈ 0. Chronos-ESM therefore subtracts, at each latitude, a uniform meridional velocity so the integrated transport vanishes (`v_new` $-=$ `net_v/area_v`). This is a first-order corrector, not a true C-grid transport streamfunction; see Chapter 8.
↪ *Code:* `chronos_esm/ocean/veros_driver.py` — `step_ocean`: per-latitude net-transport corrector

Vertical velocity from continuity. Given the horizontal flow, w is diagnosed by integrating (6.10) upward from the rigid floor ($w = 0$ at $z = -H$):

$$w(z) = -\int_{-H}^{z} \boldsymbol{\nabla}\cdot\mathbf{u} \, \mathrm{d}z' \xrightarrow{\text{discrete}} w_{k+\frac{1}{2}} = -\sum_{j \ge k} (\boldsymbol{\nabla}\cdot\mathbf{u})_j \, \Delta z_j. \tag{4.8}$$

For *tracer advection* the model first projects out the depth-mean horizontal divergence (a rigid-lid projection) so the depth-integrated flow is non-divergent and $w = 0$ at both surface and floor; this makes the discrete 3-D advecting flow exactly mass-conserving and prevents a spurious $\sim +0.25\,\mathrm{K\,yr}^{-1}$ surface-cell heat leak.
↪ *Code:* `chronos_esm/ocean/veros_driver.py` — `step_ocean`: `div_e` rigid-lid projection, `w_if`; and the diagnostic `w_new`

4.2 Tracer transport: potential temperature and salinity

Potential temperature θ and salinity S (and a passive dissolved-inorganic-carbon tracer C) are advected by the resolved flow and mixed by sub-grid diffusion. The continuous tracer equation for any θ is

$$\frac{\partial \theta}{\partial t} = \underbrace{-\,\mathbf{u}\cdot\boldsymbol{\nabla}_h\theta - w\,\frac{\partial\theta}{\partial z}}_{\text{advection}} + \underbrace{\boldsymbol{\nabla}_h\cdot(\kappa_h\boldsymbol{\nabla}_h\theta)}_{\text{horizontal diffusion}} + \underbrace{\frac{\partial}{\partial z}\Big(\kappa_z\,\frac{\partial\theta}{\partial z}\Big)}_{\text{vertical diffusion}} + \mathcal{S}_\theta, \tag{4.9}$$

with κ_h the horizontal (eddy) diffusivity, κ_z the vertical diffusivity, and $\mathcal{S}_\theta$ the surface forcing applied to the top wet cell only. The advecting velocity used for tracers includes the Gent–McWilliams bolus velocity (Gent and McWilliams, 1990), $\mathbf{u}_{\text{eff}} = \mathbf{u} + \mathbf{u}^*$, which parameterizes the adiabatic stirring by mesoscale eddies (Chapter 5).

Discrete tracer step. The code uses first-order *upwind* horizontal advection (stable, but diffusive), a centred Laplacian for horizontal diffusion, an implicit-free centred vertical-diffusion flux, and upwind vertical advection in *advective* form (consistent with the horizontal form, so no spurious compression source). The full tendency $\mathcal{T}(\theta) = \text{horiz} + \text{vadv} + \text{vdiff} + \mathcal{S}$ is advanced with a classical fourth-order Runge–Kutta (RK4) step,

$$\theta^{n+1} = \theta^n + \frac{\Delta t}{6}\big(k_1 + 2k_2 + 2k_3 + k_4\big), \qquad \Delta t = 900\,\text{s}, \tag{4.10}$$

where the k_i are tendency evaluations at the RK4 stages and $\Delta t =$ `DT_OCEAN`. The horizontal upwind/diffusion kernel is, schematically,

```
dF_dx = jnp.where(u_eff > 0, F - FW, FE - F) / dx      # upwind on sign of u
adv   = -(u_eff * dF_dx + v_eff * dF_dy)               # advective form
lap   = ((FE - F) + (FW - F)) / dx**2 + ((FN - F) + (FS - F)) / dy**2
return adv + lap * diff_coef                           # diff_coef = kappa_h (
    interior)
```

Listing 4.1: Upwind advection + Laplacian diffusion (one tracer face direction).

↪ *Code:* `chronos_esm/ocean/veros_driver.py` — `step_ocean`: `_horiz`, `_vadv`, `_vdiff`, `tendencies`, and the RK4 update

Remark (Honest limitations). Three numerical/physical caveats matter for interpretation. (i) Upwind advection adds implicit numerical diffusion of order $|u|\,\Delta x/2$, which at T31 is comparable to the explicit κ_h; sharp fronts are smeared. (ii) Near the poles ($|\,\text{row}\,| < 5$ from each edge) the horizontal diffusivity is raised to $2 \times 10^4\,\text{m}^2\,\text{s}^{-1}$ as a "sponge" (`diff_coef`), and the GM coefficient is masked to zero there, to suppress grid-converging noise. (iii) The metric terms ($\cos\phi$ factors of a true spherical operator) are omitted from the diagnostic divergence used for w_{new}; the operators are Cartesian on the lat–lon grid. These are differentiability- and stability-driven trade-offs, not physical claims.

Surface forcing. Surface heat flux Q, freshwater flux F_w, and a carbon flux enter only the top wet cell, scaled by the layer heat capacity / thickness:

$$\mathcal{S}_\theta^{\text{sfc}} = \frac{Q}{\rho_0\, c_p\, \Delta z_0}, \qquad \mathcal{S}_S^{\text{sfc}} = -\frac{F_w\, S_{\text{ref}}}{\rho_0\, \Delta z_0}, \tag{4.11}$$

with $c_p =$ `CP_WATER` $= 3985\,\text{J}\,\text{kg}^{-1}\,\text{K}^{-1}$ and a virtual-salt-flux reference $S_{\text{ref}} = 35$. An optional subpolar North-Atlantic freshwater "hosing" term (in Sv) is added to $\mathcal{S}_S$ for AMOC experiments (Chapter 17).
↪ *Code:* `chronos_esm/ocean/veros_driver.py` — `step_ocean`: `fT_s`, `fS_s`, `fD_s`; the `.at[0].add(...)` surface injection

Salinity restoring, clamps, and conservation. Because the model lacks a prognostic sea-ice/brine cycle and uses a virtual salt flux, salinity is restored and renormalized to prevent drift into the physics-stability clip. The code applies a strong high-latitude sponge (poleward of 65°, 6-month timescale) plus a weak global relaxation to $S_{\text{ref}} = 35$ (3-year timescale) (Haney, 1971), then hard-clips $S \in [30, 38]$ psu and $\theta \in [250, 320]\,\text{K}$, floors θ at the -1.8°C freezing point (271.35 K, a sea-ice stub), and finally shifts the volume-mean salinity back to the observed 34.7 psu each step. The mean-shift removes net drift without touching the spatial gradient that drives the circulation. A differentiable soft clip (`soft_clip`, a tanh saturation) is available as a gradient-friendly alternative to the hard `jnp.clip`.
↪ *Code:* `chronos_esm/ocean/veros_driver.py` — `step_ocean`: sponge/global relaxation, `jnp.clip`, freezing floor, volume-mean renorm
↪ *Code:* `chronos_esm/ocean/utils.py` — `soft_clip`: differentiable tanh saturation

4.3 Equation of state and density

Density closes the system by linking ρ to (θ, S). The real ocean obeys a strongly nonlinear equation of state (TEOS-10); Chronos-ESM uses a *linear* EOS, adequate for a differentiable T31 model and easy to differentiate exactly:

$$\rho(\theta, S) = \rho_0 - \alpha_T\,(\theta - \theta_0) + \beta_S\,(S - S_0), \tag{4.12}$$

with $\rho_0 = 1025\,\mathrm{kg\,m^{-3}}$, thermal-expansion proxy $\alpha_T = 0.2$, haline-contraction proxy $\beta_S = 0.8$, and reference state $\theta_0 = 283.15\,\mathrm{K}$, $S_0 = 35$. Density is recomputed at the start of every ocean step (entering the thermal-wind pressure) and again at the end from the updated tracers.
↪ *Code:* `chronos_esm/ocean/veros_driver.py` — `equation_of_state`

Remark. The coefficients in (6.1) are tuned tank-model values, not laboratory α, β (which are $\sim 2 \times 10^{-4}\,\mathrm{K^{-1}}$ and $\sim 8 \times 10^{-4}\,\mathrm{psu^{-1}}$ times ρ_0). The linear EOS omits cabbeling and thermobaricity and the pressure dependence of density. For the diagnostic thermal-wind and overturning closures used here, only the *sign and gradient* of $\rho(\theta, S)$ matter, and those are correct: warmer, fresher water is lighter.

4.4 Vertical coordinate, bathymetry, and masks

z-levels with a stretched grid. Chronos-ESM is a fixed z-level model with $n_z = 15$ levels over a 5000 m column. A uniform Δz would make the surface layer $\sim 333\,\mathrm{m}$ thick — far too sluggish a SST response, since forcing enters as $Q/(\rho_0 c_p \Delta z_0)$ (Eq. 4.11). Instead the interfaces are stretched so the surface layer is thin ($\sim 50\,\mathrm{m}$) and layers thicken with depth:

$$z_i = H_{\mathrm{tot}} \left(\tfrac{i}{n_z}\right)^s, \quad i = 0, \ldots, n_z, \qquad \Delta z_k = z_{k+1} - z_k, \quad s = 1.7. \tag{4.13}$$

The layer-centre depths $z_k^c = \frac{1}{2}(z_k + z_{k+1})$ are also the depths onto which the WOA18 initial conditions are interpolated (Locarnini et al., 2018; Zweng et al., 2018), so dynamics and ICs stay consistent.
↪ *Code:* `chronos_esm/config.py` — `_ocean_vertical_grid`: `OCEAN_DZ`, `OCEAN_DEPTH_CENTERS`, `stretch=1.7`

Bathymetry and the wet mask. The land/sea mask is derived from where WOA18 surface temperature is valid; a positive-down depth field $H(x, y)$ is built from ETOPO bathymetry (Amante and Eakins, 2009) (block-mean coarsened, regridded, Gaussian-smoothed, capped at 5000 m, floored at 500 m in wet cells, with isolated one-cell ponds removed). From $H(x, y)$ and the level interfaces the code forms a 3-D *wet-cell mask* $\chi_{k,j,i} \in \{0, 1\}$ (1 ocean, 0 land or below the floor).
↪ *Code:* `chronos_esm/data.py` — `load_bathymetry_mask`, `load_ocean_depth`, `load_initial_conditions`

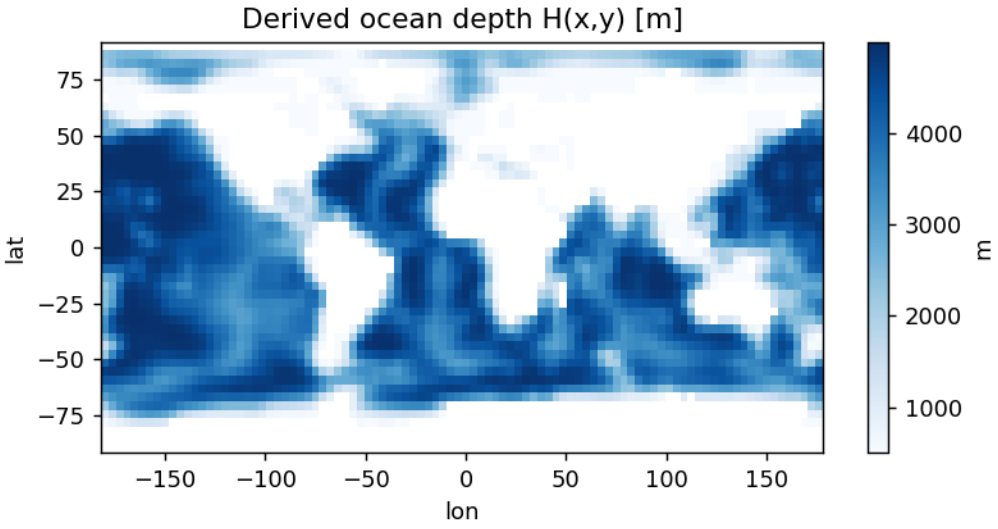

Figure 4.1: Model ocean depth $H(x, y)$ at T31, derived from ETOPO bathymetry (block-mean coarsened, smoothed, capped at 5000 m and floored at 500 m in wet cells). The land/sea mask follows where the WOA18 surface field is valid.

Table 4.1: Key ocean parameters and their source (T31 default; see `config.py` and `step_ocean` defaults).

Symbol	Value	Where
ρ_0 (`RHO_WATER`)	$1025\,\mathrm{kg\,m^{-3}}$	`config.py`
c_p (`CP_WATER`)	$3985\,\mathrm{J\,kg^{-1}\,K^{-1}}$	`config.py`
Ω (`OMEGA`)	$7.2921 \times 10^{-5}\,\mathrm{s^{-1}}$	`config.py`
Δt (`DT_OCEAN`)	$900\,\mathrm{s}$	`config.py`
$n_z,\ H_{\mathrm{tot}},\ s$	$15,\ 5000\,\mathrm{m},\ 1.7$	`config.py`
κ_h (interior)	$500\,\mathrm{m^2\,s^{-1}}$	`step_ocean kappa_h`
κ_{GM}	$1000\,\mathrm{m^2\,s^{-1}}$	`step_ocean kappa_gm`
(α_T, β_S)	$(0.2,\ 0.8)$	`equation_of_state`
r_{bt}	$(25\,\mathrm{d})^{-1}$	`step_ocean`

No-flux boundary conditions via face masks. Boundaries are enforced with the MITgcm *hFac* rule (Adcroft et al., 1997): a cell face is open only if *both* adjacent centres are wet. The code builds west, east, south, north and vertical-interface masks

$$f^{\mathrm{E}}_{k,j,i} = \chi_{k,j,i}\,\chi_{k,j,i+1}, \quad \ldots, \quad f^{\mathrm{iface}}_{k} = \chi_k\,\chi_{k+1}, \tag{4.14}$$

with the polar rows ($j = 0$, $j = n_y - 1$) and the surface/floor interfaces forced closed. A closed face returns the cell's own value (zero gradient), so neither advection nor diffusion ever exchanges with a dry cell. The surface interface is closed (rigid lid: no tracer advected through the ocean surface), and the floor interface is closed (no flux through the sea bed). All masks are *static*, hence safe for automatic differentiation.

↪ *Code:* `chronos_esm/ocean/veros_driver.py` — `step_ocean`: `maskC`, `fW/fE/fS/fN`, `iface`, `_nbrs`; tendencies frozen on dry cells

Proposition 4.1 (Discrete tracer conservation under the rigid-lid projection)**.** *With the depth-mean horizontal divergence projected out (so the discrete 3-D flow $(\mathbf{u}_{\mathrm{eff}}, w)$ is non-divergent with $w = 0$ at surface and floor) and no-flux face masks, the advective-form vertical advection conserves the volume-weighted tracer integral $\sum_{k,j,i} \theta_{k,j,i}\,\Delta z_k\,\chi_{k,j,i}$ in the absence of surface forcing and restoring. The volume-mean salinity renormalization then removes any residual drift from the clips and upwind numerics.*

Summary

The Chronos-ESM ocean solves Boussinesq tracer transport (Eq. 6.12) for (θ, S, C) with RK4 (Eq. 6.14), closes density with a linear EOS (Eq. 6.1), and *diagnoses* velocity from a barotropic Stommel streamfunction (Eq. 7.11) plus a baroclinic thermal-wind shear (Eq. 18.1), with w from continuity (Eq. 4.8). The vertical grid is stretched z-levels (Eq. 4.13); bathymetry, masks, and the hFac no-flux face rule (Eq. 4.14) impose the coastal and sea-floor boundaries. The mixing closures (GM, vertical diffusivity) are detailed in Chapter 5, the discrete operators and solver in Chapter 6, and the overturning closure in Chapter 8.

Exercise 4.1. Using `equation_of_state`, compute the density difference between a warm subtropical surface parcel ($\theta = 298\,\mathrm{K}$, $S = 36.5$) and a cold subpolar parcel ($\theta = 275\,\mathrm{K}$, $S = 34$). Which is denser, and by how much ($\mathrm{kg\,m^{-3}}$)? Now repeat with realistic coefficients $\alpha_T = \rho_0 \cdot 2 \times 10^{-4}$, $\beta_S = \rho_0 \cdot 8 \times 10^{-4}$. Discuss how the model's tuned α, β exaggerate or understate the thermal vs. haline control of density.

Exercise 4.2. The surface heat-flux tendency is $Q/(\rho_0 c_p \Delta z_0)$. Using the stretched-grid formula (4.13) with $n_z = 15$, $s = 1.7$, compute Δz_0 and the SST tendency for a net flux $Q = 50\,\mathrm{W\,m^{-2}}$, in $\mathrm{K\,day^{-1}}$. Then recompute for a *uniform* grid ($\Delta z_0 = 5000/15\,\mathrm{m}$). Explain quantitatively why the stretched grid is essential for a realistic SST response.

Exercise 4.3. Run a short ocean integration (a few model days) and diagnose the zonally-and-vertically integrated meridional transport $\sum_{k,i} v\,\Delta x\,\Delta z_k\,\chi$ at each latitude *before* and *after* the per-latitude net-transport corrector (comment it out to see the "before"). Confirm the corrector removes the spurious

$\mathcal{O}(\pm 350\,\mathrm{Sv})$ net transport. Why is a vanishing net meridional transport a prerequisite for a clean AMOC overturning (Chapter 8)?

Exercise 4.4. Chronos-ESM is differentiable. Treat the EOS coefficient α_T as a control parameter and, using JAX automatic differentiation, compute the gradient of the global-mean SST after one ocean step with respect to α_T. Where in the diagnostic velocity split (Eq. 4.4) does α_T enter, and would you expect $\partial(\mathrm{AMOC})/\partial\alpha_T$ to be non-zero given that velocity is *diagnosed*, not prognostic? Relate your answer to the thermal-wind pathway in Eq. (18.1).

Chapter 5

The Ocean: Subgrid Physics and Mixing

A model run at the T31 ocean grid of Chronos-ESM ($\approx 3.75°$, see Chapter 4) cannot resolve the mesoscale eddy field: the first baroclinic Rossby radius in the mid-latitude ocean is $\mathcal{O}(20\text{–}50)\,\mathrm{km}$, an order of magnitude below a grid box. The eddies are nonetheless dynamically essential—they flatten isopycnals, set the stratification, and carry a large fraction of the meridional heat transport. This chapter documents how the unresolved physics is *parameterized*: the Gent–McWilliams (GM) bolus advection (Gent and McWilliams, 1990), along-isopycnal (Redi) mixing, horizontal and vertical diffusion, the smooth convective adjustment, and the Shapiro/biharmonic filter used purely for numerical hygiene. Throughout, the governing continuous form precedes the discrete operator that the code actually integrates, with the source pinned by a *Code* marker. The canonical reference for the rotated tracer-mixing tensor is Griffies (2004); the GM closure is Gent and McWilliams (1990); the standard textbook treatment is Vallis (2017).

Remark. All operators below are written in JAX and are differentiable. Where a naive formulation would introduce a non-smooth max, hard clip, or `where` step (zero gradient outside the bound), the code substitutes a smooth surrogate so that $\partial(\text{output})/\partial(\text{parameter})$ stays well defined—this is a deliberate design choice, not an accident (Chapter 3).

5.1 Isopycnal slopes

Both GM and Redi act *along* surfaces of constant density. The local slope of those surfaces, relative to the geopotential, is the slope vector $\mathbf{S} = (S_x, S_y)$ with

$$S_x = -\frac{\partial_x \rho}{\partial_z \rho}, \qquad S_y = -\frac{\partial_y \rho}{\partial_z \rho}, \tag{5.1}$$

where ρ is the in-situ density and z increases downward, so a statically stable column has $\partial_z \rho > 0$. Each gradient is a centred finite difference; the horizontal differences use `jnp.roll` (periodic in λ), and the vertical gradient is taken at interfaces and averaged back to cell centres using the centre-to-centre spacing $\frac{1}{2}(\Delta z_k + \Delta z_{k+1})$.

↪ *Code:* `chronos_esm/ocean/mixing.py` — `compute_isopycnal_slopes`

The denominator is regularized to keep the slope finite in weakly stratified columns:

$$\partial_z \rho \;\longmapsto\; \begin{cases} \partial_z \rho, & |\partial_z \rho| \geq \varepsilon, \\ \operatorname{sign}(\partial_z \rho)\,\varepsilon, & |\partial_z \rho| < \varepsilon, \end{cases} \qquad \varepsilon = 10^{-8}\,\mathrm{kg\,m^{-4}}. \tag{5.2}$$

This is an honest approximation: in a near-neutral or unstably stratified column the "isopycnal" slope is not physically meaningful, and (5.2) simply prevents a blow-up while convective adjustment (§5.6) handles the genuine instability.

5.2 The slope limiter

Steep isopycnals near the surface mixed layer and at boundary currents produce very large $|\mathbf{S}|$, which would make the GM and Redi terms numerically explosive. The standard remedy is to taper the slope

(Danabasoglu and McWilliams, 1995; Large et al., 1997). Chronos-ESM replaces the conventional hard clip $|\mathbf{S}| \le S_{\max}$ with a *smooth, differentiable* soft clip:

$$\widetilde{S} \;=\; \frac{S}{\sqrt{1+(S/S_{\max})^2}}, \qquad S_{\max} = 10^{-2}, \tag{5.3}$$

applied component-wise to S_x, S_y. As $S \to 0$, $\widetilde{S} \to S$ (no distortion of gentle slopes); as $S \to \pm\infty$, $\widetilde{S} \to \pm S_{\max}$ (bounded). Crucially, $\mathrm{d}\widetilde{S}/\mathrm{d}S > 0$ everywhere, so the limiter never zeroes the gradient—a hard clip would, breaking adjoint sensitivity through steep regions.
↪ *Code:* `chronos_esm/ocean/mixing.py` — `slope_limiter`

5.3 Gent–McWilliams bolus transport

The GM parameterization (Gent and McWilliams, 1990) represents the eddy-induced ("bolus") transport as an extra advective velocity $\mathbf{u}^\star = (u^\star, v^\star, w^\star)$ that adiabatically flattens isopycnals, extracting available potential energy as a mesoscale eddy field would. With a thickness diffusivity κ_{gm} the eddy streamfunction is $\boldsymbol{\Psi} = \kappa_{\mathrm{gm}}\,\mathbf{S}$, and the bolus velocity is its curl:

$$u^\star = \partial_z\Big(\kappa_{\mathrm{gm}}\,\widetilde{S}_x\Big), \qquad v^\star = \partial_z\Big(\kappa_{\mathrm{gm}}\,\widetilde{S}_y\Big), \tag{5.4}$$

$$w^\star = -\Big[\partial_x\Big(\kappa_{\mathrm{gm}}\,\widetilde{S}_x\Big) + \partial_y\Big(\kappa_{\mathrm{gm}}\,\widetilde{S}_y\Big)\Big]. \tag{5.5}$$

By construction $\boldsymbol{\nabla}\cdot\mathbf{u}^\star = 0$, so GM transports tracers without creating or destroying mass.

In the code, the horizontal bolus fluxes $F_x = \kappa_{\mathrm{gm}}\widetilde{S}_x$, $F_y = \kappa_{\mathrm{gm}}\widetilde{S}_y$ are interpolated to vertical interfaces and set to zero at the surface and floor (no bolus flux through the boundaries); the vertical derivative (5.4) is then a centred difference over the layer thickness Δz_k:

$$u^\star_k = \frac{F_x^{k+1/2} - F_x^{k-1/2}}{\Delta z_k}, \qquad F_x^{k+1/2} = \tfrac{1}{2}\big(F_x^{k} + F_x^{k+1}\big), \tag{5.6}$$

and likewise for $v^\star$.
↪ *Code:* `chronos_esm/ocean/mixing.py` — `compute_gm_bolus_velocity`

Remark (Honest limitation: $w^\star$ is not formed here). The function returns $w^\star \equiv 0$. The vertical bolus velocity is *not* diagnosed from (5.5) in `mixing.py`; instead the bolus velocity is *added to the resolved flow*—$\mathbf{u}_{\mathrm{eff}} = \mathbf{u} + \mathbf{u}^\star$ with $u^\star, v^\star$ from (5.6)—and w is recovered from continuity of that combined field, so the three-dimensional transport that advects tracers is nondivergent by construction.
↪ *Code:* `chronos_esm/ocean/veros_driver.py` — $\mathbf{u}_{\mathrm{eff}} = \mathbf{u} + \mathbf{u}^\star$; w from $\boldsymbol{\nabla}\cdot\mathbf{u}_{\mathrm{eff}} = 0$

This is consistent but means GM enters the model as a velocity, not as a skew-flux on each tracer separately—adequate for a coarse climate run, but not the full Griffies skew-diffusion form.

The diffusivity κ_{gm} defaults to $1000\,\mathrm{m^2\,s^{-1}}$ in physics mode (versus 3000 in the legacy "tank" configuration; see the parameter table in Appendix B and the discussion in CLAUDE-context Phase 6). It is masked to zero in the five polar rows at each end of the grid, $\kappa_{\mathrm{gm}}^{\mathrm{eff}} = \kappa_{\mathrm{gm}}\,\chi_{\mathrm{int}}$, where $\chi_{\mathrm{int}} = 0$ for $j < 5$ or $j \ge n_y - 5$ and 1 otherwise, because the slope estimate degrades where the grid converges.
↪ *Code:* `chronos_esm/ocean/veros_driver.py` — `kappa_gm_eff = kappa_gm * interior_mask_3d`

5.4 Along-isopycnal (Redi) mixing tensor

Diffusion of heat and salt in the ocean interior is overwhelmingly *along* isopycnals, not across them; diapycnal mixing is weaker by $\mathcal{O}(10^7)$. The Redi tensor (Redi, 1982) rotates an isotropic diffusivity κ_{iso} into the local isopycnal plane. In the small-slope approximation (Griffies, 2004),

$$\mathbf{K} \;=\; \frac{\kappa_{\mathrm{iso}}}{1+S^2}\begin{pmatrix} 1 & 0 & \widetilde{S}_x \\ 0 & 1 & \widetilde{S}_y \\ \widetilde{S}_x & \widetilde{S}_y & S^2 \end{pmatrix}, \qquad S^2 = \widetilde{S}_x^{\,2} + \widetilde{S}_y^{\,2}, \tag{5.7}$$

so the diffusive tracer flux is $\mathbf{F} = -\mathbf{K}\,\boldsymbol{\nabla}C$ for a tracer C. The diagonal $(1,1)$ block gives near-horizontal diffusion; the off-diagonal $\widetilde{S}_x, \widetilde{S}_y$ couple horizontal gradients to the vertical flux (so mixing follows the

tilted surface); and the $(3,3)$ entry $\propto S^2$ is the small residual along-slope vertical component. The prefactor $1/(1+S^2)$ is the small-slope normalization. The tensor is assembled exactly as (5.7) (symmetric, built from the limited slopes).
↪ *Code:* `chronos_esm/ocean/mixing.py` — `rotate_tensor`

5.5 Horizontal and vertical diffusion as integrated

While `rotate_tensor` provides the full Redi operator, the tracer time-stepper in the active driver uses a *simpler* along-axis diffusion: a horizontal Laplacian with coefficient κ_h plus a vertical diffusion with the convective κ_z of §5.6. This is an honest simplification—it is robust on the coarse grid and the dominant interior balance there is set by GM, not Redi—and it is what the model integrates today.

The horizontal operator combines upwind advection by $\mathbf{u}_{\text{eff}}$ with a five-point Laplacian,

$$\mathcal{H}(C) = -\big(u_{\text{eff}}\,\partial_x C + v_{\text{eff}}\,\partial_y C\big) + \kappa_h\left(\frac{C_E - 2C + C_W}{\Delta x^2} + \frac{C_N - 2C + C_S}{\Delta y^2}\right), \tag{5.8}$$

where face neighbours $C_{E,W,N,S}$ revert to the centre value across any *closed* (land/floor) face, enforcing a no-flux boundary with no exchange into dry cells.
↪ *Code:* `chronos_esm/ocean/veros_driver.py` — `_horiz`, `_nbrs`

The effective coefficient is $\kappa_h = 500\,\mathrm{m^2\,s^{-1}}$ in the interior but is forced up to $2\times10^4\,\mathrm{m^2\,s^{-1}}$ in the polar rows, $\kappa_h^{\text{eff}} = 2{\times}10^4\,\chi_{\text{pole}} + \kappa_h\,\chi_{\text{int}}$, to damp grid-scale noise where the meridian convergence is worst.
↪ *Code:* `chronos_esm/ocean/veros_driver.py` — `diff_coef = 20000.*pole_mask + kappa_h*interior_mask`

Vertical diffusion is flux-form across interfaces,

$$\mathcal{V}(C)_k = \frac{1}{\Delta z_k}\Big(\mathcal{F}^{k-1/2} - \mathcal{F}^{k+1/2}\Big), \qquad \mathcal{F}^{k+1/2} = -\,\kappa_z^{k+1/2}\,\frac{C_{k+1} - C_k}{\tfrac{1}{2}(\Delta z_k + \Delta z_{k+1})}, \tag{5.9}$$

with $\mathcal{F} = 0$ at the surface and floor interfaces. The whole tracer update ($\mathcal{H} + \mathcal{V}$ plus vertical advection and surface fluxes) is advanced by RK4 with a $\Delta t = 900\,\mathrm{s}$ ocean step.
↪ *Code:* `chronos_esm/ocean/veros_driver.py` — `_vdiff`; RK4 `tendencies`

5.6 Smooth convective adjustment

When a column becomes statically unstable ($\partial_z \rho < 0$, denser water over lighter) the ocean overturns and mixes rapidly. The classical implementation is a hard switch—set κ_z to a huge value wherever $\Delta\rho < 0$ (Rahmstorf, 1993). Chronos-ESM rejects this for two reasons documented in the source: (i) a 0/1 switch fires patchily column-by-column and produces grid-scale SST noise that grows toward the poles, and (ii) a large κ_z at the thin surface layer violates the explicit-diffusion stability limit $\kappa_z \Delta t/\Delta z^2 \le \mathcal{O}(1)$.

Instead the diffusivity *ramps smoothly* with the degree of instability $I = \max(-\Delta\rho, 0)$ at each interface:

$$\kappa_z = \kappa_{\text{bg}} + (\kappa_{\text{cv}} - \kappa_{\text{bg}})\Big(1 - e^{-I/I_0}\Big), \tag{5.10}$$

with background $\kappa_{\text{bg}} = 10^{-5}\,\mathrm{m^2\,s^{-1}}$, convective ceiling $\kappa_{\text{cv}} = 10\,\mathrm{m^2\,s^{-1}}$, and instability scale $I_0 = 0.02\,\mathrm{kg\,m^{-3}}$. The factor $1 - e^{-I/I_0}$ runs continuously from 0 (stable) to 1 (strongly unstable), so neighbouring columns receive continuously-varying mixing and the operator is smooth (AD-friendly).

Each interface is then capped at its local explicit-stability limit, using the thinner of the two adjacent layers,

$$\kappa_z \;\longmapsto\; \min\left(\kappa_z,\; 0.45\,\frac{\big(\min(\Delta z_k, \Delta z_{k+1})\big)^2}{\Delta t}\right), \tag{5.11}$$

so thin surface layers stay numerically stable while thick abyssal layers can still mix strongly—genuine deep convection is preserved.
↪ *Code:* `chronos_esm/ocean/mixing.py` — `compute_vertical_diffusivity`

Example 5.1 (Why the cap matters)**.** With $\Delta t = 900\,\mathrm{s}$ and a surface layer $\Delta z_0 = 10\,\mathrm{m}$, the cap is $0.45\cdot10^2/900 = 0.05\,\mathrm{m^2\,s^{-1}}$—two hundred times *smaller* than $\kappa_{\text{cv}} = 10$. Without (5.11) the explicit scheme would amplify a surface perturbation each step. For a deep layer $\Delta z = 500\,\mathrm{m}$ the cap is $0.45\cdot500^2/900 \approx 125\,\mathrm{m^2\,s^{-1}} \gg \kappa_{\text{cv}}$, so deep convection is untouched. The single line (5.11) thus reconciles surface stability with deep mixing.

5.7 The Shapiro / biharmonic filter

Centred advection on a C-grid admits a $2\Delta x$ "checkerboard" mode that carries no physical signal but is invisible to Laplacian diffusion at large scales. Chronos-ESM removes it with a scale-selective *biharmonic* (Shapiro) filter (Shapiro, 1970) applied to the new θ and S after each RK4 update:

$$C \longmapsto C - \frac{\sigma}{16}\nabla^2\big(\nabla^2 C\big), \tag{5.12}$$

where ∇^2 is the five-point Laplacian and σ is `shapiro_strength`. The eigenvalue of ∇^2 for a pure $2\Delta x$ mode is -4 per direction, so the $2\Delta x$ checkerboard has $\nabla^2\nabla^2$-eigenvalue ≈ 16 and the choice $\sigma = 0.25$ *exactly* annihilates it in one pass, while resolved gradients (small $\nabla^2\nabla^2$) are nearly untouched.
↪ *Code:* `chronos_esm/ocean/veros_driver.py` — `compute_shapiro_filter`

Remark (An honest correction to the legacy default). An earlier ∇^2 form ($C + \frac{\sigma}{4}\nabla^2 C$) applied every step was a strong low-pass smoother that erased real SST/SSS structure—the WOA initial gradients collapsed toward uniform within days. The biharmonic form (5.12) is far gentler and is **off by default** ($\sigma = 0$): the primary grid-noise control is the physical Laplacian diffusion (κ_h on tracers, A_h on momentum, with $A_h = 2 \times 10^5\,\mathrm{m^2\,s^{-1}}$). The biharmonic filter is a light $2\Delta x$ cleanup to be switched on only if checkerboarding reappears. Biharmonic momentum mixing (A_b, `kappa_bi`) is likewise available and defaults to zero.

5.8 Parameter summary

Table 5.1 collects the diffusivities and their roles. Physics-mode values (the realistic configuration) are shown; the legacy "tank" values appear in Appendix B.

Table 5.1: Mixing and filter parameters (physics mode).

Symbol	Value	Role
κ_{gm}	$1000\ \mathrm{m^2\,s^{-1}}$	GM thickness (bolus) diffusivity
κ_{iso}	set by caller	Redi isopycnal diffusivity (tensor form)
κ_h	$500\ \mathrm{m^2\,s^{-1}}$	horizontal tracer Laplacian (interior)
κ_h^{pole}	$2 \times 10^4\ \mathrm{m^2\,s^{-1}}$	polar-row tracer diffusion
κ_{bg}	$10^{-5}\ \mathrm{m^2\,s^{-1}}$	background vertical diffusivity
κ_{cv}	$10\ \mathrm{m^2\,s^{-1}}$	convective ceiling
I_0	$0.02\ \mathrm{kg\,m^{-3}}$	convective ramp scale
A_h	$2 \times 10^5\ \mathrm{m^2\,s^{-1}}$	horizontal momentum viscosity
σ	0 (default)	biharmonic (Shapiro) filter strength
$S_{\max}$	10^{-2}	slope-limiter ceiling

The eddy heat transport that GM supplies is the quantity tied most directly to the overturning (Chapter 8) and to AMOC stability (Chapter 17); because every operator here is differentiable, $\partial(\mathrm{AMOC})/\partial\kappa_{\mathrm{gm}}$ is computable by the adjoint, which the calibration applications of Chapter 15 exploit.

Exercises

Exercise 5.1 (Limiter versus hard clip)**.** Implement both the smooth limiter (5.3) and a hard clip $\widetilde{S} = \mathrm{clip}(S, -S_{\max}, S_{\max})$ in a small JAX script and use `jax.grad` to plot $\mathrm{d}\widetilde{S}/\mathrm{d}S$ for $S \in [-3S_{\max}, 3S_{\max}]$. Confirm that the hard clip has zero gradient for $|S| > S_{\max}$ while (5.3) stays strictly positive. Explain, in terms of the adjoint, why a 4D-Var sensitivity through a steep western-boundary current would be lost with the hard clip.

Exercise 5.2 (GM flattens isopycnals)**.** Initialize a two-layer channel with tilted isopycnals (constant S_x) and integrate the ocean with $\kappa_{\mathrm{gm}} \in \{0, 500, 2000\}\,\mathrm{m^2\,s^{-1}}$, all other terms off. Measure the e-folding time of the slope. Verify it scales as $\tau \sim L^2/\kappa_{\mathrm{gm}}$ for a slope of horizontal scale L, and discuss why $\kappa_{\mathrm{gm}} = 3000$ (tank mode) overflattens the stratification relative to $\kappa_{\mathrm{gm}} = 1000$.

Exercise 5.3 (Convective cap by layer thickness)**.** Using (5.11) with $\Delta t = 900\,\mathrm{s}$, tabulate the capped κ_z for layer thicknesses $\Delta z \in \{5, 10, 50, 200, 500\}\,\mathrm{m}$ and the ramp value of (5.10) for $I \in \{0, 0.01, 0.05, 0.2\}\,\mathrm{kg\,m^{-3}}$. Identify the thickness at which the convective ceiling $\kappa_{\mathrm{cv}} = 10$ stops being the binding constraint. Why does this design let deep convection coexist with a stable thin surface layer?

Exercise 5.4 (Is the filter eating the signal?)**.** Run the ocean for 30 days from a WOA initial state with the biharmonic filter at $\sigma \in \{0, 0.25, 1.0\}$ and compute the global SST pattern correlation against the WOA field. Show that $\sigma = 0.25$ removes a $2\Delta x$ checkerboard you inject by hand while leaving the large-scale pattern correlation essentially unchanged, and that large σ degrades it—motivating the default $\sigma = 0$ and reliance on physical κ_h.

Chapter 6

The Ocean: Numerics and Solvers

Chapter 4 laid out the continuous primitive equations the ocean component aspires to solve, and Chapter 5 filled in the subgrid closures (isopycnal/Gent–McWilliams mixing, vertical diffusivity). This chapter is about the *discretisation* that actually runs: how one ocean time step is assembled in code, which terms are stepped prognostically and which are diagnosed, and the linear-algebra machinery (a differentiable conjugate-gradient elliptic solver) underneath it.

The central, load-bearing fact of this chapter — stated plainly so the reader is never misled — is that in the current model *only the tracers are prognostic*. Temperature θ, salinity S, and dissolved inorganic carbon are advanced in time by Runge–Kutta integration of an advection–diffusion equation. The velocity field (u, v, w) is *diagnosed* every step from the density and wind-stress fields via geostrophy, thermal wind, a flat-bottom barotropic streamfunction, and continuity. There is no prognostic momentum equation: the model never integrates $\partial \mathbf{u}/\partial t$. This is a deliberate, honest simplification with real consequences for the overturning circulation, and it is precisely what motivates the prognostic-momentum core developed in Chapter 7. The whole step lives in one JAX-jitted function.

↪ *Code:* `chronos_esm/ocean/veros_driver.py` — `step_ocean` — the entire ocean time step

6.1 Anatomy of one ocean time step

A single call to `step_ocean` executes, in order: (1) update density from the equation of state; (2) build the diagnostic horizontal velocity from wind stress (barotropic) plus thermal wind (baroclinic), then correct its net transport; (3) diagnose the vertical velocity from continuity under a rigid lid; (4) advance the tracers one step with RK4; (5) apply salinity sponges, hard physical clips, and a global-mean conservation renormalisation. Steps (2)–(3) are pure diagnostics recomputed from scratch each step; only step (4) carries state forward in time. The ocean time step is $\Delta t = 900\,\mathrm{s}$ (15 minutes).

↪ *Code:* `chronos_esm/config.py` — `DT_OCEAN`

6.1.1 Equation of state

Density is a linear function of potential temperature and salinity,

$$\rho(\theta, S) = \rho_0 - \alpha\,(\theta - \theta_0) + \beta_S\,(S - S_0), \tag{6.1}$$

with $\rho_0 = 1025\,\mathrm{kg\,m^{-3}}$, thermal expansion $\alpha = 0.2\,\mathrm{kg\,m^{-3}\,K^{-1}}$, haline contraction $\beta_S = 0.8\,\mathrm{kg\,m^{-3}\,psu^{-1}}$, and reference state $(\theta_0, S_0) = (283.15\ \mathrm{K},\ 35\ \mathrm{psu})$. This is the simplest seawater EOS; it omits the pressure dependence and the nonlinear cabbeling/thermobaricity terms of a full TEOS-10 formulation (Griffies, 2004), but it keeps the density a smooth, exactly differentiable function of the prognostic tracers — which matters because the entire diagnostic velocity, and hence the gradient $\partial(\mathrm{AMOC})/\partial(\mathrm{density})$, flows back through (6.1).

↪ *Code:* `chronos_esm/ocean/veros_driver.py` — `equation_of_state`

6.2 The diagnostic velocity field

The horizontal velocity is split, in the classical way, into a depth-independent *barotropic* part and a depth-varying *baroclinic* part, $\mathbf{u} = \mathbf{u}_{\mathrm{bt}} + \mathbf{u}_{\mathrm{bc}}$.

6.2.1 Barotropic streamfunction: a Stommel-style balance

For the barotropic flow the model does not integrate a momentum equation; it solves a steady, linearly damped vorticity balance in the spirit of Stommel (1961). Taking the curl of a wind-driven, bottom-drag-balanced depth-integrated momentum equation and writing the transport as a streamfunction ψ, with $U_{\rm bt} = -\partial\psi/\partial y$, $V_{\rm bt} = \partial\psi/\partial x$, gives a Poisson problem

$$\nabla^2\psi \;=\; \frac{(\boldsymbol{\nabla}\times\boldsymbol{\tau})_z}{\rho_0\, H\, r_{\rm bt}}, \qquad (\boldsymbol{\nabla}\times\boldsymbol{\tau})_z = \frac{\partial\tau_y}{\partial x} - \frac{\partial\tau_x}{\partial y}, \tag{6.2}$$

where $\boldsymbol{\tau} = (\tau_x, \tau_y)$ is the surface wind stress, $H = \sum_k \Delta z_k$ the total depth, and $r_{\rm bt} = 1/(25 \times 86400\,\mathrm{s})$ a tuned 25-day linear drag. The wind-stress curl is evaluated with centred differences (`jnp.roll`), and (6.2) is solved by the conjugate-gradient Poisson solver of §6.5 with the previous step's ψ as a warm start. The barotropic velocity is then recovered by centred differencing of ψ:

$$U_{\rm bt} = -\frac{\psi_{j+1} - \psi_{j-1}}{2\,\Delta y}, \qquad V_{\rm bt} = \frac{\psi_{i+1} - \psi_{i-1}}{2\,\Delta x}. \tag{6.3}$$

↪ *Code:* `chronos_esm/ocean/veros_driver.py` — step 3, `solve_poisson_2d` call + ψ differencing

Remark (Flat bottom). The Laplacian in (6.2) is constant-coefficient: it implicitly assumes a *flat bottom* (depth H pulled out of the divergence). Real bathymetry enters the tracer transport through wet-cell masks but *not* into the streamfunction operator. This mismatch is the source of the spurious net transport corrected in §6.2.3, and the variable-coefficient $\boldsymbol{\nabla}\cdot(H^{-1}\boldsymbol{\nabla}\psi)$ operator that fixes it properly is exactly the JEBAR-type solver discussed in §6.5.1 and built upon in Chapter 7.

6.2.2 Baroclinic flow: hydrostatic pressure and damped thermal wind

The baroclinic shear comes from the horizontal density gradient. The model first forms the hydrostatic pressure anomaly by integrating the density anomaly down from the surface,

$$p'(z) = g\int_0^z (\rho - \rho_0)\,\mathrm{d}z' \;\approx\; g\sum_{k'\le k} (\rho_{k'} - \rho_0)\,\Delta z_{k'}, \tag{6.4}$$

a cumulative sum over levels (`jnp.cumsum`). Rather than the bare geostrophic balance (which is singular at the equator where $f \to 0$), the model uses a *Rayleigh-damped* geostrophy — the steady, linearly-drag-balanced momentum equation $f\,\mathbf{k}\times\mathbf{u} + r\,\mathbf{u} = -\rho_0^{-1}\boldsymbol{\nabla}p'$ — which inverts to

$$u_{\rm geo} = -\frac{r\,\partial_x p' + f\,\partial_y p'}{\rho_0\,(f^2 + r^2)}, \tag{6.5}$$

$$v_{\rm geo} = \;\;\frac{f\,\partial_x p' - r\,\partial_y p'}{\rho_0\,(f^2 + r^2)}, \tag{6.6}$$

with $f = 2\Omega\sin\phi$. The denominator $f^2 + r^2$ never vanishes, so the equatorial singularity of pure geostrophy is removed by the drag r; the Coriolis parameter is additionally floored at $|f| = 10^{-5}\,\mathrm{s}^{-1}$ for safety. To split off the depth-mean (which the barotropic streamfunction already carries), the wet-column average is subtracted:

$$\mathbf{u}_{\rm bc} = \big(\mathbf{u}_{\rm geo} - \overline{\mathbf{u}}_{\rm geo}\big)\, m_C, \qquad \overline{\mathbf{u}}_{\rm geo} = \frac{1}{H_{\rm col}}\sum_k \mathbf{u}_{{\rm geo},k}\,\Delta z_k\, m_{C,k}, \tag{6.7}$$

where $m_C \in \{0, 1\}$ is the 3-D wet-cell mask (MITgcm-style `hFac` rule: a cell face is open only if both adjacent centres are wet) (Adcroft et al., 1997) and $H_{\rm col} = \sum_k \Delta z_k\, m_{C,k}$ the per-column wet depth. The full diagnostic velocity is $\mathbf{u} = \mathbf{u}_{\rm bt} + \mathbf{u}_{\rm bc}$, masked to wet cells.

↪ *Code:* `chronos_esm/ocean/veros_driver.py` — step 4, hydrostatic pressure + damped thermal wind + wet-column anomaly

Remark (Why this is only diagnostic, and why it matters). Equations (6.6)–(6.7) give a velocity that is *algebraically slaved* to the instantaneous density field. There is no time derivative of momentum, no advection of momentum, no JEBAR/bottom-pressure-torque coupling between topography and the flow. The thermal-wind shear is local in latitude and carries essentially no *net* meridional overturning to a fixed section: a direct test gives $\partial(\text{AMOC at } 26.5°\text{N})/\partial(\text{subpolar salt}) \approx 0$. Closing a realistic Atlantic overturning therefore needs an extra mechanism (§6.2.4 and Chapter 8) and, ultimately, prognostic momentum (Chapter 7).

6.2.3 Per-latitude barotropic net-transport corrector

A flat-bottom velocity streamfunction applied over *variable* bathymetry is inconsistent: the depth-integrated meridional transport is $V_{\mathrm{bt}} H_{\mathrm{col}}$, but $\partial(\psi H_{\mathrm{col}})/\partial x \neq H_{\mathrm{col}}\,\partial\psi/\partial x$ when H_{col} varies in x. The leftover divergence shows up as a large spurious *net* meridional transport — order $\pm 350\,\mathrm{Sv}$ at some latitudes, where a closed ocean basin requires ≈ 0. With that much fictitious net flow, no clean overturning cell can survive. As a first correction (it is honestly only a first one), the model removes the net transport latitude by latitude: subtract a latitude-uniform meridional velocity so that the zonally- and vertically-integrated transport vanishes at every ϕ,

$$v \;\leftarrow\; \Big(v - \frac{\sum_{x,z} v\,\Delta z\, m_C}{\sum_{x,z} \Delta z\, m_C}\Big)\, m_C, \tag{6.8}$$

(Δx is uniform in x, so it cancels in the ratio). This drives the net per-latitude transport to $\approx 0\,\mathrm{Sv}$ and is stable in long runs. It is a *patch*, not a cure: a proper per-basin / C-grid transport streamfunction (the variable-coefficient solve of §6.5.1) is the fuller fix.

↪ *Code:* `chronos_esm/ocean/veros_driver.py` — "Barotropic mass-conservation corrector" block

6.2.4 Thermohaline overturning closure

Because the diagnosed thermal wind carries no net overturning, a depth-integral-zero Atlantic overturning velocity v_{thc} is added as an interim, box-model-style closure in the spirit of Marotzke (1991). Its strength scales with the subpolar-minus-subtropical upper-ocean density contrast, so denser (colder/saltier) subpolar water drives a stronger cell:

$$v_{\mathrm{thc}} = k_{\mathrm{vel}}\ \mathrm{softplus}\Big(\frac{\Delta\rho_{\mathrm{c}}}{\Delta\rho_{\mathrm{scale}}}\Big)\, G(z)\, m_{\mathrm{Atl}}, \qquad \Delta\rho_{\mathrm{c}} = \Delta\rho + (\gamma_h - 1)\,\beta_S\,\Delta S, \tag{6.9}$$

where $G(z)$ is a fixed vertical shape that is $+1$ in the upper limb and negative below, with $\sum_k G_k \Delta z_k = 0$ (so the closure carries *no* net transport and survives the corrector (6.8)). The softplus keeps $v_{\mathrm{thc}} \geq 0$ and differentiable; γ_h (`haline_gain`) amplifies the haline term to lift the salt-advection feedback gain above unity, making the AMOC bistable for the tipping experiments of Chapter 17. This finally gives density a working pathway to the overturning ($\partial(\mathrm{AMOC})/\partial(\mathrm{density}) \neq 0$, sign-correct). The closure and its feedback are derived in full in Chapter 8.

↪ *Code:* `chronos_esm/ocean/overturning.py` — `thc_overturning_velocity`

6.3 Vertical velocity from continuity (rigid lid)

An overturning cell only transports heat and salt if w actually advects tracers, so w must be consistent with the same horizontal flow that does the advecting (including the Gent–McWilliams bolus velocity, $\mathbf{u}_{\mathrm{eff}} = \mathbf{u} + \mathbf{u}_{\mathrm{bolus}}$). The hydrostatic continuity equation,

$$\frac{\partial w}{\partial z} = -\boldsymbol{\nabla}\cdot\mathbf{u}_{\mathrm{eff}} = -\Big(\frac{\partial u_{\mathrm{eff}}}{\partial x} + \frac{\partial v_{\mathrm{eff}}}{\partial y}\Big), \tag{6.10}$$

is integrated *upward* from a rigid floor ($w = 0$ at the bottom). For the column to be closed at *both* ends — a rigid lid means no tracer is advected through the ocean surface either — the depth-mean horizontal divergence is removed first (a rigid-lid projection):

$$\boldsymbol{\nabla}\cdot\mathbf{u}_{\mathrm{eff}} \;\leftarrow\; \boldsymbol{\nabla}\cdot\mathbf{u}_{\mathrm{eff}} - \frac{1}{H_{\mathrm{col}}}\sum_k (\boldsymbol{\nabla}\cdot\mathbf{u}_{\mathrm{eff}})_k\,\Delta z_k, \qquad w_{\mathrm{if},k} = -\sum_{k'\geq k} (\boldsymbol{\nabla}\cdot\mathbf{u}_{\mathrm{eff}})_{k'}\,\Delta z_{k'}. \tag{6.11}$$

After projection the discrete 3-D flow $(\mathbf{u}_{\mathrm{eff}}, w)$ is exactly nondivergent, so the *advective*-form vertical advection conserves the tracer mean. This projection is not cosmetic: without it the residual column divergence piles into the surface cell and leaks the global mean at roughly $+0.25\ \mathrm{K\,yr^{-1}}$ of spurious heating, because the advective form (unlike flux form) has no compensating compression term to absorb it. The interface vertical velocity w_{if} is masked so that interior interfaces are open only where both adjacent centres are wet, and the surface and floor interfaces are closed.

↪ *Code:* `chronos_esm/ocean/veros_driver.py` — "Vertical velocity for tracer advection" block

Remark (Two vertical velocities). The model computes w twice. The version above (w_{if} from $\mathbf{u}_{\mathrm{eff}}$) is the one that advects tracers in the RK4. A separate, purely diagnostic w is also stored in the returned state, computed from the final (u, v) with the same upward continuity integral but *without* the rigid-lid projection and with no $\cos\phi$ metric term. The stored field is for inspection only; the tracer transport uses the projected one.

6.4 Tracer time stepping: RK4 advection–diffusion

The prognostic tracers θ, S, DIC obey

$$\frac{\mathrm{D}C}{\mathrm{D}t} = \underbrace{-\,\mathbf{u}_{\mathrm{eff}}\cdot\boldsymbol{\nabla}_h C}_{\text{horiz. advection}} \;-\; w\,\frac{\partial C}{\partial z} \;+\; \boldsymbol{\nabla}_h\cdot(\kappa_h\boldsymbol{\nabla}_h C) \;+\; \frac{\partial}{\partial z}\Big(\kappa_z\,\frac{\partial C}{\partial z}\Big) \;+\; F_{\mathrm{surf}}\,\delta_{k0}, \tag{6.12}$$

written so that all terms appear as a single tendency $\mathcal{T}(C)$. The right-hand side is built from four masked operators:

- **Horizontal advection** is first-order *upwind*: the upstream difference is selected by the sign of $u_{\mathrm{eff}}, v_{\mathrm{eff}}$ (`jnp.where`). Closed faces return the centre value (zero gradient), so no flux crosses a coast.
- **Horizontal diffusion** is a 5-point Laplacian with coefficient κ_h in the interior and a large $2\times10^4\,\mathrm{m^2\,s^{-1}}$ near the poles (a numerical sponge against polar grid-convergence noise; see Chapter 5).
- **Vertical advection** $-w\,\partial_z C$ is upwind on the sign of the cell-centred w, in the *same advective form* as the horizontal term. Mixing advective horizontal with flux-form vertical introduces a spurious $-C\,\partial_z w$ source that blows the temperature up; keeping both advective avoids it. The vertical CFL is tiny ($|w|\Delta t/\Delta z \sim 10^{-3}$), so explicit upwind is comfortably stable.
- **Vertical diffusion** uses the stability-dependent κ_z of Chapter 5, with zero flux through dry interfaces and the floor.
- **Surface forcing** F_{surf} enters only the top wet cell: $F_\theta = Q/(\rho_0 c_p \Delta z_0)$ with $c_p = 3985\,\mathrm{J\,kg^{-1}\,K^{-1}}$, a virtual-salt term for freshwater, and an optional subpolar freshwater *hosing* (Sverdrups) added directly so it bypasses the coupler's ocean-mean balancing — the bifurcation forcing of Chapter 17.

This tendency is integrated with the classical fourth-order Runge–Kutta scheme,

$$C^{n+1} = C^n + \frac{\Delta t}{6}\,(k_1 + 2k_2 + 2k_3 + k_4), \tag{6.13}$$

$$k_1 = \mathcal{T}(C^n), \quad k_2 = \mathcal{T}\big(C^n + \tfrac{\Delta t}{2}k_1\big), \quad k_3 = \mathcal{T}\big(C^n + \tfrac{\Delta t}{2}k_2\big), \quad k_4 = \mathcal{T}\big(C^n + \Delta t\,k_3\big), \tag{6.14}$$

applied simultaneously to $(\theta, S, \mathrm{DIC})$, with every tendency frozen to zero in dry cells. RK4 is fourth-order accurate and has a comfortable stability region for the advection–diffusion operator at $\Delta t = 900\,\mathrm{s}$; it is also smooth, so reverse-mode AD through the whole step is clean.
↪ *Code:* `chronos_esm/ocean/veros_driver.py` — `tendencies`, `_horiz`, `_vadv`, `_vdiff` + the RK4 combination

6.4.1 Post-step stabilisers, clips, and conservation

After the RK4 update the model applies, in order: an optional biharmonic (∇^4) Shapiro filter (Shapiro, 1970) (off by default, a light 2-grid-point cleanup; the physical Laplacian diffusion is the primary noise control); a two-tier salinity relaxation (a strong 6-month high-latitude sponge poleward of 65° plus a weak 3-year global relaxation toward 35 psu); hard physical clips $S \in [30, 38]$ psu, $\theta \in [250, 320]$ K and a 271.35 K (-1.8°C) freezing floor standing in for absent sea-ice latent heat; and finally a volume-weighted global-mean salinity renormalisation back to 34.7 psu,

$$S \;\leftarrow\; \Big(S - \big(\langle S\rangle_{\mathrm{wet}} - 34.7\big)\Big)\,m_C, \tag{6.15}$$

which removes slow numerical/brine drift as a uniform additive shift, leaving the spatial *gradient* structure (which drives the circulation) untouched. The clips use JAX's straight-through subgradient: gradient 1 in

the interior, 0 at a boundary; in normal runs the clips are not hit, so the differentiability of the step is preserved in practice.
↪ *Code:* `chronos_esm/ocean/veros_driver.py` — Shapiro filter, salinity sponges, clips, global-mean renorm

6.5 The conjugate-gradient elliptic solver

The barotropic streamfunction (6.2) (and, in Chapter 7, the variable-coefficient vorticity inversion) requires solving a large sparse linear system $A\mathbf{x} = \mathbf{b}$ every step. Because the solver sits *inside* the differentiated time step, it must itself be differentiable — ruling out `scipy.sparse.linalg.cg`. The model uses a hand-written, JAX-native preconditioned conjugate gradient (CG).

For a symmetric positive-definite A, CG generates iterates that minimise the A-norm of the error over an expanding Krylov subspace. With Jacobi (diagonal) preconditioner $M \approx \mathrm{diag}(A)^{-1}$ the recurrence is

$$\alpha_k = \frac{\mathbf{r}_k^\top \mathbf{z}_k}{\mathbf{p}_k^\top A \mathbf{p}_k}, \qquad \mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k, \qquad \mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k A \mathbf{p}_k,$$
$$\mathbf{z}_{k+1} = M \mathbf{r}_{k+1}, \qquad \beta_k = \frac{\mathbf{r}_{k+1}^\top \mathbf{z}_{k+1}}{\mathbf{r}_k^\top \mathbf{z}_k}, \qquad \mathbf{p}_{k+1} = \mathbf{z}_{k+1} + \beta_k \mathbf{p}_k. \tag{6.16}$$

The implementation has two design choices worth highlighting:

- **Fixed-length `scan`, not `while_loop`.** JAX's `while_loop` is not reverse-mode differentiable, so the loop runs a fixed `max_iter` steps via `jax.lax.scan`, and once the residual drops below $\mathrm{tol} \cdot \|\mathbf{b}\|$ each further iteration is a no-op (the update is masked back to the converged state with `jnp.where`). This keeps the solver both differentiable and statically shaped.
- **Matrix-free linear operator.** A is never assembled; it is a function $\mathbf{x} \mapsto A\mathbf{x}$. For the 2-D Poisson solve the operator is the 5-point Laplacian with periodic boundaries in x (`jnp.roll`) and Dirichlet $\psi = 0$ in y and on land. Land is handled by an *identity-row* trick: on a land cell the operator returns $\mathbf{x}$ itself, so $b = 0$ there forces $\psi = 0$ without making the matrix singular. The sign is flipped to $-\nabla^2$ so A is positive definite, and the Jacobi diagonal is the constant $2/\Delta x^2 + 2/\Delta y^2$.

A short excerpt shows the differentiable masked update at the heart of the loop:

```
def scan_body(state, _):
    x, r, p, rz_old, r_norm, it = state
    converged = r_norm <= tol_abs
    new_state = body_fun(state)          # one CG step (Eq. for CG)
    out = jax.tree.map(                   # freeze once converged -> no-op
        lambda old, new: jnp.where(converged, old, new), state, new_state)
    return out, None
```

↪ *Code:* `chronos_esm/ocean/solver.py` — `solve_cg`, `solve_poisson_2d`, `jacobi_preconditioner`

6.5.1 Variable-coefficient (JEBAR-ready) operators

The same CG kernel backs two further elliptic solvers built for the prognostic core of Chapter 7: a Cartesian $\boldsymbol{\nabla}\cdot(c\,\boldsymbol{\nabla}\psi) = \mathrm{rhs}$ and its spherical counterpart

$$\frac{1}{a^2 \cos\phi}\left[\frac{\partial}{\partial\lambda}\left(\frac{c}{\cos\phi}\frac{\partial\psi}{\partial\lambda}\right) + \frac{\partial}{\partial\phi}\left(c\cos\phi\,\frac{\partial\psi}{\partial\phi}\right)\right] = \mathrm{rhs}. \tag{6.17}$$

With $c = H^{-1}$ over real bathymetry this is precisely the topographic / JEBAR operator a barotropic-vorticity ocean needs; with $c = \text{const}$ it reduces to the Poisson solve above. The spherical version is discretised in symmetric *face-conductance* form (multiplying each cell equation by its area $a^2 \cos\phi\,\Delta\lambda\,\Delta\phi$) so the matrix stays SPD even as the cell area varies with latitude, and a polar floor $\cos\phi \geq 0.087$ avoids the $1/\cos\phi$ blow-up at the lat–lon pole singularity. Face coefficients are the arithmetic mean of adjacent cells and are *not* masked, so ocean cells couple to land cells pinned to $\psi = 0$ — the correct Dirichlet streamfunction boundary (masking the faces would impose Neumann and make the operator singular). Both solvers are differentiable in c and in the right-hand side.
↪ *Code:* `chronos_esm/ocean/solver.py` — `solve_elliptic_varcoef`, `solve_elliptic_varcoef_sphere`

6.6 Honest summary of the numerics

Tracers are the only prognostic ocean variables; they are advanced by RK4 on an upwind-advection / Laplacian-diffusion operator that is carefully made divergence-consistent and mass-conserving. The velocity that does the advecting is *diagnosed* each step from density (damped thermal wind) and wind stress (flat-bottom Stommel streamfunction), then patched with a per-latitude net-transport corrector and an explicit thermohaline overturning closure to give density a working pathway to the AMOC. The diagnostic-velocity design is differentiable and stable, but it is fundamentally limited: it has no momentum memory, no topographic vorticity torque, and no self-consistent overturning. Removing these limitations is the job of the prognostic-momentum core in Chapter 7, which reuses the variable-coefficient CG solver of §6.5.1.

Exercise 6.1 (Net-transport corrector)**.** Disable the per-latitude corrector (6.8) (set the subtracted term to zero in a copy of `step_ocean`) and run a short spin-up. Diagnose the zonally-/vertically-integrated meridional transport as a function of latitude and confirm it reaches order $\pm 10^2$ Sv. Re-enable the corrector and verify it drops to ≈ 0 Sv. Why does a closed basin require zero net transport at every latitude?

Exercise 6.2 (Rigid-lid projection and tracer conservation)**.** Remove the depth-mean divergence subtraction in (6.11) so that w_{if} is integrated up from the floor without projection. Run a multi-decade tracer-only experiment and measure the drift in global-mean θ. Compare against the documented $\sim +0.25$ K yr^{-1} leak. Explain why the advective form of vertical advection (rather than flux form) makes this projection necessary.

Exercise 6.3 (CG convergence and warm starts)**.** Instrument `solve_poisson_2d` to record the residual norm versus iteration for the barotropic solve. Compare convergence when starting from $\psi = 0$ versus warm starting from the previous step's ψ (as `step_ocean` does). How many iterations does the warm start save once the model is near equilibrium, and why?

Exercise 6.4 (Density sensitivity of the AMOC)**.** Using JAX's `jax.grad`, compute ∂(AMOC at 26.5°N)/∂(subpolar salinity
for the diagnostic thermal wind alone ($k_{\text{vel}} = 0$ in (6.9)) and again with the thermohaline closure active. Confirm that the first is ≈ 0 and the second is nonzero and sign-correct. What does this tell you about which term actually couples density to the overturning in this model?

Chapter 7

The Ocean: Prognostic-Momentum Core

The ocean velocities used by the live, coupled Chronos-ESM are *diagnostic*: the horizontal flow is read off the density field through an algebraic thermal-wind inversion, and the overturning is closed with an interim Stommel/box parameterization (Chapter 8). That design is numerically robust but it breaks a property we want for tipping-point science: in the diagnostic core the sensitivity of the Atlantic overturning to a density perturbation, $\partial(\mathrm{AMOC})/\partial\rho$, is controlled by the closure rather than by momentum dynamics, so a bifurcation cannot be *earned* by the physics. This chapter documents the prognostic-momentum ocean core that is being built to replace that closure. It is a set of standalone, fully differentiable building blocks — a variable-coefficient elliptic solver, a prognostic barotropic vorticity equation, and a prognostic baroclinic momentum equation — each validated against an analytic balance. **None of it is yet wired into `veros_driver/main`;** it lives apart precisely so that it can be validated with zero regression risk to the running model. The standard reference for every balance below is Vallis (2017).

7.1 The variable-coefficient elliptic invert

A streamfunction ψ is the natural prognostic object for a rotating, nearly-nondivergent ocean: the horizontal transport is $\mathbf{u} = \mathbf{k} \times \boldsymbol{\nabla}\psi$, which is divergence-free by construction. The vorticity of this transport defines ψ through an elliptic equation. For a flat-bottom ocean this is Poisson's equation $\nabla^2\psi = \zeta$; over real bathymetry it becomes a *variable-coefficient* elliptic problem,

$$\boldsymbol{\nabla}\cdot\big(\, c\,\boldsymbol{\nabla}\psi \,\big) = \zeta, \qquad c = \frac{1}{H}. \tag{7.1}$$

The coefficient $c = 1/H$ encodes topography: where the column is shallow, a given transport streamfunction implies a larger vorticity, and the steering of flow along f/H contours is exactly the joint-effect-of-baroclinicity-and-relief (JEBAR) mechanism that makes deep western boundary currents follow the continental slope (Vallis, 2017). Setting $c = 1$ recovers the flat-bottom Poisson limit. The same operator with $c = \text{const}$ also serves as a Laplacian for a viscous $\nu\nabla^2\zeta$ term.

Discrete operator (Cartesian)

The code discretizes Eq. (7.1) in flux form on a regular grid that is periodic in x and Dirichlet ($\psi = 0$) in y and on land. The coefficient is averaged to cell faces (east/west/north/south) as the arithmetic mean of the two adjacent cells,

$$c_{\mathrm{E}} = \tfrac{1}{2}\big(c_{i,j} + c_{i,j+1}\big), \quad c_{\mathrm{W}} = \tfrac{1}{2}\big(c_{i,j} + c_{i,j-1}\big), \quad (\text{and likewise } c_{\mathrm{N}}, c_{\mathrm{S}}), \tag{7.2}$$

and the operator is the net face-flux divergence,

$$(L\psi)_{i,j} = \frac{c_{\mathrm{E}}(\psi_{i,j+1} - \psi_{i,j}) - c_{\mathrm{W}}(\psi_{i,j} - \psi_{i,j-1})}{\Delta x^2} + \frac{c_{\mathrm{N}}(\psi_{i+1,j} - \psi_{i,j}) - c_{\mathrm{S}}(\psi_{i,j} - \psi_{i-1,j})}{\Delta y^2}. \tag{7.3}$$

↪ *Code:* `chronos_esm/ocean/solver.py` — `_varcoef_faces`, `_elliptic_op`, `solve_elliptic_varcoef`

Two design choices deserve emphasis because they are easy to get wrong. First, the faces are *not* masked: an ocean cell adjacent to land still couples to the land cell, which is pinned to $\psi = 0$ by an identity row in the matrix. This realizes the correct Dirichlet (no-streamfunction) coastal condition. Masking the faces instead would impose a Neumann (no-flux) condition, which leaves the operator singular up to an additive constant and makes the iterative solver diverge — the source comments call this out explicitly. Second, $c = 1/H$ must be regularized to a finite value on land before the solve, since $H \to 0$ there.

The system is made symmetric positive-definite by solving $-L\psi = -\zeta$ on ocean cells and the identity $\psi = 0$ on land, then inverted with a Jacobi-preconditioned conjugate-gradient iteration whose diagonal is $(c_\mathrm{E} + c_\mathrm{W})/\Delta x^2 + (c_\mathrm{N} + c_\mathrm{S})/\Delta y^2$. The CG loop is written with `jax.lax.scan` (not `while_loop`) so that it is differentiable in reverse mode — the solver is exact in ψ and exposes a clean gradient $\partial\psi/\partial c$, $\partial\psi/\partial\zeta$ (Chapter 3).

↪ *Code:* `chronos_esm/ocean/solver.py` — `solve_cg`, `jacobi_preconditioner`

Spherical face-conductance form

On the lat–lon grid the elliptic operator carries the spherical metric,

$$\frac{1}{a^2\cos\phi}\left[\frac{\partial}{\partial\lambda}\left(\frac{c}{\cos\phi}\frac{\partial\psi}{\partial\lambda}\right) + \frac{\partial}{\partial\phi}\left(c\cos\phi\,\frac{\partial\psi}{\partial\phi}\right)\right] = \zeta. \tag{7.4}$$

Discretizing the area-divided form of Eq. (7.4) directly would yield a *non-symmetric* matrix, because the cell area $a^2\cos\phi\,\Delta\lambda\,\Delta\phi$ varies with latitude. The code therefore multiplies through by the cell area and works in symmetric *face-conductance* form. A conductance is (face coefficient)×(face length)/(centre-to-centre distance); the Earth radius cancels, leaving

$$C_\mathrm{E} = \bar{c}_\mathrm{E}\,\frac{\Delta\phi}{\cos\phi\,\Delta\lambda}, \qquad C_\mathrm{N} = \bar{c}_\mathrm{N}\,\frac{\cos\phi_\mathrm{N}\,\Delta\lambda}{\Delta\phi}, \tag{7.5}$$

with $\bar{c}$ the face-averaged coefficient and ϕ_N the north-face latitude (symmetrically for W, S). The symmetric operator and right-hand side are

$$(M\psi)_{i,j} = \sum_{\text{faces}} C_\text{face}\,(\psi_{i,j} - \psi_\text{nbr}), \qquad b = -\zeta\,\left(a^2\cos\phi\,\Delta\lambda\,\Delta\phi\right), \tag{7.6}$$

which is exactly $-\,\text{area}\times\boldsymbol{\nabla}\cdot(c\boldsymbol{\nabla}\psi)$ and is SPD, so the same CG applies.

↪ *Code:* `chronos_esm/ocean/solver.py` — `_sphere_conductances`, `_sphere_op`, `solve_elliptic_varcoef_sphere`

Remark. The factor $1/\cos\phi$ in the zonal conductance blows up at the pole, the same lat–lon singularity that destabilized the single-level atmosphere (Chapter 10). The fix is identical in spirit: $\cos\phi$ is floored at `cos_min`$= 0.087$ ($\approx \phi = 85°$) so the conductance stays finite. This is a deliberate, quantified approximation in the polar caps, not a bug.

A manufactured-solution test confirms the round trip $\psi \to \zeta \to \psi$ to relative error $< 2\times10^{-3}$ (Cartesian, variable c) and $< 10^{-4}$ (spherical), and that the constant-c solver reproduces the plain Poisson solve.

↪ *Code:* `tests/test_barotropic_gyre.py` — manufactured-solution and Poisson-consistency tests

7.2 The prognostic barotropic vorticity equation

With the invert in place, the depth-integrated (barotropic) circulation is advanced by a *prognostic* relative-vorticity equation rather than diagnosed. In the linear, quasi-geostrophic form the model integrates,

$$\frac{\partial\zeta}{\partial t} = -\beta\,\frac{\partial\psi}{\partial x} - r\,\zeta + F + \nu\nabla^2\zeta, \qquad u = -\frac{\partial\psi}{\partial y},\quad v = \frac{\partial\psi}{\partial x}, \tag{7.7}$$

where $\beta = \mathrm{d}f/\mathrm{d}y$ is the planetary-vorticity gradient, r a linear bottom drag, ν a lateral viscosity, and the forcing is the wind-stress curl

$$F = \frac{1}{\rho_0\,H}\left(\boldsymbol{\nabla}\times\boldsymbol{\tau}\right)_z = \frac{1}{\rho_0\,H}\left(\frac{\partial\tau^y}{\partial x} - \frac{\partial\tau^x}{\partial y}\right). \tag{7.8}$$

The term $-\beta\,\partial\psi/\partial x = -\beta v$ is the advection of planetary vorticity by the meridional transport — the source of the β-effect and of western intensification (Vallis, 2017). Each step is forward Euler in ζ, after which ψ is re-diagnosed from $\zeta = \boldsymbol{\nabla}\cdot(c\boldsymbol{\nabla}\psi)$ using the elliptic invert of the previous section, warm-started from the previous ψ:

$$\zeta^{n+1} = \zeta^{n} + \Delta t\big(-\beta\,\partial_x\psi^{n} - r\zeta^{n} + F + \nu\nabla^2\zeta^{n}\big), \qquad \psi^{n+1} = L_c^{-1}\,\zeta^{n+1}. \tag{7.9}$$

↪ *Code:* `chronos_esm/ocean/barotropic.py` — `step_barotropic`, `velocities_from_psi`, `wind_stress_curl`

On the sphere the planetary-vorticity term simplifies elegantly. Since $\beta = 2\Omega\cos\phi/a$ and $v = (a\cos\phi)^{-1}\partial\psi/\partial\lambda$, the $\cos\phi$ factors cancel:

$$\beta\,v = \frac{2\Omega\cos\phi}{a}\cdot\frac{1}{a\cos\phi}\frac{\partial\psi}{\partial\lambda} = \frac{2\Omega}{a^2}\frac{\partial\psi}{\partial\lambda}, \tag{7.10}$$

so the spherical step advances $\partial_t\zeta = -(2\Omega/a^2)\,\partial_\lambda\psi - r\zeta + F$ with the spherical curl and metric velocities $u = -a^{-1}\partial_\phi\psi$, $v = (a\cos\phi)^{-1}\partial_\lambda\psi$.

↪ *Code:* `chronos_esm/ocean/barotropic.py` — `step_barotropic_sphere`, `velocities_sphere`, `wind_stress_curl_sphere`

Analytic validation: the Stommel gyre

At steady state ($\partial_t\zeta = 0$, $\nu = 0$, $c = 1$) Eq. (8.10) reduces to the Stommel balance (Stommel, 1961),

$$r\nabla^2\psi + \beta\,\frac{\partial\psi}{\partial x} = F. \tag{7.11}$$

Away from the western wall the friction term is negligible and the interior obeys the Sverdrup relation $\beta\,\partial_x\psi = F$, i.e. $v = F/\beta$ (Sverdrup, 1947). The return flow is collected into a thin western boundary layer of width

$$\delta_S = \frac{r}{\beta}, \tag{7.12}$$

which is what produces western intensification (a strong, narrow Gulf-Stream-like jet on the west, weak broad return flow in the interior). The validation spins the gyre up from rest under a single-gyre wind-stress curl and checks both signatures: the meridional jet $|v| = |\partial_x\psi|$ peaks in the western third of the basin, and the interior $\partial_x\psi$ matches F/β to better than 35% (Cartesian) and 45% (spherical). With $r = 5\times10^{-6}\,\mathrm{s^{-1}}$ and $\beta \approx 2\times10^{-11}\,\mathrm{m^{-1}s^{-1}}$ this gives $\delta_S \approx 250\,\mathrm{km}$, a few grid cells — just resolved.

↪ *Code:* `tests/test_barotropic_gyre.py` — `test_stommel_gyre_sverdrup_and_western_intensification`, spherical variant

Example 7.1 (Sverdrup transport from the wind)**.** For the subtropical North Atlantic, a curl of order $(\boldsymbol{\nabla}\times\boldsymbol{\tau})_z \sim -2\times10^{-7}\,\mathrm{N\,m^{-3}}$ over $\beta \approx 2\times10^{-11}\,\mathrm{m^{-1}s^{-1}}$ gives an interior southward transport per unit width $vH = (\boldsymbol{\nabla}\times\boldsymbol{\tau})_z/(\rho_0\beta) \sim -10\,\mathrm{m^2\,s^{-1}}$. Integrated across an ocean width of $\sim 6000\,\mathrm{km}$ this is $\mathcal{O}(30)\,\mathrm{Sv}$, the right order for the wind-driven gyre — and the same 30 Sv must return in the western boundary current. The model reproduces this partition; verify it yourself in Exercise 7.2.

7.3 The prognostic baroclinic momentum equation

The barotropic core fixes the gyre circulation but not the overturning, which is set by *density gradients* acting through the vertical structure of the flow. This is the role of the baroclinic momentum core. The continuous equation, in the hydrostatic, linear-drag form the code integrates, is

$$\frac{\partial\mathbf{u}}{\partial t} + f\,\mathbf{k}\times\mathbf{u} = -\frac{1}{\rho_0}\boldsymbol{\nabla}p - r\,\mathbf{u} + \nu\nabla^2\mathbf{u}, \qquad \frac{\partial p}{\partial z} = -\rho\,g, \tag{7.13}$$

with p the hydrostatic pressure. The Coriolis term $f\,\mathbf{k}\times\mathbf{u}$ is the load-bearing balance: away from boundaries and friction, Eq. (7.13) at steady state is geostrophy, $f\,\mathbf{k}\times\mathbf{u} = -\rho_0^{-1}\boldsymbol{\nabla}p$, whose vertical derivative is *thermal wind*,

$$f\,\frac{\partial v}{\partial z} = \frac{g}{\rho_0}\frac{\partial\rho}{\partial x}, \qquad f\,\frac{\partial u}{\partial z} = -\frac{g}{\rho_0}\frac{\partial\rho}{\partial y}. \tag{7.14}$$

Equation (7.14) is precisely how density drives the flow: a horizontal density gradient is balanced by a vertical shear of the horizontal velocity. The interim diagnostic core imposes Eq. (7.14) algebraically; the prognostic core lets it *emerge* as the steady limit of Eq. (7.13), so the same density field also drives the ageostrophic, overturning component through the friction and time-tendency terms.

Discrete pressure-gradient force

The hydrostatic pressure is the running vertical integral of density from the surface down,

$$p(z) = g\int_z^0 \rho \, \mathrm{d}z' \;\longrightarrow\; p_k = g\sum_{k'\le k} \rho_{k'}\,\Delta z_{k'} \quad \text{(a } \texttt{cumsum} \text{ from the surface)}, \tag{7.15}$$

defined up to an irrelevant surface constant. The horizontal pressure-gradient acceleration is $-\rho_0^{-1}\boldsymbol{\nabla} p$, with centred differences (periodic in x, one-sided at the y-edges).
↪ *Code:* `chronos_esm/ocean/momentum.py` — `hydrostatic_pressure`, `pressure_gradient_accel`

Semi-implicit Coriolis

The one numerical subtlety is the Coriolis term. An *explicit* treatment of $f\,\mathbf{k}\times\mathbf{u}$ is unconditionally unstable — it rotates the velocity vector by a finite angle each step and the amplitude grows without bound for any Δt. The code therefore treats Coriolis *implicitly*, which (with all other forcing collected into F_u, F_v) is a 2×2 rotation solve:

$$\frac{u'-u}{\Delta t} = f\,v' + F_u, \quad \frac{v'-v}{\Delta t} = -f\,u' + F_v \;\Longrightarrow\; \begin{cases} u' = \dfrac{r_u + \Delta t\, f\, r_v}{1+(\Delta t\, f)^2}, \\[2ex] v' = \dfrac{r_v - \Delta t\, f\, r_u}{1+(\Delta t\, f)^2}, \end{cases} \tag{7.16}$$

where $r_u = u + \Delta t\, F_u$, $r_v = v + \Delta t\, F_v$. The denominator $1+(\Delta t\, f)^2$ is the key: it is ≥ 1 for every Δt, so the update is a contraction and is stable for arbitrarily large $\Delta t\, f$. The full step assembles $F_u = a_x - ru + \nu\nabla^2 u$ from the pressure-gradient acceleration a_x, linear drag, and optional viscosity, then applies Eq. (7.16).
↪ *Code:* `chronos_esm/ocean/momentum.py` — `coriolis_semi_implicit`, `step_momentum`

```
ru = u + dt * Fu;   rv = v + dt * Fv
d  = 1.0 + (dt * f)**2
u_new = (ru + dt*f*rv) / d
v_new = (rv - dt*f*ru) / d
```

Listing 7.1: Semi-implicit Coriolis: the $1+(\Delta t f)^2$ denominator makes it stable for any timestep.

Analytic validation: thermal wind

The momentum core is spun up from rest under a fixed, horizontally-varying density field and the emergent vertical shear is compared with the thermal-wind prediction of Eq. (7.14). In the interior the model shear matches analytic thermal wind to relative error $< 12\%$, and a separate test confirms the semi-implicit Coriolis stays bounded ($|u| < 5\,\mathrm{m\,s^{-1}}$) at a timestep that would explode under explicit Coriolis.
↪ *Code:* `tests/test_momentum.py` — `test_thermal_wind_balance`, `test_coriolis_semi_implicit_stable`

Proposition 7.1 (Density drives a clean overturning)**.** *Couple the baroclinic shear of Eq. (7.14) to mass continuity. A meridional density (hence buoyancy) contrast sets, via thermal wind, a sheared meridional flow; continuity closes it in the vertical, producing an overturning cell whose strength scales monotonically with the density contrast. Because the velocities here are* prognostic *solutions of Eq. (7.13), the map (density)→(overturning) is a genuine, differentiable function of the dynamics, so* $\partial(\mathrm{AMOC})/\partial\rho \neq 0$ *for the right physical reason — the precondition for a clean, rather than closure-imposed, AMOC bifurcation (Chapter 17). Contrast this with Marotzke (1991)-style box models, where the overturning–density relation is* postulated.

7.4 Status: validated standalone, not yet wired in

To be unambiguous: the three components above (`solver.py`, `barotropic.py`, `momentum.py`) are validated against the Stommel/Sverdrup gyre balance, manufactured elliptic solutions, and thermal-wind balance, and every piece is differentiable end-to-end (the gyre amplitude has a finite, nonzero gradient with

respect to the wind-stress amplitude; the momentum solution with respect to the density forcing). They are **not** called by the live ocean driver `veros_driver` or by `main.step_coupled`. The coupled model still uses the diagnostic thermal-wind velocities plus the interim box overturning closure of Chapter 8. Wiring the prognostic core into the live model — replacing the algebraic inversion with the time-integrated barotropic+baroclinic split, on the real bathymetric $c = 1/H$ over the spherical grid — is the next major ocean milestone. Until then this chapter documents a tested, trustworthy laboratory, not a production path.

The differentiability is not incidental. Because both inverts and both time-steppers are written in pure JAX with `lax.scan` loops, the entire (density→velocity→transport) chain admits reverse-mode gradients, which is what will let later chapters fit the core to observations and probe the tipping manifold (Chapters 15, 17). For the GM/Redi eddy parameterization that the prognostic core will ultimately interact with, see Chapter 5 and Gent and McWilliams (1990); Griffies (2004).

Exercises

Exercise 7.1 (Western boundary-layer width)**.** Using `spin_up_gyre` from
↪ *Code:* `chronos_esm/ocean/barotropic.py` — barotropic gyre
, spin up the Stommel gyre for three values of the bottom drag, $r \in \{2.5, 5, 10\} \times 10^{-6}\,\mathrm{s}^{-1}$, holding β fixed. Locate the column of peak $|v| = |\partial_x \psi|$ and estimate the boundary-layer width in each case. Verify that it scales as $\delta_S = r/\beta$ (Eq. 7.12). At what r does δ_S shrink below one grid cell, and what happens to the solution then?

Exercise 7.2 (Sverdrup balance in the interior)**.** For the spun-up gyre, plot $\partial_x \psi$ along a mid-basin latitude and overlay F/β. Confirm the Sverdrup balance holds in the interior and fails inside δ_S of the west wall. By how much does the basin-integrated interior (Sverdrup) transport differ from the western boundary-current return transport? They should be equal and opposite; explain any residual in terms of the friction and discretization.

Exercise 7.3 (Thermal wind from a front)**.** With `spin_up` from
↪ *Code:* `chronos_esm/ocean/momentum.py` — baroclinic momentum
, impose a density field with a single meridional front (e.g. a tanh in y, uniform in x) and integrate to steady state. Diagnose the vertical shear $\partial u/\partial z$ and compare it with $-(g/\rho_0 f)\,\partial\rho/\partial y$ from Eq. (7.14). Then double the front strength and confirm the shear — and the implied overturning — doubles. Why is this the mechanism a box model cannot capture from first principles (Marotzke, 1991)?

Exercise 7.4 (Why explicit Coriolis explodes)**.** Disable the semi-implicit solve and step u, v with an *explicit* Coriolis term, $u' = u + \Delta t(fv + F_u)$, $v' = v + \Delta t(-fu + F_v)$. Show analytically that the amplification factor of the unforced rotation is $\sqrt{1 + (\Delta t f)^2} > 1$, so the kinetic energy grows every step regardless of Δt. Then show the semi-implicit update of Eq. (7.16) has amplification exactly 1 for the unforced case. Confirm numerically with `test_coriolis_semi_implicit_stable` as a starting point.

Chapter 8

The Ocean: Overturning and the Thermohaline Circulation

The Atlantic Meridional Overturning Circulation (AMOC) is the global ocean's single most consequential dynamical mode: a basin-scale conveyor that carries warm, salty surface water poleward, sinks it in the subpolar North Atlantic to form North Atlantic Deep Water (NADW), and returns it equatorward at depth. Its existence and strength depend on a competition between thermal and haline buoyancy forcing, and Stommel (1961) showed in a famous two-box model that this competition admits *multiple stable equilibria* – a strong, salt-driven "on" state and a weak, fresh-capped "off" state. Marotzke (1991) sharpened this into the salt-advection feedback that underlies the modern picture of AMOC tipping. This chapter documents how Chronos-ESM represents that physics, and the chapter is deliberately honest about what the representation is: an *interim, mechanistic closure*, not a prognostic-momentum overturning.

8.1 Why a closure is needed

Chronos-ESM's ocean (Chapters 4–6) diagnoses horizontal velocities from the density field via a thermal-wind / geostrophic balance. That balance is *local in latitude*: it relates the vertical shear of $\mathbf{u}$ to the local horizontal density gradient, but it does not constrain the basin-integrated, depth-overturning streamfunction that reaches 26.5°N. The practical symptom is that the sensitivity of the diagnosed AMOC to a subpolar buoyancy perturbation is essentially zero,

$$\frac{\partial \Psi_{\mathrm{AMOC}}(26.5^{\circ}\mathrm{N})}{\partial S_{\mathrm{subpolar}}} \approx 0, \tag{8.1}$$

so the diagnostic ocean carries no coherent overturning response to forcing and cannot host an AMOC bifurcation. The prognostic-momentum core that would close this honestly (a variable-coefficient JEBAR barotropic-vorticity solver plus a $\partial\mathbf{u}/\partial t$ equation) is the subject of Chapter 7 and is future work. In the interim we install the cheapest *differentiable* change that makes Eq. (8.1) nonzero and sign-correct, so that the tipping experiments of Chapter 17 can be run at all.

Remark. The closure prescribes the vertical *shape* of the overturning cell (as a box model does) but makes its *strength* and, crucially, its *sensitivity* to density physical. It is set to zero by $k_{\mathrm{vel}} = 0$, which recovers the pure diagnostic ocean.

↪ *Code:* `chronos_esm/ocean/overturning.py` — module docstring: rationale and scope

8.2 A density-driven, depth-integral-zero overturning velocity

The closure adds a meridional overturning velocity $v_{\mathrm{thc}}(z, \phi, \lambda)$ confined to the Atlantic basin. Two design constraints fix its form.

(i) It must carry no net meridional transport. The model's barotropic corrector (Chapter 7) removes any depth-uniform meridional flow per latitude, so a closure velocity with a nonzero depth

integral would simply be annihilated. We therefore require

$$\int_{-H}^{0} v_{\mathrm{thc}}\,\mathrm{d}z = 0 \qquad \Longleftrightarrow \qquad \sum_k v_{\mathrm{thc}}^{(k)}\,\Delta z_k = 0, \tag{8.2}$$

which guarantees v_{thc} drives only *overturning* (and the associated vertical velocity w and tracer transport), never net throughflow.

(ii) Its strength must scale with the subpolar buoyancy excess. Deep-water formation is gated by how much denser the subpolar convection region is than the subtropics. Chronos-ESM reads an upper-ocean density contrast

$$\Delta\rho \;=\; \langle\rho\rangle_{\mathrm{subpolar}} - \langle\rho\rangle_{\mathrm{subtropical}}, \tag{8.3}$$

where $\langle\cdot\rangle$ denotes an area- and depth-weighted mean over Atlantic ocean cells in a latitude band (45–65°N for subpolar, 10–30°N for subtropical) and over the convection layer $z \geq -h_{\mathrm{c}}$. A salinity contrast ΔS is read the same way.

8.2.1 Vertical shape

Let the depth-mid coordinates be z_k and define the upper-limb indicator $\mathbb{1}_{\mathrm{up}}(z) = 1$ for $z \geq -h_{\mathrm{up}}$ and 0 below, with h_{up} the limb thickness (`upper_depth_m`, default 1100 m). Writing $d_{\mathrm{up}} = \sum_k \mathbb{1}_{\mathrm{up}}(z_k)\,\Delta z_k$ and $d_{\mathrm{dn}} = H - d_{\mathrm{up}}$, the dimensionless shape

$$G(z) \;=\; \mathbb{1}_{\mathrm{up}}(z) \;-\; \big(1 - \mathbb{1}_{\mathrm{up}}(z)\big)\,\frac{d_{\mathrm{up}}}{d_{\mathrm{dn}}} \tag{8.4}$$

is $+1$ in the upper limb and negative below, and is constructed so that $\sum_k G(z_k)\,\Delta z_k = 0$ exactly, satisfying Eq. (8.2).

↪ *Code:* `chronos_esm/ocean/overturning.py` — `thc_overturning_velocity`: `G = upper - (1-upper)*(up_d/dn_d)`

8.2.2 Strength, contrast and the haline amplification

The overturning amplitude is a smooth, one-sided (≥ 0) function of a haline-amplified contrast,

$$\mathrm{contrast} = \Delta\rho + (\gamma_{\mathrm{S}} - 1)\,\beta_{\mathrm{S}}\,\Delta S, \tag{8.5}$$

$$A = k_{\mathrm{vel}}\,\mathrm{softplus}\!\left(\frac{\mathrm{contrast}}{\Delta\rho_{\mathrm{scale}}}\right), \qquad \mathrm{softplus}(x) = \ln(1 + e^{x}), \tag{8.6}$$

with $\beta_{\mathrm{S}} \approx 0.78\ \mathrm{kg\,m^{-3}\,psu^{-1}}$ the haline contraction coefficient $\partial\rho/\partial S$ of the equation of state near 35 psu, $\Delta\rho_{\mathrm{scale}}$ the contrast scale (`drho_scale`, default $0.1\ \mathrm{kg\,m^{-3}}$), and k_{vel} the velocity gain. The full overturning velocity is then

$$v_{\mathrm{thc}}(z,\phi,\lambda) \;=\; A\,G(z)\,\mathcal{M}_{\mathrm{Atl}}(\phi,\lambda)\,\mathcal{M}_{\mathrm{ocn}}, \tag{8.7}$$

where $\mathcal{M}_{\mathrm{Atl}}$ is the Atlantic basin mask (280–360°E ∪ 0–20°E, 34°S–80°N) and $\mathcal{M}_{\mathrm{ocn}}$ the wet-cell mask.

↪ *Code:* `chronos_esm/ocean/overturning.py` — `amp = k_vel*jax.nn.softplus(contrast/drho_scale)`; `v_thc = amp*G*atl*maskC`

The softplus rectifier replaces a hard $\max(0,\cdot)$: it is everywhere differentiable, so $\partial A/\partial(\text{state})$ exists for the gradient-based calibration and bifurcation diagnostics that motivate the whole model, yet it saturates to a linear ramp for large positive contrast (denser subpolar water ⇒ stronger AMOC) and decays smoothly toward zero for negative contrast (a fresh/warm subpolar cap ⇒ collapsed cell).

Example 8.1 (Reading the knobs)**.** With $\gamma_{\mathrm{S}} = 1$ the haline term in Eq. (8.5) vanishes and the strength depends on the *total* density contrast only – the conservative default. Setting $k_{\mathrm{vel}} = 0$ removes the closure entirely. Raising k_{vel} scales the cell strength linearly in the linear regime of softplus; $\Delta\rho_{\mathrm{scale}}$ sets how sharply strength turns on with contrast.

8.2.3 Two decoupled depth scales

A subtlety that matters for tipping: the depth over which the cell *flows* (h_{up}, the limb thickness in Eq. (8.4)) is decoupled from the depth over which the gating *contrast* is read (h_c = `contrast_depth_m`, default equal to h_{up}). Physically, deep-water formation responds to the buoyancy of the *upper few hundred metres* of the convection region, so a shallow $h_c \sim 300$ m is the correct choice – and it is also the numerically consequential one. A surface-trapped freshwater anomaly averaged over 0–1100 m is diluted roughly 20× and barely equilibrates within a decadal hold; averaged over 0–300 m it is diluted only $\sim 6\times$ and flushes in a few years. The shallow contrast layer thus gives a surface hosing far more leverage on the contrast, which is exactly the control variable in the bifurcation study.
↪ *Code:* `chronos_esm/ocean/overturning.py` — `upper_depth_m` (shape) vs `contrast_depth_m` (gating)

8.3 The salt-advection feedback (γ_S)

The closure is not merely diagnostic: because v_{thc} advects the prognostic salinity field, it carries salt poleward in its upper limb. A stronger AMOC therefore makes the subpolar Atlantic saltier and denser, which by Eqs. (8.5)–(8.6) strengthens the AMOC further. This is precisely the Stommel (1961)/Marotzke (1991) salt-advection feedback, realised here through the model's own tracer transport rather than imposed.

The loop gain is tunable through γ_S (`haline_gain`). When $\gamma_S > 1$ the contrast in Eq. (8.5) weights the subpolar salinity excess *above* its raw density contribution, $\beta_S \Delta S$, by the extra factor $(\gamma_S - 1)\beta_S \Delta S$. This amplifies the overturning's sensitivity to the salinity contrast specifically, raising the feedback-loop gain above unity and turning the monostable diagnostic AMOC into a *bistable, tip-capable* one. The bifurcation that results – the hysteresis loop swept by a slowly varied freshwater hosing – is the object of study in Chapter 17.

Remark. Because the feedback runs through the model's prognostic salinity, the sign of $\partial \Psi_{AMOC}/\partial S_{subpolar}$ is now positive (saltier subpolar ⇒ stronger AMOC), repairing Eq. (8.1). γ_S controls only *how strong* that feedback is, i.e. whether the system is monostable or bistable.

8.4 Subpolar freshwater hosing

The standard AMOC-bifurcation forcing is a freshwater "hosing" of the subpolar North Atlantic surface. Chronos-ESM implements it as a virtual surface-salinity tendency. Spreading a freshwater volume flux of $Q_{fw} = H_{Sv} \times 10^6\,\mathrm{m^3\,s^{-1}}$ (H_{Sv} in Sverdrups) uniformly over the subpolar Atlantic surface region $\mathcal{R}$ (band 45–65°N, area $\mathcal{A} = \sum_{\mathcal{R}} \Delta x\, \Delta y$, surface-layer thickness Δz_0) dilutes a reference salinity $S_{ref} = 35$ psu, giving the virtual salt-removal rate

$$\left.\frac{\partial S}{\partial t}\right|_{\text{hose}} = -\frac{Q_{fw}\, S_{ref}}{\mathcal{A}\, \Delta z_0} \quad \text{on } \mathcal{R}, \qquad 0 \text{ elsewhere.} \tag{8.8}$$

↪ *Code:* `chronos_esm/ocean/overturning.py` — `subpolar_hosing_salt_tendency: -(hosing_sv*1e6*s_ref)/(area*dz0)`

Freshening lowers the subpolar density, weakens the contrast $\Delta\rho$ in Eq. (8.3), and so weakens the AMOC through Eq. (8.6); the salt-advection feedback then either damps the perturbation (monostable regime) or amplifies it past a saddle-node (bistable regime). Eq. (8.8) is differentiable in H_{Sv} – so $\partial \Psi_{AMOC}/\partial H_{Sv}$ is available by automatic differentiation – and $H_{Sv} = 0$ is an exact no-op.

8.5 Diagnosing the overturning streamfunction

The overturning is reported as the meridional streamfunction in depth–latitude space. For the global MOC,

$$\Psi(\phi, z) \;=\; \int_{-H}^{z} \int_{\lambda_w}^{\lambda_e} v(\lambda, \phi, z')\; \mathrm{d}x\, \mathrm{d}z', \tag{8.9}$$

discretised as a longitudinal transport sum followed by a vertical cumulative sum, $\Psi_{jk} = \sum_i v_{ijk}\, \Delta x_j$ accumulated over z, reported in Sverdrups ($1\,\mathrm{Sv} = 10^6\,\mathrm{m^3\,s^{-1}}$).
↪ *Code:* `chronos_esm/ocean/diagnostics.py` — `compute_moc`

8.5.1 The Atlantic basin section and the barotropic correction

For the AMOC the integral in Eq. (8.9) is restricted to wet Atlantic cells. But the model's coarse longitude-box "Atlantic" is open at its southern edge and to throughflow, so the raw v contains a large depth-uniform (barotropic) component – the wind-driven gyre / ACC passing *through* the box – which would otherwise swamp the diagnostic at the level of $\sim -200\,\mathrm{Sv}$. Two steps fix this: (i) restrict the section to `atlantic_mask` $\cap$ `ocean_mask`; and (ii) remove the section depth-mean transport at each latitude,

$$\widehat{V}(\phi) = \frac{1}{H}\sum_k \Big(\sum_i v_{ijk}\,\Delta x_j\Big)\Delta z_k, \qquad V'_{jk} = \Big(\sum_i v_{ijk}\Delta x_j\Big) - \widehat{V}(\phi), \tag{8.10}$$

so the corrected section carries zero net meridional transport and the streamfunction closes at the bottom. The depth-integral-zero construction of v_{thc} in Eq. (8.2) is what lets the closure survive this correction intact.

↪ *Code:* `chronos_esm/ocean/diagnostics.py` — `compute_amoc`, `remove_barotropic`

8.5.2 Sign convention

The AMOC streamfunction is accumulated as a *positive* cumulative sum,

$$\Psi_{\mathrm{AMOC}}(\phi, z) = \frac{1}{10^6}\sum_{k' \le k} V'_{jk'}\,\Delta z_{k'}\ [\mathrm{Sv}], \tag{8.11}$$

following the RAPID convention (Smeed et al., 2018): **positive Ψ is the AMOC**, i.e. northward NADW in the upper limb with an equatorward deep return. The upper-cell strength at 26.5°N is then $\max_z \Psi_{\mathrm{AMOC}}$ and the AABW lower cell is $\min_z \Psi_{\mathrm{AMOC}}$.

↪ *Code:* `chronos_esm/ocean/diagnostics.py` — `amoc_sv = cumsum(...)`; `upper_cell_26N = max`, `lower_cell_26N = min`

Remark (A real sign bug, fixed). An earlier version negated Eq. (8.11) ($\Psi = -\sum$). With that sign a physically correct overturning cell is entirely non-positive, so $\max_z \Psi$ scored a genuine $\sim 15\,\mathrm{Sv}$ AMOC as 0 Sv (and an *anti*-AMOC as large positive). Injecting a clean 15 Sv cell confirms it: $\max(-\mathrm{cumsum}) = 0.0$ while $\max(+\mathrm{cumsum}) = +15.0$. This sign error – not only the diagnostic-velocity ocean – is why the AMOC once read ~ 0 / "incoherent." (See `tests/test_amoc_metric.py`.)

8.6 Honest limitations

This closure prescribes the cell's vertical structure and confines it to a fixed longitude box; it is therefore not a substitute for prognostic overturning. The strength's *magnitude* is a tuning of k_{vel} rather than an emergent balance, and the basin geometry is crude at T31. What the closure does buy – and all it claims to buy – is a differentiable, sign-correct, physically motivated dependence of the overturning on the subpolar buoyancy contrast, with a tunable salt-advection feedback. That is exactly the minimal ingredient needed to study AMOC bistability and tipping, which we take up in Chapter 17.

Exercise 8.1 (Turning the feedback on)**.** Using `thc_overturning_velocity`, hold a fixed density/salinity field and sweep `haline_gain` $\in \{1.0, 1.5, 2.0\}$ at fixed k_{vel}. For each, plot the overturning amplitude A (Eq. 8.6) as a function of an imposed subpolar salinity contrast ΔS. Identify which γ_S first produces a loop-gain > 1 (slope of A vs ΔS exceeding the contrast restoring rate) and relate it to the onset of bistability.

Exercise 8.2 (Why depth-integral-zero?)**.** Construct a test overturning velocity $v_{\mathrm{thc}} = A\,G(z)$ on the model grid with G from Eq. (8.4), and a second one with $G \to G + c$ for a constant $c \neq 0$. Pass both through `compute_amoc` with `remove_barotropic=True`. Explain quantitatively why the barotropic correction (Eq. 8.10) leaves the first cell intact but annihilates the constant offset of the second.

Exercise 8.3 (The hosing transfer function)**.** Apply `subpolar_hosing_salt_tendency` for $H_{\mathrm{Sv}} \in [0, 0.3]$ Sv in a multi-year integration with the closure active ($k_{\mathrm{vel}} > 0$, `contrast_depth_m` $= 300\,\mathrm{m}$). Plot the equilibrated AMOC upper-cell strength (Eq. 8.11) against H_{Sv}. Then repeat with `contrast_depth_m` $= 1100\,\mathrm{m}$ and explain, using the dilution argument of Section 3.4, why the shallow contrast layer gives the hosing far more leverage.

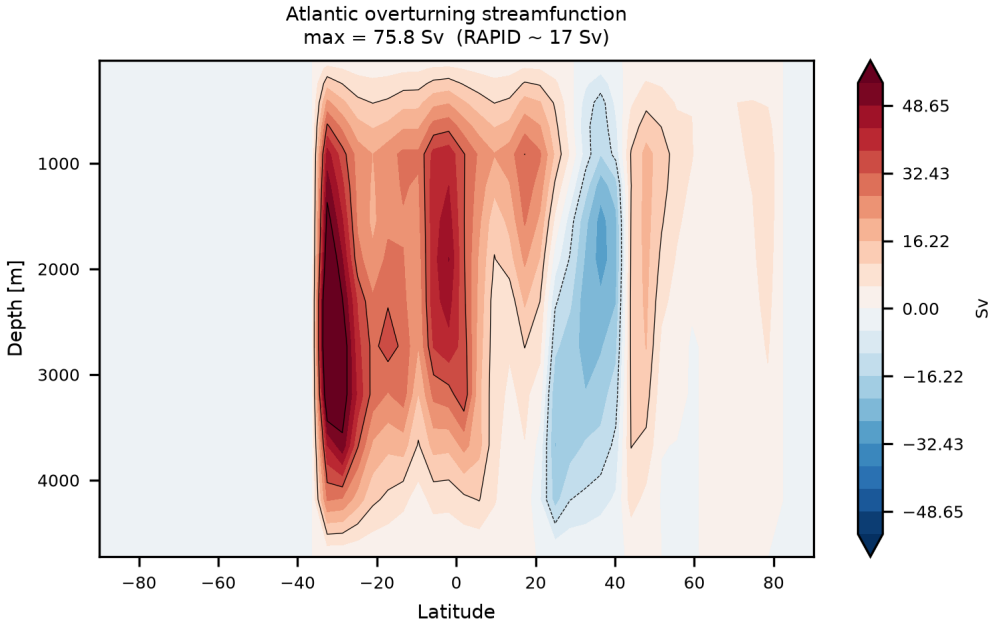

Figure 8.1: Atlantic meridional overturning streamfunction $\Psi_{\mathrm{AMOC}}(\phi, z)$ diagnosed from a coupled control run (positive = clockwise upper cell, RAPID convention). The upper-cell maximum near 26.5°N is the headline AMOC metric. This figure predates the current closure and is refreshed from the latest control run.

Exercise 8.4 (Verifying the sign convention by differentiation)**.** Compute $\partial\,\Psi_{\mathrm{AMOC}}(26.5°\mathrm{N})/\partial H_{\mathrm{Sv}}$ by automatic differentiation through `subpolar_hosing_salt_tendency` and a short ocean step. Confirm the sign is negative (freshening weakens the AMOC) and contrast it with the result one would get under the buggy $\Psi = -\mathrm{cumsum}$ convention of the final remark.

Chapter 9

The Atmosphere: Multi-Level Spectral Dynamical Core

The single-level barotropic atmosphere of Chapter 10 has a fatal structural limitation: with no vertical degree of freedom it cannot support *baroclinic instability*, the conversion of the equator–pole temperature gradient into eddy kinetic energy that builds mid-latitude storm tracks, the eddy-driven jet, and the surface westerlies. Chronos-ESM's production atmosphere therefore replaces it with a genuine multi-level spectral primitive-equation dynamical core. Rather than re-implement a spectral transform core from scratch, Chronos-ESM wraps Google's DINOSAUR dycore — the differentiable JAX spectral primitive-equation solver (Google Research, 2024) that underlies NeuralGCM (Kochkov et al., 2024) — and supplies the column physics (thermal forcing, drag, and a moisture cycle) as additional explicit tendencies. Everything in this chapter lives in

↪ *Code:* `chronos_esm/atmos/dino_atmos.py` — the `DinoAtmosphere` wrapper class

.

This is a deliberate architectural choice. The dycore handles the parts that are hard to get right and easy to get wrong — the spherical-harmonic transforms, the semi-implicit treatment of gravity waves, and tracer advection — while Chronos-ESM owns the physics that couples the atmosphere to the rest of the Earth system. The result is differentiable end to end, so the adjoint of an atmospheric trajectory can be taken with respect to forcing or initial conditions (Chapter 3).

Remark. The standalone single-file `spectral.py` in the same directory (

↪ *Code:* `chronos_esm/atmos/spectral.py` — FFT-in-λ / finite-difference-in-ϕ transforms, `solve_helmholtz`

) is a *pseudo*-spectral hybrid written for the legacy single-level model and the QTCM experiments. Its header is candid: a full pure-JAX Legendre transform is "1000+ lines," so it uses an exact zonal FFT but fourth-order finite differences in latitude. The multi-level atmosphere described here does *not* use it; DINOSAUR provides true spherical-harmonic transforms. We mention `spectral.py` only to map the repository honestly — its `solve_helmholtz` is the building block of the semi-implicit scheme the legacy model never wired in.

9.1 Governing equations in σ coordinates

The dycore integrates the hydrostatic primitive equations on the sphere in a terrain-following vertical coordinate

$$\sigma = \frac{p}{p_s}, \tag{9.1}$$

where $p_s(\lambda, \phi, t)$ is the surface pressure, so σ runs from 0 at the model top to 1 at the surface and the lower boundary $\sigma = 1$ is a coordinate surface even over orography (Vallis, 2017, Ch. 12). Chronos-ESM uses `SigmaCoordinates.equidistant(24)`: 24 equally spaced layers of thickness $\Delta\sigma = 1/24$, with layer centres σ_l stored in `self.sigma` (

↪ *Code:* `chronos_esm/atmos/dino_atmos.py` — `DinoAtmosphere.__init__`, vertical grid

). The horizontal grid is the spectral T31 triangular truncation (`Grid.T31`, $\approx 3.75°$ Gaussian nodal mesh).

Following standard spectral practice, the horizontal momentum is carried not as (u, v) but as its rotational and divergent parts — the relative vorticity ζ and the horizontal divergence δ:

$$\zeta \;=\; \mathbf{k}\cdot(\boldsymbol{\nabla}\times\mathbf{u}), \qquad \delta \;=\; \boldsymbol{\nabla}\cdot\mathbf{u}. \tag{9.2}$$

The four prognostic fields, stored as complex spherical-harmonic coefficients in the DINOSAUR `State` object, are

$$\big(\,\zeta,\ \delta,\ T',\ \ln p_s\,\big), \tag{9.3}$$

where $T' = T - \overline{T}_l$ is the temperature *variation* about a fixed reference profile $\overline{T}_l$ (the layer-mean temperatures $\approx$ 288 K returned by `isothermal_rest_atmosphere`), and $\ln p_s$ is the log of surface pressure (carrying $\ln p_s$ rather than p_s linearises the surface- pressure tendency and keeps $p_s > 0$ exactly). A specific-humidity tracer q is advected alongside them. The continuous evolution is, schematically,

$$\frac{\partial\zeta}{\partial t} = -\,\mathbf{k}\cdot\boldsymbol{\nabla}\times\big(\text{absolute-vorticity flux}\big) + F_\zeta, \tag{9.4}$$

$$\frac{\partial\delta}{\partial t} = \mathbf{k}\cdot\boldsymbol{\nabla}\times(\dots) - \nabla^2\Big(\tfrac{1}{2}|\mathbf{u}|^2 + \Phi\Big) + F_\delta, \tag{9.5}$$

$$\frac{\partial T'}{\partial t} = -\mathbf{u}\cdot\boldsymbol{\nabla}T - \dot\sigma\,\frac{\partial T}{\partial\sigma} + \frac{\kappa T\,\omega}{p} + F_T, \tag{9.6}$$

$$\frac{\partial \ln p_s}{\partial t} = -\,\frac{1}{p_s}\int_0^1 \boldsymbol{\nabla}\cdot(p_s\,\mathbf{u})\,\mathrm{d}\sigma, \tag{9.7}$$

with geopotential Φ from hydrostatic balance, $\kappa = R_d/c_p$, vertical σ-velocity $\dot\sigma$ diagnosed from the continuity constraint, and $\omega = \frac{\mathrm{D}p}{\mathrm{D}t}$. The F terms are the Chronos-ESM-supplied forcings of Section 9.5. The exact discrete tendencies are DINOSAUR's `PrimitiveEquations.explicit_terms` and `implicit_terms`; Chronos-ESM constructs the operator with the reference profile, the modal orography, and the coordinate/spec objects (

↪ *Code:* `chronos_esm/atmos/dino_atmos.py` — `pe.PrimitiveEquations(...)`

). Diagnostic (u, v) are recovered from (ζ, δ) by inverting the streamfunction/velocity-potential relations inside `pe.compute_diagnostic_state`, which returns $u\cos\phi$ and $v\cos\phi$; Chronos-ESM divides by $\cos\phi$ to get physical winds (

↪ *Code:* `chronos_esm/atmos/dino_atmos.py` — `DinoAtmosphere.diagnostics`

).

9.2 The spectral transform method

The spectral method represents each horizontal field as a truncated sum of spherical harmonics $Y_l^m(\phi,\lambda) = P_l^m(\sin\phi)\,e^{im\lambda}$,

$$f(\lambda,\phi) \;=\; \sum_{m=-M}^{M}\ \sum_{l=|m|}^{L} \hat f_l^m\, Y_l^m(\phi,\lambda), \qquad \text{T31: } L = M = 31. \tag{9.8}$$

Two operators motivate the choice. First, horizontal derivatives are *exact and local* in spectral space: the Laplacian is diagonal,

$$\nabla^2 Y_l^m \;=\; -\,\frac{l(l+1)}{a^2}\,Y_l^m, \tag{9.9}$$

so the Poisson inversions needed to get $\mathbf{u}$ from (ζ,δ) and Φ from temperature become mere multiplications — no iterative solver, no polar finite-difference singularity. Second, the nonlinear products (advection, kinetic energy) are cheapest to evaluate pointwise on the grid. The dycore therefore *transforms* between the two representations every step: it computes derivatives spectrally, transforms to the Gaussian nodal mesh (`to_nodal`), forms products there, and transforms back (`to_modal`). Chronos-ESM's physics, which is naturally pointwise (it depends on local T, q, p_s, and the underlying SST), is computed entirely on the nodal grid and projected back to modal coefficients with `to_modal` before being handed to the integrator; the vorticity/divergence parts of a velocity tendency are formed with the spectral operators `curl_cos_lat` and `div_cos_lat` (

↪ *Code:* `chronos_esm/atmos/dino_atmos.py` — `_forcing_explicit`, return value

).

Example 9.1 (Why a velocity seed must be made rotational first)**.** The symmetry-breaking perturbation (Section 9.6) is built as a random nodal velocity field $\mathbf{u}'$, multiplied by $\cos\phi$, transformed to modal space, and then passed through `curl_cos_lat` to extract only its rotational (vorticity) part:

```
clu = jnp.stack([cl*amp*randn, cl*amp*randn])      # cos(lat)*(u',v')
seed_vort = grid.curl_cos_lat(grid.to_modal(clu)) # rotational part only
```

The seed is injected purely into `vorticity`; the divergent part is discarded because divergent perturbations radiate away as gravity waves rather than projecting onto the slowly growing baroclinic mode.

9.3 Time stepping: implicit–explicit Runge–Kutta

The primitive equations are *stiff*: fast external and internal gravity waves ($\sim 300\ \mathrm{m\,s^{-1}}$) coexist with the slow advective flow ($\sim 10\ \mathrm{m\,s^{-1}}$). A fully explicit scheme would have to resolve the gravity-wave CFL with a tiny step. The dycore instead splits the right-hand side into the linear terms responsible for the fast waves (handled *implicitly*, where they are unconditionally stable) and the nonlinear advective/forcing terms (handled *explicitly*). Chronos-ESM assembles this split as

$$\frac{\mathrm{d}\mathbf{s}}{\mathrm{d}t} = \underbrace{E(\mathbf{s})}_{\text{explicit}} + \underbrace{I(\mathbf{s})}_{\text{implicit}}, \qquad E(\mathbf{s}) = E_{\mathrm{dyn}}(\mathbf{s}) + F(\mathbf{s}), \tag{9.10}$$

where E_{dyn} is `eq.explicit_terms`, F is the Chronos-ESM forcing of Section 9.5, I is `eq.implicit_terms`, and the implicit solve uses `eq.implicit_inverse`. The integrator is the three-stage implicit–explicit Runge–Kutta scheme `imex_rk_sil3` ("semi-implicit, low-storage, 3rd order"):

```
ode  = ti.ImplicitExplicitODE.from_functions(
           explicit, eq.implicit_terms, eq.implicit_inverse)
step = ti.step_with_filters(ti.imex_rk_sil3(ode, dt), [self.diff])
```

(
↪ *Code:* `chronos_esm/atmos/dino_atmos.py` — `_run_interval`
). The time step is $\Delta t = 20$ min, nondimensionalised through the dycore's unit system (`specs.nondimensionalize`); ≈ 72 steps make a model day (`steps_per_day`). One ocean coupling interval calls `step(state, sst, n_days)`, which runs $72\,n_{\mathrm{days}}$ inner steps with the SST held fixed and jitted as a traced argument so the whole interval compiles once.

Remark (A unit-handling foot-gun)*.* All physics is done in plain JAX with scalar scale factors precomputed in `__init__` (e.g. `t0_seconds`, `v_scale_ms`); no `pint` objects ever enter a jitted region. SI rates in $\mathrm{s^{-1}}$ are made nondimensional by multiplying by `t0_seconds`. When a saved state is reloaded, `sim_time` must be restored as `None`, not as the stored scalar: the ImEx integrator forms tendencies with `sim_time=None` and adds them to the state by `tree_map`, so a scalar clock would trigger an `array + None` mismatch on the first resumed step (
↪ *Code:* `chronos_esm/atmos/dino_atmos.py` — `load_state`
).

9.4 Scale-selective hyperdiffusion

The truncation discards scales smaller than the grid; energy that cascades toward that cutoff must be removed or it piles up as grid-scale noise. Chronos-ESM applies a ∇^4 (del^4, order $= 2$) hyperdiffusion as a post-step filter. Using the Laplacian eigenvalue (9.9), mode (l, m) is multiplied each step by a damping factor that scales like

$$\hat{f}_l^m \longleftarrow \hat{f}_l^m \left[1 + c\left(l(l+1)\right)^2\right]^{-1}, \qquad c \propto \frac{\Delta t}{\tau\,[\,L(L+1)\,]^2}, \tag{9.11}$$

so the very top (grid-scale) mode is damped on the chosen e-folding time τ while the largest scales are left almost untouched (
↪ *Code:* `chronos_esm/atmos/dino_atmos.py` — `ti.horizontal_diffusion_step_filter`
). The crucial tuning is τ. Chronos-ESM sets $\tau = 6$ h, deliberately *weaker* than the $\tau \approx 2$ h common in idealized cores. At T31 the baroclinic eddies live at fairly high total wavenumbers ($l \sim 10$–15); a 2 h

diffusion damps *those* modes on 1–15 days — competitive with their 1–3 day baroclinic growth — which would suppress the eddies and with them the eddy momentum flux that drives the surface westerlies. At $\tau = 6$ h the eddies grow while grid-scale noise is still killed quickly.

Remark (The opposite lesson from the single-level model). In the legacy single-level atmosphere (Chapter 10) eddies were *spurious* and hyperdiffusion was kept strong to suppress them. Here eddies *are* the physics we want, so diffusion must be weak enough to let them grow. The two regimes call for opposite settings; copying tuning from one to the other is a trap.

9.5 Held–Suarez forcing with an SST-slaved equilibrium

DINOSAUR integrates the adiabatic, frictionless dynamics; the diabatic physics is Chronos-ESM's. The thermal forcing follows the Held–Suarez benchmark (Held and Suarez, 1994): a Newtonian relaxation of temperature toward a prescribed radiative-equilibrium field T_{eq}, plus Rayleigh drag in the boundary layer. The key Chronos-ESM departure is that T_{eq} is *anchored to the underlying sea surface temperature*, so the atmosphere responds to the ocean.

Equilibrium temperature. The classical Held–Suarez T_{eq} uses a fixed analytic equator–pole profile $315 - 60\sin^2\phi$. Chronos-ESM keeps the benchmark's vertical lapse structure but replaces the surface anchor with the local SST T_s:

$$T_{\mathrm{eq}}(\phi,\sigma) \;=\; \Big(\tfrac{\sigma\, p_s}{p_0}\Big)^{\kappa}\Big[\,T_s(\phi,\lambda) \;-\; \Delta_z\,\ln\!\Big(\tfrac{\sigma\, p_s}{p_0}\Big)\,\cos^2\phi\,\Big], \qquad T_{\mathrm{eq}} \ge T_{\min}, \tag{9.12}$$

with $p_0 = 10^5$ Pa, the static-stability parameter $\Delta_z = 10$ K, and a floor $T_{\min} = 200$ K (
↪ *Code:* `chronos_esm/atmos/dino_atmos.py` — `_equilibrium_temperature`
). The $(\sigma p_s/p_0)^{\kappa}$ factor converts the surface (potential) anchor into actual temperature on each σ level; the $\Delta_z \ln(\cdot)$ term sets the vertical lapse rate, tapered to the tropics by $\cos^2\phi$. Because T_s is the coupled ocean's SST, the equator–pole *temperature gradient that drives the jet is whatever the ocean produces* — with the honest consequence that a too-weak modelled SST gradient yields weaker-than-observed surface westerlies.

Newtonian relaxation and drag. The temperature and momentum forcings are

$$F_T \;=\; -\,k_T(\phi,\sigma)\,\big(T - T_{\mathrm{eq}}\big), \tag{9.13}$$

$$F_{\mathbf{u}} \;=\; -\,k_v(\sigma)\,\mathbf{u}, \tag{9.14}$$

with Held–Suarez vertical profiles built from $\mathrm{cut}(\sigma) = \max\big(0,\,(\sigma-\sigma_b)/(1-\sigma_b)\big)$, $\sigma_b = 0.7$:

$$k_T \;=\; k_a + (k_s - k_a)\,\mathrm{cut}(\sigma)\,\cos^4\phi, \qquad k_v \;=\; k_f\,\mathrm{cut}(\sigma), \tag{9.15}$$

with $k_a = (40\text{ d})^{-1}$ (free-atmosphere radiative timescale), $k_s = (4\text{ d})^{-1}$ (faster boundary-layer relaxation), and $k_f = (1\text{ d})^{-1}$ (boundary-layer drag), all nondimensionalised (
↪ *Code:* `chronos_esm/atmos/dino_atmos.py` — `_kt`, `_kv` profiles; `_forcing_explicit`
). The drag acts only below σ_b (the boundary layer) and the strong thermal relaxation k_s acts there and equatorward, exactly as in Held and Suarez (1994).

Moisture and the ITCZ. On top of dry Held–Suarez, Chronos-ESM carries a prognostic specific-humidity tracer q with a surface evaporation source and a large-scale condensation sink. Evaporation into the lowest layer uses a bulk formula driven by SST and near-surface wind, capped to 1–25 $\mathrm{m\,s^{-1}}$:

$$E \;=\; \rho_a\, C_E\,|\mathbf{u}_{\mathrm{sfc}}|\,\max\big(q_{\mathrm{sat}}(T_s) - q_{\mathrm{bot}},\,0\big) \quad [\mathrm{kg\,m^{-2}s^{-1}}], \tag{9.16}$$

and supersaturation is rained out on a $\tau_c = 3$ h timescale, latent-heating the column:

$$C \;=\; \frac{\max(q - q_{\mathrm{sat}}(T),\,0)}{\tau_c}, \qquad F_T \mathrel{+}= \frac{L_v}{c_p}\,C, \qquad \frac{\partial q}{\partial t} \mathrel{+}= E\,\frac{g}{p_s\,\Delta\sigma} - C, \tag{9.17}$$

with q_{sat} from Clausius–Clapeyron (
↪ *Code:* `chronos_esm/atmos/dino_atmos.py` — `_qsat`, `_forcing_explicit` moisture block

). Column-integrated C is reported as precipitation (
↪ *Code:* `chronos_esm/atmos/dino_atmos.py` — `diagnostics`
). Moisture converging in the tropics condenses and releases latent heat, producing an ITCZ that the dry single-level model could never represent.

Remark (Conservation fixers). Two pragmatic fixes keep long runs honest. (i) Surface pressure is rescaled every interval to conserve the initial global-mean dry mass; without it, Gibbs ripples from spectrally representing orography leak ~ 0.3 hPa/day (
↪ *Code:* `chronos_esm/atmos/dino_atmos.py` — `_fix_mass`
). (ii) The ETOPO orography is Gaussian-smoothed before use, because raw T31 topography has sharp gradients that produced a ~ 1.8 hPa/day mass drift through those ripples. Reported MSLP reduces p_s to mean sea level with a fixed standard-atmosphere scale height, so it is noise-free and comparable to ERA5.

9.6 How the jet and storm tracks emerge

The point of going multi-level is that the mid-latitude circulation is now *earned*, not prescribed. The SST-anchored T_{eq} (9.12) sets up an equator–pole temperature gradient; thermal-wind balance turns that gradient into a westerly jet with vertical shear; the sheared, stratified flow is baroclinically unstable; the growing eddies flux momentum into the jet core and heat poleward, and their surface signature is the trade-easterly / mid-latitude-westerly / polar-easterly tripole. None of this is imposed — it is the saturation of baroclinic instability, exactly the textbook life cycle (Vallis, 2017, Ch. 9, 12).

Getting there in finite wall-clock time needed two non-obvious choices.

(1) Initialise near radiative equilibrium. DINOSAUR's `isothermal_rest_atmosphere` is an isothermal fluid at rest — and *exactly* zonally symmetric. Chronos-ESM instead initialises the temperature field at the SST-anchored T_{eq} so the baroclinic jet exists from day 0 rather than taking ~ 2 months to build through the slow 40-day thermal relaxation (
↪ *Code:* `chronos_esm/atmos/dino_atmos.py` — `initial_state`
).

(2) Break zonal symmetry with a *rotational* seed. An exactly axisymmetric state is an unstable equilibrium the flow never leaves on its own — only floating-point roundoff breaks it, and only after ~ 100 days. Without an asymmetric seed *no* baroclinic eddies form, there is no eddy momentum-flux convergence, and the surface stays easterly everywhere (the u_{sfc} pattern correlation against ERA5 is ~ 0). Chronos-ESM adds a small random *vorticity* perturbation ($\approx 2\ \mathrm{m\,s^{-1}}$ amplitude). It must be rotational, not thermal: a temperature perturbation is simply relaxed away by F_T before it can grow, whereas a rotational seed projects directly onto the growing baroclinic mode (
↪ *Code:* `chronos_esm/atmos/dino_atmos.py` — `__init__`, `seed_vort` via `curl_cos_lat`
). With (1), (2), and the weak 6 h diffusion, eddies, storm tracks, and surface westerlies appear in $\sim$ 2–3 weeks instead of $\sim$ 2–3 months. The validated dry-core spin-up (
↪ *Code:* `experiments/dino_held_suarez.py` — Phase-1 Held–Suarez verification
) reproduces the textbook twin $\sim 22\ \mathrm{m\,s^{-1}}$ upper-tropospheric midlatitude jets with surface trades.

Proposition 9.1 (The surface westerlies are eddy-driven)**.** *In a statistically steady, zonally averaged state the surface zonal-momentum budget below the boundary layer balances the eddy momentum-flux convergence against surface drag,*

$$-\frac{1}{a\cos^2\phi}\,\frac{\partial}{\partial\phi}\left(\overline{u'v'}\cos^2\phi\right)\;\approx\;k_v\,\overline{u}_{\mathrm{sfc}}. \tag{9.18}$$

Hence $\overline{u}_{\mathrm{sfc}} > 0$ (westerly) requires a convergence of eddy momentum flux at that latitude. With the rotational seed switched off (`seed_wind_ms=0`) the eddies vanish, $\overline{u'v'} \to 0$, and the surface westerlies collapse to easterlies everywhere — the failure mode the seed was introduced to cure.

9.7 Status and honest limitations

The dino atmosphere is the validated control-run path through the standalone harness

↪ *Code:* `experiments/run_dino_coupled.py` — checkpointing/resumable control harness

, *not yet* the path inside `main.py:step_coupled`; the single-level atmosphere of Chapter 10 is still the coupled default. Two honest caveats carry into the validation chapters (16, 18). First, the surface wind and pressure *pattern* are only as good as the SST gradient the T31 ocean supplies; a weak modelled gradient yields weaker westerlies than ERA5, a limitation of the *coupling*, not the dycore. Second, resume restores the prognostic modal state exactly but recomputes diagnostic fields, so a restarted run is state-complete but not bit-reproducible (~ 0.03 K/week drift, negligible). The ITCZ, by contrast, is a clear win over the single-level model: moisture convergence makes it rain at the equator rather than in the subtropics.

Exercise 9.1 (Switch off the symmetry-breaking seed)**.** Construct `DinoAtmosphere(seed_wind_ms=0.0)` and a second instance with the default 2 $\mathrm{m\,s^{-1}}$ seed. Spin both up for 60 days at a fixed Earth-like SST and plot the zonal-mean surface zonal wind $\overline{u}_{\mathrm{sfc}}(\phi)$ from `diagnostics`. Confirm that the seed-free run has easterlies at all latitudes while the seeded run develops mid-latitude westerlies. Relate the difference to the eddy momentum-flux budget of the proposition above.

Exercise 9.2 (The diffusion timescale controls the eddies)**.** Re-run a 60-day spin-up at $\tau \in \{2, 4, 6, 12\}$ h (`diffusion_tau_hours`). For each, measure eddy kinetic energy $\frac{1}{2}\,\overline{(u'^2 + v'^2)}$ at $\sim 45°$ and the peak surface westerly. Show that $\tau = 2$ h damps the eddies and weakens the westerlies, and explain why this is the *opposite* of the correct setting for the single-level model.

Exercise 9.3 (SST gradient sets the jet)**.** Replace the default SST $T_s = 300 - 40\sin^2\phi$ with a weaker gradient $300 - 20\sin^2\phi$ and a stronger one $300 - 60\sin^2\phi$, passing each to `initial_state` and `step`. Using thermal-wind balance, predict how the upper-tropospheric jet speed should change, then verify against the model's $\overline{u}(\phi, \sigma)$. Use this to argue why a cold-biased tropical SST in the coupled run (a known Chronos-ESM bias) weakens the simulated westerlies.

Exercise 9.4 (Differentiable sensitivity of the jet)**.** Because the whole trajectory is JAX-differentiable (Chapter 3), use `jax.grad` to compute the sensitivity of a 30-day-mean jet-speed diagnostic to the static-stability parameter Δ_z (`dThz`) in (9.12). Compare the adjoint gradient against a finite-difference estimate and comment on what the sign of $\partial(\text{jet})/\partial\Delta_z$ tells you about the static-stability control of baroclinicity.

Chapter 10

The Atmosphere: Single-Level Spectral Model

This chapter documents Chronos-ESM's *legacy* atmosphere: a single horizontal level (`ATMOS_GRID.nz` = 1), shallow-water-like spectral model integrated with forward Euler. It is the atmosphere that produced the validation dashboard in the README, and it remains the only atmosphere wired into the coupled time step (`main.py:step_coupled`). Its successor — a multi-level spectral dynamical core built on the DINOSAUR dycore — is the subject of Chapter 9; that core fixes the structural limitations we are honest about here (no baroclinic eddies, weak Intertropical Convergence Zone). The single-level model is nonetheless worth understanding in detail: it is small, fully differentiable, and its design choices are a clinic in what one *can* and *cannot* ask of a barotropic atmosphere.

We follow the textbook structure: continuous governing equation first, then the discrete form the code actually integrates, including every clamp, filter and limiter, with a pointer to the source. The reference for the underlying dynamics is Vallis (2017).

10.1 State variables and the prognostic–diagnostic split

The atmospheric state (
↪ *Code:* `chronos_esm/atmos/dynamics.py` — class `AtmosState`
) carries six *prognostic* fields, each a single horizontal slab on the nlat × nlon lat–lon grid: relative vorticity ζ, horizontal divergence δ, temperature T, log surface pressure $\pi = \ln p_s$, specific humidity q, and a carbon dioxide concentration field (ppm). Velocity is *not* prognostic; instead the model carries ζ and δ and reconstructs the wind $\mathbf{u} = (u, v)$ each step via a Helmholtz (vorticity–divergence) decomposition. The diagnostic fields u, v, the streamfunction ψ, the velocity potential χ, and the static surface geopotential $\phi_s = g z_s$ round out the state.

This is the classic vorticity–divergence formulation of a single-layer atmosphere (Vallis, 2017, Ch. 5). Working in (ζ, δ) rather than (u, v) makes the rotational and divergent parts of the flow cleanly separable and lets us solve two scalar Poisson problems instead of a coupled vector system.

10.2 Wind recovery: the Helmholtz decomposition

By the Helmholtz theorem any horizontal wind field splits into a non-divergent part (from a streamfunction ψ) and an irrotational part (from a velocity potential χ):

$$\mathbf{u} = \mathbf{k} \times \boldsymbol{\nabla}\psi + \boldsymbol{\nabla}\chi, \qquad u = -\frac{\partial \psi}{\partial y} + \frac{\partial \chi}{\partial x}, \qquad v = \frac{\partial \psi}{\partial x} + \frac{\partial \chi}{\partial y}. \tag{10.1}$$

The potentials are recovered from the prognostic fields by inverting two Poisson equations,

$$\nabla^2 \psi = \zeta, \qquad \nabla^2 \chi = \delta. \tag{10.2}$$

The code does exactly this. Before inversion both ζ and δ are passed through a polar filter (below); the inversions in Eq. (10.2) are solved *directly* by an FFT in longitude combined with a tridiagonal (Thomas) solve in latitude, not iteratively.

↪ *Code:* `chronos_esm/atmos/spectral.py` — `inverse_laplacian` — rFFT in x, batched tridiagonal solve in y for each zonal wavenumber

↪ *Code:* `chronos_esm/atmos/dynamics.py` — `step_atmos` — `psi = inverse_laplacian(vort)`, `chi = inverse_laplacian(div)`, then Eq. (10.1)

Remark. The longitudinal transform is exact (a real FFT), but the meridional direction uses fourth-order centred finite differences for gradients and second-order differences for the Laplacian. Chronos-ESM is therefore a *spectral–finite- difference hybrid*, not a true spherical-harmonic model: there is no Legendre transform and no triangular truncation. The module docstring is candid that a full pure-JAX spherical-harmonic transform "is 1000+ lines" and was deliberately avoided here. This is the single biggest difference from the multi-level core of Chapter 9.

After the inversion the diagnostic winds from Eq. (10.1) are clipped to $\pm 80\ \mathrm{m\,s^{-1}}$ and polar-filtered. The clip is a safety net, not a physical cap: with the wind relaxed toward a $\sim 10\ \mathrm{m\,s^{-1}}$ climatological jet (Section 10.6) the field should never approach $80\ \mathrm{m\,s^{-1}}$, so hitting the clip flags a developing instability rather than silently saturating an unphysical jet.

10.3 The governing tendencies

On a single layer the prognostic equations are the shallow-water vorticity and divergence equations augmented with thermodynamic and moisture conservation. The continuous forms, with $\eta = \zeta + f$ the absolute vorticity and $f = 2\Omega \sin\phi$ the Coriolis parameter, are

$$\frac{\partial \zeta}{\partial t} = -\mathbf{u}\cdot\boldsymbol{\nabla}\eta - \eta\,\delta + \boldsymbol{\nabla}\times\mathbf{F}, \tag{10.3}$$

$$\frac{\partial \delta}{\partial t} = -\boldsymbol{\nabla}\cdot\big(R_d\,T\,\boldsymbol{\nabla}\pi\big) - \nabla^2\phi_s + f\,\zeta + \boldsymbol{\nabla}\cdot\mathbf{F}, \tag{10.4}$$

$$\frac{\partial T}{\partial t} = -\mathbf{u}\cdot\boldsymbol{\nabla}T + Q_T, \tag{10.5}$$

$$\frac{\partial \pi}{\partial t} = -\mathbf{u}\cdot\boldsymbol{\nabla}\pi - \delta, \tag{10.6}$$

$$\frac{\partial q}{\partial t} = -\mathbf{u}\cdot\boldsymbol{\nabla}q + Q_q, \qquad \frac{\partial C}{\partial t} = -\mathbf{u}\cdot\boldsymbol{\nabla}C + Q_C, \tag{10.7}$$

where $R_d = 287\ \mathrm{J\,kg^{-1}\,K^{-1}}$ is the gas constant for dry air, $\mathbf{F}$ is the momentum forcing (friction/relaxation, Section 10.6), Q_T, Q_q, Q_C are the thermodynamic, moisture and carbon sources, and C is the CO_2 field. Equation (10.6) is the layer continuity equation: surface pressure tendency is set by mass convergence $-\delta$ plus advection of π.

A few honest remarks on what these are. The pressure-gradient term in Eq. (10.4) is written in code as $\boldsymbol{\nabla}\cdot(R_dT\,\boldsymbol{\nabla}\pi) = R_d\big(\boldsymbol{\nabla}T\cdot\boldsymbol{\nabla}\pi + T\,\nabla^2\pi\big)$ plus the orographic forcing $-\nabla^2\phi_s$; this is the single-level analogue of the geopotential gradient driving the divergent flow. Because there is only one level, there is *no* vertical advection, *no* vertical shear, and hence no thermal-wind balance — a fact that drives the central design compromise of Section 10.6.

↪ *Code:* `chronos_esm/atmos/dynamics.py` — `step_atmos`: `dzeta_dt`, `ddiv_dt`, `dt_dt`, `dlnps_dt`, `dq_dt`, `dco2_dt`

The advection operator is implemented in flux-free advective form, $\mathbf{u}\cdot\boldsymbol{\nabla}f = u\,\partial_x f + v\,\partial_y f$, with gradients from the spectral–FD routine:

```
def advect(f):
    df_dx, df_dy = spectral.compute_gradients(f, dx, dy)
    return -(u * df_dx + v * df_dy)

eta = state.vorticity + f_cor              # absolute vorticity
dzeta_dt = advect(eta) - eta * state.divergence + curl_forcing
```

Listing 10.1: Advection and the vorticity tendency (abbreviated).

10.4 Forward-Euler time stepping with hyperdiffusion

The model marches every field with a single forward (explicit) Euler step at $\Delta t =$ `DT_ATMOS`, adding scale-selective ∇^4 hyperdiffusion and a polar-only ∇^2 term. For a generic field f with tendency G_f from Eqs. (10.3)–(10.7),

$$f^{n+1} = f^n + \Delta t\Big(G_f - \nu_4^{(f)}\,\nabla^4 f^n + \nu_{\mathrm{pol}}\,\nabla^2 f^n\Big). \tag{10.8}$$

The ∇^4 (bi-Laplacian) operator is the Laplacian applied twice (
↪ *Code:* `chronos_esm/atmos/spectral.py` — `compute_bilaplacian`
); in spectral space it damps a mode of wavenumber k as k^4, so it bites hard at the grid scale ($2\Delta x$ noise) and is essentially invisible to the large-scale circulation. The coefficients are deliberately *strong* — the so-called "tank" values $\nu_4^\zeta = 3\times10^{14}$, $\nu_4^\delta = 6\times10^{14}$, $\nu_4^T = 2\times10^{13}$, with weaker values for q, C and π (all in SI units of $\mathrm{m^4\,s^{-1}}$).

Remark (The relaxation/diffusion decoupling). Why keep diffusion strong when one usually weakens it to "let the jet develop"? Because in this model the jet is *not* developed by the dynamics — it is *set* by the momentum relaxation of Section 10.6. Since ∇^4 leaves the large scale untouched, strong hyperdiffusion does not fight the relaxed jet; it only kills small-scale noise and buys stability. A four-times-weaker setting was found to let a slow eddy instability grow over ~ 25 days until $|u|$ hit the clamp. This decoupling — *relaxation → realism, hyperdiffusion → stability* — is the key organising idea of the single-level model. (Contrast the multi-level core, where the eddies *are* the physics and diffusion must be weak enough to let them grow.)

Implicit divergence damping. Linear divergence damping is the standard device for suppressing spurious gravity-wave and external-mode noise. The naive explicit form $-r\,\delta$ with $r = 5\times10^{-2}\ \mathrm{s^{-1}}$ gives $\Delta t\, r = 1.5 > 1$, so forward Euler *overshoots* and flips the sign of δ every step — a $2\Delta t$ oscillation that pumps grid-scale checkerboard noise into π. The fix is to apply the damping *implicitly*, which is unconditionally stable for any rate:

$$\delta^{n+1} \;\leftarrow\; \frac{\delta^{n+1}}{1 + \Delta t\, r_{\mathrm{div}}}, \qquad r_{\mathrm{div}} = 10^{-2}\ \mathrm{s^{-1}}. \tag{10.9}$$

This absorbs divergent-mode noise while preserving the large-scale divergent (Hadley/Walker) circulation that organises precipitation.

10.5 The polar Δx-floor: a stability fix, not a physics choice

A lat–lon grid has a fatal pathology at the poles: the zonal grid spacing

$$\Delta x(\phi) = \frac{2\pi a\cos\phi}{\mathrm{nlon}} \tag{10.10}$$

collapses to ~ 4 m on the pole-most rows (where $\cos\phi$ is floored to 10^{-5} to avoid division by zero). Any zonal structure then makes ∂_x and ∂_x^2 explode: the $\boldsymbol{\nabla}\cdot(R_dT\boldsymbol{\nabla}\pi)$ pressure term and $\mathbf{u}\cdot\boldsymbol{\nabla}\pi$ advection detonate *at* the poles within a few steps. The original symptom was diagnosed precisely: surface pressure swung to 100–1200 hPa at $\phi = \pm90°$ over flat topography, with $\sigma(p_s) \sim 150$ hPa against an observed ~ 12 hPa.

The fix is to floor the *dynamical* Δx at its 80° value,

$$\Delta x_{\mathrm{dyn}}(\phi) = \frac{2\pi a}{\mathrm{nlon}}\,\max\big(\cos\phi,\ \cos 80°\big), \tag{10.11}$$

so the handful of pole-most rows integrate with the $\sim 80°$ metric instead of the collapsing one. A separate, identical floor Δx_{diff} is applied to the diffusion operators, whose $1/\Delta x^4$ scaling would otherwise exceed the explicit stability limit by ~ 14 orders of magnitude at the poles. The *accurate* $\cos\phi$ is retained for area weighting and the Coriolis parameter. This is unapologetically a numerical band-aid for the grid choice; it is local to a few rows and does not touch the physics elsewhere.
↪ *Code:* `chronos_esm/atmos/dynamics.py` — `step_atmos`: `cos_lat_dyn`, `cos_lat_diff`, and `polar_filter` usage

The polar *filter* (

↪ *Code:* `chronos_esm/atmos/spectral.py` — `polar_filter`

) is the complementary device: it applies a smooth Gaussian taper to zonal modes above the CFL cutoff $m_{\max}(\phi) = \frac{\text{nlon}}{2}\cos\phi$, always preserving at least the zonal mean ($m = 0$). It *damps* rather than removes high modes — an earlier hard cutoff killed all polar circulation every step.

10.6 Wind relaxation: an eddy-momentum-flux parameterization

Here is the central honesty of the chapter. A single-level (barotropic) atmosphere has no vertical shear, hence no thermal wind and no baroclinic instability. A meridional temperature gradient therefore drives *no* sustained pressure gradient: the winds geostrophically adjust toward rest. In diagnosis, $|u|$ decayed from 10 to $< 1\ \mathrm{m\,s^{-1}}$ and the correlation against ERA5 surface wind (Hersbach et al., 2020) fell to ~ 0. The eddy-momentum-flux convergence that builds the real mid-latitude surface westerlies, and the Hadley overturning that drives the trades, both live in vertical structure this model does not resolve.

Chronos-ESM parameterizes their *net* effect as a Rayleigh relaxation of the wind toward an observed zonal-mean surface profile, rather than toward zero:

$$\mathbf{F} = -\frac{u - u_{\text{tgt}}(\phi)}{\tau_f}\,\hat{\mathbf{x}} - \frac{v}{\tau_f}\,\hat{\mathbf{y}}, \qquad \tau_f = 2 \text{ days}. \tag{10.12}$$

The target is a three-Gaussian fit to the observed surface wind: mid-latitude westerlies, tropical trade easterlies, and weak polar easterlies, with ϕ in degrees,

$$u_{\text{tgt}}(\phi) = 9.0\,e^{-\left(\frac{|\phi|-48}{18}\right)^2} - 5.5\,e^{-\left(\frac{|\phi|-13}{11}\right)^2} - 2.0\,e^{-\left(\frac{|\phi|-80}{10}\right)^2}. \tag{10.13}$$

The friction enters the dynamics through its curl and divergence: $\boldsymbol{\nabla}\times\mathbf{F}$ forces ζ in Eq. (10.3) and $\boldsymbol{\nabla}\cdot\mathbf{F}$ forces δ in Eq. (10.4).

↪ *Code:* `chronos_esm/atmos/dynamics.py` — `step_atmos`: `u_target`, `drag_u`, `drag_v`, `curl_forcing`, `div_forcing`

Remark (Why $\tau_f = 2$ days, not 6). The relaxed jet in Eq. (10.13) has meridional shear that is itself barotropically unstable. At a 6-day timescale, localized eddies grew faster than they were damped and $|u|$ ran to the clamp by $\sim$ day 14. A 2-day relaxation damps deviations from the target faster than the instability grows, pinning the field near the (stable, realistic) climatological jet. So the timescale is set by a numerical stability margin, not by an eddy lifetime. This is a parameterization in the strictest sense — if you want *emergent* jets and storm tracks, you need the multi-level core (Chapter 18).

10.7 Physics: precipitation, radiation, CO_2

The column physics lives in `chronos_esm/atmos/physics.py` and is written to be differentiable end-to-end (smooth or piecewise-linear triggers, no hard branches that break the gradient).

Saturation and precipitation. Saturation specific humidity follows Clausius–Clapeyron with the Tetens form,

$$e_s(T) = 611\,\exp\!\left(\frac{17.27\,(T - 273.15)}{(T - 273.15) + 237.3}\right), \qquad q_s = 0.622\,\frac{e_s}{p}. \tag{10.14}$$

Precipitation uses a relative-humidity threshold $q_c = q_c^{\text{ref}}\,q_s$ and a smoothed convective trigger. The intended form is a softplus, $P = \text{softplus}\big((q - q_c)/\epsilon\big)\,\epsilon/\tau_{\text{conv}}$, but the code as shipped uses a rectified-linear (ReLU) trigger,

$$P = \text{ReLU}\!\left(\frac{q - q_c}{\epsilon}\right)\frac{\epsilon}{\tau_{\text{conv}}}, \qquad Q_{\text{lat}} = \frac{L_v}{c_p}\,P, \tag{10.15}$$

with $\tau_{\text{conv}} = 1800$ s, $L_v = 2.5\times10^6\ \mathrm{J\,kg^{-1}}$, $c_p = 1004\ \mathrm{J\,kg^{-1}\,K^{-1}}$, and $q_c^{\text{ref}} = 0.4915$, $\epsilon = 4.71\times10^{-2}$ both tuned (the docstring's stated target was an idealized ITCZ). The smoothing parameter ϵ exists precisely so the trigger has a well-defined derivative for the differentiable-calibration applications of Chapter 15.

↪ *Code:* `chronos_esm/atmos/physics.py` — `compute_saturation_humidity`, `compute_precipitation`

Radiation and CO_2 forcing. Longwave physics is a Newtonian (Newtonian-cooling) relaxation toward a radiative equilibrium profile T_{eq}, plus a column-distributed CO_2 forcing:

$$Q_{\text{rad}} = -\frac{T - T_{\text{eq}}}{\tau_{\text{rad}}} + \frac{F_{\text{CO}_2}}{\mathcal{C}}, \qquad F_{\text{CO}_2} = \alpha_{\text{CO}_2} \ln\frac{C}{C_0}. \tag{10.16}$$

The CO_2 term is the standard logarithmic radiative-forcing law (Myhre et al., 1998) with $\alpha_{\text{CO}_2} = 5.35\ \text{W m}^{-2}$ and pre-industrial reference $C_0 = 280$ ppm, divided by a column heat capacity $\mathcal{C} = 10^7\ \text{J m}^{-2}\,\text{K}^{-1}$ to convert flux to a heating rate. The radiative-equilibrium target T_{eq} is driven by the (seasonal) downward shortwave flux when supplied, $T_{\text{eq}} = 245 + 0.28\,S_\downarrow$ (clipped to $[180, 350]$ K), or otherwise by an annual-mean profile $T_{\text{eq}} = 315 - 60\sin^2\phi$ scaled as $(S_0/1361)^{1/4}$.
↪ *Code:* `chronos_esm/atmos/physics.py` — `compute_radiative_forcing`, `compute_solar_insolation`, `compute_albedo`

Remark (An aggressive cooling timescale). The implemented $\tau_{\text{rad}} = 1$ day is far shorter than the ~ 20-day radiative damping the module constant `RAD_COOL_RATE` advertises. The comment is explicit about why: the single-level dynamics mix heat *faster* than a realistic radiative relaxation could maintain the equator–pole gradient, so the cooling rate is cranked up to force the gradient by brute relaxation. This is a symptom of the missing vertical structure, not a free physical choice — and it is one more reason this model's tropical SST and ITCZ are biased (cold tongue, weak equatorial rain).

Surface fluxes. Sensible and latent heat use bulk aerodynamic formulae (Large and Yeager, 2004), $\text{SH} = \rho_a c_p C_h\,|\mathbf{u}|\,(T_s - T_a)$ and $\text{LH} = \rho_a L_v C_e\,|\mathbf{u}|\,(q_s - q_a)\,\beta$, with a gustiness floor $|\mathbf{u}| \geq 1\ \text{m s}^{-1}$ and a cap $|\mathbf{u}| \leq 25\ \text{m s}^{-1}$ to forestall a wind-evaporation-feedback (WISHE) runaway, and an evaporation efficiency β (1 over ocean, < 1 over land).
↪ *Code:* `chronos_esm/atmos/physics.py` — `compute_surface_fluxes`

10.8 Conservation fixers, clamps, and limiters

Forward Euler conserves neither energy nor enstrophy, so over century-scale runs tiny per-step errors compound. The code applies a *gentle* energy fixer that nudges temperature to offset the spurious change in total energy $E = \int(\frac{1}{2}|\mathbf{u}|^2 + c_p T)\cos\phi\ \text{d}A$, $T \leftarrow T - \alpha\,\delta T$ with $\alpha = 0.01$ (reduced from 0.1 once the polar instability was removed). An enstrophy fixer exists analogously; jointly conserving energy and enstrophy is the design goal of the classic energy–enstrophy-conserving discretizations (Arakawa and Lamb, 1981).
↪ *Code:* `chronos_esm/atmos/dynamics.py` — `apply_energy_fixer`, `apply_enstrophy_fixer`

For robustness the integrator also imposes physics-stability clamps each step: $q \in [0, 0.05]$, $T \in [150, 350]$ K, $|\zeta|, |\delta| \leq 5 \times 10^{-4}\ \text{s}^{-1}$, $\pi \in [9.2, 11.7]$ (i.e. $p_s \in [100, 1200]$ hPa), and $C \in [0, 10^6]$ ppm. Moisture and temperature *tendencies* are also throttled (e.g. $|Q_T| \leq 5 \times 10^{-3}\ \text{K s}^{-1}$) to damp rain/heating shocks. These are safety rails: in a healthy run none should be active, and a clamp that *is* hit is a diagnostic that something upstream is wrong.

10.9 Honest limitations

The single-level model reproduces a respectable zonal-mean climate — after the v3 overhaul, $\sigma(p_s)$ ratio improved from 14.7 to 2.1, the p_s bias from -71.8 to $+2.9$ hPa, and surface-wind correlation against ERA5 from -0.08 to 0.72. But two limitations are structural, not tuning issues:

1. **No baroclinic eddies, hence no synoptic systems.** A barotropic layer cannot extract available potential energy from the temperature gradient. The mid-latitude p_s and meridional-wind pattern correlations are ~ 0; the surface westerlies are *imposed* (Eq. (10.13)), not earned. Storm tracks require the multi-level core (Chapter 18).
2. **A weak ITCZ.** With one level there is no Hadley overturning to converge moisture at the equator; precipitation is too dry and rains in the subtropics rather than at the equator. The aggressive radiative cooling and cold tropical SST compound this (Chapter 19).

Both are the explicit motivation for the multi-level dynamical core of Chapter 9. The single-level model remains the production atmosphere for coupled runs because it is cheap, stable, and differentiable; it is best read as a deliberately minimal baseline against which the multi-level core's gains are measured.

Exercises

Exercise 10.1. Using `step_atmos`, initialize the atmosphere from `init_atmos_state`, set the momentum-forcing timescale τ_f in Eq. (10.12) to ∞ (i.e. remove the relaxation by zeroing `drag_u`, `drag_v`), and integrate for 30 days from a $10\cos\phi$ m s^{-1} jet. Plot the zonal-mean u each 5 days. Confirm the prediction of Section 10.6: the barotropic layer spins down toward rest. Roughly what e-folding time do you measure, and how does it compare with $\tau_f = 2$ days?

Exercise 10.2. Reproduce the polar blow-up of Section 10.5. Temporarily replace Δx_{dyn} in Eq. (10.11) by the bare $2\pi a\cos\phi/\mathrm{nlon}$ (no floor), integrate from rest with flat topography, and record $\sigma(p_s)$ at the pole rows versus the floored run. How many steps until p_s leaves the $[100, 1200]$ hPa clamp? This is a hands-on demonstration of why a lat–lon dynamical core needs a polar treatment.

Exercise 10.3. The CO_2 forcing in Eq. (10.16) is logarithmic (Myhre et al., 1998). Holding all dynamics fixed, drive `compute_radiative_forcing` with $C \in \{280, 560, 1120\}$ ppm and verify that each doubling adds a constant $\alpha_{CO_2}\ln 2 \approx 3.7$ W m^{-2} to F_{CO_2}. Then use JAX's `jax.grad` to compute $\partial T_{\mathrm{eq,column}}/\partial C$ at $C = 400$ ppm — this is the kind of exact sensitivity that motivates the differentiable applications of Chapter 15.

Exercise 10.4. The precipitation trigger Eq. (10.15) ships as a ReLU even though the docstring describes a softplus. (a) Show analytically that $\mathrm{softplus}(x) \to \mathrm{ReLU}(x)$ as the softplus sharpness increases, and that the two differ by at most $\ln 2$ at $x = 0$. (b) Swap the ReLU for a true softplus in `compute_precipitation`, run a short integration, and report the change in global-mean precipitation and in the smoothness of $\partial P/\partial q$ near the threshold. Which form is friendlier to gradient-based calibration, and why might the ReLU still have been chosen?

Chapter 11

Sea Ice Thermodynamics

Sea ice is the climate system's most leveraged component per unit mass. A thin film of frozen seawater—typically 1–3 m thick—sits between a cold polar atmosphere and an ocean held at its freezing point, and it controls the heat, freshwater, and momentum exchanged across that interface. Two feedbacks make it dynamically central. First, the *ice–albedo feedback*: open water absorbs roughly 90% of incident shortwave radiation, bare ice roughly 30%, so a small retreat of the ice edge warms the surface and promotes further retreat. Second, the *insulation feedback*: ice is a poor conductor, so a growing ice cover throttles the ocean-to-atmosphere heat loss that drives convection and, ultimately, the deep-water formation discussed in Chapter 8. Sea ice also rejects brine as it freezes and releases freshwater as it melts, coupling directly to the salinity budget and hence to the overturning circulation studied in Chapter 17.

Chronos-ESM represents these processes with a *Semtner zero-layer* thermodynamic model (Semtner, 1976): the simplest scheme that captures the surface energy balance, conductive growth, and the albedo/insulation feedbacks without resolving the internal temperature profile of the ice or its dynamics (drift, ridging, rheology). This chapter derives the governing equations, then shows exactly which discrete equations the code integrates, where each approximation, clamp, and limiter lives, and where the scheme departs from observations. The two source modules are

↪ *Code:* `chronos_esm/ice/thermodynamics.py` — surface energy balance & albedo

and

↪ *Code:* `chronos_esm/ice/driver.py` — growth/melt, frazil, fluxes, state

.

Remark. "Zero-layer" means the ice slab has *no* prognostic heat capacity: heat conducts through it instantaneously and linearly. The full Semtner 1976 model also offers three-layer and zero-layer variants with a snow layer and brine heat storage; Chronos-ESM implements the bare zero-layer core (no snow, no internal heat reservoir). This is honest to keep in mind: the model cannot represent the thermal-inertia lag of thick multiyear ice.

11.1 State variables and the freezing point

The prognostic state per ocean grid cell is the triple

↪ *Code:* `chronos_esm/ice/driver.py` — `IceState`

$$\big(h,\ A,\ T_s\big), \tag{11.1}$$

the grid-cell mean ice *thickness* $h\,[\mathrm{m}]$, the fractional ice *concentration* $A \in [0,1]$, and the ice *surface temperature* $T_s\,[^\circ\mathrm{C}]$. The model is initialised ice-free ($h = A = 0$, $T_s = T_f$) and forms ice dynamically.

Seawater freezes below 0°C because dissolved salt depresses the freezing point. Chronos-ESM uses a constant

$$T_f = -1.8\,^\circ\mathrm{C}, \tag{11.2}$$

the standard value for ocean salinity near 34–35 psu. This is an approximation: the true freezing point is salinity-dependent, $T_f \approx -0.054\,S$, so Eq. (11.2) is exact only near 33 psu and biased by up to ± 0.1 K across the realistic salinity range—negligible for the energy budget but worth noting for the brackish Baltic or the salty subtropics. The constant is `T_FREEZE` in `thermodynamics.py`.

11.2 Surface energy balance

Over an ice surface the skin temperature T_s adjusts so that the net heat flux into the surface vanishes (an infinitesimally thin skin cannot store energy). The continuous balance, all fluxes positive *into* the surface, is

$$F_{\mathrm{net}}(T_s) = \underbrace{(1-\alpha)\,S^{\downarrow}}_{\text{absorbed SW}} + \underbrace{L^{\downarrow}}_{\text{down LW}} - \underbrace{\varepsilon\sigma (T_s + T_0)^4}_{\text{up LW}} + \underbrace{\frac{k_i}{h}\,(T_f - T_s)}_{\text{conduction}} + \underbrace{C_b\,(T_a - T_s)}_{\text{turbulent}} = 0, \tag{11.3}$$

where α is the surface albedo, $S^{\downarrow}$ and $L^{\downarrow}$ are the downward shortwave and longwave fluxes [$\mathrm{W\,m^{-2}}$], $\varepsilon = 0.97$ is the emissivity, $\sigma = 5.67 \times 10^{-8}\,\mathrm{W\,m^{-2}\,K^{-4}}$ the Stefan–Boltzmann constant, $T_0 = 273.15\,\mathrm{K}$, T_a the air temperature, and $k_i = 2.03\,\mathrm{W\,m^{-1}\,K^{-1}}$ the conductivity of ice. The conduction term is the zero-layer heart of the model: it linearises the temperature profile across the slab between the freezing ocean at the base (T_f) and the radiating skin (T_s), so the conductive flux is simply $k_i\,(T_f - T_s)/h$.

The turbulent (sensible + latent) fluxes are bulk-linearised to $C_b(T_a - T_s)$ with a lumped exchange coefficient $C_b = 10\,\mathrm{W\,m^{-2}\,K^{-1}}$. This is a deliberate simplification: it absorbs the sensible and latent heat into one temperature-difference term and omits the wind-speed and humidity dependence a full bulk formula carries (contrast the coupler bulk fluxes in Chapter 13).

Albedo. The albedo is not a step function but a doubly-smooth sigmoid blend, chosen for differentiability (see Chapter 3). Writing $s(x) = (1 + e^{-x})^{-1}$ for the logistic function,
↪ *Code:* `chronos_esm/ice/thermodynamics.py` — `compute_albedo`

$$m = s\!\left(\frac{T_s + 0.5}{0.5}\right), \qquad \alpha_{\mathrm{ice}} = (1-m)\,\alpha_i + m\,\alpha_m, \tag{11.4}$$

$$f_{\mathrm{thin}} = s\!\left(\frac{0.5 - h}{0.1}\right), \qquad \alpha = (1 - f_{\mathrm{thin}})\,\alpha_{\mathrm{ice}} + f_{\mathrm{thin}}\,\alpha_{\mathrm{ocean}}, \tag{11.5}$$

with $\alpha_i = 0.7$ (cold bare ice), $\alpha_m = 0.5$ (melting ice/melt ponds), and $\alpha_{\mathrm{ocean}} = 0.1$ (open water). The first sigmoid melts the albedo down as $T_s \to 0\,^{\circ}\mathrm{C}$ over a 0.5 K width; the second blends toward the ocean value for thin ice ($h \lesssim 0.5\,\mathrm{m}$), so a thin, broken cover is not given the full reflectivity of a thick floe. Both transitions are smooth and therefore differentiable—a hard threshold would zero the gradient $\partial\alpha/\partial h$ used by the adjoint applications of Chapter 15.

Solving for T_s. Equation (11.3) is nonlinear in T_s through the quartic longwave term, so the code takes a fixed number of Newton steps—three—rather than iterating to a tolerance. A fixed step count keeps the solver a static, differentiable computational graph (JAX can trace and back-propagate through it). With $F'_{\mathrm{net}}(T_s) = -4\varepsilon\sigma(T_s + T_0)^3 - k_i/h - C_b$,
↪ *Code:* `chronos_esm/ice/thermodynamics.py` — `solve_surface_temp`

$$T_s^{(n+1)} = T_s^{(n)} - \frac{F_{\mathrm{net}}(T_s^{(n)})}{F'_{\mathrm{net}}(T_s^{(n)}) - 10^{-5}}, \qquad n = 0, 1, 2, \tag{11.6}$$

seeded with $T_s^{(0)} = T_a$. The -10^{-5} guard keeps the denominator away from zero. The thickness entering k_i/h is floored at $h_{\mathrm{safe}} = \max(h,\, 0.1\,\mathrm{m})$ to avoid the singular conduction of vanishing ice. Finally the skin temperature is capped at the melting point,

$$T_s \leftarrow \min(T_s,\ 0\,^{\circ}\mathrm{C}), \tag{11.7}$$

and F_{net} is re-evaluated at the capped T_s. This cap is the mechanism that distinguishes growth from melt: if the radiative balance would drive $T_s > 0$, the surface cannot warm past melting, and the leftover positive F_{net} becomes melt energy (Section 11.3).

Remark. Three Newton steps from $T_s^{(0)} = T_a$ is usually enough because the air temperature is a good first guess and the quartic is gently curved over the relevant range. It is *not* guaranteed to converge under extreme forcing; the melt cap (11.7) and the downstream clamps (Section 11.3) provide the safety net that an exact solve would otherwise give. This trades a little physical fidelity for a fixed, fast, differentiable graph.

11.3 Ice growth and melt

The slab thickness changes through three pathways, all phrased as a Stefan condition—heat exchanged at a phase boundary converts to a thickness rate via the latent heat of fusion $L_f = 3.34 \times 10^5\,\mathrm{J\,kg^{-1}}$ and ice density $\rho_i = 917\,\mathrm{kg\,m^{-3}}$:

$$\frac{\mathrm{d}h}{\mathrm{d}t} = \underbrace{\frac{k_i(T_f - T_s)/h_{\mathrm{safe}}}{\rho_i L_f}}_{\text{bottom growth}} - \underbrace{\frac{Q_o}{\rho_i L_f}}_{\text{bottom melt}} - \underbrace{\frac{\max(F_{\mathrm{net}}, 0)}{\rho_i L_f}}_{\text{top melt}}. \tag{11.8}$$

Each term has a clear physical reading.

Bottom (congelation) growth. When the surface is cold ($T_s < 0$) the energy balance is satisfied and $F_{\mathrm{net}} \approx 0$; the ice grows from below because the conductive flux $k_i(T_f - T_s)/h$ carries the ocean's latent heat away through the slab to the cold sky. Thicker ice conducts less, so growth self-limits as h^{-1}—the classic $\sqrt{t}$ thickening law of Stefan.

Top melt. When the radiative balance would push T_s above 0°C, the cap (11.7) holds it at melting and the residual $\max(F_{\mathrm{net}}, 0) > 0$ is spent melting ice from the top.

Bottom melt by ocean heat. If the ocean mixed layer is above freezing, it delivers heat to the ice base. Chronos-ESM parameterises this ocean heat flux as a relaxation of the mixed layer's excess temperature toward T_f,
↪ *Code:* `chronos_esm/ice/driver.py` — ocean heat flux, `step_ice`

$$Q_o = \frac{\rho_w c_w\, \max(T_{\mathrm{sst}} - T_f, 0)\, H_{\mathrm{ml}}}{\tau_{\mathrm{mix}}}, \tag{11.9}$$

with $\rho_w = 1025\,\mathrm{kg\,m^{-3}}$, $c_w = 3985\,\mathrm{J\,kg^{-1}\,K^{-1}}$, mixed-layer depth $H_{\mathrm{ml}} = 50\,\mathrm{m}$, and exchange timescale $\tau_{\mathrm{mix}} = 10\,\mathrm{days}$. This yields $O(10\,\mathrm{W\,m^{-2}})$ for a few tenths of a degree of excess heat, in the observed polar range, but the fixed H_{ml} and τ_{mix} are crude—there is no prognostic mixed layer here.

Discrete update. The driver integrates Eq. (11.8) with one explicit Euler step at the ocean timestep $\Delta t = 900\,\mathrm{s}$ (`DT_OCEAN`):

$$h^{n+1} = h^n + \Delta t\, \frac{\mathrm{d}h}{\mathrm{d}t}. \tag{11.10}$$

11.4 Frazil: new-ice formation

Where there is no ice yet but the ocean is supercooled ($T_{\mathrm{sst}} < T_f$), Chronos-ESM forms new ice (*frazil*) by converting the heat deficit of the mixed layer directly into thickness. The supercooling is clamped to 2 K before conversion, then the new ice is rate-limited to 0.1 m per step,
↪ *Code:* `chronos_esm/ice/driver.py` — frazil block, `step_ice`

$$\Delta T_{sc} = \min\big(\max(T_f - T_{\mathrm{sst}}, 0),\ 2\,\mathrm{K}\big), \tag{11.11}$$

$$h_{\mathrm{new}} = \min\left(\frac{\rho_w c_w\, \Delta T_{sc}\, H_{\mathrm{ml}}}{\rho_i L_f},\ 0.1\,\mathrm{m}\right), \tag{11.12}$$

which is added to h^{n+1}. The two limiters are numerical hygiene, not physics: without them a single cold step over a 50 m mixed layer could dump several metres of ice instantaneously (a 5 K deficit $\times$ 50 m is $\sim$ 6 m of ice), destabilising the coupled run. The comments in the source record that the supercooling clamp was tightened from 5 K to 2 K for exactly this reason. The trade-off is honest: real frazil in a strongly supercooled lead would form faster than 0.1 m/900 s, so the model lags rapid opening events.

After growth, melt, and frazil, the thickness is clamped to a physically realistic ceiling and floor,

$$h^{n+1} \leftarrow \max\big(\min(h^{n+1},\ 5\,\mathrm{m}),\ 0\big), \tag{11.13}$$

and masked to ocean cells ($h \leftarrow h \cdot \mathrm{mask}$, land = 0). The 5 m cap is a guard against an earlier failure mode in which unbounded conductive growth produced $\sim$ 39 m ice; 5 m is at the upper end of multiyear Arctic ice. Capping thickness this way suppresses pressure ridges and the thickest deformed ice, a known low bias of the scheme.

11.5 Concentration

The zero-layer model carries no ice dynamics, so concentration is not advected or ridged; it is diagnosed from thickness by a smooth saturating map,
↪ *Code:* `chronos_esm/ice/driver.py` — concentration, `step_ice`

$$A = \tanh\left(\frac{h}{0.5\,\mathrm{m}}\right). \tag{11.14}$$

A cell with $h \gtrsim 1\,\mathrm{m}$ is essentially fully covered ($A \to 1$), while thin ice gives partial cover. This is a Hibler-style parameterisation in spirit but far simpler: there is no separate open-water (lead) fraction equation and no thermodynamic redistribution. Because A is a monotone function of h, the model cannot represent thick, low-concentration fields (broken pack) or thin, fully consolidated new ice. Treat A here as a diagnostic of how "complete" the cover is, not an independent dynamical variable.

11.6 How ice modifies the ocean–atmosphere fluxes

The reason a thermodynamic ice model matters to the rest of Chronos-ESM is the heat and freshwater it hands to the ocean. `step_ice` returns the pair $(Q_{\mathrm{ocn}}, F_{\mathrm{fw}})$.

Heat flux to the ocean. Two ice processes exchange heat with the ocean below:
↪ *Code:* `chronos_esm/ice/driver.py` — flux block, `step_ice`

$$Q_{\mathrm{ocn}} = \underbrace{-\frac{k_i(T_f - T_s)}{h_{\mathrm{safe}}}}_{\text{conductive extraction}} + \underbrace{\frac{h_{\mathrm{new}}\,\rho_i L_f}{\Delta t}}_{\text{frazil latent release}}. \tag{11.15}$$

Congelation growth *removes* heat from the ocean–ice interface (the first, negative term: the ocean must give up latent heat to freeze), whereas frazil formation *releases* latent heat back to the mixed layer as the supercooled water freezes (the second, positive term). The sign convention is that Q_{ocn} is energy delivered to the ocean. Crucially, while ice is present the ocean is insulated from the atmospheric radiative and turbulent fluxes—that masking is applied in the coupler (Chapter 13), and Eq. (11.15) is the *only* thermal channel left open between ocean and the surface there. This is the insulation feedback in code form.

Freshwater flux. Ice growth concentrates the seawater it leaves behind (brine rejection raises ocean salinity); melt does the reverse. The model expresses this as a freshwater flux proportional to the thickness tendency,

$$F_{\mathrm{fw}} = -\,\rho_i\,\frac{h^{n+1} - h^n}{\Delta t} \qquad [\mathrm{kg\,m^{-2}\,s^{-1}}]. \tag{11.16}$$

Growth (h increasing) gives $F_{\mathrm{fw}} < 0$: freshwater is locked into the ice, leaving the ocean saltier. Melt (h decreasing) gives $F_{\mathrm{fw}} > 0$: meltwater freshens the surface. This coupling to the salinity field is precisely the high-latitude freshwater forcing that the overturning circulation is sensitive to (Chapters 8 and 17).

Remark (Honest accounting note). Equation (13.8) converts *all* of the net thickness change—including the frazil term (11.12)—into a freshwater flux, while the heat flux (11.15) accounts for frazil latent heat separately. Because frazil freezing also rejects brine, treating its mass as freshwater here is consistent; but note that the scheme uses pure-ice density ρ_i and assumes the ice is salt-free (zero ice salinity). Real sea ice retains ~ 4–$8\,\mathrm{psu}$ of brine, so the model slightly overstates the freshwater (and brine-rejection) signal per metre of ice. The bias is second-order for the energy budget but relevant for precise salinity tendencies.

11.7 Summary of the time step

Per ocean step Δt, `step_ice` executes, in order: (1) solve the surface energy balance for T_s by three Newton iterations and cap at melting [Eqs. (11.3)–(11.7)]; (2) form the bottom-growth, top-melt, and ocean-heat-melt terms and Euler-step the thickness [Eq. (11.8)]; (3) add rate-limited frazil where the ocean is supercooled [Eq. (11.12)]; (4) clamp thickness to $[0, 5]\,\mathrm{m}$ and mask land [Eq. (11.13)]; (5) diagnose concentration [Eq. (11.14)]; and (6) return the ocean heat and freshwater fluxes [Eqs. (11.15)–(13.8)]. The scheme is deliberately minimal: it omits snow, internal heat storage, ice dynamics/advection, ridging,

Table 11.1: Physical constants in the Chronos-ESM sea-ice thermodynamics (`thermodynamics.py`, `driver.py`).

Symbol	Value	Meaning
ρ_i	$917\,\mathrm{kg\,m^{-3}}$	ice density
L_f	$3.34\times10^{5}\,\mathrm{J\,kg^{-1}}$	latent heat of fusion
k_i	$2.03\,\mathrm{W\,m^{-1}\,K^{-1}}$	ice thermal conductivity
T_f	$-1.8\,^{\circ}\mathrm{C}$	freezing point of seawater
ε	0.97	ice emissivity
C_b	$10\,\mathrm{W\,m^{-2}\,K^{-1}}$	bulk turbulent coefficient
$\alpha_i, \alpha_m, \alpha_{\mathrm{ocean}}$	0.7, 0.5, 0.1	cold/melt/ocean albedo
τ_{mix}	10 days	ice–ocean exchange timescale
H_{ml}	50 m	assumed mixed-layer depth

a prognostic lead fraction, and a salinity-dependent freezing point. Within those limits it faithfully reproduces the two feedbacks that make sea ice climatically important—albedo and insulation—and remains end-to-end differentiable.

Exercises

Exercise 11.1 (The Stefan thickening law)**.** Set the ocean heat flux $Q_o = 0$ and the surface fluxes so that $F_{\mathrm{net}} = 0$ (perpetual polar night, T_s fixed at some $T_s < T_f$ by strong longwave cooling). Equation (11.8) then reduces to $\dot{h} = k_i(T_f - T_s)/(\rho_i L_f\, h)$. Integrate analytically to show $h(t) = \sqrt{2k_i(T_f - T_s)t/(\rho_i L_f)}$. Now run `step_ice` in a single-column driver with constant forcing and $h^0 = 0.1\,\mathrm{m}$, $T_s = -20^{\circ}\mathrm{C}$, and compare the simulated $h(t)$ to the $\sqrt{t}$ law. At what thickness does the 5 m clamp [Eq. (11.13)] start to distort the curve, and how many days does that take?

Exercise 11.2 (Albedo feedback and differentiability)**.** Using Eq. (12.20), plot α as a function of h at fixed $T_s = -5^{\circ}\mathrm{C}$ and as a function of T_s at fixed $h = 2\,\mathrm{m}$. Then use JAX's `jax.grad` on `compute_albedo` to obtain $\partial\alpha/\partial h$ and $\partial\alpha/\partial T_s$. Where are the gradients largest, and why would a hard ($\tanh \to$ step) threshold break the adjoint applications of Chapter 15?

Exercise 11.3 (The insulation feedback in the coupled model)**.** Insulation enters through the conduction term k_i/h. Run a short coupled simulation, then artificially *double* k_i in `thermodynamics.py` and rerun from the same initial state. Predict, then verify, the sign of the change in (i) equilibrium ice thickness, (ii) the conductive heat flux to the ocean Q_{ocn} [Eq. (11.15)], and (iii) high-latitude ocean convection. Explain why more conductive ice can paradoxically be *thinner* ice.

Exercise 11.4 (Brine rejection and the overturning)**.** The freshwater flux (13.8) ties sea-ice growth to ocean salinity. Estimate the brine flux from freezing 1 m of ice over a winter in a 3.75° T31 polar cell, expressing it as an equivalent salinity tendency for a 50 m mixed layer. Then discuss qualitatively how the zero-ice-salinity assumption (Remark, Section 6) biases this estimate, and connect it to the high-latitude freshwater sensitivity of the AMOC in Chapter 17.

Chapter 12

The Land Surface

The land surface in Chronos-ESM is deliberately the simplest of the three media. Where the ocean (4) carries a full three-dimensional primitive-equation state and the atmosphere (9) carries a spectral dynamical core, land is a *single-layer slab*: one prognostic skin temperature, one column of soil water held in a leaky bucket, a prognostic leaf-area index, a soil-carbon pool, and a snow reservoir. There is no horizontal transport within the land — each grid column is an independent "tile" — and the only land-to-land coupling is through the shared atmosphere above it. This is the classic *bucket model* of Manabe (1969), dressed with a dynamic-vegetation albedo and a one-bucket snow scheme. Its purpose is to close the surface energy and water budgets seen by the atmosphere, not to be a land-carbon-cycle laboratory; we are explicit below about where the simplifications bite.

The implementation is a single JAX, fully differentiable time step,

↪ *Code:* `chronos_esm/land/driver.py` — `step_land`

, with the vegetation sub-step in

↪ *Code:* `chronos_esm/land/vegetation.py` — `step_vegetation`

.

12.1 State variables and timescales

The land state is the tuple

$$\mathcal{L} = \big(T_\mathrm{s},\ W,\ L,\ C_\mathrm{s},\ h_\mathrm{sn}\big), \tag{12.1}$$

namely the surface temperature T_s [K], the soil-moisture depth W [m] (bounded by the bucket capacity $W_\mathrm{max} = 0.15$ m), the leaf-area index L [$\mathrm{m^2\,m^{-2}}$], a soil-carbon density C_s [$\mathrm{kg\,C\,m^{-2}}$], and a snow water-equivalent depth h_sn [m]. The component is integrated with the *coupling interval* as its time step (the daily coupling cadence, passed as `dt`), not the atmospheric dynamics step — a deliberate choice that exploits the long thermal and hydrological memory of the slab.

Remark. The slab has a single thermal layer of depth $d = 1$ m, soil density $\rho_\mathrm{s} = 1600\ \mathrm{kg\,m^{-3}}$ and heat capacity $c_\mathrm{s} = 800\ \mathrm{J\,kg^{-1}\,K^{-1}}$, giving a column heat capacity $\mathcal{C}_0 = \rho_\mathrm{s} c_\mathrm{s} d \approx 1.28 \times 10^6\ \mathrm{J\,m^{-2}\,K^{-1}}$. The thermal depth was raised from typical $\sim$0.1 m bucket values to 1 m *for numerical stability of the semi-implicit step*, which makes the diurnal response unphysically sluggish — acceptable here because the model is run at daily coupling and validated against monthly/annual climatology, not the diurnal cycle.

12.2 Surface energy balance

12.2.1 Net radiation

Land absorbs shortwave according to its albedo and emits longwave as a grey body. With downwelling shortwave $S^\downarrow$ and longwave $L^\downarrow$ supplied by the atmosphere,

$$S_\mathrm{net} = (1 - \alpha)\, S^\downarrow, \tag{12.2}$$

$$L_\mathrm{net} = L^\downarrow - \varepsilon\, \sigma\, T_\mathrm{s}^4, \tag{12.3}$$

$$R_\mathrm{n} = S_\mathrm{net} + L_\mathrm{net}, \tag{12.4}$$

where σ is the Stefan–Boltzmann constant, ε the land emissivity, and α the (dynamic) surface albedo of Section 12.5. See
↪ *Code:* `chronos_esm/land/driver.py` — net-radiation block in `step_land`
.

12.2.2 Turbulent fluxes

Sensible and latent heat are bulk-aerodynamic. With air density $\rho_a = 1.225\ \mathrm{kg\,m^{-3}}$, $c_p = 1004\ \mathrm{J\,kg^{-1}\,K^{-1}}$, wind speed U, and a drag coefficient $C_h = C_e$ supplied by the coupler (which folds in the surface-roughness logic),

$$H = \rho_a\, c_p\, C_h\, U\,(T_\mathrm{s} - T_\mathrm{a}). \tag{12.5}$$

The latent flux uses a Monteith-style saturation deficit. Saturation specific humidity is built from the August–Roche–Magnus vapour-pressure formula at the surface pressure $p_0 = 101325$ Pa,

$$e_s(T_\mathrm{s}) = 611\,\exp\!\Big[\frac{17.67\,(T_\mathrm{s} - 273.15)}{(T_\mathrm{s} - 273.15) + 243.5}\Big], \qquad q_\mathrm{sat} = 0.622\,\frac{e_s}{p_0}. \tag{12.6}$$

The actual evaporation is the *potential* evaporation throttled by a soil- and vegetation-dependent moisture availability β:

$$E_\mathrm{pot} = \max\!\big[\rho_a\, C_e\, U\,(q_\mathrm{sat} - q_a),\ 0\big], \qquad E = \beta\, E_\mathrm{pot}, \qquad L_v E = L_v\, E, \tag{12.7}$$

with the latent heat of vapourisation $L_v = 2.5 \times 10^6\ \mathrm{J\,kg^{-1}}$ (sublimation over snow is treated with the same L_v, a known $\sim 13\%$ underestimate of the true sublimation enthalpy). The $\max(\cdot, 0)$ forbids dew/condensation. The availability factor is the *bucket beta*, enhanced by vegetation cover and forced to unity over snow:

$$\beta = \mathrm{clip}\Big[\underbrace{\mathrm{clip}\big(\tfrac{W}{W_\mathrm{max}}, 0, 1\big)}_{\beta_\mathrm{bucket}}\,(1 + f_\mathrm{veg}),\ 0,\ 1\Big], \qquad \beta \leftarrow (1 - c_\mathrm{sn})\,\beta + c_\mathrm{sn}\cdot 1, \tag{12.8}$$

where f_veg is the vegetated fraction (Section 12.5) and c_sn the snow-cover fraction. The factor $(1 + f_\mathrm{veg})$ crudely represents transpiration drawing water faster than bare-soil evaporation, but because the result is clipped to $[0, 1]$ it saturates: a fully-vegetated, half-full bucket already evaporates at the potential rate. See
↪ *Code:* `chronos_esm/land/driver.py` — turbulent-flux and β block
.

12.2.3 Ground heat flux and the semi-implicit temperature update

The energy reaching the slab is the residual

$$G = R_\mathrm{n} - H - L_v E, \tag{12.9}$$

and the slab obeys $\mathcal{C}\,\frac{\mathrm{d}T_\mathrm{s}}{\mathrm{d}t} = G$, with the total heat capacity augmented by a snow term, $\mathcal{C} = \mathcal{C}_0 + \rho_\mathrm{sn} c_\mathrm{sn,heat} h_\mathrm{sn}$ ($\rho_\mathrm{sn} = 300$, $c_\mathrm{sn,heat} = 2000$). Because G depends strongly and nonlinearly on T_s (through T_s^4, H and q_sat), an explicit Euler step would need a tiny Δt. The code instead linearises G about the current temperature and takes one *semi-implicit* (backward-Euler-in-the-linearised-source) step. Writing $G(T_\mathrm{s} + \Delta T_\mathrm{s}) \approx G + G'\,\Delta T_\mathrm{s}$ with

$$G' = \frac{\partial G}{\partial T_\mathrm{s}} = \underbrace{-4\varepsilon\sigma T_\mathrm{s}^3}_{\text{longwave}} \underbrace{-\,\rho_a c_p C_h U}_{\text{sensible}} \underbrace{-\,\rho_a L_v C_e U\,\beta\,\frac{\partial q_\mathrm{sat}}{\partial T_\mathrm{s}}}_{\text{latent}}, \qquad \frac{\partial q_\mathrm{sat}}{\partial T_\mathrm{s}} = q_\mathrm{sat}\,\frac{17.67 \cdot 243.5}{(T_\mathrm{s} - 273.15 + 243.5)^2}, \tag{12.10}$$

the implicit balance $\mathcal{C}\,\Delta T_\mathrm{s}/\Delta t = G + G'\Delta T_\mathrm{s}$ gives the increment actually coded,

$$\boxed{\Delta T_\mathrm{s} = \frac{G}{\mathcal{C}/\Delta t - G'}} \tag{12.11}$$

Since $G' < 0$ the denominator is always positive and larger than the explicit $\mathcal{C}/\Delta t$, so the step is unconditionally damped — this is what permits the long Δt. The update $T_\mathrm{s} \mapsto T_\mathrm{s} + \Delta T_\mathrm{s}$ is applied only on land (multiplied by the land mask). See
↪ *Code:* `chronos_esm/land/driver.py` — semi-implicit temperature update
.

12.3 Snow and the phase-change correction

Precipitation is partitioned by the *air* temperature: it is snow where $T_{\mathrm{a}} < 273.15$ K and rain otherwise (a hard threshold, no mixed phase). After the temperature update, any column that is both warm ($T_{\mathrm{s}} > 273.15$) and snow-covered melts. The available melt energy $(T_{\mathrm{s}} - 273.15)\,\mathcal{C}$ is converted to a melt depth through the latent heat of fusion $L_f = 3.34 \times 10^5$ J kg^{-1},

$$\Delta h_{\mathrm{melt}} = \min\Big[\max\big(0,\ (T_{\mathrm{s}} - 273.15)\,\mathcal{C}/L_f/\rho_w\big),\ h_{\mathrm{sn}}\Big], \tag{12.12}$$

and the slab temperature is corrected *downward* by the latent heat consumed, $T_{\mathrm{s}} \mapsto T_{\mathrm{s}} - \Delta h_{\mathrm{melt}}\,\rho_w L_f/\mathcal{C}$, which pins a still-snowy column to the freezing point. The snow reservoir evolves as

$$h_{\mathrm{sn}}^{\,n+1} = \max\big[0,\ h_{\mathrm{sn}}^{\,n} + \Delta t\,P_{\mathrm{snow}} - \Delta h_{\mathrm{melt}}\big]. \tag{12.13}$$

A final safety clamp $T_{\mathrm{s}} \in [180, 340]$ K guards the differentiable step against transient overshoots. See
↪ *Code:* `chronos_esm/land/driver.py` — snow/melt block
.

12.4 Bucket hydrology and runoff

The soil-water column is a leaky bucket. Inflow is rain plus snowmelt (the melt depth is converted from a per-step amount back to a rate $\Delta h_{\mathrm{melt}}/\Delta t$); the only sink is evaporation $E_{\mathrm{soil}} = E/\rho_w$:

$$\frac{\mathrm{d}W}{\mathrm{d}t} = \big(P_{\mathrm{rain}} + \Delta h_{\mathrm{melt}}/\Delta t\big) - E/\rho_w. \tag{12.14}$$

After the explicit update $W^* = W + \Delta t\,\frac{\mathrm{d}W}{\mathrm{d}t}$ the bucket is capped at $W_{\max}$, and *everything above the rim is runoff*:

$$R = \max(W^* - W_{\max},\ 0), \qquad\qquad W^{\,n+1} = \mathrm{clip}(W^*,\ 0,\ W_{\max}). \tag{12.15}$$

This is instantaneous saturation-excess runoff: there is no infiltration delay, no sub-surface drainage, and no base-flow recession. Dry-end behaviour is similarly simple — the floor at 0 silently discards any negative residual, a small non-conservation that is negligible at the validated drift levels. See
↪ *Code:* `chronos_esm/land/driver.py` — soil-moisture update and runoff
.

12.4.1 Routing runoff to the coastal ocean

Runoff must reach the ocean or the global water budget leaks. The coupler does not solve a river network; it performs a one-step *nearest-neighbour spread* from each land cell to its four neighbours, depositing only into ocean cells. With the overflow expressed as a freshwater flux $r = R\,\rho_w/\Delta t$ [kg m^{-2} s^{-1}] masked to land, the spread is

$$r_{\mathrm{spread}} = \tfrac{1}{4}\big(r_{i-1,j} + r_{i+1,j} + r_{i,j-1} + r_{i,j+1}\big), \qquad r_{\mathrm{ocean}} = r_{\mathrm{spread}} \cdot m_{\mathrm{ocean}}, \tag{12.16}$$

and r_{ocean} is added to the ocean's freshwater forcing. The longitude axis is periodic (a true `roll`); the latitude axis uses *zero-filled* shifts so polar land cannot wrap runoff from the south-pole row into the north-pole row. See
↪ *Code:* `chronos_esm/main.py` — runoff routing in the coupled step
.

Remark. This scheme conserves freshwater *only* for coastal land. Runoff generated in a deep continental interior — whose four neighbours are all land — is spread one cell, masked to ocean, and the part landing back on land is discarded. At T31 most large land masses have such interiors, so a fraction of terrestrial runoff never reaches the ocean. A genuine flow-accumulation router (downhill drainage to a single coastal mouth) is the natural upgrade; it is not yet implemented.

12.5 Vegetation and dynamic albedo

Leaf-area index is prognostic and follows a logistic growth law with temperature- and moisture-limited growth competing against senescence:

$$\frac{\mathrm{d}L}{\mathrm{d}t} = \underbrace{r\,g\,L\Big(1-\frac{L}{L_{\max}}\Big)}_{\text{logistic growth}} + \underbrace{s\,g}_{\text{seed}} - \underbrace{\Big(\frac{1}{\tau_d}+\frac{1-\beta_{\text{bucket}}}{\tau_d/2}\Big)L}_{\text{decay + drought stress}}, \tag{12.17}$$

with $L_{\max} = 6$, growth rate $r = 1/\tau_g$ ($\tau_g = 10$ d), decay timescale $\tau_d = 30$ d, and a tiny seed term ($s = 10^{-8}$) so that bare desert can re-green when conditions improve. The growth potential $g \in [0,1]$ multiplies a Gaussian temperature window centred on $T_{\text{opt}} = 298.15$ K (width 15 K, hard-cut to zero below 278.15 K) with the moisture factor β_{bucket}:

$$g(T_{\text{s}}, \beta_{\text{bucket}}) = \beta_{\text{bucket}}\,\exp\Big[-\frac{(T_{\text{s}}-T_{\text{opt}})^2}{2\cdot 15^2}\Big]\cdot \mathbb{1}[T_{\text{s}} \geq 278.15]. \tag{12.18}$$

The result is clipped to $[L_{\min}, L_{\max}] = [0.1, 6]$. Note this growth sub-step uses the atmospheric step `DT_ATMOS` internally even though the thermodynamic step uses the coupling interval — an inconsistency that makes the vegetation timescales effectively slower than the nominal τ_g, τ_d; it is harmless for climate equilibria but worth knowing before tuning. See

↪ *Code:* `chronos_esm/land/vegetation.py` — `step_vegetation`

.

LAI controls albedo through a Beer–Lambert canopy-closure factor. The vegetated fraction and the area-weighted albedo are

$$f_{\text{veg}} = 1 - e^{-kL}, \quad k = 0.5, \qquad\qquad \alpha_{\text{veg}} = f_{\text{veg}}\,\alpha_{\text{leaf}} + (1-f_{\text{veg}})\,\alpha_{\text{soil}}, \tag{12.19}$$

with a dark canopy $\alpha_{\text{leaf}} = 0.12$ over bright bare soil $\alpha_{\text{soil}} = 0.35$. The snow-cover fraction then over-rides this, $c_{\text{sn}} = \text{clip}(h_{\text{sn}}/0.05, 0, 1)$ (full cover at 5 cm), giving the final albedo fed back to Eq. (12.2),

$$\alpha = (1-c_{\text{sn}})\,\alpha_{\text{veg}} + c_{\text{sn}}\,\alpha_{\text{snow}}, \qquad \alpha_{\text{snow}} = 0.80. \tag{12.20}$$

This closes a genuine *albedo feedback*: cooling promotes snow, snow raises α, which cools further — the same positive loop that drives high-latitude sensitivity in full models, here in its barest form. See

↪ *Code:* `chronos_esm/land/vegetation.py` — `compute_land_properties`

.

Example 12.1 (Why the surface is so reflective by default)**.** At $L = 1$ (the initial sparse state of `init_land_state`), $f_{\text{veg}} = 1 - e^{-0.5} \approx 0.39$, so $\alpha_{\text{veg}} = 0.39\cdot 0.12 + 0.61\cdot 0.35 \approx 0.26$. Even mature forest ($L = 6$) gives $f_{\text{veg}} \approx 0.95$ and $\alpha_{\text{veg}} \approx 0.13$. Bare/desert tiles therefore sit near $\alpha = 0.35$ — bright, which is realistic for deserts but means any column that fails to green stays cold, reinforcing the limit on growth.

12.6 Soil carbon: a passive diagnostic

A single-pool soil-carbon budget is integrated for completeness but is not coupled back to the atmosphere's CO_2 (which is prescribed; see 14). Gross primary production is light-use-efficiency limited, autotrophic respiration is half of GPP, and heterotrophic respiration is Q_{10}-temperature-sensitive,

$$\text{GPP} = \text{LUE}\,S^{\downarrow}\,L\,\beta, \qquad R_a = 0.5\,\text{GPP}, \qquad R_h = k\,C_{\text{s}}\,2^{(T_{\text{s}}-T_{\text{ref}})/10}\,\beta, \tag{12.21}$$

$$\frac{\mathrm{d}C_{\text{s}}}{\mathrm{d}t} = (\text{GPP}-R_a) - R_h, \qquad \text{NEE} = R_a + R_h - \text{GPP}, \tag{12.22}$$

with $\text{LUE} = 10^{-9}$, $k = 10^{-8}$, $T_{\text{ref}} = 288.15$ K, $C_{\text{s}} \geq 0$. NEE is returned for diagnostics only. Treat this pool as an indicator of where a carbon cycle *would* respond, not as a calibrated flux. See

↪ *Code:* `chronos_esm/land/driver.py` — carbon-cycle block

.

Process	Treatment in Chronos-ESM
Heat storage	1 m slab, semi-implicit Eq. (12.11); no diurnal cycle
Soil water	single leaky bucket, $W_{\max} = 0.15$ m, instant saturation runoff
Runoff routing	4-neighbour spread to coastal ocean; interiors leak
Snow	one reservoir, T_a phase split, L_v used for sublimation
Vegetation	prognostic LAI (logistic), Beer–Lambert albedo
Carbon	passive single pool, not fed back to atmosphere
Lateral land transport	none (independent tiles)

Table 12.1: The land surface as a closure for the atmospheric boundary, not a stand-alone biosphere model.

12.7 Summary of approximations

Exercise 12.1 (The bucket time constant)**.** Using Eqs. (12.7)–(12.14), linearise the bucket about a half-full state and estimate the e-folding drying time of W under constant potential evaporation with no precipitation. Take E_{pot} for a 5 $\text{m}\,\text{s}^{-1}$ wind, $C_e = 1.5 \times 10^{-3}$, and a 0.5% humidity deficit. Compare your estimate to $W_{\max}/E_{\text{soil}}$ and explain why the $(1 + f_{\text{veg}})$ enhancement in β does *not* shorten it once β saturates at 1.

Exercise 12.2 (Snow–albedo feedback as a laboratory)**.** Initialise a high-latitude column at $T_s = 274$ K with $h_{\text{sn}} = 0$ and step `step_land` forward with a fixed weak $S^{\downarrow}$ and a cold $T_a = 270$ K. Show numerically that the column transitions to a snow-covered, high-albedo state and that the transition is hysteretic: once snowed in (Eq. (12.20) pins $\alpha = 0.80$ above 5 cm), a larger $S^{\downarrow}$ is needed to melt out than was needed to snow in. Relate this to the bistability ideas of 17.

Exercise 12.3 (Closing the freshwater budget)**.** Build a synthetic continent (a block of land cells with a land interior) and verify by direct summation over Eq. (12.16) that the freshwater delivered to the ocean is *less* than the total runoff R generated, by the fraction of runoff produced in interior cells. Sketch the modification needed to make `step_land`'s runoff routing globally conservative (hint: accumulate interior runoff downstream rather than discarding it).

Exercise 12.4 (Differentiable land calibration)**.** The entire land step is written in JAX and is differentiable. Pick a single scalar parameter (e.g. α_{soil} or $W_{\max}$), define a loss as the squared error of seasonal-mean T_s against a target, and use `jax.grad` to compute $\partial\mathcal{L}/\partial\theta$. Comment on which clamps (the $T_s \in [180, 340]$ guard, the $\max(\cdot, 0)$ on evaporation/runoff) produce zero-gradient regions, and how that constrains gradient-based tuning.

Chapter 13

The Coupler and Surface Fluxes

A coupled Earth System Model is, at bottom, a contract between subsystems that otherwise know nothing of one another. The atmosphere does not know what the sea surface temperature (SST) is; the ocean does not know how hard the wind is blowing. The *coupler* is the layer that brokers this exchange: it moves the ocean's surface state up to the atmosphere, runs the atmosphere, computes the fluxes of heat, freshwater and momentum across the air–sea interface from bulk formulae, and hands those fluxes back down to drive the ocean. This chapter documents how Chronos-ESM does this, derived directly from the source.

Two facts about the Chronos-ESM coupler are worth stating up front, because they shape everything below. First, the *active* coupled path is the multi-level DINOSAUR atmosphere coupled to the z-level ocean (Chapters 4 and 9), expressed as a single differentiable JAX function so that the whole coupled step flows through `jax.jit`, `jax.grad` and `jax.lax.scan` (Bradbury et al., 2018). Second, the coupler does *not* conserve energy by construction; it uses a *flux correction* to hold the mean climate against model drift, and a substantial part of this chapter is an honest accounting of what that correction does and what it costs.

13.1 The coupling cycle

A single coupling interval (default one model day) realizes the data flow

$$\mathrm{SST}^{\,\mathrm{lin}} \xrightarrow{\text{regrid}} \mathrm{SST}^{\,\mathrm{Gauss}} \xrightarrow{\text{DINOSAUR}} \text{surface fields}^{\,\mathrm{Gauss}} \xrightarrow{\text{regrid}} \text{bulk fluxes}^{\,\mathrm{lin}} \xrightarrow{\text{ocean scan}} \mathrm{ocean}(t{+}\Delta t), \tag{13.1}$$

where "lin" is the regular latitude–longitude (ocean/land) grid and "Gauss" is the DINOSAUR Gaussian spectral-transform grid. The lower boundary the atmosphere actually sees is not the bare SST but a surface skin temperature that blends ocean, sea ice and land,

$$T_{\mathrm{surf}} = \begin{cases} (1-A)\,T_{\mathrm{sst}} + A\,(T_{\mathrm{ice}} + 273.15) & \text{over ocean,} \\ T_{\mathrm{land}} & \text{over land,} \end{cases} \tag{13.2}$$

with A the (lagged) sea-ice concentration. Equation (13.2) uses the *previous* interval's ice and land skin temperatures: the components are coupled *explicitly* (sequentially), not implicitly, so each subsystem is forced by its partners' state at the start of the interval.

↪ *Code:* `chronos_esm/coupler/dino_step.py` — `DinoCoupledModel._advance` assembles T_{surf} and drives the cycle of Eq. (13.1)

13.2 Bulk aerodynamic surface fluxes

The turbulent exchange of heat, moisture and momentum through the atmospheric surface layer is not resolved at T31; it is *parameterized* by bulk aerodynamic formulae (Vallis, 2017; Large and Yeager, 2004). These express each surface flux as the product of a transfer coefficient, the near-surface wind speed, and an air–sea contrast in the transported quantity.

Let $\mathbf{u}_a = (u_a, v_a)$ be the atmospheric bottom-level wind, T_a and q_a the bottom-level air temperature and specific humidity, and T_s the surface (SST) temperature. The effective surface wind speed used in

the fluxes is floored to represent sub-grid gustiness and capped to suppress a wind–evaporation feedback runaway (WISHE),

$$|\mathbf{u}_a|_{\text{eff}} = \min\big(\max(\sqrt{u_a^2 + v_a^2},\, 1),\, 25\big)\ \text{m s}^{-1}. \tag{13.3}$$

The sensible and latent heat fluxes (positive *upward*, out of the ocean) are

$$Q_{\text{sens}} = \rho_a\, c_{p,a}\, C_H\, |\mathbf{u}_a|_{\text{eff}}\, (T_s - T_a), \tag{13.4}$$

$$Q_{\text{lat}} = \rho_a\, L_v\, C_E\, |\mathbf{u}_a|_{\text{eff}}\, \big(q_{\text{sat}}(T_s) - q_a\big)\, \beta, \tag{13.5}$$

with $\rho_a = 1.2\ \text{kg m}^{-3}$, $c_{p,a} = 1004\ \text{J kg}^{-1}\,\text{K}^{-1}$, $L_v = 2.5 \times 10^6\ \text{J kg}^{-1}$, transfer coefficients $C_H = C_E = 1.2 \times 10^{-3}$, and an evaporation efficiency $\beta = 1$ over the ocean. The saturation specific humidity follows the Clausius–Clapeyron form

$$q_{\text{sat}}(T_s) = \frac{0.622\, e_{\text{sat}}(T_s)}{p_0}, \qquad e_{\text{sat}}(T_s) = 611\, \exp\!\left(\frac{17.27\,(T_s - 273.15)}{(T_s - 273.15) + 237.3}\right)\ \text{Pa}, \tag{13.6}$$

with a fixed reference $p_0 = 101325$ Pa. The fixed p_0 is an honest simplification: it ignores the (modest) dependence of q_{sat} on actual surface pressure, an error of at most a few percent away from sea level.
↪ *Code:* `chronos_esm/atmos/physics.py` — `compute_surface_fluxes` implements Eqs. (13.3)–(13.6)

Net surface heat flux. The bulk turbulent fluxes are combined with radiation to form the net heat flux into the ocean,

$$Q_{\text{net}} = \underbrace{(1 - \alpha_o)\, S^{\downarrow}}_{\text{shortwave in}} + \underbrace{0.8\,\sigma\, \max(T_a - 10, 150)^4}_{\text{longwave down}} - \underbrace{0.98\,\sigma\, T_s^4}_{\text{longwave up}} - Q_{\text{sens}} - Q_{\text{lat}}, \tag{13.7}$$

where $\sigma = 5.67 \times 10^{-8}\ \text{W m}^{-2}\,\text{K}^{-4}$ is the Stefan–Boltzmann constant, α_o is the ocean albedo, and $S^{\downarrow}$ is the daily-mean top-of-atmosphere insolation reduced for cloud/atmospheric absorption (a fixed perpetual day-of-year 80 in the coupled harness, so there is *no seasonal cycle* in the shortwave forcing — a known limitation). The downwelling longwave uses a fixed effective emissivity of 0.8 and a floored air temperature; the upwelling term uses ocean emissivity 0.98. This is a grey, single-band radiation closure — adequate to set the surface energy balance, but not a spectral radiative-transfer scheme.
↪ *Code:* `chronos_esm/coupler/dino_coupling.py` — `ocean_fluxes_jax` assembles Q_{net} from Eq. (13.7)

Freshwater flux. Evaporation is recovered from the latent heat flux, $E = Q_{\text{lat}}/L_v$, and the surface freshwater flux (precipitation minus evaporation, the ocean's salinity forcing of Chapter 4) is

$$F_w = P - E = P - Q_{\text{lat}}/L_v, \tag{13.8}$$

with P the precipitation diagnosed by the atmosphere's large-scale condensation. Both Q_{net} and F_w are subsequently constrained for global balance (Section 13.4).

13.3 Wind stress

Momentum is transferred to the ocean by the surface wind stress $\boldsymbol{\tau} = (\tau_x, \tau_y)$, computed from the same bulk-drag idea but with its own quadratic form,

$$\tau_x = \rho_a\, C_D\, |\mathbf{u}_a|_w\, u_a, \qquad \tau_y = \rho_a\, C_D\, |\mathbf{u}_a|_w\, v_a, \qquad |\mathbf{u}_a|_w = \max\big(\sqrt{u_a^2 + v_a^2},\, 1\big), \tag{13.9}$$

with drag coefficient $C_D = 1.3 \times 10^{-3}$. Note that the wind speed floor here ($1\ \text{m s}^{-1}$) matches the heat-flux floor but there is *no* $25\ \text{m s}^{-1}$ cap; instead each stress component is hard-clamped,

$$\tau_x, \tau_y \in [-0.3,\, 0.3]\ \text{Pa}. \tag{13.10}$$

The clamp is a numerical safety net: realistic stresses rarely exceed ~ 0.3 Pa, so saturating it flags an instability rather than shaping the climatology. This stress is the boundary forcing of the ocean momentum equation (Chapter 7) and drives the wind-driven gyres and, indirectly, the overturning of Chapter 8.
↪ *Code:* `chronos_esm/coupler/dino_coupling.py` — `ocean_fluxes_jax`, Eqs. (13.9)–(13.10)

13.4 Flux correction: control mode vs. free mode

A coupled model with imperfect components drifts: small biases in the bulk fluxes accumulate into large SST errors over years. Chronos-ESM controls this with a surface-heat *flux correction* on the SST, and it offers two distinct modes selected at construction time. This is the single most important design choice in the coupler, and it has direct consequences for forcing experiments (Chapter 14).

13.4.1 Strong Haney restoring (control mode)

In control mode the net heat flux receives a Haney restoring term (Haney, 1971) toward a fixed target SST field $T^\star$ (the World Ocean Atlas climatology captured at initialization),

$$Q_{\mathrm{corr}} = \lambda\,(T^\star - T_s), \qquad \lambda = \frac{\rho_0\, c_{p,w}\, \Delta z_1}{\tau}, \tag{13.11}$$

where λ is the restoring coefficient, $\rho_0 = 1025\ \mathrm{kg\,m^{-3}}$ and $c_{p,w} = 3985\ \mathrm{J\,kg^{-1}\,K^{-1}}$ are seawater density and heat capacity, Δz_1 is the top model-layer thickness, and τ is the restoring timescale. With the default $\tau = 30$ d this gives $\lambda \approx 50\ \mathrm{W\,m^{-2}\,K^{-1}}$ — a *strong* relaxation that pins the surface to climatology within roughly a month. The corrected net flux that drives the ocean is $Q_{\mathrm{net}} + Q_{\mathrm{corr}}$.

↪ *Code:* `chronos_esm/ocean/flux_correction.py` — `restoring_lambda`, `restoring_flux`, `heat_correction` (`q_flux=None`)

Remark. The strong restoring is what makes the control climatology look good, but it does so by *absorbing* any imposed forcing: if you add CO_2 forcing while $\lambda \approx 50\ \mathrm{W\,m^{-2}\,K^{-1}}$ is acting, an SST anomaly of order $\Delta T = F/\lambda$ develops — a 3.7 $\mathrm{W\,m^{-2}}$ forcing yields only $\sim$0.07 K. Control mode is therefore the wrong tool for climate sensitivity; it is the right tool for spin-up and for diagnosing the implied correction below.

13.4.2 Frozen q-flux + weak anomaly restoring (free mode)

Free mode breaks the suppression without re-opening the cold-tongue drift. The recipe is the classic mixed-layer flux correction:

1. Run a strongly-restored control to equilibrium and accumulate the time-mean restoring flux $\bar{Q} = \langle \lambda\,(T^\star - T_s) \rangle$. This is the *implied* flux correction the restoring was silently supplying to hold the mean state.

2. In the free run, *prescribe* the frozen field $\bar{Q}$ plus only a *weak*, long-τ anomaly restoring:

$$Q_{\mathrm{corr}} = \underbrace{\lambda_{\mathrm{weak}}\,(T^\star - T_s)}_{\text{weak anomaly restoring}} + \underbrace{\bar{Q}}_{\text{frozen q-flux}}, \qquad \tau \sim 3650\ \mathrm{d}. \tag{13.12}$$

The mean state is now held by the fixed field $\bar{Q}$, residual drift is bounded by the weak term, but SST is free to respond to an imposed forcing on the slow anomaly timescale. Crucially the entire correction is pure JAX, so d(climate)/d(forcing) survives it — the differentiability the model is built for is not severed by the flux correction.

↪ *Code:* `chronos_esm/ocean/flux_correction.py` — `heat_correction` with `q_flux` provided, Eq. (13.12)

Example 13.1 (Selecting the mode)**.** A `DinoCoupledModel` constructed with `q_flux=None, restore_tau_days=30` runs in control mode; passing `q_flux=<field>, restore_tau_days=3650` switches to free mode. The same call also balances the *uncorrected* branch: if no target is given, Q_{net} has its area-weighted ocean mean removed so the global ocean neither gains nor loses heat,

$$Q_{\mathrm{net}} \leftarrow Q_{\mathrm{net}} - \frac{\sum_{\mathrm{ocean}} w\, Q_{\mathrm{net}}}{\sum_{\mathrm{ocean}} w}, \qquad w = \cos\phi, \tag{13.13}$$

and the freshwater flux F_w is de-meaned identically so it neither adds nor removes net mass. Finally Q_{net} is clipped to $[-1500, 1500]\ \mathrm{W\,m^{-2}}$ as a stability guard.

↪ *Code:* `chronos_esm/coupler/dino_coupling.py` — `ocean_fluxes_jax` mean-removal and clip

13.5 CO_2 radiative forcing into the ocean heat budget

The model's single anthropogenic forcing channel is CO_2, entered through the Myhre logarithmic radiative-forcing law (Myhre et al., 1998),

$$F_{CO_2}(C) = 5.35 \ln\left(\frac{C}{C_0}\right) \text{ W m}^{-2}, \qquad C_0 = 280 \text{ ppm}, \tag{13.14}$$

which vanishes at the pre-industrial reference C_0 and gives the canonical ≈ 3.7 W m^{-2} at doubling. A deliberate design decision is *where* this enters: F_{CO_2} is added to the ocean surface heat budget as extra downwelling longwave, *before* the flux correction, rather than as an offset to the SST-slaved atmospheric radiative-equilibrium temperature. The latter would be nearly inert (the atmosphere is slaved to SST) and would double-count. Concretely, Eq. (13.7) becomes $Q_{\text{net}} \leftarrow Q_{\text{net}} + F_{CO_2}(C)$ and only *then* receives Q_{corr}. Because F_{CO_2} is differentiable in C, the sensitivity d(climate)/d(CO_2) is directly available for calibration and sensitivity studies (Chapter 14).

↪ *Code:* `chronos_esm/coupler/dino_coupling.py` — `co2_forcing_wm2` (Eq. (13.14)) added in `ocean_fluxes_jax` before the flux correction

Remark. Putting F_{CO_2} before the correction makes the mode choice load-bearing: under the strong control-mode $\lambda \approx 50$ W m^{-2} K^{-1} the response is suppressed to $\sim F/\lambda$; only free mode (Eq. (13.12)) lets F_{CO_2} drive a real warming. The historical forcing record (CO_2 in ppm, plus a small TSI cycle) is supplied by `ForcingManager`, which linearly interpolates annual-mean concentrations between 1850 and 2025.

↪ *Code:* `chronos_esm/forcing.py` — `ForcingManager.get_forcing` interpolates $C(t)$ used in Eq. (13.14)

13.6 Regridding between atmosphere and ocean

The atmosphere and ocean live on different horizontal grids: the DINOSAUR atmosphere uses a Gaussian latitude grid (the natural collocation for the spectral transform), while the ocean and land use a regular latitude–longitude grid. The longitudes coincide, so coupling reduces to a *one-dimensional* interpolation in latitude. Chronos-ESM uses differentiable piecewise-linear interpolation in both directions,

$$f^{\,\text{Gauss}} = \mathcal{I}\big[\phi_{\text{lin}} \to \phi_{\text{Gauss}}\big]\, f^{\,\text{lin}}, \qquad f^{\,\text{lin}} = \mathcal{I}\big[\phi_{\text{Gauss}} \to \phi_{\text{lin}}\big]\, f^{\,\text{Gauss}}, \tag{13.15}$$

implemented with `jnp.interp` vectorized over longitude columns (`jax.vmap`). Using `jnp.interp` rather than a looped `np.interp` is what keeps the regrid differentiable, so gradients flow across the grid boundary in both directions.

↪ *Code:* `chronos_esm/coupler/dino_coupling.py` — `make_regridders_jax` builds $\mathcal{I}$ of Eq. (13.15)

Remark (Linear vs. conservative remapping). The latitudinal interpolation of Eq. (13.15) is *not* flux-conserving: it does not guarantee that the area integral of a flux is preserved across the grid change. A separate dense/sparse matrix remapper exists (`regrid_field`, Dest $= W$ Src) intended for conservative weights, but in the current configuration it short-circuits to identity for collocated grids. The de-meaning step of Section 13.4 (which enforces zero global-mean heat and freshwater on the ocean grid) is what actually guards global balance, not the regrid.

↪ *Code:* `chronos_esm/coupler/regrid.py` — `regrid_field`, `Regridder` (matrix remap, identity for collocated grids)

13.7 Coupling time-stepping: `step` vs. `step_fast`

A coupling interval contains many sub-steps: the atmosphere is advanced with its ImEx integrator (Chapter 9) over n_{atm} steps, and the ocean is advanced over n_{sub} sub-steps via `jax.lax.scan`, where

$$n_{\text{atm}} = \lceil s_{\text{atm}}\, \Delta T_{\text{int}} \rceil, \qquad n_{\text{sub}} = \lceil 86400\, \Delta T_{\text{int}} / \Delta t_{\text{ocean}} \rceil, \tag{13.16}$$

with ΔT_{int} the interval in days, s_{atm} the atmosphere steps-per-day, and Δt_{ocean} the ocean time step. Chronos-ESM exposes the same interval body through two entry points with different performance/differentiability trade-offs:

`step` — the differentiable, variable-interval path. The atmosphere advance and the ocean `scan` are wrapped in gradient checkpointing (`jax.checkpoint`, "remat") so the reverse-mode adjoint fits in memory. This is the path used for `jax.grad` and data assimilation (Chapter 15).

`step_fast` — the forward-only path for long control and forced runs. The *entire* interval (regrid, spectral transforms, atmosphere steps, ocean scan) is fused into one `jax.jit` program at a *fixed* interval, with no remat. Removing the checkpointing lets XLA fuse the program, which the source reports is $\sim$10$\times$ faster than the eager `step`.

Both call the shared `_advance` body of Eq. (13.1); they differ only in the `remat` flag and whether the interval is static.
↪ *Code:* `chronos_esm/coupler/dino_step.py` — `DinoCoupledModel.step` (remat) and `step_fast` (fused jit)

Remark (Honest status). The differentiable `DinoCoupledModel` is the library promotion of the standalone control harness `experiments/run_dino_coupled.py`; it is the `atmos_backend='dino'` model. The legacy single-level coupler in `main.step_coupled` (Chapter 10) is untouched and is the path the older `jax.grad` demos exercise. A second honesty note inherited from the AMOC metric: on the present diagnostic-velocity ocean, d(AMOC)/d(density) = 0, so a CO_2 or hosing perturbation entering through the heat/freshwater budget here does not yet drive overturning gradients — closing that is the prognostic-momentum work of Chapter 7.

13.8 Exercises

Exercise 13.1 (Restoring strength and forcing suppression)**.** Using Eq. (13.11), compute the Haney coefficient λ for restoring timescales $\tau = 10, 30, 90, 365$ days (take Δz_1 from the model's top layer, $\rho_0 = 1025$, $c_{p,w} = 3985$). For each, predict the equilibrium SST anomaly $\Delta T = F/\lambda$ to a doubled-CO_2 forcing $F = 3.7\ \mathrm{W\,m^{-2}}$ (Eq. (13.14)). At what τ does ΔT exceed 1 K? Explain why control mode is unusable for climate sensitivity and free mode (Eq. (13.12)) is required.

Exercise 13.2 (Diagnosing the implied q-flux)**.** Construct a `DinoCoupledModel` in control mode (`q_flux=None`, `restore_tau_days=30`) and integrate to a quasi-steady state with `step_fast`. At each interval accumulate the restoring flux $\lambda\,(T^\star - T_s)$ (Eq. (13.11)) and form its time mean $\bar{Q}$. Map $\bar{Q}$: where is the implied correction largest, and what known SST biases (e.g. the tropical cold tongue) does its sign reveal? Then rebuild the model in free mode passing this $\bar{Q}$ as `q_flux` with `restore_tau_days=3650` and verify the mean SST stays close to the control.

Exercise 13.3 (Wind-stress clamp as an instability detector)**.** The stress components are clamped to $[-0.3, 0.3]$ Pa (Eq. (13.10)). Instrument `ocean_fluxes_jax` to record the fraction of ocean points hitting the clamp each interval during a multi-year run. Argue why a *rising* clamp-hit fraction is a leading indicator of an atmospheric instability rather than a feature of the climatology, and relate it to the 25 $\mathrm{m\,s^{-1}}$ WISHE cap in the heat fluxes (Eq. (13.3)).

Exercise 13.4 (Differentiable CO_2 sensitivity)**.** Because F_{CO_2} (Eq. (13.14)) and the whole coupling interval are pure JAX, the map $C \mapsto \overline{\mathrm{SST}}$ after N intervals is differentiable. In free mode, use `jax.grad` through `step` to compute $\mathrm{d}\,\overline{\mathrm{SST}}/\mathrm{d}C$ at $C = 400$ ppm, and compare it to a finite-difference estimate. Then explain, using the structure of Section 13.5, why the same gradient computed in *control* mode is much smaller, and what that implies for using gradients to calibrate climate sensitivity.

Part III

Forcing, Sensitivity, and Data Assimilation

Chapter 14

Radiative Forcing and Climate Response

A climate model earns its name only if it *responds*: if you push it with an external forcing, the simulated climate must move, and move by an amount you can defend. This chapter is about giving Chronos-ESM that capability for the single most important anthropogenic lever, atmospheric carbon dioxide. We proceed in three steps that mirror the code. First, we convert a CO_2 concentration into a radiative forcing through the Myhre logarithmic law (§14.1). Second, we confront a structural obstacle: the surface flux correction that keeps the control climate from drifting also *absorbs* any forcing we impose, so we must free the ocean surface with the q-flux method (§14.2). Third, we design the abrupt-2×CO_2 experiment as a difference of two branches (§14.3) and read off the response (§14.4). Throughout, the central honesty of the chapter is that what Chronos-ESM produces is a *transient, surface-forcing proxy* for the transient climate response (TCR), not the equilibrium climate sensitivity (ECS); §14.4.1 makes that distinction quantitative.

This chapter builds directly on the coupler and surface-flux budget of Chapter 13; the ocean that does the warming is the prognostic core of Chapter 7, and the overturning circulation that the same forcing can perturb is the subject of Chapter 17.

14.1 The Myhre CO_2 radiative forcing

The radiative effect of a well-mixed greenhouse gas is, to leading order, not linear in its concentration but logarithmic: each *doubling* of CO_2 adds roughly the same forcing because the strong absorption lines saturate and only the band wings contribute further. Myhre et al. (1998) fitted line-by-line radiative-transfer calculations to the compact expression

$$F(C) \;=\; 5.35\,\ln\!\left(\frac{C}{C_0}\right) \qquad [\mathrm{W\,m^{-2}}], \tag{14.1}$$

where C is the CO_2 mixing ratio in parts per million by volume (ppm), $C_0 = 280\,\mathrm{ppm}$ is the pre-industrial reference, and the coefficient $5.35\,\mathrm{W\,m^{-2}}$ carries the radiative physics. By construction $F(C_0) = 0$, and a doubling $C = 2C_0$ gives the canonical

$$F(2C_0) \;=\; 5.35\,\ln 2 \;\approx\; 3.71\,\mathrm{W\,m^{-2}}. \tag{14.2}$$

Chronos-ESM implements Equation (14.1) verbatim as a pure-JAX function, so it is differentiable in C — a property we will exploit when we ask for d(climate)/d(CO_2) in Chapter 15.

```
def co2_forcing_wm2(co2_ppm, co2_ref=280.0):
    return 5.35 * jnp.log(jnp.asarray(co2_ppm) / co2_ref)
```

Listing 14.1: Myhre forcing, the single radiative channel.

↪ *Code:* `chronos_esm/coupler/dino_coupling.py` — `co2_forcing_wm2` — Equation (14.1)

Remark (A single forcing channel, and where it enters). Chronos-ESM injects F at exactly one point: it is *added to the ocean surface heat budget* as an extra downwelling term, alongside the shortwave, longwave,

sensible and latent fluxes assembled in Chapter 13. Writing the net downward surface heat flux of the control as Q_{net}, the forced budget is simply

$$Q_{\text{net}}^{\text{forced}} \;=\; Q_{\text{net}} \;+\; F(C). \tag{14.3}$$

A tempting alternative — shifting the SST-slaved atmospheric radiative equilibrium temperature T_{eq} — would be nearly inert (the single-/multi-level atmosphere relaxes back) and would risk double-counting, so it is deliberately *not* used. The code comment in `co2_forcing_wm2` states this design choice explicitly.

The forcing is added in `ocean_fluxes_jax` *before* the flux-correction term, because — as we now show — that correction is precisely the artifact that would otherwise swallow it.

```
net_heat = sw_net + lw_down - lw_up - sens - lat
if co2_ppm is not None:
    net_heat = net_heat + co2_forcing_wm2(co2_ppm)   # single channel
if sst_target is not None:
    net_heat = net_heat + flux_correction.heat_correction(
        sst_K, sst_target, restore_tau_days, q_flux=q_flux)
```

Listing 14.2: Forcing enters the surface budget before flux correction.

↪ *Code:* `chronos_esm/coupler/dino_coupling.py` — `ocean_fluxes_jax` — forcing injected before the SST flux correction

14.2 The flux-correction obstacle and the q-flux cure

14.2.1 Why strong restoring eats the forcing

The control climate of Chronos-ESM is held against drift by a *Haney restoring* (Haney, 1971) of SST toward the WOA18 climatology $T^{\star}$ (Locarnini et al., 2018). The correction is a linear damping of the SST error,

$$Q_{\text{corr}}(T_{\text{s}}) \;=\; \lambda\,\big(T^{\star} - T_{\text{s}}\big), \qquad \lambda \;=\; \frac{\rho_0\, c_p\, \Delta z_1}{\tau}, \tag{14.4}$$

where λ is the restoring coefficient [$\mathrm{W\,m^{-2}\,K^{-1}}$], $\rho_0 = 1025\,\mathrm{kg\,m^{-3}}$ and $c_p = 3985\,\mathrm{J\,kg^{-1}K^{-1}}$ are seawater density and heat capacity, $\Delta z_1 \approx 50\,\mathrm{m}$ is the thickness of the top ocean layer, and τ is the restoring timescale. The control uses $\tau = 30\,\mathrm{d}$, which gives a *strong*

$$\lambda \;=\; \frac{1025 \times 3985 \times 50}{30 \times 86400} \;\approx\; 79\,\mathrm{W\,m^{-2}\,K^{-1}}. \tag{14.5}$$

↪ *Code:* `chronos_esm/ocean/flux_correction.py` — `restoring_lambda`, `restoring_flux` — Equations (14.4)–(14.5)

Now superimpose a forcing F on this surface budget and ask for the *steady* SST anomaly ΔT_{s} it can sustain. The surface heat balance of the perturbed top layer requires the extra input F to be removed by the restoring it provokes,

$$F \;-\; \lambda\,\Delta T_{\text{s}} \;=\; 0 \quad\Longrightarrow\quad \Delta T_{\text{s}} \;=\; \frac{F}{\lambda}. \tag{14.6}$$

With $F = 3.71\,\mathrm{W\,m^{-2}}$ and $\lambda \approx 79\,\mathrm{W\,m^{-2}\,K^{-1}}$, Equation (14.6) permits only $\Delta T_{\text{s}} \approx 0.047\,\mathrm{K}$ — a forty-seventh of a degree. The strong restoring is, in effect, an infinite heat reservoir clamped to the present climate: it silently supplies whatever flux is needed to hold $T_{\text{s}} \to T^{\star}$ and therefore *absorbs the forcing entirely*. This is the central problem of forced experiments with a flux-corrected model.

14.2.2 The q-flux methodology

The cure is the classical mixed-layer *q-flux* (flux-anomaly correction). The key realization is that the time-mean of the restoring flux over a converged control *is itself the flux correction the model needs* to hold its mean state:

$$\overline{Q}(\phi,\lambda) \;\equiv\; \Big\langle\, \lambda\,\big(T^{\star} - T_{\text{s}}\big)\,\Big\rangle_{t}, \tag{14.7}$$

the angle brackets denoting a time average over the equilibrated control. We *freeze* this field. In the forced ("free") run we then prescribe the fixed $\overline{Q}$ and replace the strong restoring with only a *weak, long-timescale anomaly* restoring:

$$Q_{\mathrm{corr}}^{\mathrm{free}}(T_\mathrm{s}) \;=\; \underbrace{\overline{Q}}_{\text{frozen, holds mean}} \;+\; \underbrace{\lambda_{\mathrm{weak}}\left(T^\star - T_\mathrm{s}\right)}_{\text{bounds drift}}, \qquad \lambda_{\mathrm{weak}} = \frac{\rho_0 c_p \Delta z_1}{\tau_{\mathrm{weak}}}. \tag{14.8}$$

With $\tau_{\mathrm{weak}} = 3650\,\mathrm{d}$ (ten years) the weak coefficient collapses to

$$\lambda_{\mathrm{weak}} \;=\; \frac{1025 \times 3985 \times 50}{3650 \times 86400} \;\approx\; 0.65\,\mathrm{W\,m^{-2}\,K^{-1}}, \tag{14.9}$$

a factor of $\sim$120 weaker than the control restoring. The division of labour is exact: the frozen $\overline{Q}$ carries the mean state (so the cold-tongue drift documented in Chapter 16 does *not* re-open), the weak term merely bounds slow drift, and SST is now *free to respond* to F on the multi-year anomaly timescale. The same single function selects the two modes by the presence or absence of `q_flux`:

```
def heat_correction(sst_K, sst_target_K, restore_tau_days, q_flux=None):
    corr = restoring_flux(sst_K, sst_target_K, restore_tau_days)  # weak or
        strong
    if q_flux is not None:           # FREE mode: add frozen q-flux
        corr = corr + jnp.asarray(q_flux)
    return corr
```

Listing 14.3: One function, two modes: control restoring vs free q-flux.

↪ *Code:* `chronos_esm/ocean/flux_correction.py` — `heat_correction` — Equation (14.8); `q_flux=None` recovers Equation (14.4)

Remark (Differentiability survives the correction). Every operation in Equations (14.1)–(14.8) is expressed in `jnp`, with no `numpy` or `pint` hops. Consequently d(climate)/d(forcing) survives the flux correction: the q-flux is a *constant additive field*, so it contributes nothing to the gradient with respect to C, while the weak restoring contributes a bounded, differentiable damping. This is what makes the forced experiment a legitimate target for the inverse methods of Chapter 15.

14.3 The abrupt-2×CO_2 experiment

The experiment, following the spirit of the CMIP `abrupt-2xCO2` protocol (Eyring et al., 2016), is driven by
↪ *Code:* `experiments/run_dino_co2.py` — the forced-CO_2 driver
. Its design has three deliberate features.

Branching from a common state. Both branches start from the *same* converged control checkpoint and its frozen $\overline{Q}$. We integrate two free-mode trajectories that differ only in the prescribed concentration:

$$\text{baseline:}\quad C = C_0 = 280\,\mathrm{ppm} \qquad \Rightarrow\; F = 0, \tag{14.10}$$

$$\text{forced:}\quad C = 2C_0 = 560\,\mathrm{ppm} \qquad \Rightarrow\; F = 5.35 \ln 2 \approx 3.71\,\mathrm{W\,m^{-2}}. \tag{14.11}$$

Warming as a difference (drift cancellation). The free model still has a small residual drift (the weak restoring does not pin the mean exactly). The CO_2 warming is defined as the *difference* of the two ocean-mean SST trajectories,

$$\Delta T_\mathrm{s}(t) \;=\; T_\mathrm{s}^{\mathrm{forced}}(t) - T_\mathrm{s}^{\mathrm{baseline}}(t), \tag{14.12}$$

because both branches inherit the *same* drift from the same initial state and the same $\overline{Q}$, so subtracting one from the other cancels it to leading order and isolates the forced signal. The ocean-area-mean SST of each branch is

$$\overline{T_\mathrm{s}}(t) \;=\; \frac{\sum_{i\in\mathrm{ocean}} T_\mathrm{s}^{(i)}(t)}{\sum_{i\in\mathrm{ocean}} 1} \;-\; 273.15 \quad [^\circ\mathrm{C}], \tag{14.13}$$

the sum running over the surface ocean grid cells (`omask`); see `_run_branch` / `sst_mean_C` in the driver.

Time horizon. The branches run for ∼50 model years (`-years 50`, the experiment's default), stepping the fast coupled model at the coupling interval and recording $\overline{T_s}$ annually. Fifty years is short relative to the deep-ocean equilibration timescale ($\sim 10^3$ yr), which is exactly why the result is a transient, not an equilibrium, measure — the point of §14.4.1.

Example 14.1 (Two lines, one number)**.** The driver prints the forcing F from Equation (14.2), the end-of-run warming ΔT_s(end) from Equation (14.12), and their ratio $\Delta T_s/F$ in $\mathrm{K\,(W\,m^{-2})^{-1}}$. That ratio is the headline climate-response number, and it is what we interpret next.

14.4 Result and its honest interpretation

A 50-year free-mode abrupt-2×CO_2 run yields a global ocean-mean surface warming of

$$\Delta T_s \;\approx\; +1.58\,\mathrm{K}, \qquad \frac{\Delta T_s}{F} \;\approx\; \frac{1.58}{3.71} \;\approx\; 0.43\,\mathrm{K\,(W\,m^{-2})^{-1}}. \tag{14.14}$$

The model has moved, in the right direction, by a physically plausible amount: the sign is correct, the magnitude is within the order of magnitude of observed estimates (IPCC, 2021), and (crucially) the response is *nonzero*, confirming that the q-flux successfully freed the SST from the restoring clamp of Equation (14.6).

14.4.1 What this number is — and is not

It would be easy, and wrong, to call $0.43\,\mathrm{K\,(W\,m^{-2})^{-1}}$ a climate sensitivity. Three structural limitations make it a *proxy* only, and a textbook must state them plainly.

1. **Transient, not equilibrium.** Fifty years samples only the upper ocean's adjustment; the deep ocean continues to take up heat for centuries. ΔT_s in Equation (14.14) is therefore a TCR-like *transient* response, structurally smaller than the equilibrium climate sensitivity (ECS) the same forcing would eventually produce.

2. **No atmospheric radiative feedback amplification.** The forcing enters as a fixed surface-heat term (Equation (14.3)); there is *no top-of-atmosphere energy closure*. The water-vapour, lapse-rate, cloud and surface-albedo feedbacks that amplify (or damp) the response in a full GCM are simply absent. The number is a *surface-forcing* response, with the net feedback parameter effectively set by the weak restoring λ_{weak} and the ocean's heat uptake rather than by radiative physics. The driver itself prints the caveat "TCR-like proxy, NOT equilibrium ECS".

3. **Cold-tropics base state.** The control climate carries a cold tropical bias (the cold-tongue / dead-ITCZ issue of Chapter 16). Surface fluxes — the longwave $\propto T_s^4$ and the Clausius–Clapeyron-controlled latent flux — are nonlinear in SST, so a response computed about a biased base state is itself biased; the slope in Equation (14.14) should not be read to two significant figures of accuracy.

Proposition 14.1 (The result is a lower-bound-flavoured TCR proxy)**.** *Because the dominant missing ingredients — deep-ocean equilibration time and positive water-vapour/cloud feedbacks — both act to* increase *the eventual warming relative to Equation* (14.14)*, while no comparably large omitted process is known to decrease it, the model's* $0.43\,\mathrm{K\,(W\,m^{-2})^{-1}}$ *is best read as a conservative, transient-flavoured estimate. It is a demonstration that the coupled model* responds correctly in sign and order of magnitude*, not a calibrated sensitivity.*

Remark (Why this is still scientifically useful)*.* A differentiable model whose forced response is honest about its scope is precisely the right laboratory for *sensitivity analysis* and *calibration*. The very next chapter (15) uses the differentiability preserved through the q-flux to compute $\mathrm{d}\Delta T_s/\mathrm{d}C$ by automatic differentiation rather than by finite differencing two 50-year runs — turning the experiment of this chapter into a single gradient evaluation.

Table 14.1: Key quantities of the abrupt-2×CO_2 experiment.

Quantity	Value	Source
Forcing coefficient	$5.35\,\mathrm{W\,m^{-2}}$	Myhre et al. (1998), Eq. (14.1)
$F(2\times \mathrm{CO_2})$	$3.71\,\mathrm{W\,m^{-2}}$	Eq. (14.2)
Control restoring λ (τ=30 d)	$79\,\mathrm{W\,m^{-2}K^{-1}}$	Eq. (14.5)
Free restoring λ_{weak} (τ=3650 d)	$0.65\,\mathrm{W\,m^{-2}K^{-1}}$	Eq. (14.9)
Warming ΔT_{s} (50 yr)	+1.58 K	Eq. (14.14)
Response $\Delta T_{\mathrm{s}}/F$	$0.43\,\mathrm{K\,(W\,m^{-2})^{-1}}$	Eq. (14.14)

14.5 Adding 4×CO_2: a sublinear response

Running the same protocol at 4×CO_2 (1120 ppm) gives a second point on the forcing-response curve (Figure 14.1). The Myhre forcing doubles, $F(4\times) = 5.35 \ln 4 = 7.42\,\mathrm{W\,m^{-2}}$, but the warming does *not*:

$$\Delta T_{\mathrm{s}}(4\times) \approx +2.35\,\mathrm{K}, \qquad \frac{2.35}{7.42} \approx 0.32\,\mathrm{K\,(W\,m^{-2})^{-1}}. \tag{14.15}$$

The second doubling adds only ≈ 0.77 K against the first doubling's 1.58 K, so the response is *sub-logarithmic* in CO_2: the warming per unit forcing falls from 0.43 to $0.32\,\mathrm{K\,(W\,m^{-2})^{-1}}$. A truly linear-in-forcing model would warm ≈ 3.16 K at 4×; the model warms 0.8 K less. This is consistent with a proxy that lacks the amplifying atmospheric radiative feedbacks (which in nature make the response closer to linear in lnCO_2) and with the surface-flux response saturating as the freed mixed layer warms. The end-of-run ΔT_{s} carries $\sim\pm 0.3$ K of interannual noise, smaller than the 0.8 K departure from linearity, so the curvature is a genuine model property, not noise. Both points should still be read as transient TCR-like *proxies*, not ECS.

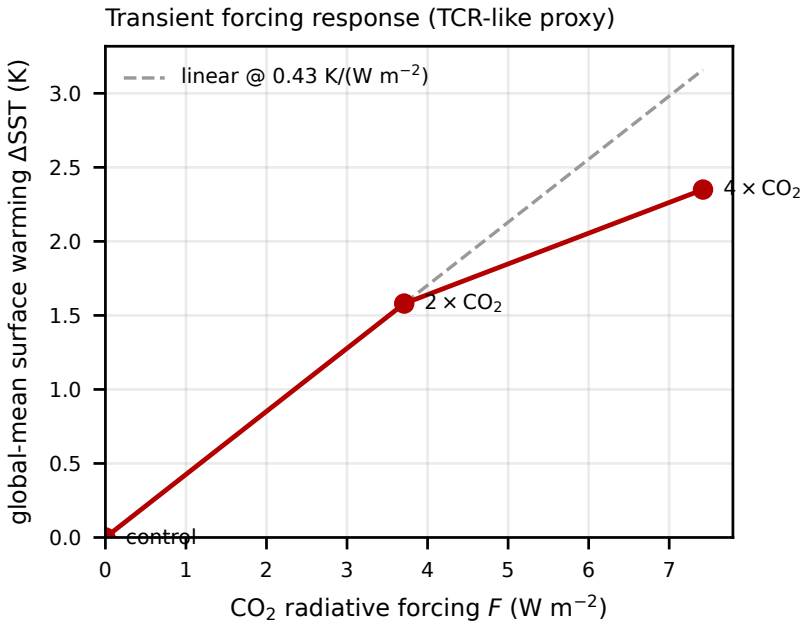

Figure 14.1: Transient global-mean surface warming ΔT_{s} versus CO_2 radiative forcing for the abrupt-2× and abrupt-4×CO_2 experiments. The dashed line is the linear response calibrated on the 2× point; the 4× point falls below it, showing the sub-logarithmic (saturating) forcing response of the proxy. Regenerate with `experiments/plot_forcing_response.py`.

Exercises

Exercise 14.1 (The forcing trap, quantified)**.** Using Equation (14.6), compute the steady SST anomaly that the *control* restoring ($\lambda = 79\,\mathrm{W\,m^{-2}K^{-1}}$) would permit under 2×$CO_2$, and compare it with the $0.65\,\mathrm{W\,m^{-2}K^{-1}}$ free mode. By what factor does freeing the ocean increase the permitted steady response? Explain in one sentence why this factor is *not* the same as the ratio of the actual simulated warmings, and which two physical processes (in the transient run) break the simple proportionality.

Exercise 14.2 (A forcing sweep)**.** Run `experiments/run_dino_co2.py` for $C \in \{420, 560, 1120\}$ ppm against the 280 ppm baseline. For each, record F from Equation (14.1) and the simulated ΔT_s. Plot ΔT_s against F (not against C). Is the relationship linear? Fit a slope and compare it with Equation (14.14). What does the logarithmic form of Equation (14.1) imply for the spacing of your three points on the *concentration* axis versus the *forcing* axis?

Exercise 14.3 (Drift cancellation matters)**.** The experiment defines warming as a difference of two branches (Equation (14.12)). Re-derive ΔT_s from the single forced branch alone (do *not* subtract the baseline) and compare. Estimate the residual drift of the free model from the baseline curve and argue, with numbers, whether ignoring it would have changed the headline $0.43\,\mathrm{K\,(W\,m^{-2})^{-1}}$ by more than its honest uncertainty.

Exercise 14.4 (From two runs to one gradient)**.** The Myhre forcing is differentiable in C. Sketch how you would obtain $\mathrm{d}\Delta T_\mathrm{s}/\mathrm{d}C$ at $C = 560$ ppm using `jax.grad` through `ocean_fluxes_jax` rather than by finite-differencing two 50-year integrations. Which term in the surface budget carries the gradient, and which two terms (the frozen $\overline{Q}$ and the area-mean removal) contribute zero or a constant to it? Cross-reference your answer with Chapter 15.

Chapter 15

Differentiable Applications: Calibration, Sensitivity, Data Assimilation

The defining feature of Chronos-ESM is that every prognostic step — ocean tracer advection, the DINOSAUR spectral atmosphere, the bulk flux coupler, the overturning closure — is written as a composition of JAX primitives, so the *entire* coupled trajectory is a differentiable function of its inputs (Bradbury et al., 2018). Chapters 3–14 built this machine; this chapter shows what the derivatives are *for*. We treat four concrete uses — gradient-based parameter calibration, sensitivity maps, four-dimensional variational data assimilation (4D-Var), and the calibration of a tipping point — and we are deliberately honest about the one place where naive backpropagation *fails*: a bifurcation is non-smooth, and the gradient through a long integration that crosses it diverges. The remedy, fold continuation by implicit differentiation, closes the chapter and forward-references the AMOC tipping study of Chapter 17.

15.1 The differentiable model as a function

Write the coupled step (Chapter 13) as a map $\mathbf{x}_{n+1} = \mathcal{M}(\mathbf{x}_n; \boldsymbol{\theta})$, where $\mathbf{x}$ is the state pytree (ocean `OceanState`, DINOSAUR modal `pe.State`, land, ice) and $\boldsymbol{\theta}$ collects tunable parameters. The forward model over N coupling intervals is the composition $\mathcal{M}^N$. A scalar objective $\mathcal{J}(\mathbf{x}_0, \boldsymbol{\theta})$ — a misfit to observations, a climatology error, or a target diagnostic — is therefore an ordinary differentiable function, and JAX returns its exact gradient by reverse-mode AD (the adjoint):

$$\nabla_{\boldsymbol{\theta}} \mathcal{J} \;=\; \texttt{jax.grad}(\mathcal{J})(\boldsymbol{\theta}), \qquad \nabla_{\mathbf{x}_0} \mathcal{J} \;=\; \texttt{jax.grad}(\mathcal{J}, \text{argnums=0})(\mathbf{x}_0). \tag{15.1}$$

This is not a finite-difference estimate: it is the analytic derivative of the discrete program, accurate to floating point. The coupled step is built so this works in practice. Each interval is

↪ *Code:* `chronos_esm/coupler/dino_step.py` — `DinoCoupledModel._advance`: SST → regrid → DINOSAUR interval → flux coupler → ocean `scan`

, and the differentiable variant `step()` wraps both the atmosphere interval and the inner ocean sub-stepping in `jax.checkpoint` (rematerialisation) so the reverse pass stores only interval-boundary states and recomputes the rest:

$$\underbrace{\text{store every step}}_{\text{memory } \mathcal{O}(N_{\text{sub}})} \longrightarrow \underbrace{\text{store boundaries, recompute}}_{\text{memory } \mathcal{O}(1),\ \text{cost } \times 2}. \tag{15.2}$$

The two code paths are intentionally split: `step_fast` is one fused `jit` program with *no* remat (used for forward control/forced runs, $\sim 10\times$ faster), while `step` carries the remat for gradients and DA.

Remark. Reverse mode costs $\mathcal{O}(1)$ adjoints regardless of the number of parameters, so it is the right tool for high-dimensional control vectors (an initial-condition field, a 3-D flux correction). For a handful of scalar parameters, forward-mode (`jax.jvp`) or even central differences are competitive and easier to validate against.

15.2 Gradient-based parameter calibration

Calibration tunes $\boldsymbol{\theta}$ so the model reproduces data. The classical approach perturbs each parameter and reruns; with P parameters this is $P{+}1$ runs per gradient step. Differentiability collapses that to *one* run plus one adjoint, *independent of* P. Given targets $\mathbf{y}_k$ (e.g. WOA18 SST/SSS (Locarnini et al., 2018; Zweng et al., 2018), ERA5 winds (Hersbach et al., 2020)) sampled by an observation operator $\mathcal{H}$, minimise

$$\mathcal{J}(\boldsymbol{\theta}) = \sum_k \left\| \mathcal{H}\big(\mathcal{M}^k(\mathbf{x}_0;\boldsymbol{\theta})\big) - \mathbf{y}_k \right\|^2_{\mathbf{R}^{-1}} \;+\; \tfrac{1}{2}\left\|\boldsymbol{\theta}-\boldsymbol{\theta}_b\right\|^2_{\mathbf{B}^{-1}}, \tag{15.3}$$

with $\mathbf{R}$ the observation-error covariance and the second term a background (prior) regulariser around $\boldsymbol{\theta}_b$ with covariance $\mathbf{B}$. Any JAX-compatible optimiser (L-BFGS, Adam) then iterates $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha\,\nabla_{\boldsymbol{\theta}}\mathcal{J}$ using (15.1). Natural targets in Chronos-ESM are the eddy-diffusivity κ_{GM} (Gent and McWilliams, 1990), the bulk drag coefficient, hyperdiffusion coefficients, and the overturning closure constants (`thc_k_vel`, `thc_haline_gain`) threaded through `_advance`.

Remark (Gradient hygiene over long windows). The adjoint of a chaotic trajectory has a Lyapunov-amplified component: the gradient is correct for the *discrete* cost, but its variance grows with window length and it stops being a useful descent direction for a *climatological* target. Keep calibration windows short (days–weeks for the atmosphere), or target time-mean / low-frequency statistics, or use the steady-state route of §15.5. This is the same caveat that motivates fold continuation below.

15.3 Sensitivity maps

A sensitivity map is the spatial gradient field of a scalar diagnostic with respect to a field-valued input — exactly the by-product of one adjoint. Let J be, say, the AMOC strength at 26.5°N (Chapter 8) and let the input be the surface freshwater flux $F(\phi,\lambda)$. Then

$$\frac{\delta J}{\delta F(\phi,\lambda)} = \big[\texttt{jax.grad}(J)(F)\big](\phi,\lambda) \tag{15.4}$$

is a 2-D field telling you *where* a unit of freshwater most weakens (or strengthens) the overturning. One reverse pass yields the sensitivity to every grid point at once — the property that makes adjoint sensitivity the workhorse of ocean state estimation. Because the barotropic streamfunction solve sits inside the ocean step, these maps propagate through an *implicit* linear solve; the next section explains why that is differentiable for free.

Differentiating through the elliptic solve

The ocean streamfunction is obtained by solving a variable-coefficient elliptic problem (Chapter 7),

$$\boldsymbol{\nabla}\cdot\big(\kappa\,\boldsymbol{\nabla}\psi\big) = \zeta, \qquad \kappa = 1/H, \tag{15.5}$$

on the sphere, discretised in symmetric face-conductance form so the operator is SPD and solved by a custom conjugate-gradient (CG) iteration,

↪ *Code:* `chronos_esm/ocean/solver.py` — `solve_elliptic_varcoef_sphere` / `solve_cg`

. Two design choices make (15.5) differentiable. First, the CG loop uses `jax.lax.scan` for a *fixed* `max_iter` with a `jnp.where` that turns converged steps into no-ops — a `while_loop` would not be reverse-differentiable:

$$\mathbf{x}^{(i+1)} = \begin{cases} \mathbf{x}^{(i)}, & r^{(i)} \le \mathrm{tol}\cdot\|b\| \quad \text{(no-op)},\\ \mathbf{x}^{(i)} + \alpha_i\,\mathbf{p}^{(i)}, & \text{otherwise}, \end{cases} \qquad \alpha_i = \frac{\mathbf{r}^{(i)\top}\mathbf{z}^{(i)}}{\mathbf{p}^{(i)\top} A\mathbf{p}^{(i)}}. \tag{15.6}$$

Second, the operator pins $\psi = 0$ on land via an identity row ($A_{ii} = 1$ there) rather than masking faces, which would make A singular and the CG diverge — a Dirichlet ($\psi = 0$) coastline is the correct streamfunction boundary condition. Because the solution satisfies $A\psi = b$ with A, b depending on $\boldsymbol{\theta}$, the adjoint of the solve is itself a solve of $A^{\top}\boldsymbol{\lambda} = \partial J/\partial\psi$; AD through the unrolled `scan` computes this automatically (a small floor `1e-12` guards the Jacobi preconditioner and the α_i denominator against division by zero).

15.4 Variational data assimilation (4D-Var) on the atmosphere

4D-Var seeks the initial state $\mathbf{x}_0$ that best fits a window of observations subject to the model dynamics as a strong constraint. With the DINOSAUR atmosphere as the active dynamical core (
↪ *Code:* `chronos_esm/coupler/dino_step.py` — `DinoCoupledModel.step`, the remat'd differentiable interval
), the standard incremental cost is

$$\mathcal{J}(\mathbf{x}_0) = \tfrac{1}{2}\|\mathbf{x}_0 - \mathbf{x}_b\|^2_{\mathbf{B}^{-1}} + \tfrac{1}{2}\sum_{k=0}^{K} \big\|\mathcal{H}_k\big(\mathcal{M}^k(\mathbf{x}_0)\big) - \mathbf{y}_k\big\|^2_{\mathbf{R}_k^{-1}}, \tag{15.7}$$

with $\mathbf{x}_b$ the background, $\mathbf{B}, \mathbf{R}_k$ the background and observation error covariances. The control variable is the modal DINOSAUR state (vorticity / divergence / temperature variation / log-surface-pressure), which is a registered JAX pytree and therefore a legal argument to `jax.grad`. The gradient

$$\nabla_{\mathbf{x}_0}\mathcal{J} = \mathbf{B}^{-1}(\mathbf{x}_0 - \mathbf{x}_b) + \sum_k \mathbf{M}_k^\top\, \mathbf{H}_k^\top\, \mathbf{R}_k^{-1}\big(\mathcal{H}_k(\mathbf{x}_k) - \mathbf{y}_k\big) \tag{15.8}$$

involves the tangent-linear $\mathbf{M}_k = \partial\mathcal{M}^k/\partial\mathbf{x}_0$ and its adjoint $\mathbf{M}_k^\top$ — in a hand-coded DA system these are thousands of lines of separately maintained code that must be re-validated every time the forward model changes. Here they are *generated* by reverse-mode AD of the same forward step, so they can never drift out of sync with the model.

Example 15.1 (A minimal atmospheric 4D-Var)**.** Build the model once, then minimise (15.7) over a short window:

```
model = DinoCoupledModel()                    # dino atmosphere active
def J(atmos0):
    st = state0._replace(atmos=atmos0)
    miss = 0.0
    for k in range(K):
        st = model.step(st, interval=0.25)   # 6-h slot, remat'd
        miss += loss(model.diagnostics_lin(st), y[k])
    return Jb(atmos0) + miss
g = jax.grad(J)(state0.atmos)                 # adjoint of the whole window
```

The atmosphere is the cleanest 4D-Var target in Chronos-ESM: short windows, a genuine multi-level dynamical core, and well-behaved tangent linearity over a few days. Ocean and fully coupled DA are gated by the slower, stiffer ocean adjoint and are out of scope here.

15.5 Bifurcations are non-smooth: calibrate the fold, not the run

The applications above all assume the objective is a smooth function of the trajectory. A *tipping point* breaks that assumption. The AMOC 'on'/'off' collapse is a saddle-node (fold) bifurcation (Stommel, 1961; Rahmstorf et al., 2005): as a control parameter — here the North-Atlantic freshwater (hosing) flux F — crosses a critical value F_{crit}, the stable on-state *ceases to exist* and the system jumps to the off-state. Two facts make "`jax.grad` through a 200-year hosing sweep" the wrong tool:

1. the trajectory sensitivity $\partial\mathbf{x}/\partial F$ *diverges* as $F \to F_{\text{crit}}$ (the linearisation has a zero eigenvalue at the fold), so the backpropagated gradient blows up exactly where you need it;

2. even away from the fold you would differentiate through hundreds of model years and a discrete collapse event — expensive and dominated by the chaotic tail of §3.

The rigorous differentiable handle on a fold is *continuation*: implicitly differentiate the joint conditions that *define* the fold rather than backpropagating a trajectory.

The reduced box and its fold

Chronos-ESM's overturning closure (
↪ *Code:* `chronos_esm/ocean/overturning.py` — `thc_overturning_velocity`
) sets the AMOC amplitude from an Atlantic upper-ocean density contrast through a softplus rectifier,

$$A \;=\; C_{\text{eff}}\, k_{\text{rel}}\, \text{softplus}\Big(\frac{\Delta\rho + (\gamma-1)\,\beta_S\,\Delta S}{\delta\rho}\Big), \qquad \text{softplus}(z) = \log(1+e^{z}), \tag{15.9}$$

where $\Delta\rho$ is the thermal density contrast, ΔS the subpolar-minus- subtropical salinity contrast, $\gamma =$ `haline_gain` amplifies the haline feedback, $\beta_S = 0.78\ \text{kg}\,\text{m}^{-3}\,\text{psu}^{-1}$, $\delta\rho = 0.1$, and $k_{\text{rel}} = k_{\text{vel}}/10^{-4}$. The calibration script
↪ *Code:* `experiments/calibrate_amoc_fold.py` — `amoc` / `fcrit` / `newton_invert`
recognises that, with a salt-advection feedback, this closure *is* a Stommel two-box model. Reducing to the salinity contrast $s \equiv \Delta S$ with thermal contrast held fixed,

$$\frac{\mathrm{d}s}{\mathrm{d}t} = -\kappa F \;+\; \mu\, A(s)\,(-s), \tag{15.10}$$

$$A(s) = C_{\text{eff}}\, k_{\text{rel}}\, \text{softplus}\Big(\frac{\rho_T + \gamma\,\beta_S\, s}{\delta\rho}\Big), \tag{15.11}$$

where hosing F freshens the box ($-\kappa F$) and the overturning imports salt at a rate $\propto A(s)(-s)$ (advective feedback: a stronger AMOC carries more salt poleward, deepening the contrast). The steady on-state balances these, $\kappa F = \mu A(s)(-s) =: R(s)$, and since $R(s) \to 0$ as $s \to 0^-$ and as $s \to -\infty$, R has an interior maximum. That maximum is the fold:

$$\boxed{F_{\text{crit}}(\boldsymbol{\theta}) \;=\; \max_{s}\ \frac{R(s)}{\kappa} \;=\; \frac{\mu}{\kappa}\ \max_{s}\ A(s)\,(-s)} \tag{15.12}$$

For $F > F_{\text{crit}}$ no steady on-state exists: the saddle and node have annihilated.

Why AD of the fold is exact: the envelope theorem

The key step is that F_{crit} is itself a *maximum* over the internal coordinate s. By the envelope theorem, the parameter-derivative of a maximised function equals the partial derivative at the optimum, with the $\partial(\cdot)/\partial s \cdot\ \mathrm{d}s^\star/\,\mathrm{d}\boldsymbol{\theta}$ term vanishing because $\partial R/\partial s = 0$ there (that vanishing condition is exactly the fold's zero-eigenvalue condition):

$$\frac{\mathrm{d}F_{\text{crit}}}{\mathrm{d}\boldsymbol{\theta}} = \frac{1}{\kappa}\,\frac{\partial R}{\partial \boldsymbol{\theta}}\bigg|_{s=s^\star(\boldsymbol{\theta})}. \tag{15.13}$$

So differentiating the *discretised* max over a fixed grid *_S_GRID* ($s \in [-3, -0.02]$, 4000 points; `jnp.max` after a `vmap`) gives the correct fold gradient to machine precision — no integration, no divergence, one `jax.grad`. The script then *inverts*: given a target F_{crit} (default 0.6 Sv), Newton's method on (15.12) with an AD-supplied derivative solves for the parameter (here `haline_gain`) that places the fold there,

$$\gamma^{(i+1)} = \gamma^{(i)} - \frac{F_{\text{crit}}(\gamma^{(i)}) - F_{\text{crit}}^{\text{target}}}{\mathrm{d}F_{\text{crit}}/\,\mathrm{d}\gamma\big|_{\gamma^{(i)}}}, \tag{15.14}$$

↪ *Code:* `experiments/calibrate_amoc_fold.py` — `newton_invert`: 15 AD-Newton iterations
. The two lumped box constants C_{eff} and μ/κ are first fitted to GCM anchors — the on-state AMOC at F=0 fixes C_{eff}, a near-marginal hosing run fixes μ/κ — read from saved hysteresis sweeps (`_load_anchors`). One confirmatory full GCM hosing run then validates the predicted F_{crit}, replacing an entire multi-job parameter grid with a single run.

Remark (Honest scope). The reduction (15.10)–(15.11) is a *caricature*: a single salinity contrast, fixed thermal contrast, lumped constants. Its value is that it shares the *functional form* (15.9) of the GCM closure, so the fold it predicts is a calibrated *guess* for where the full model tips — not a substitute for the full hysteresis experiment. The envelope-theorem gradient is exact for the box; whether the box's F_{crit} matches the GCM's is an empirical question answered by the confirmatory run. Chapter 17 carries this through to the full coupled hysteresis and its interpretation.

Remark (softplus as a differentiable rectifier). A hard threshold $\max(0,\cdot)$ on the density contrast would give a non-differentiable (and, with a hard on/off, a genuinely discontinuous) overturning. The softplus in (15.9) is a smooth surrogate: positive and C^∞, with $\mathrm{softplus}(z) \approx z$ for $z \gg 0$ and $\mathrm{softplus}(z) \approx e^z \to 0$ for $z \ll 0$. It keeps the closure differentiable everywhere *within* a branch; the non-smoothness that matters is the fold (15.12) between branches, which is why it needs continuation, not a smoother rectifier.

Summary

Differentiability turns calibration from a parameter grid into a gradient descent, turns sensitivity studies into a single adjoint, and supplies the tangent-linear and adjoint operators of 4D-Var for free (Bradbury et al., 2018; Kochkov et al., 2024). The one structural caveat is the bifurcation: backpropagation through a trajectory that crosses a fold diverges, and the correct differentiable object is the fold condition itself, handled by implicit differentiation and the envelope theorem on a reduced box. The same adjoint that makes the smooth applications cheap is what forces this distinction to be made explicit.

Exercise 15.1 (Sensitivity to the drag coefficient)**.** Using `DinoCoupledModel.step`, define J as the global-mean kinetic energy of the surface ocean after a 30-day integration and compute $\partial J/\partial C_d$ with `jax.grad`. Verify the AD gradient against a central finite difference $\big(J(C_d{+}\epsilon) - J(C_d{-}\epsilon)\big)/2\epsilon$ for $\epsilon = 10^{-3}$. Explain any disagreement in terms of the remat path and the CG tolerance in `solve_cg`.

Exercise 15.2 (Freshwater sensitivity map)**.** Take J = AMOC at 26.5°N (Chapter 8) and compute the field $\delta J/\delta F(\phi,\lambda)$ via (15.4) over a 5-year window. Where is the AMOC most sensitive to freshwater forcing? Compare the location of the maximum-magnitude sensitivity with the subpolar North-Atlantic deep-water-formation region implied by the convection layer in `thc_overturning_velocity`.

Exercise 15.3 (Reproduce the fold calibration)**.** Run `python experiments/calibrate_amoc_fold.py -target 0.6`. (i) From the printed F_{crit} table over (`haline_gain`, k_{vel}), confirm that F_{crit} increases with the haline gain and decreases with k_{vel}, and explain both signs from (15.12). (ii) Re-derive the AD gradient $\mathrm{d}F_{\mathrm{crit}}/\,\mathrm{d}\gamma$ at the solution using (15.13) and check it against the script's value. (iii) Why does refining `_S_GRID` from 4000 to 40 000 points barely change F_{crit}, whereas *shrinking* its range to exclude $s^\star$ corrupts it?

Exercise 15.4 (Why not backprop the sweep?)**.** Argue quantitatively that $\partial(\mathrm{AMOC}_{26.5})/\partial F$ from a hosing-sweep adjoint must diverge as $F \to F_{\mathrm{crit}}^{-}$. Relate the divergence to the zero eigenvalue of the steady-state Jacobian at the fold, and contrast with the *finite* envelope-theorem derivative (15.13). What does this imply for the variance of a 4D-Var gradient (15.8) computed over a window that straddles a tipping event?

Part IV

Canonical Climate Test Cases

Chapter 16

Mean Climatology versus Observations

The previous chapters built Chronos-ESM component by component: the ocean primitive equations (Ch. 4–7), the overturning closure (Ch. 8), the multi-level DINOSAUR and single-level spectral atmospheres (Ch. 9–10), and the bulk-flux coupler (Ch. 13). A model that runs, conserves, and stays finite has cleared only the lowest bar. The question this Part IV chapter asks is the harder one: *is it right?* We answer it for the time-mean state — the *climatology* — by confronting the coupled control run with two observational references and reporting area-weighted skill, honestly, including where the model is wrong and why that matters.

The climatology is the cleanest target for a first evaluation because it averages away weather and interannual noise: a sufficiently long time mean of a stable run should reproduce the large-scale geographical patterns of temperature, salinity, and pressure that a textbook climate atlas shows. It is also the most unforgiving target, because every structural bias survives the averaging. We proceed by (i) defining the diagnostic and its observational references (§16.1), (ii) stating the four area-weighted metrics the validation framework computes (§16.2), (iii) presenting the ocean and atmosphere bias-map and zonal-mean dashboards (§16.3–§16.4), and (iv) reading the mean state honestly — the closed salt budget and realistic large-scale structure on the one hand, the documented cold-tropics bias and its downstream consequence on the other (§16.5).

16.1 The diagnostic and its observational references

16.1.1 What a climatology is

Let $X(\lambda, \phi, t)$ be a model field. Its climatology is the time mean

$$\overline{X}(\lambda, \phi) \;=\; \frac{1}{T} \int_0^T X(\lambda, \phi, t)\, \mathrm{d}t, \tag{16.1}$$

taken over an *equilibrated* window T long enough that the weather spectrum and any residual spin-up transient are averaged out. In practice this is a discrete mean over a glob of saved checkpoints. The dashboard harness

↪ *Code:* `experiments/make_readme_figures.py` — averages a glob of saved states into a climatology before scoring

explicitly averages a sequence of yearly checkpoints into $\overline{X}$ so that weather noise and spin-up do not swamp the time-mean signal. The quantitative scorecard reported below is refreshed from the latest control mean; a 30-yr time mean of the coupled DINOSAUR↔ocean control is being generated for the headline numbers, and the figures in this chapter are the current validation dashboard.

16.1.2 The two observational references

Chronos-ESM is graded against two community reference datasets, each on its native grid, loaded by

↪ *Code:* `chronos_esm/validation/obs.py` — WOA18 and ERA5 loaders, returning native-grid NumPy arrays

:

Ocean — WOA18.
The World Ocean Atlas 2018 annual climatology supplies sea-surface temperature (θ, the topmost level) and sea-surface salinity (S) (Locarnini et al., 2018; Zweng et al., 2018). WOA18 is also the dataset used to *initialise* the ocean (Ch. 4), so the surface-temperature comparison must be read with care: in the flux-corrected control the surface heat flux *restores* SST toward WOA (Haney restoring (Haney, 1971); Ch. 13), so SST skill is partly imposed, not predicted. The genuinely emergent ocean field is salinity, which evolves freely.

Atmosphere — ERA5.
The ECMWF ERA5 reanalysis (Hersbach et al., 2020) (monthly means, averaged to an annual climatology) supplies 2 m air temperature (T_{a}), 10 m zonal and meridional winds ($u_{\mathrm{sfc}}, v_{\mathrm{sfc}}$), total precipitation, and mean sea-level pressure (`mslp`). ERA5 is an independent reference — the model is never initialised from it — so every atmospheric metric is an honest skill score.

Both references are regridded onto the model's T31 grid before scoring (`grid.regrid_to_model`), and each field is converted to a human-friendly display unit (precipitation in mm/day, pressure in hPa) by the per-variable display table in

↪ *Code:* `chronos_esm/validation/scorecard.py` — the _DISPLAY table maps each field to units, scale, colour map

. The canonical model fields the scorecard reads are `sst`, `sss`, `t2m`, `u_sfc`, `v_sfc`, `precip`, and `mslp`.

16.2 Area-weighted skill metrics

All metrics are area weighted. On a latitude–longitude grid the cells shrink toward the poles, so each cell carries the weight $w_{ij} \propto \cos\phi_i$ (`grid.area_weights`); a cell contributes only where *both* model and observation are finite, which masks land for ocean fields automatically. Writing $\langle\cdot\rangle$ for the area-weighted average over valid cells, the four diagnostics in

↪ *Code:* `chronos_esm/validation/metrics.py` — area_weighted_stats and taylor_stats compute bias, RMSE, pattern correlation, and the std ratio

are:

$$\text{bias} \quad \overline{\Delta} = \langle X_{\mathrm{mod}} \rangle - \langle X_{\mathrm{obs}} \rangle, \tag{16.2}$$

$$\text{RMSE} \quad \mathrm{RMSE} = \sqrt{\left\langle (X_{\mathrm{mod}} - X_{\mathrm{obs}})^2 \right\rangle}, \tag{16.3}$$

$$\text{pattern correlation} \quad r = \frac{\left\langle X'_{\mathrm{mod}}\, X'_{\mathrm{obs}} \right\rangle}{\sigma_{\mathrm{mod}}\, \sigma_{\mathrm{obs}}}, \tag{16.4}$$

$$\text{std ratio} \quad \sigma_{\mathrm{r}} = \frac{\sigma_{\mathrm{mod}}}{\sigma_{\mathrm{obs}}}, \tag{16.5}$$

where a prime denotes the deviation from the area-weighted mean, $X' = X - \langle X \rangle$, and $\sigma^2 = \langle X'^2 \rangle$. These four separate the kinds of error a climatology can have. The *bias* (Eq. 16.2) is the global offset — is the model on average too warm or too salty? The *RMSE* (Eq. 16.3) is the total point-by-point error, conflating offset and pattern. The *pattern correlation* (Eq. 16.4) removes the mean from each field and asks only whether the model puts the highs and lows in the *right places*; $r = 1$ is a perfect spatial pattern regardless of amplitude. The *std ratio* (Eq. 16.5) measures whether the spatial *amplitude* is right: $\sigma_{\mathrm{r}} < 1$ means the model is too smooth (too little spatial contrast), $\sigma_{\mathrm{r}} > 1$ means it is too noisy or too sharp. A perfect model sits at $\overline{\Delta} = 0$, $r = 1$, $\sigma_{\mathrm{r}} = 1$. The pair (r, σ_{r}) are the polar coordinates of a Taylor diagram (Taylor, 2001), which the framework also emits, and they are exactly the right summary for a climatology because they are insensitive to a constant offset that a flux correction may impose.

Two reductions present the spatial detail. The *bias map* $\overline{X}_{\mathrm{mod}} - \overline{X}_{\mathrm{obs}}$ shows *where* the model is wrong, on a diverging colour scale centred at zero. The *zonal mean* $\langle X \rangle_\lambda(\phi)$, the longitude average per latitude band (`metrics.zonal_mean`), collapses the field to a single latitude profile and is the cleanest view of the large-scale meridional structure that the primitive equations are supposed to set.

16.3 Ocean climatology versus WOA18

16.3.1 Sea-surface temperature

Figure 16.1 maps the SST bias and Fig. 16.2 the SST zonal mean against WOA18. The model reproduces the canonical meridional SST structure — a warm equatorial belt, the subtropical maxima, and the steep poleward cooling — with a near-unity pattern correlation and std ratio, as expected for a field whose surface is restored toward the same climatology in the flux-corrected control. The honest reading is therefore that SST is largely *imposed* at the surface, not predicted; its value here is as a consistency check (the restoring is doing its job and the run is stable) rather than as an independent skill claim. The informative residual is the spatial pattern of the small remaining bias, which the bias map exposes.

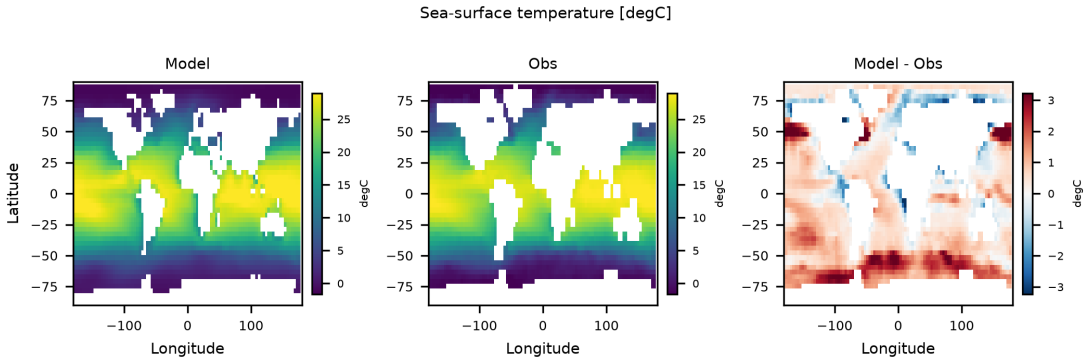

Figure 16.1: Sea-surface temperature bias, $\overline{\theta}_{\mathrm{mod}} - \overline{\theta}_{\mathrm{WOA18}}$ (°C), on the T31 grid. Diverging scale centred at zero; land masked. The large-scale SST pattern is realistic (near-unity pattern correlation), but recall the control restores surface temperature toward WOA, so the agreement is partly imposed.

16.3.2 Sea-surface salinity

Salinity is the emergent ocean test, because nothing restores it to WOA at the surface in the modern control — the global *salt budget is closed* (water conservation plus volume-mean salinity conservation), so the model no longer drifts its salinity into the physics-stability clip (Ch. 5;
↪ *Code:* `chronos_esm/ocean/veros_driver.py` — high-latitude salinity sponge and the closed volume-mean salt budget
). The zonal-mean salinity (Fig. 16.4) reproduces the textbook double-peaked structure — saline subtropical gyres flanking a fresher equatorial band, with fresh high latitudes — which is the signature of the evaporation-minus-precipitation pattern that the freshwater flux imposes. The bias map (Fig. 16.3) shows the pattern is right where the amplitude is honestly under-resolved: at T31 the std ratio is below one (the spatial salinity contrast is too smooth), and the pattern correlation, while positive in the forced-atmosphere assessment, is modest. The key honest point is conservation: the *global-mean* salinity bias is small precisely because the budget is closed, and the remaining error is a pattern/amplitude deficiency, not a runaway drift.

16.4 Atmosphere climatology versus ERA5

16.4.1 2 m air temperature

The 2 m air temperature (Figs. 16.5, 16.6) is the atmosphere's thermodynamic mean state. Its large-scale meridional structure — warm tropics, cold poles, the land/ocean contrast — follows the SST it sits over, so T_{a} inherits both the strength and the weakness of the surface ocean. In the flux-corrected coupled control the T_{a} pattern correlation against ERA5 is high (the documented control scorecard reports $r \approx 0.88$ once the continents are given an interactive land surface and no longer behave as a uniformly cold ocean), but the field carries a documented warm bias and an over-damped spatial amplitude: the control std ratio is below one, meaning the model spreads too little warm–cold contrast across the globe.

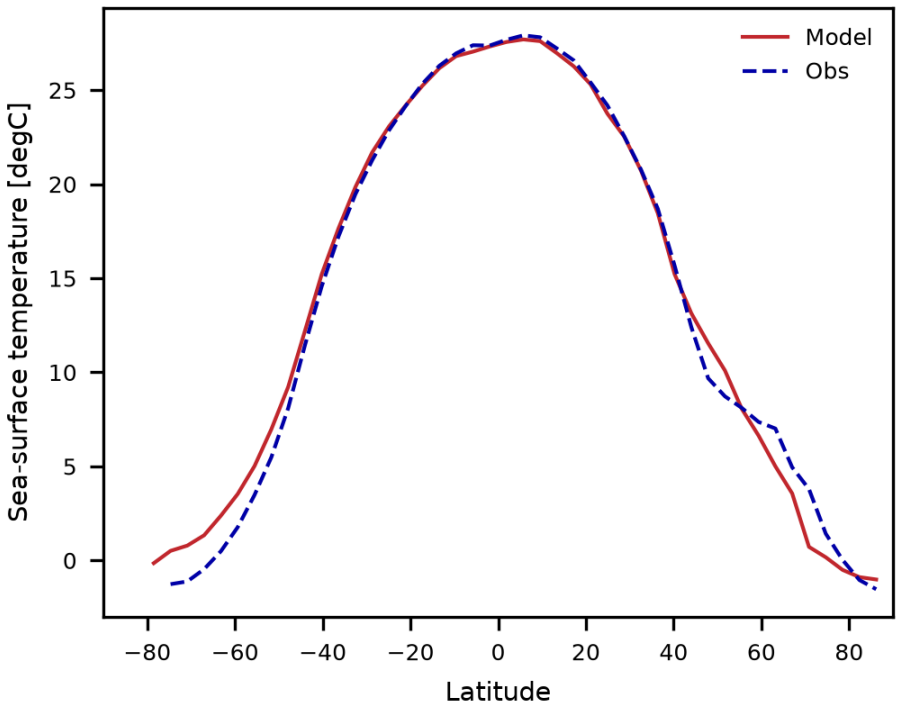

Figure 16.2: Zonal-mean sea-surface temperature versus WOA18. The model recovers the warm equatorial belt and the steep poleward decline. The residual flattening of the tropical maximum is the cold-tropics bias discussed in §16.5.

Because the SST-driven part of T_a is imposed by the SST restoring, the honestly *emergent* skill here is the land/ocean contrast and the continental pattern, not the absolute temperature.

16.4.2 Mean sea-level pressure — the weakest field

Mean sea-level pressure (Fig. 16.7) is, by design, the field on which Chronos-ESM's atmosphere is graded most harshly, and it is the weakest field in the dashboard. The two atmospheric options expose two distinct failure modes, both documented and both understood. The default *single-level barotropic* atmosphere (Ch. 10) has no baroclinic eddies, so it cannot generate the synoptic highs and lows that organise the observed pressure field; its `mslp` *pattern* correlation stays near zero. Crucially, this was hard-won: an earlier polar instability — the lat–lon $\mathrm{d}x$ collapsed at the poles and detonated the $R\theta\nabla^2(\ln p_s)$ term — had pinned the pressure field to its clamp with a variance $\sim$215$\times$ observed, until the dynamical metric was floored, bringing the variance to $\sim$2$\times$ observed (a physical, if pattern-poor, field). The multi-level DINOSAUR atmosphere (Ch. 9) *does* perform genuine baroclinic instability (Kochkov et al., 2024), yet its surface pressure/wind *pattern* skill is still not improved in the coupled control (the documented control `mslp` correlation is negative): at T31 the regridded WOA SST has only a $\sim$22.6 K equator–pole gradient, too weak for the resolved eddy momentum flux to build the observed mid-latitude surface pressure and wind pattern. The bias map should therefore be read as a *diagnosis of a known structural limit*, not a tuning failure: closing it needs a sharper SST gradient (higher resolution) or an explicit eddy-momentum parameterization.

16.5 Honest assessment of the mean state

16.5.1 What is right

Three results in the dashboard are genuine, not artefacts of restoring. First, the *salt budget is closed*: salinity no longer drifts into the clip, the global-mean bias is small, and the zonal-mean S structure is realistic — this is a conservation property of the discretisation (Ch. 6), and it is a non-trivial thing for

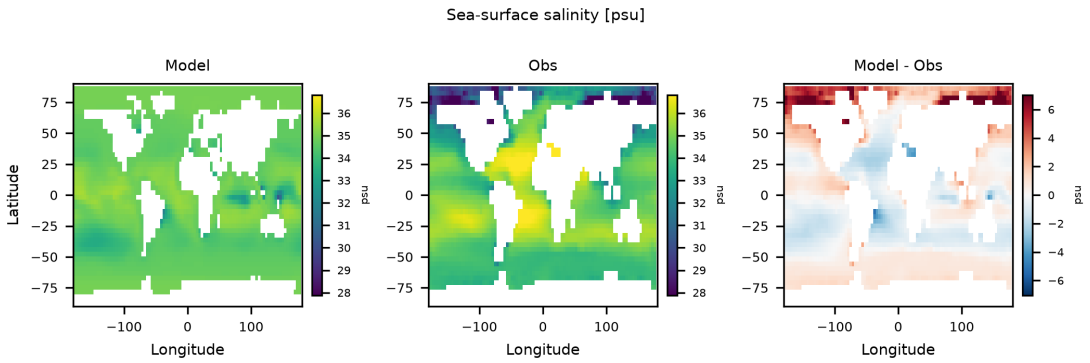

Figure 16.3: Sea-surface salinity bias, $\overline{S}_{\mathrm{mod}} - \overline{S}_{\mathrm{WOA18}}$ (psu). The small global-mean bias reflects the closed salt budget; the residual is a too-smooth spatial pattern (std ratio < 1) rather than a drift.

a free ocean to get right. Second, the *large-scale T/S structure* is correct: the warm-tropics/cold-poles temperature profile and the double-peaked salinity profile both emerge with high pattern correlation, confirming that the primitive equations plus the surface forcing set the right meridional skeleton (Vallis, 2017; Griffies, 2004). Third, when the continents are given an interactive land surface, the *land/ocean temperature contrast* is captured (T_{a} pattern correlation rises sharply in the documented control), and a *revived ITCZ* appears in precipitation once the run rains on the equator rather than the subtropics — the genuinely emergent atmospheric skill.

16.5.2 The cold-tropics bias and its consequence

The single most consequential mean-state error is a *cold-tropics bias*: a cold-tongue deficit in tropical SST relative to WOA, visible as the flattened tropical maximum in the SST and T_{a} zonal means (Figs. 16.2, 16.6). It arises from a slow, basin-wide ocean cooling tendency that the surface flux balance pins at the *global mean* but not in its *spatial pattern*: removing the global-mean net heat keeps the planet's energy budget closed without fixing where the heat is missing, so the tropics run cold while the global mean stays on target. A related subtlety surfaced and was fixed in the coupled control — the heat-flux adjustment had been balanced over land+ocean cells, leaving a residual net *ocean* cooling because heat is applied only to ocean cells; balancing the adjustment over ocean cells only flattened the SST drift (documented: a $17.3 \to 14.9\,^{\circ}\mathrm{C}$ drift became $17.3 \to 17.5\,^{\circ}\mathrm{C}$, flat).

The consequence is real and runs downstream into every other diagnostic. A cold, flat tropical SST means a weak equator–pole and equator–subtropics SST gradient, which (i) weakens the Hadley overturning and dries the ITCZ in the single-level atmosphere, leaving precipitation too dry, and (ii) deprives the multi-level DINOSAUR atmosphere of the baroclinicity it needs to build mid-latitude surface westerlies and a realistic pressure pattern — exactly the `mslp` weakness of §16.4. It is also why the CO_2 response of Ch. 14 must be read as a transient surface-forcing proxy on a cold base state rather than an equilibrium sensitivity. The mean-state cold-tropics bias is thus not a cosmetic blemish; it is the upstream limiter on the model's hydrological and dynamical realism, and it is the next calibration target. The differentiable framework of Ch. 15 is the intended lever for attacking it: a gradient-based fit of the surface flux and mixing parameters to the WOA/ERA5 climatology is exactly the kind of high-dimensional calibration that automatic differentiation makes tractable (Bradbury et al., 2018).

16.5.3 Reading the scorecard

The quantitative scorecard — one row per field with bias, RMSE, pattern correlation, std ratio, and the count of valid cells — is printed by `scorecard.format_scorecard` and refreshed from the latest 30-yr control mean as new checkpoints accumulate. Read it field by field with the metric roles of §16.2 in mind: trust r and σ_{r} over $\overline{\Delta}$ wherever a flux correction can impose the mean (SST, the SST-driven part of T_{a}); treat salinity, precipitation, and the land/ocean contrast as the emergent skill; and read `mslp` and the meridional wind as the documented structural ceiling of a single-gradient T31 atmosphere. The figures in this chapter are the current validation dashboard against which the next generation of calibrated runs will be measured.

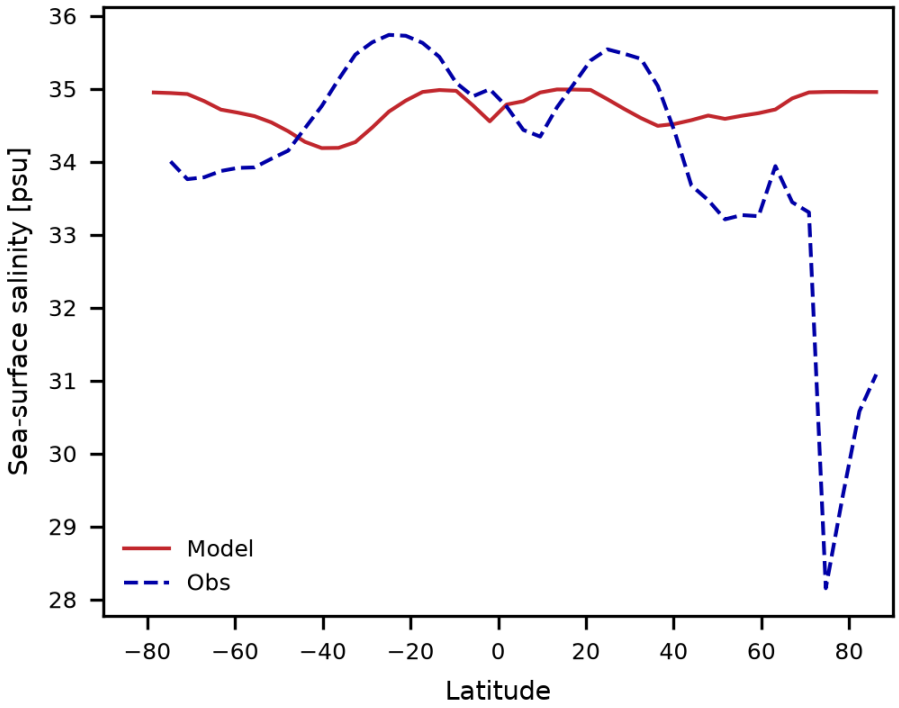

Figure 16.4: Zonal-mean sea-surface salinity versus WOA18. The double-peaked subtropical-saline / equatorial-fresh / polar-fresh structure — set by the evaporation-minus-precipitation forcing — is reproduced, with amplitude somewhat damped at T31.

Remark. A perfect score on a flux-corrected control is not a goal; it would only mean the restoring is strong. The scientific content of a climatology evaluation lives in the *residuals* that the restoring cannot reach — the salinity pattern, the precipitation, the pressure field — and in being explicit about which numbers are imposed and which are earned.

Exercises: the model as laboratory

Exercise 16.1 (Restored versus emergent skill)**.** Run the validation harness on a saved control climatology and record the scorecard for `sst` and `sss`. Now turn off the SST restoring (set the Haney coefficient $\to 0$; Ch. 13), re-run a short equilibration, and re-score. Quantify how much of the SST pattern correlation collapses without restoring, and confirm that the salinity score is essentially unchanged. Explain, using Eqs. (16.4)–(16.5), why the std ratio is a more honest summary than the bias for a restored field.

Exercise 16.2 (Diagnosing the cold-tropics bias)**.** From the SST and T_a zonal means (Figs. 16.2, 16.6), extract the model's equator-to-pole temperature gradient and compare it to the $\sim$22.6 K WOA value documented for the T31 grid. Then plot the precipitation zonal mean and test the hypothesis that a weaker tropical SST gradient produces a drier, less equatorially-peaked ITCZ. Discuss why fixing the global-mean energy balance does not, by itself, fix the tropical pattern.

Exercise 16.3 (Calibrating against the climatology)**.** Using the differentiable pipeline of Ch. 15, define the area-weighted RMSE of (16.3) between the model SST climatology and WOA18 as a scalar cost. Compute its gradient with respect to a vertical-mixing or surface-flux parameter and take a few descent steps. Report whether the tropical cold bias and the global-mean energy balance can be improved *simultaneously*, and reflect on whether a single scalar parameter is enough or whether a spatial pattern of corrections is required.

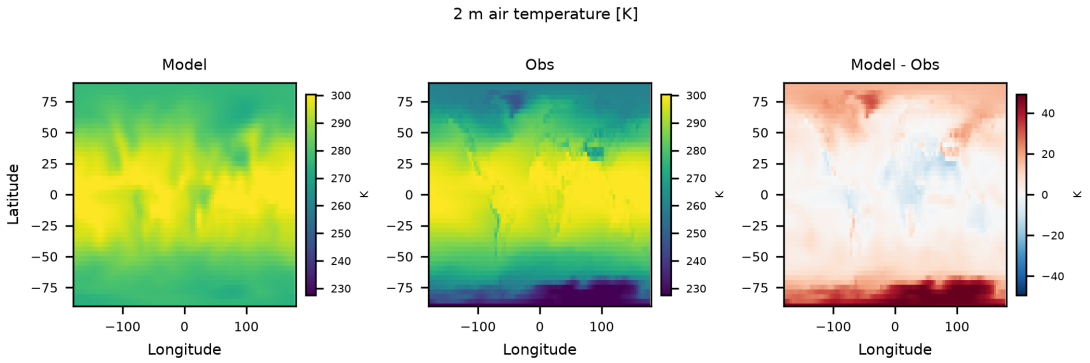

Figure 16.5: 2 m air-temperature bias, $\overline{T}_{\mathrm{amod}} - \overline{T}_{\mathrm{aERA5}}$ (K). The pattern is realistic; a warm bias and an over-damped spatial amplitude remain, and the field inherits the surface ocean's biases.

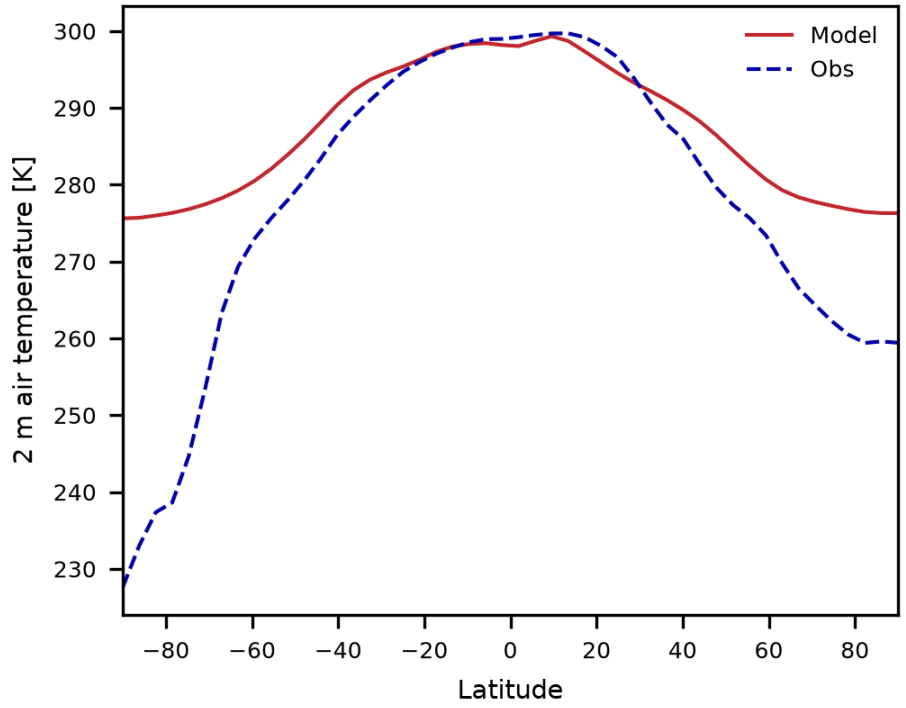

Figure 16.6: Zonal-mean 2 m air temperature versus ERA5. The model recovers the warm-tropics / cold-poles profile; the flattened tropical maximum mirrors the cold-tropics SST bias of §16.5.

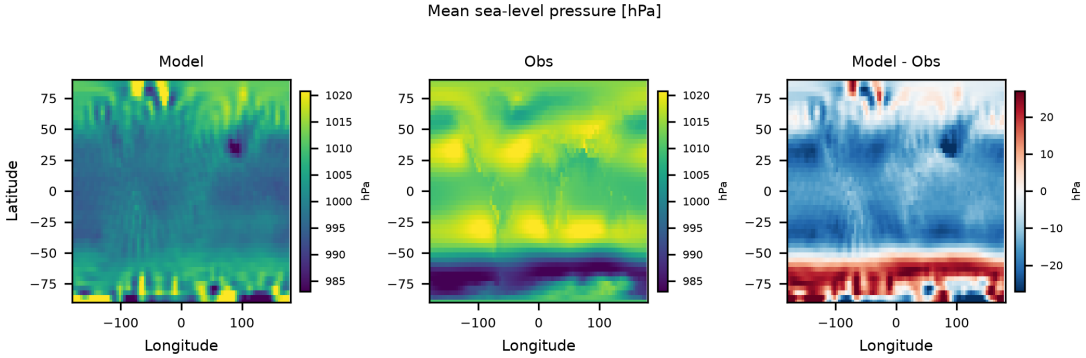

Figure 16.7: Mean sea-level-pressure bias, $\overline{p}_{s,\mathrm{mod}} - \overline{p}_{s,\mathrm{ERA5}}$ (hPa). The single-level atmosphere has no synoptic eddies (pattern correlation near zero), and at T31 the smoothed SST gradient is too weak for the multi-level DINOSAUR eddies to build the observed surface pressure pattern. The variance is nonetheless physical after the polar-metric fix; the residual is a documented structural limit, not a drift.

Chapter 17

The AMOC: Overturning, Hosing, and Tipping

The Atlantic Meridional Overturning Circulation (AMOC) carries warm, salty water poleward in the upper Atlantic and returns cold, dense water at depth. It is a textbook example of a climate *tipping element*: theory and a hierarchy of models suggest the overturning can collapse abruptly and *irreversibly* once a freshwater perturbation exceeds a critical threshold, because of the salt-advection feedback first identified by Stommel (1961). This chapter uses Chronos-ESM as a laboratory to reproduce that bifurcation, and — exploiting differentiability — to *calibrate* where it occurs. The governing equations of the overturning closure are given in Chapter 8; here we focus on the experiments and results.

17.1 The salt-advection feedback

Write the overturning strength q as an increasing function of the meridional density contrast between the subpolar and subtropical Atlantic. A stronger q imports more salt to the subpolar basin, raising its density, which strengthens q further: a positive feedback. A subpolar freshwater input ("hosing") opposes it. When the feedback loop gain exceeds unity the system becomes *bistable* — an "on" (vigorous) and an "off" (collapsed) state coexist over a finite range of forcing — and the transition between them is a saddle-node bifurcation with hysteresis. In Chronos-ESM the loop gain is set by the tunable parameter `haline_gain` (Chapter 8).

17.2 The hosing experiment

We add a freshwater flux F (in Sverdrups, $1\,\mathrm{Sv} = 10^6\ \mathrm{m^3\,s^{-1}}$) over the 45–65°N Atlantic surface and record the AMOC strength at 26.5°N. Two protocols are used:

Quasi-static sweep
Ramp F slowly up ($0 \to F_{\max}$) then down ($F_{\max} \to 0$), holding each level to near-equilibrium. An open loop (the down-leg below the up-leg) signals hysteresis. *Caveat:* near the fold the collapse timescale is multi-decadal, so finite holds give a *rate-dependent* loop that can be mistaken for, or hide, true bistability.

↪ *Code:* `experiments/run_amoc_hosing.py` — quasi-static hosing sweep

Fixed-F bistability test
The rigorous protocol: at a single fixed F, integrate one run started from an *on*-state and one from an *off*-state for ~ 100 years. If the two converge, the system is monostable at that F; if they remain separated, it is genuinely bistable. This is immune to rate-dependence.

↪ *Code:* `experiments/run_amoc_bistability.py` — fixed-F on/off initial-condition test

17.3 A verified bistable window

With the calibrated configuration `haline_gain`$= 6$, `thc_k_vel`$= 6 \times 10^{-5}$, and a convection-layer contrast depth of 300 m, the fixed-F test gives the result in Table 17.1 and Figure 17.1. At $F = 0.46$, 0.57, 0.69 Sv

the on-state and off-state branches remain 8–9 Sv apart after 100 years — a separation that is statistically significant (autocorrelation-corrected $z = 2.6$–4.4) and persistent (the off-branch does not recover). Below $F \approx 0.3$ Sv an off-state initial condition recovers to the upper branch; above $F \approx 0.8$ Sv an on-state initial condition collapses. The *hysteresis window* is therefore approximately

$$F_{\text{lower}} \approx 0.38\,\text{Sv} \;<\; F \;<\; F_{\text{upper}} \approx 0.75\,\text{Sv},$$

centered near 0.6 Sv — consistent with the freshwater-transport scale of a $\sim$ 15–20 Sv AMOC (Smeed et al., 2018) and with the collapse thresholds reported across the model hierarchy (Rahmstorf et al., 2005).

Table 17.1: Equilibrated AMOC (last-40-yr mean) at 26.5°N from on-state vs. off-state initial conditions, held at fixed hosing F for 100 years.

F (Sv)	ON-branch (Sv)	OFF-branch (Sv)	separation z
≤ 0.30	off-IC recovers to ON		—
0.46	≈ 20	≈ 12	2.6
0.57	≈ 18	≈ 10	3.1
0.69	≈ 15	≈ 6	4.4
≥ 0.80	on-IC collapses to OFF		—

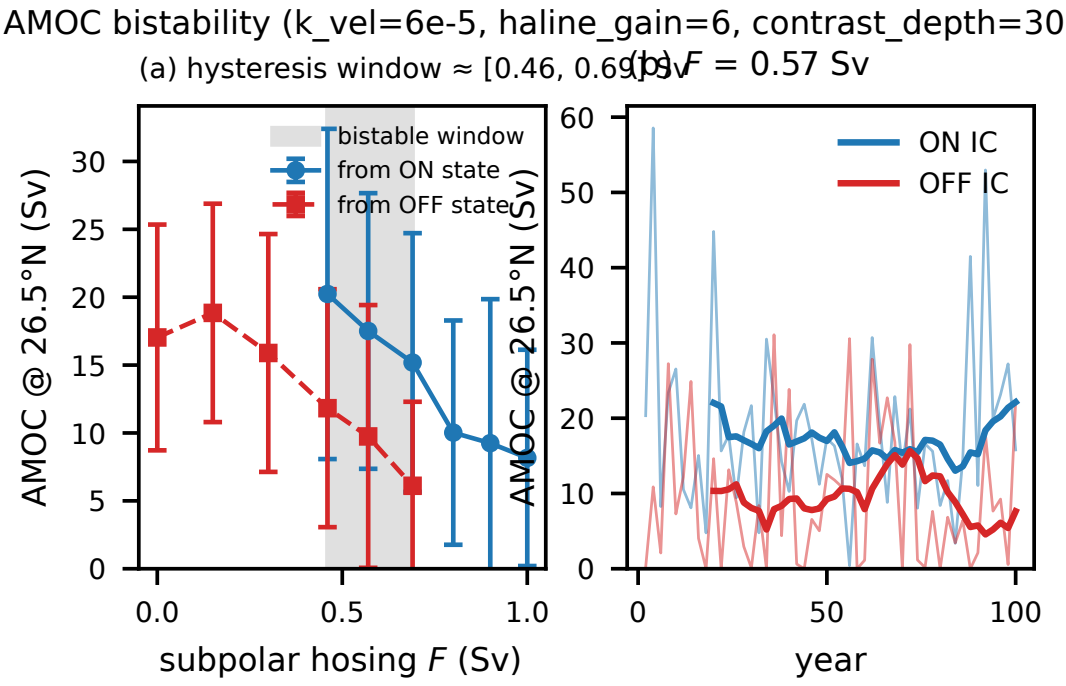

Figure 17.1: The AMOC hysteresis window. (a) On-state (blue) and off-state (red) equilibrated AMOC versus hosing F; the branches stay separated across the window. (b) At $F = 0.57$ Sv the two initial conditions settle on different branches and do not reconverge over 100 years — the signature of genuine bistability.

An honest caveat. The branches are *noisy*: the AMOC oscillates by $\sim \pm 10$ Sv (a relaxation oscillation intrinsic to the strong feedback in this interim box-style closure). The bifurcation is genuine — established by the statistically-significant, non-recovering initial-condition dependence — but it is not a clean, low-variance loop. A clean dynamical bifurcation requires the prognostic-momentum ocean core of

Chapter 7; the closure used here *works around*, rather than solves, the fact that the diagnostic-velocity ocean has $\partial(\text{AMOC})/\partial\rho \approx 0$.

17.4 Calibrating the tipping point by differentiation

A tipping threshold is a bifurcation, and therein lies a subtlety for differentiable modeling: one *cannot* obtain $\partial F_{\text{crit}}/\partial\boldsymbol{\theta}$ by backpropagating through the hosing sweep, because the trajectory's sensitivity *diverges* at the fold (that divergence *is* the bifurcation). The correct tool is *fold continuation*: reduce the closure to its governing Stommel box, where the steady-state condition $F = R(s;\boldsymbol{\theta})$ has a fold at $\partial R/\partial s = 0$, so that $F_{\text{crit}}(\boldsymbol{\theta}) = \max_s R(s;\boldsymbol{\theta})$. By the envelope theorem the gradient through this maximum is exact, and a few Newton steps invert $F_{\text{crit}}(\boldsymbol{\theta}) = F^\star$ for the parameters in milliseconds — the differentiable replacement for a brute-force parameter grid.
↪ *Code:* `experiments/calibrate_amoc_fold.py` — AD fold-continuation calibration
The gradient ranking this yields, $\partial F_{\text{crit}}/\partial(\texttt{k\_vel}) \gg \partial F_{\text{crit}}/\partial(\texttt{haline\_gain})$, shows that the AMOC on-state *strength* is the dominant control on the tipping threshold — a non-obvious result that the grid search would have taken hours to reveal.

17.5 Reproducing this chapter

```
# quasi-static hysteresis sweep (self-contained from WOA; auto-resumes):
python experiments/run_amoc_hosing.py --spinup-years 20 --fmax 0.8 --nsteps 8 \
    --hold-years 15 --haline-gain 6 --k-vel 6e-5 --contrast-depth 300

# rigorous fixed-F bistability test (run once per branch IC, then compare means):
python experiments/run_amoc_bistability.py --ckpt <on_state>  --hosing-sv 0.57 \
    --haline-gain 6 --k-vel 6e-5 --contrast-depth 300 --years 100 --outdir out/on
python experiments/run_amoc_bistability.py --ckpt <off_state> --hosing-sv 0.57 \
    --haline-gain 6 --k-vel 6e-5 --contrast-depth 300 --years 100 --outdir out/off

# differentiable calibration of the threshold, and the figure:
python experiments/calibrate_amoc_fold.py --target 0.6
python experiments/plot_amoc_bistability.py --root out --out docs/figures/amoc_bistability.pdf
```

Listing 17.1: Hosing sweep, bistability test, and figure.

Exercise 17.1. Re-run the fixed-F test at $F = 0.57\,\text{Sv}$ but lower `haline_gain` to 4. Is the system still bistable? Relate your answer to the loop-gain condition for bistability.

Exercise 17.2. Using `calibrate_amoc_fold.py`, find the `(haline_gain, k_vel)` pair that places the collapse threshold at exactly $F_{\text{crit}} = 0.5\,\text{Sv}$. Verify with one fixed-$F$ run near your predicted threshold.

Exercise 17.3 (Differentiability)**.** Explain in one paragraph why $\partial F_{\text{crit}}/\partial\boldsymbol{\theta}$ cannot be computed by automatic differentiation *through* a hosing sweep that crosses the fold, and how fold continuation circumvents the problem.

Chapter 18

Mid-Latitude Jets and Storm Tracks

The mid-latitude westerly jets and the storm tracks that flank them are among the most demanding tests of an atmospheric model. They are not directly forced — there is no external agent that blows the wind eastward at 40° latitude. Instead they are an *emergent* consequence of how the atmosphere disposes of the equator-to-pole temperature gradient. This chapter explains the phenomenon, shows how Chronos-ESM represents it with two very different atmospheres (the multi-level DINOSAUR dycore of Chapter 9 and the single-level barotropic model of Chapter 10), and presents the validation dashboard for the surface winds — honestly, including where the coarse T31 resolution defeats the dynamics.

18.1 The phenomenon: baroclinic instability and the eddy-driven jet

A rotating, differentially heated atmosphere is in approximate thermal-wind balance: the poleward temperature gradient $\partial\theta/\partial\phi$ is tied to a vertical shear of the zonal wind,

$$f\,\frac{\partial u}{\partial H} \;\approx\; -\frac{g}{\theta}\,\frac{\partial \theta}{\partial \phi}, \tag{18.1}$$

so the gradient alone produces westerlies that increase with height — a *thermal-wind jet*. This balanced state is, however, baroclinically *unstable*: small perturbations tap the available potential energy stored in the sloping isentropes and grow into the travelling synoptic eddies we call weather systems (Vallis, 2017). These eddies are not noise on top of the circulation; they *are* the circulation. Their poleward heat flux relaxes the gradient that feeds them, and — crucially for the jet — their momentum flux converges in mid-latitudes, accelerating the surface westerlies. The mature state is therefore an *eddy-driven jet* sitting over a storm track, with a surface-wind tripole: trade easterlies in the tropics, westerlies in mid-latitudes, and weak polar easterlies. No parameterization puts the westerlies there; the eddies earn them.

The key implication for modelling is structural: *baroclinic instability requires vertical structure*. Equation (18.1) is a statement about shear, and a single-level model has no shear to be unstable to. This single fact divides Chronos-ESM's two atmospheres.

18.2 Two representations in Chronos-ESM

18.2.1 The multi-level dinosaur dycore: eddies as physics

Chronos-ESM's multi-level atmosphere (Chapter 9) is built on Google's differentiable DINOSAUR spectral primitive-equation dycore — the dynamical core behind NeuralGCM (Kochkov et al., 2024). Run at T31 with 24 σ-levels, an ImEx–RK3 integrator and a ∇^4 hyperdiffusion filter, it resolves vertical shear and therefore performs *genuine* baroclinic instability: it grows transient eddies and builds an eddy-driven jet from the internal dynamics, with no jet prescribed.

The canonical proof is the dry Held–Suarez test (Held and Suarez, 1994), in which the only forcing is Newtonian relaxation toward a prescribed radiative-equilibrium temperature plus boundary-layer Rayleigh drag. Started from an isothermal rest state, the DINOSAUR core in Chronos-ESM reproduces the textbook circulation: a pair of upper-tropospheric mid-latitude westerly jets of ∼22 m/s and surface trade easterlies

(Figure 18.1). This is exactly the structure the single-level model can never generate, and it confirms the dycore is the right tool.
↪ *Code:* `experiments/dino_held_suarez.py` — Held–Suarez dry-dynamics reference + `build_held_suarez_model`

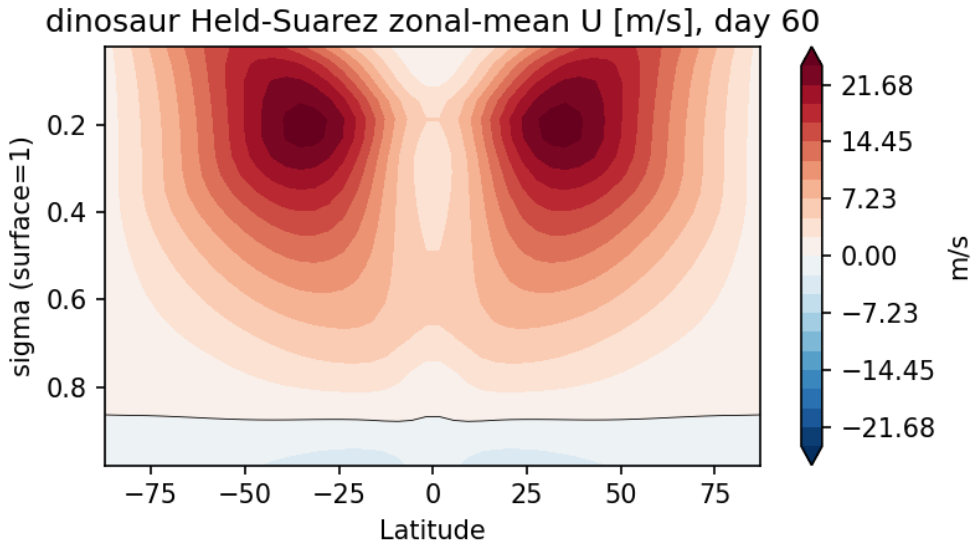

Figure 18.1: The DINOSAUR dry dycore in Chronos-ESM under Held–Suarez (Held and Suarez, 1994) forcing, T31 with 24 σ-levels. From an isothermal rest state the core spins up the textbook baroclinic circulation: twin upper-tropospheric mid-latitude westerly jets of ∼22 m/s with surface trade easterlies beneath. The eddy-driven jet is emergent, not prescribed — the defining capability the single-level barotropic model lacks.

When this same core is coupled to the ocean SST (Chapter 13), the storm track appears. To make the eddies grow on the right timescale at coarse T31, the initialization is anchored near radiative equilibrium (so the baroclinic jet exists from day zero rather than after a two-month relaxation), a small rotational vorticity seed breaks the exact zonal symmetry that an axisymmetric state would otherwise preserve indefinitely, and the hyperdiffusion is weakened (relaxation timescale 2→6 h) so the $l \sim 10$–15 eddies that carry the momentum flux are not damped before they grow. With these in place the eddies fire within ∼2–6 weeks and the atmosphere develops a ∼27 m/s eddy-driven jet, surface winds with visible synoptic structure, and an equatorial ITCZ (Figure 19.1).
↪ *Code:* `chronos_esm/atmos/dino_atmos.py` — SST-coupled DINOSAUR atmosphere, eddy spin-up

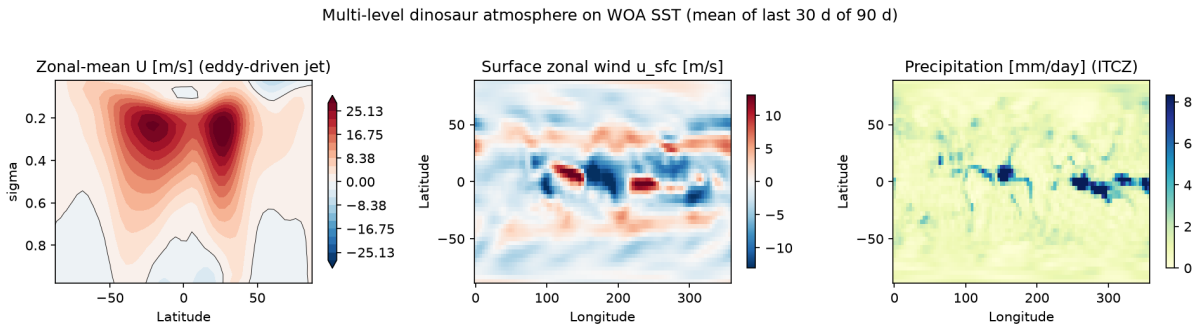

Figure 18.2: The SST-coupled multi-level DINOSAUR atmosphere on the WOA SST (time-mean of a 120-day spin-up). Left: a ∼27 m/s eddy-driven baroclinic jet in the zonal-mean zonal wind. Centre: surface winds carrying transient synoptic-eddy structure (the storm track). Right: an equatorial ITCZ in precipitation from the resolved Hadley circulation and prognostic moisture. Regenerate with `python experiments/dino_circulation_figure.py`.

18.2.2 The single-level model: a relaxed jet

The default coupled atmosphere is single-level and barotropic (Chapter 10). Having no vertical shear, it cannot satisfy the thermal-wind relation (18.1) and cannot be baroclinically unstable, so left to itself the winds geostrophically adjust to a near-standstill (and, before the v3 overhaul, even *anti*-correlated with ERA5). Chronos-ESM therefore *parameterizes* the eddy-momentum flux that the level cannot resolve: the winds are relaxed (timescale ∼2 days) toward an observed zonal-mean surface-wind profile — tropical easterlies, mid-latitude westerlies, weak polar easterlies. This is an honest stand-in, not a derivation: it imposes the *climatological* jet rather than growing it. The decoupling is deliberate — relaxation sets the realism while strong hyperdiffusion provides stability — and is discussed in Chapter 10.
↪ *Code:* `chronos_esm/atmos/dynamics.py` — single-level wind relaxation toward `u_target`

18.3 Diagnostic and metric

The jet/storm-track diagnostic is built from the near-surface zonal wind u_{sfc}, the meridional wind v_{sfc}, and the mean sea-level pressure p. We score the time-mean fields against ERA5 with the validation framework (Chapter 16), reporting area-weighted bias, RMSE, the spatial pattern correlation, and the standard-deviation ratio (a perfect model scores bias 0, corr 1, std ratio 1). The *zonal mean* of u_{sfc} versus latitude isolates the jet structure — the trade-westerly-polar tripole — while the u_{sfc} bias map exposes where the jet is misplaced. The MSLP zonal mean diagnoses the subpolar lows and subtropical highs that the storm track organises.
↪ *Code:* `chronos_esm/validation/` — area-weighted skill metrics, bias maps, zonal means

18.4 Assessment against ERA5

Figures 18.3–18.5 are the current surface-wind/pressure dashboard, generated by the atmosphere-only forced spin-up (SST held to WOA18, so coupled-ocean bias cannot mask the atmospheric skill). Two readings, one per atmosphere, are warranted.

Single-level (forced spin-up). The relaxation parameterization succeeds at what it is designed to do: the surface zonal-wind *zonal mean* matches ERA5's tripole well and the u_{sfc} pattern correlation reaches ≈ 0.72 (recovered from -0.08 before the overhaul). But this is the jet *prescribed*, not earned, and the limits show in the fields the relaxation does not set: the meridional wind has near-zero pattern correlation and the MSLP pattern correlation stays near zero, because synoptic systems — the eddies that organise the storm track in two dimensions — are simply absent.

Multi-level (SST-forced). The DINOSAUR atmosphere earns its winds dynamically and wins the thermodynamics/hydrology decisively (precipitation pattern correlation $0.16 \rightarrow 0.34$ with a real ITCZ, plus a better near-surface temperature). On the surface *wind/pressure pattern*, however, it does *not* yet beat the prescribed single-level field, for a physically understood reason rather than a bug.

The honest limitation: a weak T31 SST gradient. The eddy momentum flux that builds the mid-latitude surface westerlies scales with the equator-to-pole temperature gradient. At T31 the regridded WOA SST has only a ∼22.6 K equator–pole contrast — the sharp Gulf Stream, Kuroshio and ACC fronts are smoothed away — roughly half the ∼45 K of an idealized aquaplanet at which the textbook surface tripole emerges cleanly. At this weak gradient the resolved eddy momentum flux is too small to build a clean mid-latitude surface-westerly belt; a 22.6 K aquaplanet stays surface-easterly at mid-latitudes too, so this is the gradient, not the orography. Consequently the DINOSAUR surface dynamic-field pattern correlations stay near zero ($u_{\mathrm{sfc}} \approx -0.01$, with the coupled control near $+0.10$), and the MSLP pattern correlation is negative. Recovering the observed surface westerlies needs higher resolution (to resolve the SST fronts), an explicit eddy-momentum / surface-stress parameterization, or realistic stationary-wave forcing (monsoons, land contrast) that the Held–Suarez plus bulk-moisture physics does not yet supply. This is why the single-level relaxed jet remains the *default* coupled atmosphere: it trades dynamical honesty for a usable surface-wind climatology.

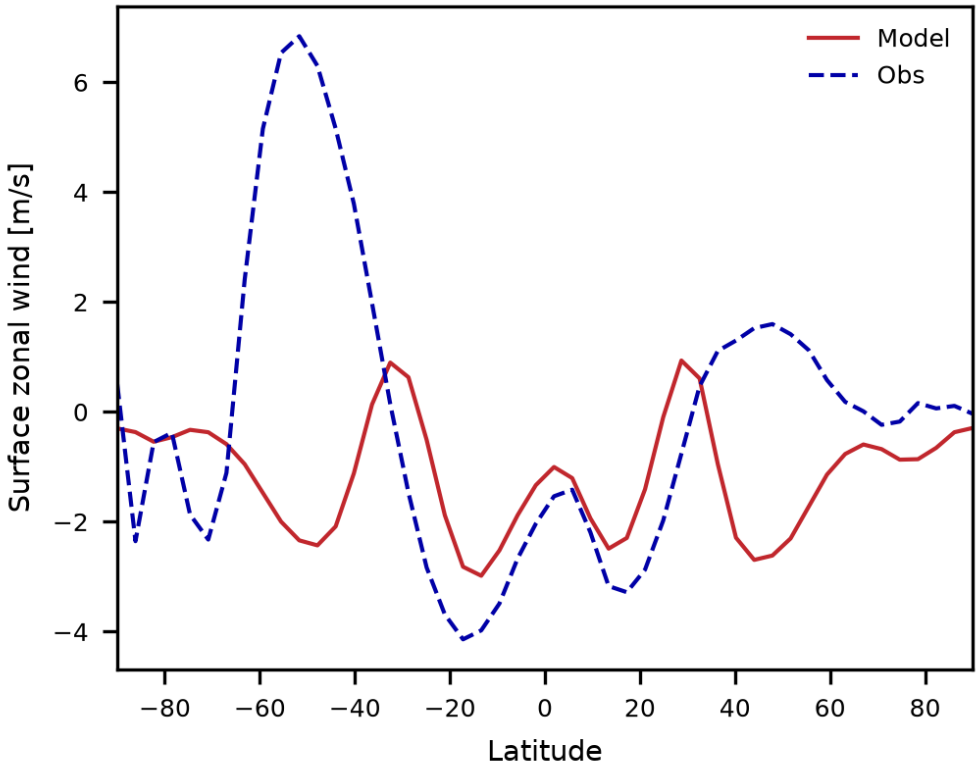

Figure 18.3: Zonal-mean surface zonal wind versus latitude, Chronos-ESM (single-level forced spin-up) versus ERA5. The relaxed single-level jet reproduces the latitudinal tripole — equatorial trade easterlies, mid-latitude westerlies, polar easterlies — which is what lifts the u_{sfc} pattern correlation to ≈ 0.72 in this configuration.

Status of the scorecard. The quantitative numbers quoted above are the documented benchmark values; the headline surface-wind/pressure scorecard is being refreshed from the latest control run (a 30-year time mean is being generated), since the dense transient eddies otherwise swamp the weak time mean in a short climatology. The figures here are the current validation dashboard for the jets and storm tracks.

18.5 Summary

Chronos-ESM represents the eddy-driven jet two ways. The multi-level DINOSAUR dycore does the *physics* — baroclinic instability, transient storm-track eddies, an emergent ∼27 m/s jet, validated against the Held–Suarez benchmark — but at T31 the SST gradient is too weak for those eddies to build realistic surface westerlies. The single-level model *parameterizes* the result, reaching a good surface-wind zonal mean (u_{sfc} corr ≈ 0.72) at the cost of having no genuine synoptic systems (near-zero MSLP and v_{sfc} pattern skill). Both are honest about what they cannot do: the storm track in two dimensions remains the frontier, awaiting higher resolution or explicit eddy parameterization.

Exercise 18.1 (Model as laboratory)**.** Run the dry Held–Suarez reference (`experiments/dino_held_suarez.py`) from isothermal rest and plot the zonal-mean zonal wind every few days. Identify the day on which the upper-tropospheric jet first exceeds 15 m/s and the day eddy kinetic energy begins to grow. Explain, using equation (18.1), why the thermal-wind jet appears *before* the eddies.

Exercise 18.2 (Model as laboratory)**.** Force the coupled DINOSAUR atmosphere with two SST fields: the real WOA SST (∼22.6 K equator–pole gradient) and an idealized aquaplanet with a ∼45 K gradient. Compare the zonal-mean surface zonal wind at ∼50° latitude. Quantify how much of the missing mid-latitude surface westerly is attributable to the weak T31 SST gradient versus other causes.

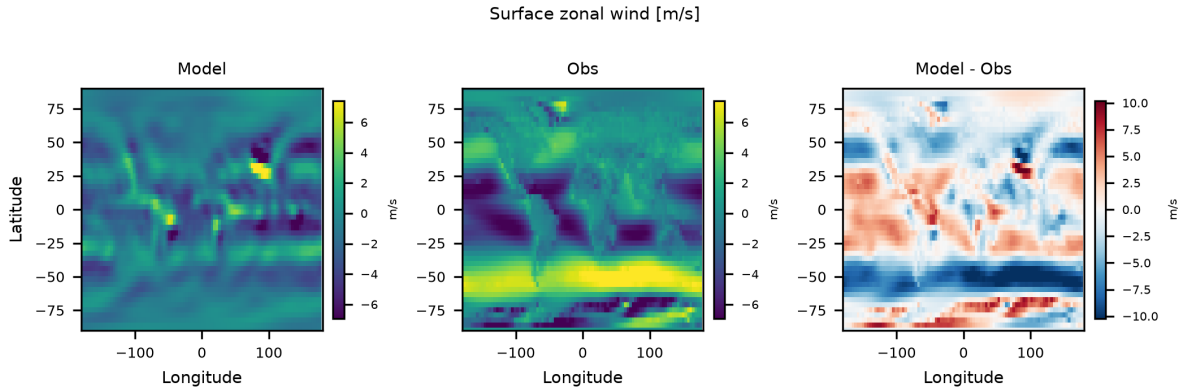

Figure 18.4: Surface zonal-wind bias map (model − ERA5). The single-level model captures the zonal-mean jet structure (Figure 18.3); the residual two-dimensional pattern is dominated by zonal asymmetries — the longitudinal positioning of the storm tracks — that a single level, lacking baroclinic eddies, cannot place.

Exercise 18.3 (Model as laboratory)**.** In the single-level model, vary the wind-relaxation timescale (e.g. 1, 2, and 6 days in `chronos_esm/atmos/dynamics.py`) and re-score u_{sfc} against ERA5 with the validation framework. Explain why too-weak relaxation lets a barotropic shear instability drive the winds toward the safety clamp, and relate this to why the single level needs strong hyperdiffusion whereas the multi-level model needs it weak.

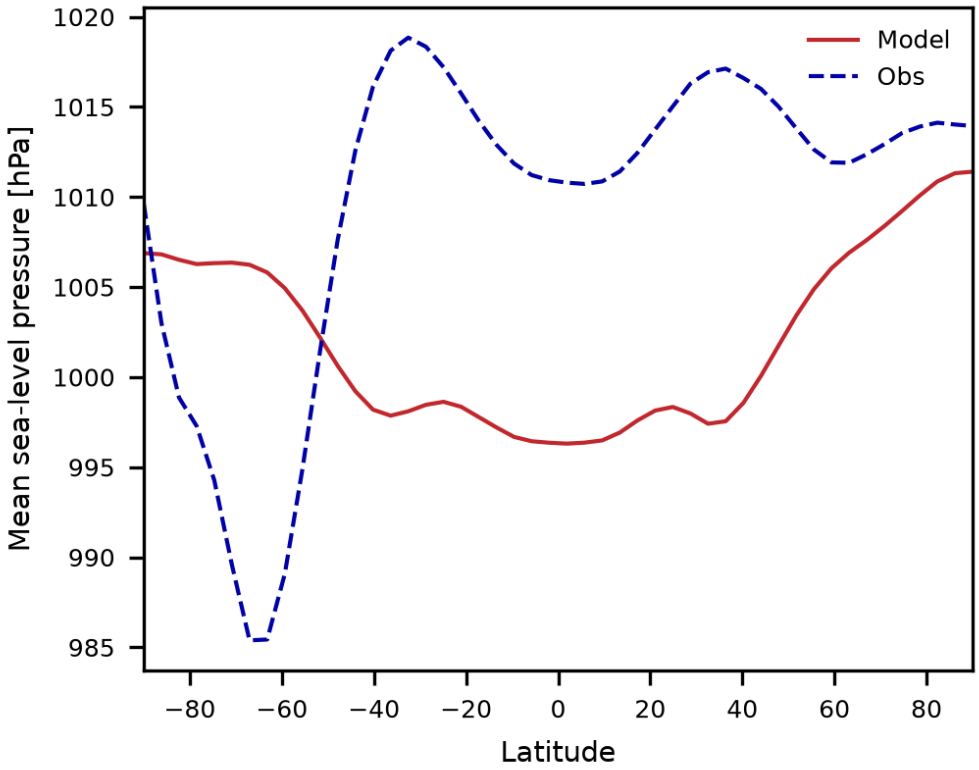

Figure 18.5: Zonal-mean mean sea-level pressure versus latitude, Chronos-ESM versus ERA5. After the polar-instability fix the pressure field is physical (variance reduced from ∼15× to ∼2× observed), but the storm-track-organised subpolar low / subtropical high pattern remains the weakest field: at T31 neither atmosphere has strong synoptic MSLP skill.

Chapter 19

The Tropics: ITCZ, Monsoons, and Variability

The tropics are the engine room of the climate system. Solar heating peaks near the equator, moist convection lofts that heat into the upper troposphere, and the resulting Hadley circulation exports it poleward. The narrow, deep-convecting rain band where the trade winds of the two hemispheres meet — the *Intertropical Convergence Zone* (ITCZ) — is the single most prominent feature of the observed precipitation field, and one of the hardest for a coarse model to capture. This chapter evaluates how Chronos-ESM represents the tropical mean state: the ITCZ and its precipitation, what role the multi-level moist atmosphere plays in producing it, and — honestly — where the representation remains weak, including the near-absence of interannual (ENSO-like) variability and the limited monsoon at this resolution.

This is a Part IV evaluation chapter. We first define the phenomenon and the physics that should produce it, recall how the relevant model components implement that physics (Chapters 9 and 10), describe the diagnostic and the skill metric, then present the validation figures and a qualitative assessment with its limitations.

19.1 The phenomenon: deep convection and the ITCZ

In the time mean, the trade-wind boundary layers of the Northern and Southern Hemispheres converge in the deep tropics, forcing ascent, condensation, and a band of intense rainfall. The ascending branch of the Hadley cell sits over this band; the descending branches dry the subtropics, producing the desert latitudes near $\pm 25°$. The position and strength of the ITCZ are set by the cross-equatorial energy transport and by the underlying sea-surface temperature (SST), since deep convection switches on only above an SST threshold of roughly 26–27°C (Vallis, 2017). A faithful tropical climate therefore requires two ingredients simultaneously: a moist atmosphere that can convect and rain, and an ocean whose tropical SST is warm enough to trigger that convection.

19.2 Why vertical structure matters: single level vs. multi-level

The distinction between Chronos-ESM's two atmospheres is sharpest in the tropics. The **single-level (barotropic) atmosphere** of Chapter 10 has no vertical coordinate: it cannot represent the ascent–condensation–latent-heating column that *is* deep tropical convection. With no resolved Hadley overturning to concentrate moisture at the equator, its precipitation is too dry and rains in the wrong place — in the subtropics and mid-latitudes rather than on the equator — and its precipitation pattern correlation against observations is near zero. This is a structural limitation, not a tuning failure: a single level simply has nowhere to put the vertical motion that makes an ITCZ.

The **multi-level spectral atmosphere** built on the DINOSAUR dycore (Chapter 9) does have the necessary vertical structure. It carries prognostic specific humidity as an advected tracer, evaporates moisture from the surface with a bulk formula (Large and Yeager, 2004), and rains out supersaturation through a large-scale condensation closure that latent-heats the column. As the resolved Hadley circulation spins up, that closure concentrates rainfall in the deep tropics and dries the subtropics — a real ITCZ forming *from the resolved circulation*, with no explicit convection scheme. Chronos-ESM therefore follows

the same lineage as NeuralGCM, whose differentiable spectral core DINOSAUR is (Kochkov et al., 2024). This is the central evaluation message of the chapter: the multi-level atmosphere produces an equatorial rain band that the single level cannot.

19.3 Diagnostic and metric

The precipitation diagnostic is the column-integrated condensation rate, P (in mm/day), output by the multi-level atmosphere's moisture physics and regridded from the Gaussian transform grid to the model's lat–lon grid. We assess it three ways against the GPCP-class observational climatology bundled with the validation framework (`chronos_esm/validation/`):

Spatial map and bias map
P, the observed climatology, and their difference $P_{\text{model}} - P_{\text{obs}}$, to localize where the model is too wet or too dry.

Zonal mean
the latitude profile of $\langle P \rangle_\lambda$, model versus observations, which isolates the ITCZ peak, its width, and the subtropical minima.

Scalar scorecard
area-weighted bias, RMSE, pattern correlation, and standard-deviation ratio (a perfect model scores bias 0, corr 1, std ratio 1).

↪ *Code:* `experiments/make_readme_figures.py` — precipitation map, zonal mean, and scorecard

19.4 The current validation dashboard

Figures 19.1–19.3 are the present tropical validation dashboard, generated from a short SST-forced spin-up of the multi-level atmosphere on the WOA18 SST climatology (Locarnini et al., 2018). The quantitative scorecard is refreshed from the latest control run; a 30-year time mean of the coupled DINOSAUR ↔ ocean control is being generated to replace the short-spin-up numbers quoted below with an equilibrated climatology.

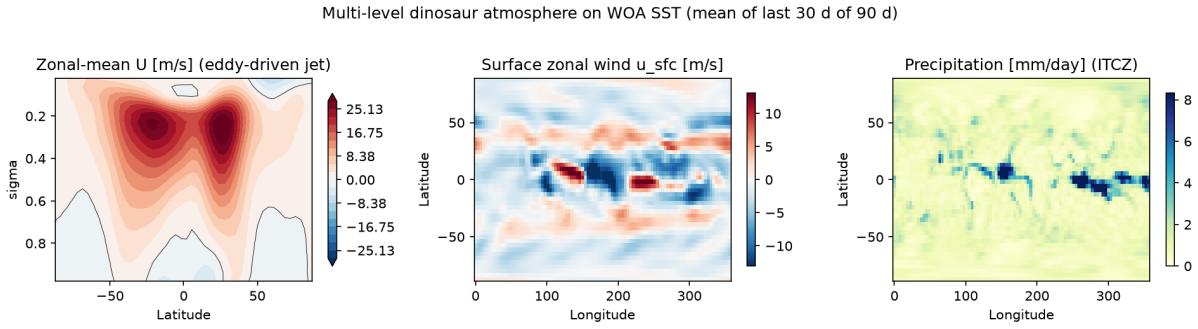

Figure 19.1: Multi-level DINOSAUR atmosphere forced by the WOA SST (time mean of a 120-day spin-up). *Left:* the zonal-mean zonal wind, with the twin upper-tropospheric eddy-driven baroclinic jets (~ 27 m/s) that the single level cannot generate. *Centre:* surface zonal wind, showing trade easterlies straddling the equator. *Right:* precipitation (mm/day) — the tropical ITCZ rain band, the feature of interest in this chapter. Regenerate with `python experiments/dino_circulation_figure.py`.

The right-hand panel of Figure 19.1 is the headline result: an equatorial band of enhanced precipitation, organized by the resolved Hadley ascent, with dry subtropical flanks. The accompanying trade easterlies (centre panel) converging toward that band are the surface signature of the same circulation. A single-level atmosphere produces neither.

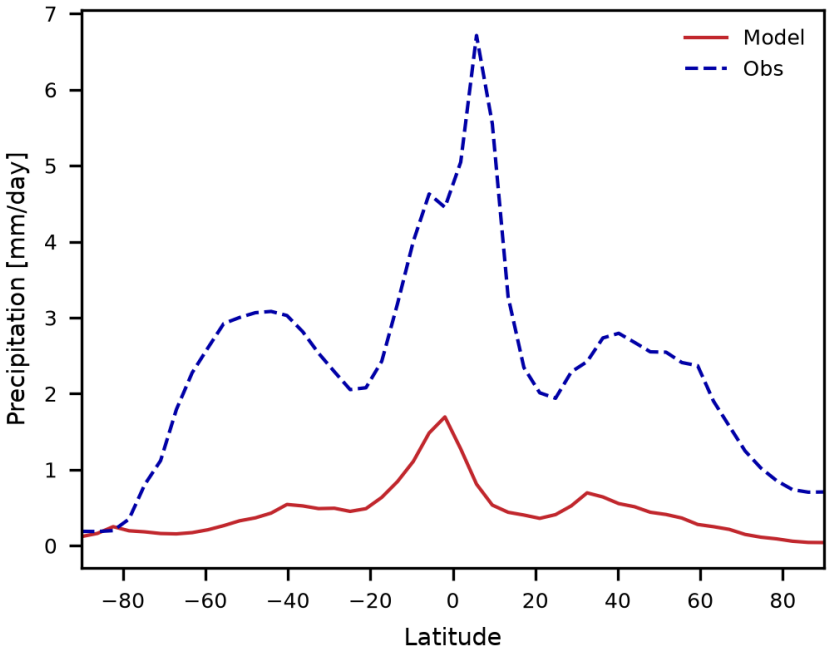

Figure 19.2: Zonal-mean precipitation versus latitude: Chronos-ESM's multi-level atmosphere (solid) and the observed climatology (dashed). The model reproduces a distinct equatorial peak near 5–15°N — a genuine ITCZ — but it is too dry: the peak undershoots the observed maximum and the subtropical/extratropical totals are low. The single-level atmosphere has no such equatorial peak.

19.5 Assessment

What works. The qualitative win is unambiguous and is documented across the model's development record. Moving from the single-level to the multi-level moist atmosphere lifts the precipitation *pattern correlation* markedly: in the SST-forced benchmark it rises from 0.16 for the single level to 0.34 for the multi-level atmosphere, and the precipitation standard-deviation ratio recovers toward observed (from 0.41 to ~ 0.8). Crucially, the correlation gain comes with a *physically correct* ITCZ — the model now rains on the equator instead of in the subtropics. In the fully coupled, flux-corrected control run (where the tropical SST is restored toward WOA so it is warm enough to convect), the same revival is even clearer: the precipitation pattern correlation reaches 0.37, and the documented narrative of that run names the "revived ITCZ" as one of its two headline gains. The chain is exactly the one the physics predicts in Section 19: warm tropical SST → deep convection → Hadley ascent → equatorial rain band.

What does not (yet). Three honest limitations stand out.

1. **The ITCZ is still too dry and too weak.** Figure 19.2 shows the equatorial peak undershooting the observed maximum, and the bias map (Figure 19.3) shows a coherent dry band straddling the equator. A pattern correlation of ~ 0.34–0.37 is a real improvement but is still modest. The large-scale condensation closure has no convective organization or explicit deep-convection parameterization, and at T31 ($\sim 3.75°$) the rain band is only a few grid cells wide, so it is unavoidably blurred and under-peaked.
2. **Tropical SST limits the convection.** The free-running coupled model carries a tropical cold-tongue bias (tropical SST several degrees below WOA), which suppresses the deep convection that should drive the ITCZ. The control run quoted above relies on a Haney SST restoring (Haney, 1971) toward WOA to remove that bias; the ITCZ is therefore partly *earned* by the flux correction rather than freely predicted. A free-running warm tropical SST remains a standing target (Chapter 13).
3. **No strong interannual (ENSO-like) variability.** At this configuration Chronos-ESM has no documented El Niño–Southern Oscillation. The coupled control is run as a stable, flux-corrected,

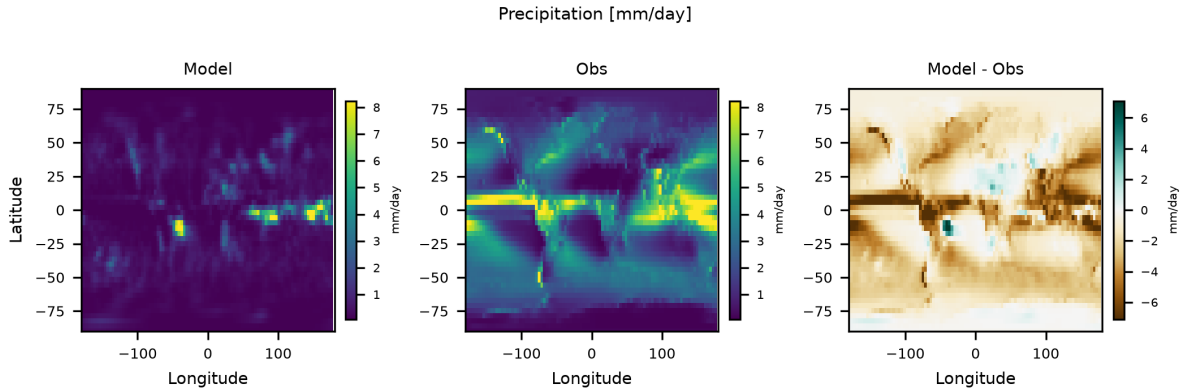

Figure 19.3: Precipitation maps: model (left), observed climatology (centre), and their difference (right). The model captures the broad equatorial rain band and the Pacific/Atlantic structure, but is systematically too dry along the deep-tropical maximum (brown band near the equator in the difference map) and lacks the sharp South Pacific and South Atlantic convergence zones.

drift-free mean state; the ocean's diagnostic-velocity dynamics and the coarse T31 atmosphere do not sustain the coupled Bjerknes feedback that produces ENSO. The tropical climate documented here is a *mean-state* evaluation, not a variability one — and the spurious noise that the AMOC closure exhibits (Chapter 17) should not be mistaken for organized tropical variability.

Monsoons. Monsoon circulations — the seasonal land–sea thermal contrast that reverses the low-level winds and drives summer rainfall over South Asia, West Africa, and the Americas — are only weakly represented. Two factors limit them. First, resolution: at $\sim 3.75°$ the land–sea coastlines, orographic barriers (the Tibetan Plateau, the Ethiopian Highlands), and narrow monsoon rain bands are smoothed to a few grid cells, and the sub-grid Central-American isthmus even has to be closed by hand in the control run (Chapter 13). Second, the land surface is a simplified slab plus bucket-hydrology model (Chapter 12): it captures the first-order land–ocean temperature contrast but not the soil-moisture–precipitation feedbacks and surface heterogeneity that sharpen real monsoon onset and intensity. The model produces a continental-scale seasonal land warming, but the resulting monsoon overturning and rainfall are muted. Sharper monsoons, like a sharper ITCZ, need higher resolution and a richer land surface.

Why the surface wind pattern lags the rain. It is worth contrasting the tropical *thermodynamic* skill (precipitation, which the multi-level atmosphere clearly improves) with the tropical *dynamic* skill (surface wind and pressure patterns, which it does not yet improve). The eddy momentum flux that builds the observed surface-wind belts scales with the equator-to-pole SST gradient; at T31 the regridded WOA SST has only a ~ 22.6 K gradient, because the sharp Gulf Stream, Kuroshio, and ACC fronts are smoothed away. That weak gradient is enough to organize moisture into an ITCZ but too weak to reproduce the observed surface-wind pattern — the same decoupling discussed for the mid-latitudes in Chapter 18. Tropical precipitation is the field where the multi-level atmosphere most clearly earns its added cost.

19.6 Summary

Chronos-ESM's multi-level moist atmosphere produces a genuine ITCZ — an equatorial rain band organized by the resolved Hadley circulation — where the single-level atmosphere structurally cannot, lifting the precipitation pattern correlation from ~ 0.16 to ~ 0.34–0.37 and moving the rain onto the equator. The honest counterweights are that the ITCZ remains too dry and too weak at T31, that it currently relies on an SST flux correction to keep the tropics warm enough to convect, that strong interannual (ENSO-like) variability is absent at this configuration, and that monsoons are limited by resolution and the simplified land surface. The tropical mean state is a documented success of the multi-level upgrade; tropical variability and a free-running warm tropics are open work.

Model-as-laboratory exercises

Exercise 19.1 (The vertical-structure argument)**.** Run the single-level atmosphere and the multi-level DINOSAUR atmosphere on the *same* WOA SST and compare their zonal-mean precipitation profiles (`make_readme_figures.py`). Explain, in terms of the vertical motion field each model can or cannot represent, why only the multi-level profile has an equatorial peak. Quantify the difference in pattern correlation.

Exercise 19.2 (SST control on the ITCZ)**.** The ITCZ should follow warm SST. Force the multi-level atmosphere with a WOA SST field to which you have added a $+2$ K anomaly in the Northern-Hemisphere tropics and -2 K in the Southern-Hemisphere tropics. Predict, then measure, how the latitude of the zonal-mean precipitation peak shifts. Relate the shift to the cross-equatorial energy-transport argument of Vallis (2017).

Exercise 19.3 (Quantifying the dry bias)**.** Using the precipitation bias map (Figure 19.3) from a control checkpoint, compute the area-weighted mean bias *within* $\pm 10^\circ$ of the equator and compare it to the global-mean bias. Is the deep-tropical dry bias larger or smaller than the global value? What does this tell you about whether the model's water budget error is concentrated in the ITCZ or spread evenly? Discuss whether raising horizontal resolution or strengthening the condensation closure is the more promising fix.

Part V

Community, Standards, and Teaching

Chapter 20

Toward CMIP7 Participation

The Coupled Model Intercomparison Project (CMIP) is the organising backbone of modern climate science: a community agreement that, whatever model you build, you will run it through a common set of experiments and publish the output in a common format, so that dozens of independently developed models can be compared apples-to-apples and synthesised into assessments such as the IPCC reports. CMIP6 (Eyring et al., 2016) formalised this into a small, mandatory core (the *DECK*) plus a federation of opt-in Model Intercomparison Projects (MIPs); CMIP7 inherits and extends that design. This chapter asks an honest, instructive question: *what would it take for Chronos-ESM — a T31, partially idealised, research-preview model — to participate in CMIP7?* We are not claiming that it can today. The value of the exercise is pedagogical: it forces us to lay the model's existing capabilities (Chapters 13, 14, 16, 17) against a community standard and to name, precisely, the gaps between a teaching model and a CMIP-class one.

We proceed in four steps. §20.1 explains the DECK and the historical/ScenarioMIP simulations. §20.2 maps Chronos-ESM's existing experiments onto these entry points: the flux-corrected control as a `piControl` analogue, and the abrupt-CO_2 / q-flux machinery of Chapter 14 as an `abrupt-4xCO2` / `1pctCO2` analogue. §20.3 is the candid accounting: data standards (CF, CMOR, the data request, ESGF) and the scientific gaps (no full radiative transfer, a TCR-proxy rather than a true ECS, coarse resolution). §20.4 frames a realistic roadmap and the genuinely distinctive value Chronos-ESM brings — speed, differentiability, and classroom accessibility.

Remark (A caveat about a moving target). CMIP7 is, at the time of writing, still being specified by the WCRP CMIP Panel and its task teams: the precise experiment list, the data-request variables, and the governance of forcing datasets are evolving. We therefore describe CMIP7 at the level of its *stable design principles* — which are continuous with CMIP6 (Eyring et al., 2016) — and flag explicitly where details remain provisional. Treat the specific experiment names and variable counts as illustrative of the *structure*, not as a frozen specification.

20.1 The CMIP DECK and the standard simulations

The CMIP design separates a small *entry card* that every participating model must run from a large menu of topical MIPs that each model may join. The entry card is the **DECK** — the "Diagnostic, Evaluation and Characterization of Klima" — four experiments chosen because, between them, they characterise a coupled model's baseline behaviour and its response to the canonical CO_2 perturbations (Eyring et al., 2016).

`piControl` — pre-industrial control.
A long (multi-century) coupled integration with forcings fixed at their pre-industrial (nominally 1850, $C_0 \approx 280\,\mathrm{ppm}$) values. It establishes the model's *unforced baseline*: its mean climate, its internal variability, and its drift. Every forced experiment is read as an anomaly *relative to* `piControl`, so a stable, well-characterised control is the foundation on which all other DECK experiments rest.

`abrupt-4xCO2`.
CO_2 is *instantaneously quadrupled* from the control value and held there while the coupled system adjusts. The transient of global-mean surface temperature against top-of-atmosphere (TOA) energy imbalance — the *Gregory regression* — yields the effective radiative forcing, the net climate-feedback

parameter λ, and an estimate of the equilibrium climate sensitivity (ECS). It is the single most diagnostic experiment for a model's sensitivity.

`1pctCO2`.
CO_2 increases at 1% per year (compounding), reaching a doubling near year 70 and a quadrupling near year 140. The warming at the year of doubling defines the *transient climate response* (TCR). Because the forcing ramps rather than jumps, it samples the coupled system's response on the decadal-to-century timescale that is most policy-relevant.

AMIP — atmosphere-only.
The atmosphere (and land) model is run with *observed* sea-surface temperature and sea-ice prescribed as lower boundary conditions over the historical period. This isolates atmospheric and land skill from coupled-ocean biases — exactly the logic of our atmosphere-only forced benchmark in Chapter 16.

Beyond the DECK, two further simulation families are so widely used that they are effectively standard:

`historical`.
A coupled run from ∼1850 to the near-present, forced by the observed evolution of greenhouse gases, aerosols, volcanic and solar forcing, and land use. It is the run compared directly against the instrumental record, and it bridges `piControl` to the future.

ScenarioMIP.
Future projections under shared socioeconomic pathways (SSPs) — emissions and concentration trajectories spanning low-mitigation to high-emission worlds — branched from the end of `historical`. These produce the spread of twenty-first-century outcomes that anchor adaptation and mitigation assessments.

The unifying logic is a chain: `piControl` fixes the baseline, `abrupt-4xCO2` and `1pctCO2` characterise sensitivity, AMIP isolates the atmosphere, `historical` validates against observations, and ScenarioMIP projects forward. A model that can credibly run this chain — and publish it in the common format — is, operationally, a CMIP model.

20.2 Mapping Chronos-ESM's existing capabilities

Chronos-ESM already possesses, in prototype form, analogues of three DECK entry points. We describe each honestly, naming both the correspondence and where the analogy breaks.

20.2.1 The flux-corrected control as a `piControl` analogue

The 100-year coupled control of Chapter 16 — the multi-level DINOSAUR atmosphere coupled to the z-level ocean, driven by
↪ *Code:* `experiments/run_dino_control.py` — the library coupled control harness
— is Chronos-ESM's structural counterpart to `piControl`. It runs at fixed $C_0 = 280\,\mathrm{ppm}$ (the driver steps the control with `co2_ppm=280.0`), it is integrated for a century with yearly checkpoints, and it equilibrates to a stable mean state (global SST $\approx 18\,^\circ\mathrm{C}$, SSS ≈ 34.7, both flat over the run). That stability and the resumable checkpoint/restart machinery are exactly the properties `piControl` demands: a baseline against which a forced anomaly can be defined.

There is, however, a load-bearing difference, and it must be stated up front: our control is *flux-corrected*. The surface heat flux restores SST toward the WOA18 climatology with a Haney term (Chapter 14, $\tau = 30\,\mathrm{d}$, $\lambda \approx 79\,\mathrm{W\,m^{-2}\,K^{-1}}$) to suppress a structural cold-tongue bias. A modern CMIP `piControl` is expected to be *free of flux correction* — the practice was largely abandoned after CMIP2 precisely because it masks model error and contaminates the forced response. So the analogy is structural (a stable, fixed-forcing baseline with the right bookkeeping) but not yet methodological (the surface is constrained, not free).

The escape from that constraint is the *q-flux* construction of Chapter 14: the time-mean restoring flux over the converged control is frozen into a fixed field $\overline{Q}$ and the strong restoring is replaced by a weak, long-τ anomaly term, freeing SST to respond. The control harness already accumulates and writes this $\overline{Q}$ per checkpoint (`state_d<DAY>_qflux.npy`). This is the bridge from a flux-corrected control to a forcing-responsive experiment — and it is exactly the machinery the next two analogues use.

20.2.2 Abrupt and ramped CO_2 as DECK-sensitivity analogues

Chronos-ESM's forced-CO_2 experiment (Chapter 14,
↪ *Code:* `experiments/run_dino_co2.py` — the q-flux forced driver
) is a direct prototype of the CMIP CO_2 experiments. It branches two free-mode trajectories from a common control state and frozen $\overline{Q}$ — a baseline at C_0 and a forced branch at an elevated C — and reads the warming as the drift-cancelling difference of their ocean-mean SST. The Myhre forcing $F(C) = 5.35 \ln(C/C_0)\,\mathrm{W\,m^{-2}}$ (Myhre et al., 1998) is the same logarithmic law CMIP models use to set the CO_2 radiative forcing.

The mapping onto the DECK is one of *protocol shape*:

`abrupt-NxCO2`.
The driver imposes a step change in C and holds it — precisely the `abrupt` protocol. The completed run is an abrupt-2×CO_2 experiment, which gave a warming

$$\Delta\mathrm{SST} \approx +1.58\,\mathrm{K}, \qquad \frac{\Delta\mathrm{SST}}{F} \approx 0.43\,\mathrm{K}\,(\mathrm{W\,m^{-2}})^{-1}, \tag{20.1}$$

with $F = 5.35 \ln 2 \approx 3.71\,\mathrm{W\,m^{-2}}$. Reaching the DECK's nominal `abrupt-4xCO2` requires only $C = 4C_0 = 1120\,\mathrm{ppm}$ (Exercise 20.1); the harness already accepts an arbitrary `-co2`, so the experiment is a one-flag change, not new physics.

`1pctCO2`.
A compounding $1\%\,\mathrm{yr^{-1}}$ ramp is a time-dependent schedule $C(t) = C_0\,(1.01)^t$ fed to the same forced driver. The model is differentiable and fast enough that a 140-year ramp is cheap; the missing piece is only the scheduling loop, not the response mechanism.

The break in the analogy is what Equation (20.1) *measures*. As Chapter 14 establishes at length, the $0.43\,\mathrm{K}\,(\mathrm{W\,m^{-2}})^{-1}$ slope is a **transient, surface-forcing proxy for TCR — not an ECS**. The forcing enters as a fixed surface-heat term with no TOA energy closure, so the Gregory regression that defines effective forcing and λ in a real `abrupt-4xCO2` cannot be performed: there is no top-of-atmosphere imbalance to regress against. Chronos-ESM can produce a number with the right *name* and a defensible sign and order of magnitude, but not the feedback decomposition that is the scientific payload of the DECK sensitivity experiments.

20.2.3 An AMIP analogue, and what is genuinely missing

The atmosphere-only forced benchmark of Chapter 16 — the DINOSAUR atmosphere run with SST held to climatology, scored against ERA5 — is methodologically an AMIP run in miniature: it isolates atmospheric skill from coupled-ocean bias. A true AMIP simulation would prescribe the *observed, time-varying* monthly SST and sea-ice over the full historical period (rather than a fixed climatology) and run the resolved seasonal cycle; the infrastructure (an SST-forced DINOSAUR atmosphere with prognostic moisture and an ITCZ) already exists, so the gap is observational forcing data and a seasonal driver, not dynamics.

`historical` and ScenarioMIP, by contrast, are not yet within reach. Both require the full suite of time-varying forcings — aerosols, ozone, volcanic and solar variability, land use — none of which Chronos-ESM currently ingests; it has a single radiative channel, CO_2 (Chapter 14). Adding even concentration-driven greenhouse-gas histories is tractable; the aerosol and land-use forcings are substantial work. These are honestly out of scope for the research preview.

20.3 What genuine participation requires

Running experiments that *resemble* the DECK is necessary but far from sufficient. CMIP is as much a data-standards project as a science project. We separate the requirements into a data-engineering tier and a scientific tier, and treat both candidly.

20.3.1 Data standards: CF, CMOR, the data request, ESGF

For output to be usable by the community — and ingestible by the analysis tools that produce IPCC figures — it must be *CMOR-ised*: rewritten by (or to the standard of) the Climate Model Output Rewriter so that it is CF-compliant. Concretely:

Table 20.1: CMIP entry experiments and Chronos-ESM's current standing. "Analogue" = a prototype with the right protocol shape but a documented methodological gap; "infrastructure" = the dynamics exist but the forcing data / driver do not; "out of scope" = missing forcings or physics.

CMIP experiment	Chronos-ESM status	Key gap
`piControl`	Analogue (flux-corrected)	not yet flux-correction-free
`abrupt-4xCO2`	Analogue (ran 2×)	TCR-proxy, no TOA/ECS
`1pctCO2`	Infrastructure	ramp scheduler only
AMIP	Analogue (climatological)	needs observed time-varying SST
`historical`	Out of scope	no aerosol/solar/volcanic/land-use
ScenarioMIP	Out of scope	SSP forcings + `historical` branch

- **CF conventions.** Every variable carries a controlled `standard_name`, canonical units, and complete coordinate metadata; the files are self-describing netCDF that any CF-aware tool can read without bespoke code. Chronos-ESM already writes netCDF, but with internal names and conventions — it is not CF-compliant out of the box.
- **The CMIP data request.** CMIP specifies, per experiment, exactly which variables to archive, at which frequencies (monthly, daily, sub-daily), on which realms (atmos, ocean, land, seaice), and on which standardised grids. This is the contract: a participating model must deliver *this* list, named and shaped *this* way. Mapping Chronos-ESM's state (modal spectral atmosphere; z-level ocean on the model grid) onto the requested variables and grids is a non-trivial diagnostics-and-regridding layer that does not yet exist.
- **Standard grids.** Output is typically requested on the native grid *and* regridded to a common grid. Chronos-ESM's atmosphere is spectral (modal state; physical fields recovered on a Gaussian grid, Chapter 9) and its ocean is on a stretched lat–lon z-grid; a conservative regridder to the request grids would be needed.
- **ESGF publication.** Finally, the CMOR-ised output is published to the Earth System Grid Federation with the full controlled vocabulary of global attributes (`source_id`, `experiment_id`, `variant_label`, tracking IDs, license, and citation). Registering a `source_id` and meeting the publication QC is itself a formal process.

None of this is scientifically deep, but all of it is mandatory, and collectively it is a real engineering project. It is also the most *tractable* part of the gap: it is well-specified, tooling exists (the CMOR library, the CF checker), and it could be added as a post-processing layer without touching the model physics.

20.3.2 Scientific gaps

The harder gaps are physical, and the model's own validation record (Chapters 16–19) names them plainly.

1. **No full radiative transfer, hence no ECS.** Chronos-ESM's CO_2 forcing is a single surface-heat channel (Chapter 14); there is no multi-band, multi-species radiation scheme and no TOA energy budget. Without TOA closure there is no Gregory regression, no effective radiative forcing, no feedback parameter λ, and therefore *no true ECS* — only the TCR-flavoured surface proxy of Equation (20.1). This is the single largest scientific gap to CMIP-class sensitivity science.
2. **The TCR-proxy is not a calibrated TCR.** Even read as a transient measure, $0.43\,\mathrm{K}\,(\mathrm{W\,m^{-2}})^{-1}$ omits the water-vapour, lapse-rate, cloud and albedo feedbacks that amplify the response in a full GCM, and is computed about a cold-tropics base state. It is a *lower-bound-flavoured* estimate, not a number that can stand beside CMIP TCR values.
3. **Resolution.** At T31 ($\sim 3.75°$) the model cannot resolve SST fronts, mesoscale ocean eddies, tropical cyclones, or sharp orographic precipitation. The smoothed equator-to-pole SST gradient is too weak to build observed mid-latitude surface westerlies from the resolved eddy momentum flux (Chapter 18),

and mean sea-level-pressure pattern skill stays low. CMIP-class models run at far finer resolution; coarse resolution is an honest, structural limitation of a model designed to run hundreds of years per GPU-day.

4. **Ocean dynamics and the AMOC.** The overturning is still produced by an interim density-driven closure rather than a fully prognostic-momentum core (Chapters 7, 8, 17); the AMOC is $\sim$ 15–34 Sv but noisy. A clean CMIP ocean response would need the prognostic core under construction.

5. **Component completeness.** The carbon cycle, atmospheric chemistry, dynamic vegetation and ice sheets that define the *Earth System* tier of CMIP are absent or stubbed. Chronos-ESM is a coupled physical climate model, not yet a full ESM in the CMIP sense.

The discipline of this section is the discipline of the whole manual: a research model earns trust by being explicit about what it does *not* do. None of these gaps is hidden; each is documented and several have a stated path forward.

20.4 A realistic roadmap, and the value Chronos-ESM brings

20.4.1 An incremental roadmap

The gaps fall into a natural difficulty ordering, and the cheapest, most instructive steps come first.

1. **Run the CO_2 DECK shape (low cost).** Extend the completed 2× run to `abrupt-4xCO2` ($C = 1120$ ppm) and add a 1% yr^{-1} ramp scheduler for a `1pctCO2` analogue. Both are driver-level changes on existing physics (Exercise 20.1).

2. **A free (flux-correction-free) control (medium).** Use the frozen q-flux as the *only* surface correction and demonstrate a stably drifting free control — the methodological step from a flux-corrected to a CMIP-shaped `piControl`.

3. **A CMOR/CF export layer (medium, well-specified).** A post-processing module that maps Chronos-ESM's diagnostics to a subset of the data request, writes CF-compliant netCDF on a standard grid, and passes the CF checker. This is the single highest-leverage engineering step.

4. **A seasonal AMIP driver (medium).** Prescribe observed time-varying SST to the DINOSAUR atmosphere and run the seasonal cycle for an AMIP-style benchmark.

5. **TOA radiation and a true sensitivity (high).** Add a radiation scheme with a TOA budget so the Gregory regression — and hence effective forcing, λ, and ECS — becomes possible. This is the deep scientific upgrade.

6. **Forcing breadth and Earth-system components (high).** Aerosol, solar, volcanic and land-use forcings for `historical`/ScenarioMIP, and the biogeochemical components for the ESM tier.

The honest framing is that steps 1–4 are achievable add-ons that would let Chronos-ESM run *CMIP-shaped* experiments and publish standards-compliant output; steps 5–6 are the research that would make it CMIP-*class*. A textbook model that completes 1–4 is already an excellent vehicle for teaching the CMIP workflow.

20.4.2 The distinctive value Chronos-ESM brings

Chronos-ESM is not, and need not be, a competitor to operational ESMs. Its value lies in three properties that the big models largely lack.

Speed.
At $\sim$280 simulated years per GPU-day (T31), the multi-century integrations that take an operational model weeks on a supercomputer take Chronos-ESM hours on a single accelerator. A student can run the entire DECK shape in an afternoon and *see* the chain of cause and effect.

Differentiability.
Built end-to-end in JAX (Chapter 3), Chronos-ESM can return the gradient of any scalar diagnostic with respect to any parameter or forcing. The 50-year abrupt-CO_2 difference of Chapter 14 collapses to a single `jax.grad` evaluation (Chapter 15). For sensitivity analysis, calibration, and inverse

problems this is a capability the CMIP ensemble does not have — and it is the same property behind differentiable emulators such as NeuralGCM (Kochkov et al., 2024), on whose DINOSAUR dycore Chronos-ESM's atmosphere is built.

Classroom accessibility.
The model is small, open (Apache-2.0), readable, and documented equation-by-equation in this manual, with each equation tied to its source line. It is a model a student can run, modify, break, and understand — a laboratory for *learning* how a coupled model and the CMIP protocol work, rather than a black box to be trusted.

The realistic conclusion, then, is not "Chronos-ESM will join CMIP7" but "Chronos-ESM is the ideal place to *learn* what joining CMIP7 means." It can run the experiments in the shape of the DECK, it makes the forcing–response logic transparent and differentiable, and it is honest about every gap to the standard. For a university course, that combination is worth more than another opaque production model.

Exercises

Exercise 20.1 (From 2× to the DECK `abrupt-4xCO2`)**.** The completed experiment quadrupled neither concentration nor forcing: it ran $C = 560\,\mathrm{ppm}$. (a) Using the Myhre law $F(C) = 5.35\ln(C/C_0)$ with $C_0 = 280\,\mathrm{ppm}$, compute the forcing for the DECK's `abrupt-4xCO2` ($C = 1120\,\mathrm{ppm}$) and confirm it is exactly twice the 2× value. (b) If the surface response stayed linear in forcing at the proxy slope of Equation (20.1), what ΔSST would you predict for 4×? (c) Give one physical reason the *true* response is unlikely to be exactly linear, and state whether your prediction is more likely an over- or under-estimate, citing the missing-feedback argument of Chapter 14.

Exercise 20.2 (Why a flux-corrected control is not a `piControl`)**.** Our control restores SST toward WOA18 with $\lambda \approx 79\,\mathrm{W\,m^{-2}\,K^{-1}}$. (a) Explain in two or three sentences why this disqualifies it as a CMIP `piControl`, referring to how the restoring would interact with an imposed forcing (use the steady-balance argument $\Delta\mathrm{SST} = F/\lambda$ from Chapter 14). (b) Describe how the frozen q-flux $\overline{Q}$, written per checkpoint by the control harness, converts this into a flux-correction-*free* control suitable as a CMIP baseline. (c) What single diagnostic would you monitor to demonstrate that the resulting free control does not drift unacceptably over a century?

Exercise 20.3 (Scoping a CMOR export)**.** Pick three output variables — one each from the atmosphere, ocean, and land — that a CMIP data request would demand (e.g. near-surface air temperature, sea-surface temperature, precipitation). For each, name (a) the CF `standard_name` and canonical unit, (b) the Chronos-ESM state field or diagnostic it would be derived from (cite a chapter or a code path from Appendix C), and (c) the regridding step needed to put the spectral-atmosphere or stretched-ocean field onto a standard lat–lon grid. Then argue, in a sentence, why this export layer is the *highest-leverage* step toward participation despite involving no new physics.

Chapter 21

Teaching with Chronos-ESM

Most graduate climate courses teach Earth System Models the way one teaches an opaque instrument: students read the governing equations on a chalkboard, then run a community model (CESM, MOM6, an EC-Earth configuration) as a black box on a remote queue, waiting hours or days for output they did not generate and cannot fully read. The pedagogy and the software pull in opposite directions — the equations are transparent but inert, the model is alive but inscrutable. Chronos-ESM was not built to be a teaching model, but several of its design choices make it an unusually good one. This chapter argues why, lays out a 12–14 week graduate course that uses the model as a weekly laboratory, gives concrete lab assignments drawn from the *real* experiment scripts in the repository, and closes with notes for instructors.

A standing caveat frames everything below. Chronos-ESM is research software — a research preview, not a production climate model (Chapter 1). Some components are validated against observations (ocean T/S structure, the single-level surface winds, the multi-level ITCZ); others are explicit simplifications or carry documented biases (the single-level atmosphere has no synoptic eddies; the interim AMOC closure is a box-model parameterization, not prognostic momentum). This is a feature for teaching, not a bug: a model whose limitations are written down is a model whose limitations a student can *discover*, which is the most durable lesson a climate course can deliver.

21.1 Why Chronos-ESM teaches well

21.1.1 Minutes-to-hours runtime closes the experiment loop within one session

The single most important pedagogical property of Chronos-ESM is speed. On a single GPU the coupled model integrates roughly 280 simulated years per day (Chapter D), so a multi-decadal control run finishes in tens of minutes and a short forced experiment in single minutes. Even on a laptop CPU the differentiable kernels are fast enough for the short probes the labs below use. The consequence is that a student can *conceive, run, plot, and interpret* a numerical experiment inside one three-hour lab. The hypothesis-to-result loop — the actual engine of scientific learning — closes before the student leaves the room, rather than across a week-long queue wait during which the original question is forgotten. This is the difference between an experiment a student *owns* and a figure a student *receives*.

21.1.2 Full source legibility

Chronos-ESM is a few tens of thousands of lines of readable JAX/Python, not millions of lines of Fortran accreted over three decades. A student can open `chronos_esm/ocean/veros_driver.py` and read, in one sitting, the actual tracer advection their experiment exercises; they can open `chronos_esm/ocean/overturning.py` and see the exact line where the salt-advection feedback gain enters (Chapter 8). The equation on the lecture slide and the line of code that integrates it are separated by minutes of reading, not by an institutional firewall. This collapses the chalkboard/black-box gap described above: the model *is* the lecture notes, executable. Throughout the book the *Code:* margin pointers exist precisely so a reader can jump from a result to its source.

21.1.3 Differentiability makes sensitivity and inverse methods first-class

Chronos-ESM is end-to-end differentiable (Chapters 3, 15): JAX's reverse-mode autodiff (Bradbury et al., 2018) returns the exact gradient of any scalar diagnostic with respect to any input — a parameter, a boundary condition, an initial state. In a conventional course, "sensitivity" is taught as a finite-difference afterthought (run the model twice, subtract, divide). Here a student can compute $\partial(\text{AMOC})/\partial(\text{subpolar salinity})$ as a single backward pass and cross-check it against a central finite difference in the same script (`experiments/verify_gradient.py`, `experiments/p0_amoc_gradient_baseline.py`). This turns abstract topics — adjoint sensitivity, 4D-Var data assimilation, gradient-based parameter calibration, the envelope theorem at a bifurcation — into things a student can *run*. Few teaching models can do this at all; fewer can do it in a script short enough to read in a lab.

21.2 A 12–14 week graduate course

The course below maps one lecture block per week to the book's chapters and pairs each with a computer lab that uses the model. It is sized for a one-semester graduate course (12 core weeks plus 2 optional project weeks) and assumes a parallel track of problem sets on the analytic material. The arc is deliberate: build the ocean first (it is the validated core), add the atmosphere, couple, then spend the back half on the three things differentiability and speed make uniquely teachable — forcing response, tipping, and inverse methods.

Week 1 — Orientation and the differentiable paradigm.
Lecture: Chapters 1, 2, 3. What an ESM is; the model hierarchy; why "differentiable" changes what you can ask. Lab: install, run a short control (`run_century_physics.py -years 5`), open three source files and trace one tracer through the code. Deliverable: a plot of global-mean SST settling, with a paragraph on what the student read in the source.

Week 2 — Ocean governing equations.
Lecture: Chapter 4. Primitive equations, hydrostatic and Boussinesq approximations, the equation of state. Lab A (below): perturb the EOS and watch the circulation respond.

Week 3 — Ocean mixing and parameterization.
Lecture: Chapter 5. Vertical mixing, isopycnal diffusion, the Gent–McWilliams eddy parameterization (Gent and McWilliams, 1990). Lab: vary `kappa_gm` and `kappa_h` in a short control; compare the resulting thermocline depth.

Week 4 — Ocean numerics.
Lecture: Chapter 6. The C-grid, flux-form advection, the z-level staircase bathymetry, stability and the CFL condition (Griffies, 2004). Lab: halve the time step and the grid masking; observe stability limits and conservation.

Week 5 — The prognostic ocean core.
Lecture: Chapter 7. Diagnostic versus prognostic momentum; why the current ocean's velocities are reconstructed and what that costs (the $\partial(\text{AMOC})/\partial\rho \approx 0$ blocker). This is the week to be honest about a research model's frontier. Lab: run `p0_amoc_gradient_baseline.py` and measure the blocker directly.

Week 6 — Overturning and the AMOC.
Lecture: Chapter 8. The overturning streamfunction; the Stommel two-box model (Stommel, 1961); the interim density-driven closure. Lab C (below): hose the AMOC.

Week 7 — The atmosphere I: multi-level dynamics.
Lecture: Chapter 9. Spectral primitive equations, sigma coordinates, baroclinic instability, the DINOSAUR dycore behind NeuralGCM (Kochkov et al., 2024); the Held–Suarez idealized benchmark (Held and Suarez, 1994). Lab: run `dino_held_suarez.py`; recover the twin midlatitude jet from rest.

Week 8 — The atmosphere II: the single level and its limits.
Lecture: Chapter 10. The barotropic single level, the eddy-momentum-flux relaxation, why a single level has no synoptic systems. Lab: score the atmosphere against ERA5 with `validate_control.py -demo`; read the skill table honestly (where it wins, where it fails).

Week 9 — Sea ice, land, and the coupler.
Lecture: Chapters 11, 12, 13. Thermodynamic ice, the bucket land model, bulk-formula surface fluxes,

flux correction. Lab: run a short coupled DINOSAUR↔ocean control (`run_dino_coupled.py -days 30`) and inspect the surface energy budget.

Week 10 — Forcing and response.
Lecture: Chapter 14. Radiative forcing of CO_2 (Myhre et al., 1998), transient versus equilibrium response, feedbacks, why Chronos-ESM's warming is a forcing-vs-warming *proxy* not an ECS. Lab D (below): double CO_2.

Week 11 — Differentiable applications.
Lecture: Chapter 15. Adjoint sensitivity, gradient-based calibration, 4D-Var, calibrating a bifurcation by AD fold-continuation. Lab E (below): take a gradient and an inverse.

Week 12 — Evaluating a model: climatology and CMIP.
Lecture: Chapters 16, 20. Skill metrics, Taylor diagrams, model intercomparison (Eyring et al., 2016). Lab B (below): build and read a validation scorecard.

Week 13–14 — Selected phenomena / project work.
Lecture (pick by interest): Chapters 17 (tipping), 18 (jets and storm tracks), 19 (tropics). Lab: supervised work on the term project (Section 21.4). Reference appendices throughout: A (notation), B (parameters), C (code map), D (running).

21.3 Lab assignments

The six labs below are drawn directly from runnable scripts in `experiments/`. Each gives a learning goal, the command to run, what to plot, and discussion questions. All are sized to finish within one lab session. Commands assume the repository root and an activated environment (Chapter D); paths and flags match the scripts as shipped.

21.3.1 Lab A — Run a control and perturb the equation of state

Learning goal. Connect the equation of state to the large-scale circulation: density sets pressure gradients, which drive flow. Establish what a "control" run is and how to read a spin-up.

Run.

```
python experiments/run_century_physics.py -years 10 -ocean-ic woa
```

Then re-run with a perturbed thermal-expansion or haline-contraction coefficient in the EOS (`chronos_esm/ocean/`; a one-line edit the source-legibility property makes easy), saving to a second output directory.
↪ *Code:* `experiments/run_century_physics.py` — WOA-seeded control / spin-up

Plot. Global-mean SST and SSS time series for control versus perturbed; a surface-current or streamfunction map for each; the difference map.

Discussion. Which way did the gyres shift when you made seawater more (or less) sensitive to temperature? Did the perturbation change the *mean* state or the *spatial pattern* more? The control still relaxes salinity in a high-latitude sponge — how does that constrain what your perturbation can do?

21.3.2 Lab B — Score the model against observations

Learning goal. Learn what "validation" means quantitatively: bias, RMSE, pattern correlation, standard-deviation ratio, and the Taylor framework (Chapter 16). Learn to read a model's skill *honestly*.

Run.

```
python experiments/validate_control.py -demo 200 -era5
```

(or, against saved checkpoints, `-states "outputs/century_physics/year_*.nc" -era5`).
↪ *Code:* `experiments/validate_control.py` — score a run against WOA18 + ERA5

Plot. The auto-generated scorecard table, the bias maps, and the zonal-mean comparisons the script writes.

Discussion. The single-level atmosphere scores surface zonal-wind correlation near 0.7 but mean-sea-level-pressure pattern correlation near 0 — why does it get winds right and pressure wrong? (Hint: eddies.) Which fields are *predicted* and which are *imposed* (in the flux-corrected coupled control, SST correlation is 1.00 by construction)? Pick the field you trust least and defend why.

21.3.3 Lab C — Hose the AMOC and find a tipping point

Learning goal. Reproduce a saddle-node bifurcation with hysteresis from the salt-advection feedback (Stommel, 1961); understand why a rate-dependent sweep is ambiguous and a fixed-forcing on/off test is rigorous (Chapter 17).

Run. A short self-contained quasi-static sweep:

```
python experiments/run_amoc_hosing.py -spinup-years 20 -fmax 0.8 -nsteps 6
-hold-years 15 -haline-gain 6 -k-vel 6e-5 -contrast-depth 300
```

↪ *Code:* `experiments/run_amoc_hosing.py` — quasi-static subpolar freshwater hosing sweep

Plot. AMOC at 26.5°N versus hosing F for the up-leg and down-leg on one axis — the hysteresis loop. The calibrated configuration opens a window $\approx [0.38, 0.75]$ Sv centered near 0.6 Sv.

Discussion. Why does the down-leg sit below the up-leg over a finite F window, and what physical feedback makes the "off" state self-sustaining? The branches are noisy ($\pm \sim 10$ Sv) — is that numerical noise or a genuine relaxation oscillation, and how would you tell? Why is the interim closure a box-model parameterization rather than prognostic momentum, and what would a clean bifurcation require (Chapter 7)?

21.3.4 Lab D — Double CO_2 and measure the warming

Learning goal. Separate radiative *forcing* from climate *response* (Myhre et al., 1998); understand what a transient forcing-vs-warming slope is and why it is *not* equilibrium climate sensitivity in this model.

Run. From a converged control checkpoint, two branches (control ppm and 2×):

```
python experiments/run_dino_co2.py -ckpt <control_ckpt> -co2 560 -years 50
```

↪ *Code:* `experiments/run_dino_co2.py` — abrupt-2×CO_2 on the freed q-flux model

Plot. The two SST time series and their difference $\Delta\mathrm{SST}(t)$ = forced − baseline (drift cancels in the difference).

Discussion. The script reports $\Delta\mathrm{SST} \approx +1.6$ K, about 0.43 K per W m^{-2}. Why is this a *proxy* and not ECS? (No top-of-atmosphere energy closure, no atmospheric radiative-feedback amplification, a cold-tropics base state.) Which feedbacks present in a full GCM are absent here, and which way would each move the number? Why is taking the *difference* of two branches more trustworthy than either branch alone?

21.3.5 Lab E — Take a gradient and run an inverse

Learning goal. Use differentiability directly: compute an exact sensitivity by reverse-mode autodiff (Bradbury et al., 2018), cross-check it against finite differences, and use a gradient to *calibrate* a target (Chapter 15).

Run. The sensitivity baseline, then a calibration:

```
python experiments/p0_amoc_gradient_baseline.py
python experiments/calibrate_amoc_fold.py -target 0.6 -kvel-rel 0.6
```

↪ *Code:* `experiments/p0_amoc_gradient_baseline.py` — $\partial(\mathrm{AMOC})/\partial(\mathrm{density})$ by grad + finite difference

↪ *Code:* `experiments/calibrate_amoc_fold.py` — AD fold-continuation calibration of F_{crit}

Plot. The grad value next to the finite-difference value (agreement check); the calibrated (`haline_gain`, `k_vel`) that places the fold at the target F_{crit}.

Discussion. The baseline finds $\partial(\mathrm{AMOC})/\partial(\mathrm{density}) \approx 0$ on the diagnostic-velocity ocean — what does a *zero* sensitivity tell you about the model's physics, and why is recording a zero a legitimate scientific result? Why can a tipping threshold *not* be tuned by backpropagating through the hosing sweep itself (the sensitivity diverges *at* the fold), and how does fold-continuation via the envelope theorem get an exact gradient anyway? Rank `k_vel` versus `haline_gain` as levers from the gradient.

21.3.6 Lab F — Recover a jet from rest (idealized dynamics)

Learning goal. See baroclinic instability and the eddy-driven jet emerge from a controlled idealized forcing, decoupled from observational boundary conditions (Held and Suarez, 1994) (Chapters 9, 18).

Run.

```
python experiments/dino_held_suarez.py
```

↪ *Code:* `experiments/dino_held_suarez.py` — DINOSAUR dry dycore with Held–Suarez forcing

Plot. The zonal-mean zonal wind: twin upper-tropospheric midlatitude westerlies (~ 22 m/s) over surface trades.

Discussion. Where does the eddy momentum flux that builds the surface westerlies come from, and why can the single-level model (Lab B) not produce it? Why does the textbook jet emerge here from an *isothermal* start but the realistic coupled run needs a symmetry-breaking seed? What does Held–Suarez deliberately leave out, and why is that the point?

21.4 An assessment / term-project idea

Project: a small intercomparison of one tunable knob. Each student (or pair) picks one model parameter — the GM coefficient `kappa_gm`, the salinity-sponge timescale, the `haline_gain`, the diffusion τ of the multi-level atmosphere — and conducts a miniature, self-contained intercomparison in the spirit of CMIP (Eyring et al., 2016). The deliverable is a short paper with: (i) a physical hypothesis for how the knob should change the climate; (ii) a 3–5 member ensemble of short runs spanning the knob; (iii) a validation scorecard (Lab B) for each member; (iv) where differentiability applies, the *gradient* of one target diagnostic with respect to the knob, cross-checked against the ensemble's finite difference; and (v) an honest discussion of which observed feature the knob improved, which it degraded, and what the model's documented limitations mean for the result. The runtime budget makes a real ensemble feasible inside a term, and the validation framework supplies a common, reproducible scoring standard so the class can pool results into one Taylor diagram.

A lighter alternative for a shorter course: reproduce one published figure from the book (the AMOC hysteresis loop, the Held–Suarez jet, the $2\times CO_2$ warming curve) from scratch, and write a one-page critique of what the figure does and does not establish.

21.5 Notes and exercises for instructors

A teaching model is only as good as the honesty with which it is taught. Chronos-ESM's greatest classroom value is that its failures are documented and reproducible: a student who measures the $\partial(\text{AMOC})/\partial\rho \approx 0$ blocker, or reads the near-zero MSLP pattern correlation, has learned more about how models work than one handed a polished CESM figure. Design the labs so the model is allowed to be wrong, and grade the interpretation, not the agreement.

Instructor Exercise 1 — Calibrate the difficulty of Lab C.
Before assigning the hosing lab, run the sweep yourself at two or three `haline_gain` values (e.g. 4, 6, 8) and time each on your teaching hardware. Identify a configuration that (a) opens a clearly visible hysteresis loop and (b) finishes inside your lab session. Write a half-page rubric distinguishing a student who found a *real* loop from one who mistook the ± 10 Sv branch noise for one.

Instructor Exercise 2 — Build an answer key for the honest-validation discussion.
Run Lab B yourself and, for each of the seven scored fields, write one sentence stating whether the model's skill is *earned*, *imposed*, or *absent*, with the physical reason. (Worked starting points: SST in the flux-corrected control is imposed; the ITCZ precipitation in the multi-level atmosphere is earned; the MSLP pattern is absent for lack of synoptic eddies.) Use it to grade interpretation rather than memorized numbers, which drift as the model is refined.

Instructor Exercise 3 — Map your syllabus onto the model's frontier.
The model is actively developed; some labs above exercise validated components and some exercise the research frontier (the prognostic ocean core, the multi-level atmosphere's surface winds). Before the term, classify each lab as *settled* or *frontier* for your offering, and decide explicitly which frontier results you will present as open questions rather than as answers. Document one place where the model has changed since this chapter was written, and adjust the affected lab.

The recurring meta-lesson across all three is the same one the labs teach the students: an Earth System Model is an argument, not an oracle. Teaching with a model whose source you can read, whose runs finish before the bell, and whose gradients you can take, lets a student examine that argument from the inside — which is the whole point.

Appendix A

Notation and Symbols

This appendix is a quick-reference for the mathematical notation used throughout the book. It is deliberately terse: each entry gives the printed symbol, the LaTeX source macro defined in the manual preamble, the meaning of the symbol, and the typical SI units in which it appears. All math in the manual is built from the macros listed here, so that a symbol always renders the same way and always means the same thing. If you are reading the source, you will find these macros defined once in `docs/manual/preamble.tex`; chapter authors are asked to use them rather than redefine local notation.

A few conventions hold book-wide. Vectors are set in bold (for example the horizontal velocity $\mathbf{u} = (u, v)$); scalars are set in italic. Horizontal operators act in the (λ, ϕ) plane on the sphere unless a chapter states otherwise. The material derivative $\frac{\mathrm{D}}{\mathrm{D}t}$ follows a fluid parcel and expands as $\frac{\mathrm{D}q}{\mathrm{D}t} = \frac{\partial q}{\partial t} + \mathbf{u}\cdot\boldsymbol{\nabla} q + w\,\frac{\partial q}{\partial z}$. Where a quantity has both a continuous (textbook) meaning and a discrete (code) counterpart, the continuous symbol is used in the equations and the mapping to the implementation is given in the relevant chapter and in Appendix C; numerical values of constants and tunable parameters are collected in Appendix B.

The governing equations that motivate this notation are introduced in 4 (ocean) and 9 (atmosphere); the standard references for the symbols and their physical meaning are (Vallis, 2017) and, for the overturning and box-model variables, (Stommel, 1961; Marotzke, 1991).

A.1 Calculus and operators

These are the differential operators used in the conservation laws. The horizontal gradient, divergence, and curl are understood on the sphere of radius a; their discrete spherical forms (with the metric factors $\cos\phi$ and the polar treatment) are given in 6.

Table A.1: Calculus and differential operators.

Symbol	Macro	Meaning	Acts on
$\frac{\partial q}{\partial t}$	`\pd`	Partial derivative (local rate of change)	scalar/field
$\frac{\partial^2 q}{\partial x^2}$	`\pdd`	Second partial derivative	scalar/field
$\frac{\mathrm{d}q}{\mathrm{d}t}$	`\dt`	Ordinary (total) time derivative	function of t
$\frac{\mathrm{D}q}{\mathrm{D}t}$	`\Dt`	Material (advective) derivative following a parcel	field
$\boldsymbol{\nabla} q$	`\grad`	Gradient operator $\boldsymbol{\nabla}$	scalar $\to$ vector
$\boldsymbol{\nabla}\cdot\mathbf{u}$	`\diver`	Divergence $\boldsymbol{\nabla}\cdot\mathbf{u}$	vector $\to$ scalar
$\boldsymbol{\nabla}\times\mathbf{u}$	`\curl`	Curl $\boldsymbol{\nabla}\times\mathbf{u}$	vector $\to$ vector
$\nabla^2 q$	`\lap`	Laplacian $\nabla^2 q$	scalar/field
$\mathrm{d}x$	`\dd`	Differential (integration / line element)	—

A.2 Fields and state variables

These are the prognostic and diagnostic fields carried in the model state. The velocity components (u, v, w) are zonal, meridional, and vertical respectively; potential temperature θ and salinity S set the density ρ through the equation of state, with ρ_0 a constant reference used in the Boussinesq approximation (see 4). The streamfunction ψ is used both for the horizontal barotropic flow and, reinterpreted as a meridional overturning streamfunction, for the AMOC diagnostic in 8.

Table A.2: Fields and state variables.

Symbol	Macro	Meaning	Typical units
$\mathbf{u}$	`\uvec`	Horizontal velocity vector (u, v)	$\mathrm{m\,s^{-1}}$
u	`\uu`	Zonal (eastward) velocity	$\mathrm{m\,s^{-1}}$
v	`\vv`	Meridional (northward) velocity	$\mathrm{m\,s^{-1}}$
w	`\ww`	Vertical velocity	$\mathrm{m\,s^{-1}}$
θ	`\Temp`	Potential temperature	K (°C)
S	`\Salt`	Salinity	psu ($\mathrm{g\,kg^{-1}}$)
ρ	`\dens`	Density	$\mathrm{kg\,m^{-3}}$
ρ_0	`\densz`	Reference (Boussinesq) density	$\mathrm{kg\,m^{-3}}$
p	`\pres`	Pressure	Pa
ψ	`\psib`	Streamfunction (barotropic / overturning)	$\mathrm{m^3\,s^{-1}}$ (Sv)
ζ	`\vort`	Relative vorticity $\boldsymbol{\nabla}\times\mathbf{u}\cdot\mathbf{k}$	$\mathrm{s^{-1}}$

Remark. The overturning streamfunction is reported in Sverdrups ($1\,\mathrm{Sv} = 10^6\,\mathrm{m^3\,s^{-1}}$, macro `\Sv`); the AMOC strength at 26.5°N is the central scalar diagnostic of 8 and 17.

A.3 Parameters and constants

These are the geometric, rotational, and gravitational quantities that parameterise the equations. The Coriolis parameter is $f = 2\Omega\sin\phi$ and its meridional gradient is $\beta = \frac{\partial f}{\partial y}$; on the sphere $y = a\,\phi$, so $\beta = (2\Omega/a)\cos\phi$. The latitude ϕ and longitude λ are the horizontal coordinates, and H denotes the ocean depth or column thickness. Standard numerical values are tabulated in Appendix B.

Table A.3: Parameters, constants, and coordinates.

Symbol	Macro	Meaning	Typical units / value
f	`\Cor`	Coriolis parameter $2\Omega\sin\phi$	$\mathrm{s^{-1}}$
β	`\beq`	Coriolis gradient $\frac{\partial f}{\partial y}$	$\mathrm{m^{-1}\,s^{-1}}$
a	`\Erad`	Earth radius	$6.371\times10^6\,\mathrm{m}$
Ω	`\Om`	Earth rotation rate	$7.292\times10^{-5}\,\mathrm{s^{-1}}$
g	`\grav`	Gravitational acceleration	$9.81\,\mathrm{m\,s^{-2}}$
ϕ	`\lat`	Latitude	rad (°)
λ	`\lon`	Longitude	rad (°)
H	`\dep`	Ocean depth / column thickness	m

A.4 Software notation

A handful of macros keep references to the implementation consistent. Inline source identifiers are set in a monospace font with the `\code` macro; the model itself is Chronos-ESM, written in JAX (the array/autodiff framework, Bradbury et al., 2018), with the multi-level atmospheric dynamical core

provided by DINOSAUR. The smooth rectifier $\mathrm{softplus}(x) = \log(1 + e^{x})$ appears where a positivity constraint must remain differentiable. Pointers from an equation to the file that implements it use the code-callout macro, for example:

↪ *Code:* `chronos_esm/ocean/diagnostics.py` — AMOC streamfunction ψ and the 26.5°N metric.

A complete map from symbols and equations to source files is given in Appendix C, and instructions for running the model in Appendix D.

Remark. Chronos-ESM is research software. Several components documented in this book are deliberate simplifications (for example the single-level atmosphere of 10 and the box-model view of the overturning in 17). Where a symbol stands for an idealised or parameterised quantity rather than a fully resolved one, the chapter that introduces it says so explicitly; this appendix only fixes the notation, not the fidelity of the physics it denotes.

Appendix B

Parameter Reference

This appendix is a single-place reference for the physical and numerical parameters that define a Chronos-ESM run. Every value listed here is the *actual* default in the source code as of writing; where a parameter is a keyword argument of a step routine, the default shown is the one that fires when the caller passes nothing. The intent is that a reader can reproduce, or sensibly perturb, a control integration without re-deriving constants from the governing equations. The physics behind each group is developed in the body chapters, which we cross-reference throughout: the ocean equations in 4, mixing and the Gent–McWilliams parameterization in 5, ocean numerics in 6, the overturning closure in 8, the multi-level atmosphere in 9, and the coupler and forcing in 13 and 14.

A standing caveat: Chronos-ESM is research software, and several of these numbers are not first-principles constants but *tuning knobs* chosen for numerical stability or for plausible large-scale behaviour at T31. The thermohaline closure (8) in particular is an interim box-model-style parameterization, and its coefficients have no observational counterpart; they are calibrated so that the AMOC sits in a realistic range and so that the salt-advection feedback can be pushed across a bifurcation in tipping experiments (17). Read those entries as control parameters, not measured quantities.

Symbols in the "Code symbol" column are the literal identifiers in the source; the home file is given so a value can be traced and changed at its definition.

B.1 Grid and resolution

The horizontal grid is selected at import time from the environment variable `CHRONOS_RESOLUTION` (default `T31`); both the ocean and the single-level atmosphere share the same regular latitude–longitude mesh, while the active DINOSAUR atmosphere runs on its own Gaussian grid at the same spectral truncation. The ocean uses a stretched vertical grid with a thin surface layer so that the surface heat capacity is realistic and SST responds to fluxes on the right timescale (6).

Parameter (code symbol)	Value	Units	Meaning
Truncation `RESOLUTION`	T31	—	spectral truncation (default)
Latitudes `NLAT`	48	—	ocean/atmos grid rows ($\approx 3.75°$)
Longitudes `NLON`	96	—	ocean/atmos grid columns
Ocean levels `NZ_OCEAN`	15	—	vertical H-levels
Total depth `OCEAN_TOTAL_DEPTH`	5000	m	ocean column depth
Stretch `stretch`	1.7	—	vertical grid stretching exponent
DINOSAUR layers `layers`	24	—	atmosphere σ-levels
DINOSAUR truncation `truncation`	T31	—	atmosphere spectral truncation
Earth radius `EARTH_RADIUS`, a	6.371×10^6	m	planetary radius

Table B.1: Grid and resolution. Interfaces are placed at $H\,(k/\mathrm{NZ})^{1.7}$, giving a ~ 50 m surface layer that thickens with depth; `OCEAN_DZ` and `OCEAN_DEPTH_CENTERS` are derived from this and stay mutually consistent.

↪ *Code:* `chronos_esm/config.py` — resolution, vertical grid, grid configs

B.2 Time steps and physical constants

Chronos-ESM sub-cycles the ocean and atmosphere inside an hourly coupling interval (13). The single-level atmosphere uses a very short 30 s step (forward Euler on gravity waves, see 10); the active DINOSAUR atmosphere uses an ImEx-RK3 scheme at a 20 min step (9). Both can be overridden from the environment for tuning.

Parameter (code symbol)	Value	Units	Meaning
Ocean step `DT_OCEAN`	900	s	ocean time step (15 min)
Atmos step `DT_ATMOS`	30	s	single-level atmosphere step
DINOSAUR step `dt_minutes`	20	min	multi-level atmosphere step
Coupling `COUPLING_INTERVAL`	3600	s	ocean–atmos exchange interval
CG tol. `CG_TOL`	10^{-6}	—	conjugate-gradient tolerance
CG max iter `CG_MAX_ITER`	1000	—	CG iteration cap
Softplus sharpness `EPSILON_SMOOTH`	10^{-3}	—	softplus scale (precip)
Gravity `GRAVITY`, g	9.81	$\mathrm{m\,s^{-2}}$	gravitational acceleration
Rotation `OMEGA`, Ω	7.2921×10^{-5}	$\mathrm{rad\,s^{-1}}$	Earth angular velocity
Seawater density `RHO_WATER`, ρ_0	1025	$\mathrm{kg\,m^{-3}}$	reference seawater density
Air density `RHO_AIR`	1.225	$\mathrm{kg\,m^{-3}}$	reference air density
$c_{p,\mathrm{air}}$ `CP_AIR`	1004	$\mathrm{J\,kg^{-1}\,K^{-1}}$	specific heat of dry air
$c_{p,\mathrm{sea}}$ `CP_WATER`	3985	$\mathrm{J\,kg^{-1}\,K^{-1}}$	specific heat of seawater
R_{air} `R_AIR`	287	$\mathrm{J\,kg^{-1}\,K^{-1}}$	gas constant, dry air
L_v `LATENT_HEAT_VAPORIZATION`	2.5×10^{6}	$\mathrm{J\,kg^{-1}}$	latent heat of vaporization
σ `STEFAN_BOLTZMANN`	5.67×10^{-8}	$\mathrm{W\,m^{-2}\,K^{-4}}$	Stefan–Boltzmann constant
p_0 `P0`	10^{5}	Pa	reference pressure

Table B.2: Time steps, solver tolerances, and physical constants. $\kappa = R_{\mathrm{air}}/c_{p,\mathrm{air}}$ is derived. Note that the hourly coupling interval is sub-cycled by both the ocean (900 s) and the atmosphere.

↪ *Code:* `chronos_esm/config.py` — time stepping, constants, tuning coefficients

B.3 Ocean physics

These are the defaults of `step_ocean` (the ocean stepper). They set the viscous and diffusive closure (5), the Gent–McWilliams eddy-bolus coefficient (Gent and McWilliams, 1990), the bottom drag entering the geostrophic balance, and the safety clamps that keep the integration physical (6). The equation of state is the linear form $\rho = \rho_0 - \alpha(\theta - \theta_0) + \beta(S - S_0)$ used throughout 4.
↪ *Code:* `chronos_esm/ocean/veros_driver.py` — step_ocean defaults, EOS, sponges, clamps

B.4 Thermohaline-closure knobs

The thermohaline overturning closure (`thc_overturning_velocity`, 8) adds a depth-integral-zero Atlantic meridional cell whose strength scales as `thc_k_vel` · softplus(contrast/`drho_scale`), where the contrast is the subpolar-minus-subtropical upper-ocean density difference, optionally haline-amplified. Because the cell carries no net transport it survives the barotropic corrector and gives density a clean pathway to the 26.5°N overturning, so that $\partial(\mathrm{AMOC})/\partial(\rho)$ is nonzero and sign-correct. Raising `thc_haline_gain` above unity amplifies the salt-advection feedback past a loop gain of one, making the AMOC bistable for the tipping experiments in 17 (Stommel, 1961; Rahmstorf et al., 2005; Marotzke, 1991). These are control knobs, not observed constants.
↪ *Code:* `chronos_esm/ocean/overturning.py` — thc_overturning_velocity, subpolar_hosing_salt_tendency

Parameter (code symbol)	Value	Units	Meaning
Lateral viscosity `Ah`	2.0×10^5	$\mathrm{m^2\,s^{-1}}$	harmonic momentum viscosity
Biharmonic visc. `Ab`	0	$\mathrm{m^4\,s^{-1}}$	biharmonic viscosity (off)
GM coefficient `kappa_gm`	1000	$\mathrm{m^2\,s^{-1}}$	Gent–McWilliams bolus
Tracer diff. `kappa_h`	500	$\mathrm{m^2\,s^{-1}}$	interior horizontal diffusivity
Biharmonic tracer `kappa_bi`	0	$\mathrm{m^4\,s^{-1}}$	biharmonic tracer diff. (off)
Smagorinsky `smag_constant`	0.1	—	dynamic-viscosity constant
Bottom drag `r_drag`	5.0×10^{-2}	$\mathrm{s^{-1}}$	linear drag in geostrophy
Barotropic drag `r_drag_bt`	$1/(86400 \cdot 25)$	$\mathrm{s^{-1}}$	Stommel 25-day drag
Shapiro filter `shapiro_strength`	0	—	$2\Delta x$ noise cleanup (off)
Polar diffusivity (interior)	20000	$\mathrm{m^2\,s^{-1}}$	enhanced diff. at $\lvert\phi\rvert$ poles
EOS α	0.2	$\mathrm{kg\,m^{-3}\,K^{-1}}$	thermal expansion
EOS β	0.8	$\mathrm{kg\,m^{-3}\,psu^{-1}}$	haline contraction
EOS θ_0	283.15	K	reference temperature
EOS S_0	35.0	psu	reference salinity
Salinity clip	[30, 38]	psu	hard stability clamp
Temperature clip	[250, 320]	K	hard stability clamp
Freezing floor	271.35	K	≈ -1.8°C sea-ice stub
Sponge salinity `S_REF`	35.0	psu	high-lat sponge target
Sponge timescale `tau_sponge`	180	days	polar ($> 65°$) relaxation
Global relax `tau_global`	3	yr	weak global salinity relaxation
Global mean `S_REF_GLOBAL`	34.7	psu	conserved volume-mean salinity

Table B.3: Ocean physics defaults from `step_ocean`. The two-tier salinity treatment (a strong 6-month polar sponge plus a weak 3-year global relaxation, with the volume mean renormalized each step) is described in 7; the clamps are stability nets and are not hit in a healthy control run.

B.5 Atmosphere

The active atmosphere is the DINOSAUR differentiable spectral primitive-equation dycore (9), run with Held–Suarez-style forcing (Held and Suarez, 1994) and the architecture behind NeuralGCM (Kochkov et al., 2024). The defaults below are the `DinoAtmosphere` constructor arguments. The eddy-permitting tuning is deliberate: the hyperdiffusion timescale is weak enough (6 h) that T31 baroclinic eddies can grow, and a small rotational wind seed breaks the exactly axisymmetric equilibrium so that mid-latitude surface westerlies are earned dynamically (18). The legacy single-level path (10) is retained but is not the active atmosphere.

↪ *Code:* `chronos_esm/atmos/dino_atmos.py` — DinoAtmosphere constructor, forcing, seed

B.6 Coupler and forcing

The coupler (13) computes bulk surface fluxes (Large and Yeager, 2004), applies an SST flux correction, and injects external radiative forcing. The CO_2 forcing follows the logarithmic Myhre form $F = 5.35\,\ln(C/C_0)\ \mathrm{W\,m^{-2}}$ (Myhre et al., 1998), zero at the preindustrial reference $C_0 = 280$ ppm, and is the channel for the forcing-response and CMIP-style experiments (14, 20) (Eyring et al., 2016). The flux correction has two modes: a strong Haney restoring of SST to the WOA target ($\tau \approx 30$ days, control mode), or a frozen q-flux with weak anomaly restoring (τ long, free / forcing-responsive mode) (Haney, 1971).

↪ *Code:* `chronos_esm/coupler/dino_coupling.py` — co2_forcing_wm2, ocean_fluxes_jax, restoring

↪ *Code:* `chronos_esm/coupler/dino_step.py` — DinoCoupledModel: interval, thc/flux-correction wiring

Parameter (code symbol)	Value	Units	Meaning
Velocity scale `thc_k_vel`	1.0×10^{-4}	$\mathrm{m\,s^{-1}}$	overturning strength sc
Haline gain `thc_haline_gain`	1.0	—	salt-feedback amplifica
Contrast depth `thc_contrast_depth_m`	None ($\rightarrow$ `upper_depth_m`)	m	layer for density contra
Limb depth `upper_depth_m`	1100	m	overturning-cell upper-
Contrast scale `drho_scale`	0.1	$\mathrm{kg\,m^{-3}}$	softplus contrast norm
Haline coeff. `BETA_S`	0.78	$\mathrm{kg\,m^{-3}\,psu^{-1}}$	$\partial\rho/\partial S$ for amplificatio
Subpolar band `subpolar`	(45, 65)	°N	dense-water-formation
Subtropical band `subtropical`	(10, 30)	°N	reference band
Hosing `hosing_sv`	0	Sv	subpolar freshwater fo
Hosing band `band`	(45, 65)	°N	hosing footprint

Table B.4: Thermohaline-closure and AMOC-hosing knobs. `thc_k_vel=0` disables the closure entirely. A shallow `thc_contrast_depth_m` ($\sim$ 300 m) gives a surface freshwater hosing more leverage on deep-water formation than the default 0–1100 m mean; see 17.

Parameter (code symbol)	Value	Units	Meaning
Diffusion timescale `diffusion_tau_hours`	6.0	h	hyperdiffusion damping time
Diffusion order `diffusion_order`	2	—	∇^2-order of the filter
Boundary-layer top `sigma_b`	0.7	—	σ of friction layer top
Friction rate `kf_per_day`	1.0	$\mathrm{day^{-1}}$	boundary-layer Rayleigh drag
Free-trop. relax `ka_per_day`	1/40	$\mathrm{day^{-1}}$	Newtonian cooling rate
Surface relax `ks_per_day`	1/4	$\mathrm{day^{-1}}$	near-surface relaxation rate
Min. equil. temp. `minT`	200	K	stratospheric θ_{eq} floor
Horizontal $\Delta\theta$ `dThz`	10	K	equator–pole θ_{eq} contrast
Condensation time `tau_cond_hours`	3.0	h	large-scale precip relaxation
Initial RH `init_rh`	0.5	—	initial relative humidity
Seed wind `seed_wind_ms`	2.0	$\mathrm{m\,s^{-1}}$	symmetry-breaking vorticity seed
Orography `orography`	True	—	ETOPO topography on/off

Table B.5: DINOSAUR atmosphere defaults (`DinoAtmosphere.__init__`). The time step and truncation appear in Tables B.1 and B.2.

B.7 A note on provenance and reproducibility

Two practical points close this reference. First, the resolution, both atmospheric time steps, and the contrast depth can be overridden at run time (`CHRONOS_RESOLUTION`, `CHRONOS_DT_ATMOS`, and constructor keywords), so a reproduced run should record the environment and the explicit keyword arguments, not merely cite this table. Second, several values — the THC knobs, the salinity sponge and global-relaxation timescales, the ocean clamps, and the hyperdiffusion time — are pragmatic choices that trade physical fidelity against the stability of a low-resolution, partly diagnostic ocean. They are documented here so that they can be changed honestly and with their consequences understood, which is the point of running a fully differentiable model (3, 15) (Bradbury et al., 2018; Vallis, 2017; Griffies, 2004; Semtner, 1976).

Parameter (code symbol)	Value	Units	Meaning
CO_2 coefficient	5.35	$\mathrm{W\,m^{-2}}$	Myhre log-forcing coefficient
CO_2 reference `co2_ref`	280	ppm	preindustrial reference (zero forcing)
Restoring time `RESTORE_TAU_DAYS`	30	days	control-mode SST restoring
Free-mode restoring `restore_tau_days`	3650	days	q-flux anomaly restoring
Drag coeff. `CD`	1.3×10^{-3}	—	bulk wind-stress drag
Coupler air density `RHO_AIR`	1.2	$\mathrm{kg\,m^{-3}}$	bulk-flux air density
Ocean drag `DRAG_COEFF_OCEAN`	1.2×10^{-3}	—	surface drag over ocean
Land drag `DRAG_COEFF_LAND`	2.0×10^{-3}	—	surface drag over land
Ocean albedo `ALBEDO_OCEAN`	0.07	—	shortwave albedo, ocean
Land albedo `ALBEDO_LAND`	0.30	—	shortwave albedo, land
Land emissivity `EMISSIVITY_LAND`	0.96	—	longwave emissivity, land
Wind-stress clip	± 0.3	Pa	τ_x, τ_y safety clamp
Net-heat clip	± 1500	$\mathrm{W\,m^{-2}}$	surface heat-flux clamp
Coupling interval `interval`	1.0	day	coupled-step interval

Table B.6: Coupler and forcing parameters. The two restoring timescales select the flux-correction mode; in control runs the model restores SST strongly to WOA (30 days), while forcing-response runs freeze a diagnosed q-flux and restore only anomalies on a multi-year timescale.

Appendix C

Code Map: Module to Chapter

This appendix is a directory: it maps every source module in the `chronos_esm/` package to the chapter(s) of this book that develop its theory and numerics, with a one-line statement of the module's role in the model. The intent is to let a reader who is staring at a piece of code jump to the place where it is explained — and, conversely, to let a reader who has just absorbed a chapter find the file that turns those equations into running JAX. Where a single module spans more than one chapter (`veros_driver.py`, for instance, touches the ocean equations, the mixing parameterizations, and the numerics) all relevant chapter links are given.

Honesty note. Chronos-ESM is research software, and the package contains code at three distinct maturity levels. (i) *Live* modules are wired into the integrated model and validated against observations. (ii) *Standalone / prototype* modules (the prognostic-momentum ocean core, the differentiable DINOSAUR coupler) are validated against analytic balances but not yet the default path; they are the building blocks of the working-ESM roadmap. (iii) *Dead / superseded* modules (`atmos/qtcm.py`, the `jcm` adapter) are kept only for historical reference. The status column below states which is which, so the reader is never misled about what the code actually does.

C.1 Top-level and infrastructure

These modules glue the components together, hold the global configuration, and handle data, I/O, and forcing. They are documented diffusely across the book; the "primary" chapter is the one where each is first put to substantive use.

C.2 Ocean

The ocean is the most developed component. The *live* ocean is the z-level, diagnostic-velocity driver in `veros_driver.py` (Chapters 4–6); the *prototype* prognostic-momentum core (`barotropic.py`, `momentum.py`) and the interim thermohaline closure (`overturning.py`) are the subject of Chapters 7–8.

C.3 Atmosphere

Two atmosphere paths coexist. The *multi-level* path wraps Google's DINOSAUR spectral primitive-equation dycore (Chapter 9) and is the active control-run atmosphere. The *single-level* barotropic path (Chapter 10) is the one inside the legacy `main.step_coupled`. The `qtcm.py` adapter is superseded.

C.4 Sea ice and land

C.5 Coupler

The coupler regrids fields between component grids and assembles the surface fluxes. The two `dino_*` modules are the differentiable multi-level coupling path (working-ESM roadmap), distinct from the numpy-helper standalone harness.

Module	Chapter(s)	Role / status
`main.py`	13	Integrated time loop coupling ocean, single-level atmosphere, ice, and land; the legacy `step_coupled` path. *Live.*
`config.py`	B	Resolution switch (T31/T63), grids, physical constants, default time steps. *Live.*
`data.py`	16, D	Downloads + regrids initial conditions and boundary data (WOA18, ETOPO) with periodic-longitude interpolation. *Live.*
`forcing.py`	14	Time-dependent external forcing (CO_2, solar, volcanic AOD) for historical/scenario runs. *Live.*
`io.py`	D	Saves/loads the coupled state to/from NetCDF (including `phi_s` topography). *Live.*
`log_utils.py`	D	Logging configuration. *Utility.*
`profile_model.py`	D	Throughput/timing harness for a coupled step. *Utility.*
`verify_stability.py`	6, D	Smoke test: integrates and checks for NaNs / clamp hits / drift. *Utility.*

Table C.1: Top-level and infrastructure modules.

Module	Chapter(s)	Role / status
`ocean/veros_driver.py`	4, 5, 6	Z-level primitive-equation ocean: tracer (θ, S) advection-diffusion (RK4), diagnostic thermal-wind $\mathbf{u}$, high-lat salinity sponge. The *live* ocean.
`ocean/mixing.py`	5	Isopycnal (Redi) diffusion + Gent–McWilliams eddy parameterization with differentiable slope limiters (Gent and McWilliams, 1990). *Live.*
`ocean/solver.py`	6	AD-safe conjugate-gradient elliptic solver for $\boldsymbol{\nabla}\cdot(H^{-1}\boldsymbol{\nabla}\psi) = \zeta$ (variable-coefficient invert). *Live.*
`ocean/bathymetry.py`	6	Turns a depth field $H(\lambda, \phi)$ into static 3-D wet masks (full-cell "staircase" coasts/floor) for no-flux boundaries (Griffies, 2004). *Live.*
`ocean/barotropic.py`	7	Prototype prognostic barotropic vorticity core: $\frac{\mathrm{d}\zeta}{\mathrm{d}t} = -\beta\frac{\partial\psi}{\partial x} - r\zeta + F + \nu\nabla^2\zeta$. Standalone, validated vs analytic gyres. *Prototype.*
`ocean/momentum.py`	7	Prototype prognostic baroclinic momentum: time-integrated $\frac{\mathrm{d}\mathbf{u}}{\mathrm{d}t}$ with Coriolis, hydrostatic PGF from ρ, friction. Standalone, validated vs thermal wind. *Prototype.*
`ocean/overturning.py`	8	Interim Stommel/Marotzke box-style thermohaline closure giving ρ a pathway to a coherent AMOC (Stommel, 1961; Marotzke, 1991). *Interim closure.*
`ocean/diagnostics.py`	8	Meridional / Atlantic overturning streamfunctions and the AMOC@26.5°N scalar metric (in Sv). *Live.*
`ocean/flux_correction.py`	13, 14	SST flux correction: Haney restoring (control) vs. frozen q-flux + weak anomaly restoring (forcing-responsive free run). *Prototype.*
`ocean/utils.py`	3	Differentiable `soft_clip` (tanh) replacing hard clips so gradients survive at bounds. *Utility.*

Table C.2: Ocean modules. The live ocean uses diagnostic velocities; the prognostic core is the standalone successor under development.

Module	Chapter(s)	Role / status
`atmos/dino_atmos.py`	9, 18, 19	Multi-level atmosphere on the DINOSAUR dycore (T31, σ-levels, ImEx-RK3): SST-coupled Held–Suarez forcing + prognostic moisture (Held and Suarez, 1994; Kochkov et al., 2024). *Active control-run atmosphere.*
`atmos/spectral.py`	9, 10	Spherical-harmonic / FFT spectral transforms and the (unused) Helmholtz solver for the single-level dynamics. *Support / partially live.*
`atmos/dynamics.py`	10	Single-level (barotropic) primitive-equation step: ζ, divergence, θ, $\ln p_s$, advected q/CO_2; wind relaxed to a climatological jet; strong hyperdiffusion. *Live (legacy path).*
`atmos/physics.py`	10, 14	Single-level physics: softplus precipitation (convection), radiative cooling, CO_2 forcing (Myhre et al., 1998). *Live (legacy path).*
`atmos/qtcm.py`	10	Quasi-equilibrium tropical circulation stepper. *Superseded / dead.*

Table C.3: Atmosphere modules. The DINOSAUR multi-level path is active for control runs; the single-level path drives the legacy coupled loop.

Module	Chapter(s)	Role / status
`ice/thermodynamics.py`	11	Semtner 0-layer thermodynamic ice: surface energy-balance solver, growth/melt, albedo (Semtner, 1976). *Live.*
`ice/driver.py`	11	Sea-ice component driver advancing the thermodynamic state each ocean step. *Live.*
`land/driver.py`	12	Land surface: bucket soil moisture, surface energy balance, runoff routed to coastal ocean. *Live.*
`land/vegetation.py`	12	Prognostic dynamic vegetation (leaf-area index) responding to temperature and soil moisture. *Live.*

Table C.4: Sea-ice and land-surface modules.

Module	Chapter(s)	Role / status
`coupler/state.py`	13	`CoupledState` / `FluxState` pytrees: the exchanged heat, freshwater, wind-stress, SST, and carbon fluxes. *Live.*
`coupler/regrid.py`	13	Dense-matrix conservative remapping between atmosphere and ocean grids. *Live.*
`coupler/dino_coupling.py`	13, 3	Pure-JAX (graph-uncut) coupling primitives: SST→DINOSAUR→bulk fluxes→ocean, fully differentiable. *Prototype (P0).*
`coupler/dino_step.py`	13, 9	One jittable/differentiable multi-level coupling interval carrying the DINOSAUR modal state alongside ocean/land. *Prototype (P1).*

Table C.5: Coupler modules. Flux correction itself lives in `ocean/flux_correction.py` (see the Ocean table) and is documented in Chapter 13.

C.6 Validation and proxy / data assimilation

The validation suite scores the model against WOA18 and ERA5 and produces the scorecard and figures; the proxy modules supply the forward operators and losses used in the differentiable data-assimilation experiments of Chapter 15.

Module	Chapter(s)	Role / status
`validation/grid.py`	16	Model grid helpers; regrids observations onto the model grid (bilinear, periodic-longitude). *Live.*
`validation/obs.py`	16	Observation/reanalysis loaders (WOA18, ERA5, RAPID AMOC) on native grids (Rahmstorf et al., 2005). *Live.*
`validation/metrics.py`	16	Area-weighted skill metrics (bias, RMSE, pattern correlation, std-ratio) with NaN masking. *Live.*
`validation/scorecard.py`	16	Orchestrates obs-vs-model scoring into the printable validation scorecard. *Live.*
`validation/plots.py`	16, 21	Validation figures styled for Nature-family submission (vector PDF). *Live.*
`proxy/forward_ops.py`	15	Forward operators mapping state (θ, S) to proxies (e.g. the δ^{18}O calcite paleotemperature equation of Bemis et al. (1998)). *Live.*
`proxy/sensor.py`	15	Sparse-observation sensor masking and interpolation. *Live.*
`proxy/loss.py`	15, 3	Differentiable loss functions for gradient-based (4D-Var-style) data assimilation (Bradbury et al., 2018). *Live.*

Table C.6: Validation and proxy / data-assimilation modules. CMIP-style output conventions are discussed in Chapter 20 (Eyring et al., 2016).

C.7 Where the climate test cases live

The canonical experiments of Part IV are driven from the `experiments/` directory rather than the package itself, but they exercise specific modules. The AMOC hysteresis/tipping case (Chapter 17) drives `ocean/overturning.py`, `ocean/diagnostics.py`, and `ocean/flux_correction.py`; the jets-and-storm-tracks case (Chapter 18) and the tropics/monsoon case (Chapter 19) both exercise `atmos/dino_atmos.py` via the DINOSAUR control harness (Vallis, 2017). The forcing-response experiments (Chapter 14) couple `forcing.py`, `atmos/physics.py`, and `ocean/flux_correction.py`.

Remark. The single source of truth is the code itself. Where this map and a module's docstring disagree about status, trust the docstring — the package evolves faster than the book, and each module's leading docstring states its current maturity and known limitations.

Appendix D

Installation and Running

This appendix is the practical companion to the conceptual chapters: how to install Chronos-ESM, stage its input data, and run it on three increasingly large targets — a laptop CPU, a single GPU, and a SLURM cluster. Every command below is taken from the repository (`README.md`, `requirements.txt`, and the SLURM scripts under `experiments/`) and should be run from the repository root unless stated otherwise. Because Chronos-ESM is research software, treat these instructions as a recipe that is validated against the pinned dependency stack (see Appendix B for parameter values and Appendix C for the code map).

D.1 Software stack and the JAX pin

Chronos-ESM is a JAX program. The single most important constraint is that the JAX version is *pinned*: the multi-level atmosphere is built on the DINOSAUR dycore (`dinosaur-dycore==1.2.1`), which uses fast-moving private JAX APIs and breaks on newer JAX releases. The repository therefore pins

```
jax==0.8.1
jaxlib==0.8.1
dinosaur-dycore==1.2.1
```

Do *not* upgrade JAX to chase a newer feature: doing so silently breaks the DINOSAUR core. The remaining dependencies are standard scientific Python — `numpy`, `scipy`, `xarray`, `netCDF4`, `matplotlib`, and `pooch` (which manages the automatic data cache, Section D.4) (Harris et al., 2020; Virtanen et al., 2020; Hoyer and Hamman, 2017; Hunter, 2007; Ueda et al., 2020; Google Research, 2024). The optional `cdsapi` is needed only for ERA5 validation downloads.

D.2 Cloning and a CPU install

The default install gives you a *CPU* build of JAX, which is the right choice for a laptop, for the test suite, and for short experiments. Clone the repository, create a virtual environment, and install the pinned requirements:

```
git clone https://github.com/bijanf/ESMB.git && cd ESMB
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
```

This installs CPU JAX. Verify the install by asking JAX which devices it can see; on a CPU-only box you should see a `CpuDevice`:

```
python -c "import jax; print(jax.devices())"
```

D.3 The GPU (CUDA) build

`requirements.txt` deliberately installs CPU JAX so the package is usable everywhere. For GPU you must additionally install the matching CUDA wheel *at the same JAX version* — mismatched versions

are the most common cause of a silent fall-back to CPU or an import error. The robust idiom reads the installed version and pins the CUDA build to it:

```
pip install "jax[cuda12]==$(python -c 'import jax; print(jax.__version__)')"
```

This pulls the bundled `nvidia-*` wheels, so you do not need a system-wide CUDA toolkit for the libraries themselves (you still need a compatible NVIDIA driver). Confirm the GPU is visible before launching any long run:

```
python -c "import jax; print(jax.devices())"   # expect a CudaDevice / gpu
```

On a single T31 GPU the model runs roughly 280 simulated years per day; the coupled DINOSAUR↔ocean control runs near 30 model-years per hour. The GPU jobs enable double precision via `JAX_ENABLE_X64=True`.

D.4 Staging the input data

The model's input datasets are *not* committed to the repository. On first use they download automatically through `pooch` and cache to `~/.cache/chronos_esm/` (`pooch.os_cache("chronos_esm")`). Two datasets are required for any run, a third only for validation:

Dataset	Used for	Size	Source
WOA18 T+S (5°)	ocean IC + validation	∼ 8 MB	NOAA NCEI
ETOPO1 bathymetry	land mask + orography	∼ 900 MB	SOCIB THREDDS
ERA5 monthly means	validation only	∼ MBs	Copernicus CDS

WOA18 supplies the ocean initial θ/S fields and the validation target; ETOPO1 supplies the variable-depth bathymetry (the ocean land mask) and the atmospheric orography. ERA5 is needed only when you pass `-era5` to a validation script, and it requires a Copernicus CDS account configured in `~/.cdsapirc`.

You rarely need to stage data by hand for a local CPU run — the first model build downloads what it needs. The explicit staging tool,

↪ *Code:* `experiments/prefetch_data.py` — the data-cache prefetcher

, exists mainly for clusters (Section D.7) but is useful anywhere:

```
python experiments/prefetch_data.py           # fetch WOA18 + ETOPO1 into the cache
python experiments/prefetch_data.py --era5    # also fetch ERA5 (needs ~/.cdsapirc)
python experiments/prefetch_data.py --build   # + build the model once (full preflight)
python experiments/prefetch_data.py --check   # verify the cache only; exit 1 if missing
```

If your `$HOME` is space-constrained, redirect the cache by exporting `XDG_CACHE_HOME` to a larger filesystem *before* both the prefetch and the run, so they agree on the path.

D.5 Running locally on CPU

For development and small experiments, run on CPU and force the platform explicitly so a stray CUDA install cannot change behaviour. The repository idiom prepends the repo root to `PYTHONPATH` (the experiment scripts also do this internally, but it makes ad-hoc invocations robust) and pins the platform with `JAX_PLATFORMS=cpu`:

```
PYTHONPATH=$PWD JAX_PLATFORMS=cpu python experiments/run_century_physics.py \
    --years 1 --ocean-ic woa
```

A one-year (or a few-day) run is the natural smoke test: it exercises the full coupled step, downloads any missing data, and finishes in minutes on a laptop. For the multi-level coupled control run, the same pattern applies:

```
# a short coupled dinosaur <-> ocean control run on CPU:
PYTHONPATH=$PWD JAX_PLATFORMS=cpu python experiments/run_dino_coupled.py --days
    5
# resume an existing run toward a larger total (here, toward day 36500):
PYTHONPATH=$PWD JAX_PLATFORMS=cpu python experiments/run_dino_coupled.py \
    --years 100 --resume 36500
```

The harness
↪ *Code:* `experiments/run_dino_coupled.py` — the coupled control-run driver
is checkpointing and resumable: `-years`/`-days` give the *absolute* total, `-resume <day>` restarts from a saved day, and `-ckpt-every-days` sets the checkpoint cadence (yearly by default). Each checkpoint writes both the ocean state (`state_d<DAY>.nc`) and the DINOSAUR modal atmosphere (`state_d<DAY>_dino.npz`). Resuming restores the prognostic state (ocean θ/S and the modal dynamics) exactly; diagnostic fields — the ocean streamfunction warm-start, density ρ, vertical velocity w, and DIC — are recomputed, so a restart is state-complete but not bit-reproducible (a divergence of order 0.03 K/week, far below any climate signal).

D.6 Running on a single GPU

On a machine with a GPU, install the CUDA build (Section D.3) and simply omit the `JAX_PLATFORMS=cpu` pin so JAX selects the GPU. The GPU jobs set double precision and the resolution explicitly:

```
export CHRONOS_RESOLUTION=T31
export JAX_PLATFORM_NAME=gpu
export JAX_ENABLE_X64=True
python experiments/run_dino_coupled.py --years 100 --outdir outputs/
    dino_control
```

Always confirm the GPU is actually in use before committing to a long run — a model that silently fell back to CPU will be orders of magnitude slower:

```
python -c "import jax; print(jax.devices())"   # must list a gpu device
```

D.7 Submitting to a SLURM cluster

Compute nodes on most clusters have *no internet*, so the workflow is: stage the data once on a login node (which does have internet), then submit the job, which reads from the shared cache. The full login-node sequence is:

```
git clone https://github.com/bijanf/ESMB.git && cd ESMB
module load anaconda cuda/12.3.1            # site-specific module names
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
# GPU JAX is REQUIRED on the cluster (the SLURM script aborts without a GPU):
pip install "jax[cuda12]==$(python -c 'import jax; print(jax.__version__)')"

# stage WOA18 + ETOPO1 into ~/.cache/chronos_esm on the LOGIN node:
python experiments/prefetch_data.py         # add --era5 to also stage
    validation data
```

Then submit. The control-run script,
↪ *Code:* `experiments/run_dino_control_slurm.sh` — the SLURM control-run wrapper
, forwards all arguments to `run_dino_coupled.py`:

```
sbatch experiments/run_dino_control_slurm.sh --years 100
tail -f logs/dino_control_*.log
```

D.7.1 What the SLURM script does for you

The wrapper encodes several cluster conventions that are worth understanding, because you will likely need to adapt them to your site. Its resource request is a single GPU node:

```
#SBATCH --partition=gpu
#SBATCH --qos=gpumedium
#SBATCH --account=poem
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --gres=gpu:1
#SBATCH --mem=64G
#SBATCH --time=23:00:00
```

Adjust for your site: the `-account`, `-partition`, and `-qos` lines (the defaults `poem`/`gpu`/`gpumedium` are PIK-specific) and the `module load` names. The script then:

1. **Loads modules and the venv.** It loads `python/3.12.3` (must match the Python the venv was built with) and `cuda/12.6.0` (must match the `jax[cuda12]` build), then activates `venv`. It also prepends the bundled `nvidia-*` wheel library paths to `LD_LIBRARY_PATH`.
2. **Runs a GPU preflight.** A tiny Python snippet asserts that JAX sees a GPU device and aborts the job immediately if not, so a multi-hour allocation is never wasted on a CPU fall-back.
3. **Runs a data-cache guard.** It calls `prefetch_data.py -check`; if WOA18/ETOPO1 are not staged, the job fails fast with instructions to run the prefetch on a login node, rather than hanging on a blocked download.
4. **Writes output to scratch, never `$HOME`.** Output goes to `$CHRONOS_OUTDIR` (PIK default `/p/tmp/$USER/dino_control`); override it by exporting `CHRONOS_OUTDIR` to your own scratch path.
5. **Auto-resumes across jobs.** If you do not pass `-resume`, the script scans the output directory for the latest `state_d<DAY>.nc` checkpoint and resumes from it. This is what lets you chain back-to-back 23-hour jobs against a single `-years` target until the run completes:

```
# first job (fresh): runs until walltime, checkpointing yearly to scratch
sbatch experiments/run_dino_control_slurm.sh --years 100
# subsequent jobs (same command): AUTO-RESUME from the latest checkpoint
sbatch experiments/run_dino_control_slurm.sh --years 100
# explicit resume overrides the auto-detect:
sbatch experiments/run_dino_control_slurm.sh --years 200 --resume 36500
```

If your `$HOME` quota is small, export `XDG_CACHE_HOME=/path/on/shared/scratch` *before both* the prefetch and the `sbatch` — the cache must be the same path, visible from both the login and compute nodes.

D.7.2 The AMOC hosing sweep

A second SLURM wrapper,
↪ *Code:* `experiments/run_amoc_hosing_slurm.sh` — the AMOC hosing/hysteresis sweep
, drives the freshwater-hosing experiment behind the bistability and tipping results of Chapter 17. It uses the same module/venv/GPU preflight preamble but requests a preemptible QOS and forwards all arguments to `run_amoc_hosing.py`, which auto-resumes from per-hosing-level checkpoints (so a `-requeue` after preemption continues rather than re-spinning up):

```
sbatch --dependency=afterok:<control_jobid> --requeue \
    experiments/run_amoc_hosing_slurm.sh \
    --ckpt /p/tmp/$USER/dino_control_thc/state_d036500 \
    --fmax 1.0 --nsteps 5 --hold-years 15 \
    --outdir /p/tmp/$USER/amoc_hosing
```

The sweep ramps the subpolar freshwater hosing up to `-fmax` (in Sv) in `-nsteps` stages, holding each for `-hold-years` so the overturning equilibrates; the `-dependency=afterok` chains it after a finished control run.

D.8 Scoring a run against observations

Once a run has written checkpoints, score them against WOA18 (ocean θ/S) and ERA5 (near-surface atmosphere) with the validation pipeline (Chapter 16). It accepts a glob of checkpoints, which it averages into a climatology:

```
# score saved checkpoints (scorecard + bias maps + zonal means):
python experiments/validate_control.py --states "outputs/dino_control/state_d*.nc" --era5
# or exercise the pipeline on a short in-process run:
python experiments/validate_control.py --demo 200 --era5
# refresh the README dashboard figures from a saved glob:
python experiments/make_readme_figures.py "outputs/dino_control/state_d*.nc" \
    --label "dino control"
```

A perfect model scores bias 0, correlation 1, and standard-deviation ratio 1; the current scorecards (and their honest limitations) are discussed in Chapters 16–19.

D.9 Running the test suite

The test suite under `tests/` is the fastest way to confirm an install is healthy and to guard changes. It runs on CPU; use the same platform pin as for local runs:

```
PYTHONPATH=$PWD JAX_PLATFORMS=cpu python -m pytest
```

The tests cover the ocean and atmosphere components, the coupler and flux correction, the AMOC and momentum diagnostics, the DINOSAUR coupling and time step, the CO_2 warming end-to-end path, and — crucially for a differentiable model — the adjoint/gradient checks. Run a single module, for example the adjoint tests, with the usual pytest selector:

```
PYTHONPATH=$PWD JAX_PLATFORMS=cpu python -m pytest tests/test_adjoint.py -q
```

The pytest configuration in `pyproject.toml` fixes the test path to `tests/` and silences deprecation warnings, so a bare `pytest` invocation from the repository root behaves identically. With the test suite green and a short coupled run completing without NaNs, the install is ready for the experiments of Part III.

Bibliography

Alistair Adcroft, Chris Hill, and John Marshall. Representation of topography by shaved cells in a height coordinate ocean model. *Monthly Weather Review*, 125(9):2293–2315, 1997.

Christopher Amante and Barry W. Eakins. Etopo1 1 arc-minute global relief model: Procedures, data sources and analysis. Technical Report NOAA Technical Memorandum NESDIS NGDC-24, National Geophysical Data Center, NOAA, 2009.

Akio Arakawa and Vivian R. Lamb. A potential enstrophy and energy conserving scheme for the shallow water equations. *Monthly Weather Review*, 109(1):18–36, 1981.

Bryan E. Bemis, Howard J. Spero, Jelle Bijma, and David W. Lea. Reevaluation of the oxygen isotopic composition of planktonic foraminifera: Experimental results and revised paleotemperature equations. *Paleoceanography*, 13(2):150–160, 1998.

James Bradbury et al. JAX: composable transformations of Python+NumPy programs. http://github.com/google/jax, 2018.

Gokhan Danabasoglu and James C. McWilliams. Sensitivity of the global ocean circulation to parameterizations of mesoscale tracer transports. *Journal of Climate*, 8(12):2967–2987, 1995.

Veronika Eyring et al. Overview of the coupled model intercomparison project phase 6 (cmip6) experimental design and organization. *Geoscientific Model Development*, 9(5):1937–1958, 2016.

Peter R. Gent and James C. McWilliams. Isopycnal mixing in ocean circulation models. *Journal of Physical Oceanography*, 20(1):150–155, 1990.

Google Research. Dinosaur: differentiable dynamics for global atmospheric modeling. https://github.com/google-research/dinosaur, 2024.

Stephen M. Griffies. *Fundamentals of Ocean Climate Models*. Princeton University Press, 2004.

Dion Häfner, René Løwe Jacobsen, Carsten Eden, Mads R. B. Kristensen, Markus Jochum, Roman Nuterman, and Brian Vinter. Veros v0.1 – a fast and versatile ocean simulator in pure python. *Geoscientific Model Development*, 11(8):3299–3312, 2018.

Robert L. Haney. Surface thermal boundary condition for ocean circulation models. *Journal of Physical Oceanography*, 1(4):241–248, 1971.

Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020.

Isaac M. Held and Max J. Suarez. A proposal for the intercomparison of the dynamical cores of atmospheric general circulation models. *Bulletin of the American Meteorological Society*, 75(10):1825–1830, 1994.

Hans Hersbach, Bill Bell, Paul Berrisford, et al. The era5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society*, 146(730):1999–2049, 2020.

Stephan Hoyer and Joseph Hamman. xarray: N-D labeled arrays and datasets in Python. *Journal of Open Research Software*, 5(1):10, 2017.

John D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3): 90–95, 2007.

IPCC. *Climate Change 2021: The Physical Science Basis. Contribution of Working Group I to the Sixth Assessment Report of the Intergovernmental Panel on Climate Change*. Cambridge University Press, 2021.

Dmitrii Kochkov et al. Neural general circulation models for weather and climate. *Nature*, 632:1060–1066, 2024.

William G. Large and Stephen G. Yeager. Diurnal to decadal global forcing for ocean and sea-ice models: The data sets and flux climatologies. NCAR Technical Note NCAR/TN-460+STR, National Center for Atmospheric Research, 2004.

William G. Large, Gokhan Danabasoglu, Scott C. Doney, and James C. McWilliams. Sensitivity to surface forcing and boundary layer mixing in a global ocean model: Annual-mean climatology. *Journal of Physical Oceanography*, 27(11):2418–2447, 1997.

Ricardo A. Locarnini, Alexey V. Mishonov, Olga K. Baranova, Timothy P. Boyer, Melissa M. Zweng, et al. World ocean atlas 2018, volume 1: Temperature. Technical Report NOAA Atlas NESDIS 81, NOAA, 2018. A. Mishonov, Technical Editor.

Syukuro Manabe. Climate and the ocean circulation: I. the atmospheric circulation and the hydrology of the earth's surface. *Monthly Weather Review*, 97(11):739–774, 1969.

Jochem Marotzke. Influence of convective adjustment on the stability of the thermohaline circulation. *Journal of Physical Oceanography*, 21(6):903–907, 1991.

Gunnar Myhre, Eleanor J. Highwood, Keith P. Shine, and Frode Stordal. New estimates of radiative forcing due to well mixed greenhouse gases. *Geophysical Research Letters*, 25(14):2715–2718, 1998.

Stefan Rahmstorf. A fast and complete convection scheme for ocean models. *Ocean Modelling (unpublished manuscript series)*, 101:9–11, 1993.

Stefan Rahmstorf et al. Thermohaline circulation hysteresis: A model intercomparison. *Geophysical Research Letters*, 32(23), 2005.

Martha H. Redi. Oceanic isopycnal mixing by coordinate rotation. *Journal of Physical Oceanography*, 12 (10):1154–1158, 1982.

Albert J. Semtner. A model for the thermodynamic growth of sea ice in numerical investigations of climate. *Journal of Physical Oceanography*, 6(3):379–389, 1976.

Ralph Shapiro. Smoothing, filtering, and boundary effects. *Reviews of Geophysics*, 8(2):359–387, 1970.

David A. Smeed, Simon A. Josey, Claudie Beaulieu, et al. The north atlantic ocean is in a state of reduced overturning. *Geophysical Research Letters*, 45(3):1527–1533, 2018.

Henry Stommel. Thermohaline convection with two stable regimes of flow. *Tellus*, 13(2):224–230, 1961.

Harald U. Sverdrup. Wind-driven currents in a baroclinic ocean; with application to the equatorial currents of the eastern pacific. *Proceedings of the National Academy of Sciences*, 33(11):318–326, 1947.

Karl E. Taylor. Summarizing multiple aspects of model performance in a single diagram. *Journal of Geophysical Research: Atmospheres*, 106(D7):7183–7192, 2001.

Leonardo Uieda, Santiago R. Soler, Rémi Rampin, et al. Pooch: A friend to fetch your data files. *Journal of Open Source Software*, 5(45):1943, 2020.

Geoffrey K. Vallis. *Atmospheric and Oceanic Fluid Dynamics: Fundamentals and Large-Scale Circulation*. Cambridge University Press, 2nd edition, 2017.

Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020.

Melissa M. Zweng, James R. Reagan, Dan Seidov, Timothy P. Boyer, Ricardo A. Locarnini, et al. World ocean atlas 2018, volume 2: Salinity. Technical Report NOAA Atlas NESDIS 82, NOAA, 2018. A. Mishonov, Technical Editor.